

THE DEFINITIVE GUIDE TO

DAX

Module 1: Foundations & Core Mechanics

*Evaluation context · Calculated columns vs Measures · CALCULATE · Filter functions
Iterator functions · RELATED/RELATEDTABLE · Time intelligence · Variables & patterns*

Advanced Developer Edition

PL-300 Exam Ready

Most Tested Area in the Exam

How to Use This Guide

Module 1 covers the most heavily tested area of the PL-300 exam. DAX is not a spreadsheet formula language — it is an expression language built on two orthogonal evaluation contexts that govern every single calculation. Without understanding context, you cannot reason about why a measure returns the wrong number or why a calculated column behaves unexpectedly. This guide explains the mechanics of every significant DAX concept: what the engine is actually doing internally, why the syntax is designed the way it is, and what the failure modes look like.

- 1 MECHANICS** The internal model — how the DAX engine evaluates each function, what it returns, and why.
- 2 ANNOTATED CODE** Every parameter explained inline so you see exactly why the output is what it is.
- 3 PATTERNS** Reusable professional patterns that compose multiple functions into real solutions.

■ How DAX Does It

Module 1 assumes you are comfortable with Power BI Desktop: you can create reports, load data, and define basic relationships. You do not need prior DAX experience.

DAX composes heavily with data model relationships — the relationship diagram is always in your head when you write a measure.

The Two Things That Govern Every DAX Calculation

- | | |
|--------------------|---|
| Row Context | Row context exists when DAX evaluates an expression one row at a time. It is created automatically in calculated columns and explicitly by iterator functions (SUMX, AVERAGEX, etc.). In row context, column references like [Price] resolve to the value in the current row. |
|--------------------|---|

Filter Context

Filter context is the set of filters active on every column of every table at any moment. It is defined by slicers, visual filters, row/column headers in a matrix, and explicit CALCULATE filter arguments. Every measure evaluates inside a filter context. Without filter context there is no such thing as a meaningful aggregate.

Lesson 1 Evaluation Context

Row Context · Filter
Context · Context
Transition

Every DAX expression evaluates inside a context. Context is not a setting you configure — it is the fundamental runtime environment that determines what any column reference, function call, or measure returns. There are two kinds of context, and they interact in a non-obvious way that trips up even experienced developers.

Row Context — One Row at a Time

Row context is the context in which DAX evaluates an expression for a specific row. It knows "which row am I on right now." Row context is created in two places: automatically for every row when a calculated column is being defined, and explicitly inside iterator functions (SUMX, AVERAGEX, MAXX, etc.) for each row they visit.

```
// Calculated column – row context is automatic

// The engine evaluates this for every row of the Sales table

Revenue =

Sales[Quantity] * Sales[UnitPrice]

// [Quantity] and [UnitPrice] resolve to THIS ROW's values
// because row context is active

// Iterator – explicit row context over a table

Total Revenue :=

SUMX(

Sales, -- iterate over every row of Sales

Sales[Quantity] * Sales[UnitPrice] -- evaluate per row

)

// Inside SUMX's second argument, row context is active for each Sales row
// [Quantity] and [UnitPrice] resolve correctly because SUMX creates row context
```

■ How DAX Does It

Row context does NOT automatically cross relationships. If Sales has a related Products table, writing Products[Category] inside a calculated column on Sales will return an error — you need RELATED() to follow the relationship.

Measures do NOT have row context by default. Inside a measure, [Column] means the entire column as a vector — not a single row. This is why SUM(Sales[Revenue]) works in a measure but Sales[Revenue] alone does not.

Filter Context — The Active Set of Filters

Filter context is the invisible hand that shapes every aggregate. When a user drops "Total Sales" onto a matrix with Region on rows and Quarter on columns, each cell evaluates the measure in a different filter context: the cell for East / Q2 has a filter on Region = "East" AND Quarter = "Q2". The measure SUM(Sales[Revenue]) returns only the rows that survive those filters.

```
// This measure is identical in every cell of every visual

Total Sales :=
SUM(Sales[Revenue])

// But it returns DIFFERENT numbers because the filter context changes:
// Cell [East, Q1] → filter context: Region=East, Quarter=Q1 → returns 12400
// Cell [West, Q2] → filter context: Region=West, Quarter=Q2 → returns 10000
// Grand Total cell → filter context: no region/quarter filter → returns all rows

// Filter context is built from:
// 1. Slicers and page-level filters
// 2. Visual filters (filter pane)
// 3. Row/column headers in a matrix or table visual
// 4. CALCULATE() filter arguments (explicit, in DAX code)
```

Context Transition — The Advanced Concept

Context transition is what happens when CALCULATE is called inside a row context. CALCULATE converts the current row context into an equivalent filter context — it adds a filter for every column of the current row. This is one of the most powerful and most misunderstood mechanics in DAX.

```
// Calculated column that calls a MEASURE – triggers context transition
// (Rare but important to understand)

Sales[RunningTotal] =
CALCULATE(
    [Total Sales],
    Sales[OrderDate] <= EARLIER(Sales[OrderDate])
)

// What happens:
// 1. For each row, row context is active (calculated column)
// 2. CALCULATE converts that row context into a filter context
// that filters ALL columns of that row to their current values
// 3. EARLIER() retrieves the value from the OUTER row context
// (the current row being iterated)

// Modern alternative: use VAR to capture the current row value
Sales[RunningTotal] =
VAR CurrentDate = Sales[OrderDate]
RETURN
CALCULATE(
    [Total Sales],
    Sales[OrderDate] <= CurrentDate
)

// VAR captures the value in the current row context.
// Inside RETURN, the row context is gone – CALCULATE works cleanly.
```

■ PL-300 Exam

Context transition is tested in PL-300 in these forms:

- "Why does a calculated column referencing a measure give unexpected results?"
- "What does CALCULATE do when called inside an iterator?"
- "What is the difference between EARLIER() and VAR in a calculated column?"

Know: CALCULATE in row context = context transition. VAR is the modern, safer way to capture row values inside nested contexts.

■■ Watch Out

A common mistake: writing a measure reference inside a calculated column WITHOUT CALCULATE. Example: Sales[Check] = [Total Sales] — this evaluates the measure in the current row context with NO context transition. The measure sees ALL rows (no filter), and every row returns the grand total. Wrap in CALCULATE to trigger context transition.

Another trap: RELATED() in an iterator. RELATED() follows the relationship from the current row context — this works correctly inside SUMX but fails inside a measure with no row context.

Lesson 2 Calculated Columns and Measures

Storage · Context ·
When to Use Each

The choice between a calculated column and a measure is architectural, not aesthetic. They are stored differently, evaluated at different times, and have access to different evaluation contexts. Choosing the wrong one creates either a model that is too large or a calculation that returns the wrong result.

Calculated Columns — Defined at Refresh Time

A calculated column is computed once when the data model is refreshed. The result is stored in memory as if it were a native column imported from the source. Every row gets a value. Because calculation happens at refresh, row context is active — the engine knows which row it is computing.

```
// Calculated column – evaluated at REFRESH time, stored per row

Sales[Revenue] = Sales[Quantity] * Sales[UnitPrice]

// Conditional tier – each row gets its own Tier value stored in memory

Sales[Tier] =
IF(Sales[Revenue] > 5000, "Gold",
IF(Sales[Revenue] > 2000, "Silver", "Bronze")
)

// Accessing related table – RELATED follows the many-to-one relationship

Sales[ProductCategory] = RELATED(Products[Category])

// Date parts for faster slicing

Calendar[Year] = YEAR(Calendar[Date])

Calendar[Quarter] = "Q" & FORMAT(Calendar[Date], "Q")

Calendar[MonthNo] = MONTH(Calendar[Date])

Calendar[MonthName] = FORMAT(Calendar[Date], "MMMM")
```

Measures — Defined at Query Time

A measure stores no data. It is a formula that is evaluated fresh every time a visual requests it, inside whatever filter context is active at that moment. A measure has no implicit row context — it operates on sets of rows that survive the current filter context. All aggregation (SUM, AVERAGE, COUNT, SUMX) happens inside the measure.

```
// Measure – evaluated at QUERY time, inside the current filter context

// The := operator defines a measure (vs = for calculated column)

[Total Sales] :=
SUM(Sales[Revenue])

// This returns different values in every cell depending on filter context
// No data is stored – calculated fresh on every report interaction

// Measures can reference other measures – composition is normal and encouraged

[Average Order Value] :=
DIVIDE([Total Sales], [Order Count], 0)

// DIVIDE(numerator, denominator, alternate_result_if_zero)
// ALWAYS use DIVIDE instead of / to handle division by zero safely

[Order Count] :=
COUNTROWS(Sales)

// Counts ALL rows that survive the current filter context
```

	Calculated Column	Measure
Evaluated at	Refresh time (model load)	Query time (visual render)
Storage	Stored in memory per row — increases model size	No storage — formula only
Context	Row context active (knows current row)	Filter context only (no row context)
Use for	Segmentation, slicing, RELATED lookups, date parts	Aggregates, KPIs, ratios, % totals
Can reference	Same-table columns, RELATED columns	Other measures, columns via aggregation
Visible in	Tables, visuals, can be used as axis/slicer	Values well only — cannot slice by a measure

■ PL-300 Exam

PL-300 traps on this topic:

- "When should you use a calculated column vs a measure?" — if you need to slice/filter by the result, it must be a calculated column. If it is an aggregate KPI, it must be a measure.
- "Why is a measure showing the same value in every row of a table visual?" — the measure has no row context. It is likely summing all rows regardless of position. The fix is to ensure the table visual uses a column (not a measure) as the axis, so each row gets a different filter context.
- "Which object increases model size when added?" — calculated columns, not measures.

■ Pro Tip

Rule of thumb: if you need it on an axis, a slicer, or as a GROUP BY column — use a calculated column. If you need it in the Values well of a visual — use a measure. If you are unsure: start with a measure. You can always add a calculated column later for filtering, but a spurious calculated column wastes memory permanently.

Lesson 3 CALCULATE — The Engine of DAX

Filter modification ·
Context override ·
Modifiers

CALCULATE is the single most important function in DAX. It is also the most frequently tested in PL-300. Almost every useful measure eventually calls CALCULATE — directly or through a time intelligence function that calls it internally. Understanding CALCULATE means understanding how to modify filter context programmatically.

What CALCULATE Does — Three Steps

```
CALCULATE( , , , ... )

// Step 1: Take the current filter context

// Step 2: Apply / modify / remove filters as specified by filter arguments

// Step 3: Evaluate inside the NEW filter context

// Simple example – filter context override

[East Sales] :=
CALCULATE(
SUM(Sales[Revenue]), -- expression to evaluate
Sales[Region] = "East" -- add a filter: only East rows
)

// Even if the visual is filtered to West, this measure ADDS the East filter,
// OVERRIDING the region filter from the visual.
// The result: always returns East sales, regardless of what the slicer shows.
```

■ How DAX Does It

CALCULATE filter arguments OVERRIDE existing filters on the same column. If a slicer filters Region to West and your CALCULATE adds Region = "East", the slicer filter is replaced — not combined. The result is East sales only.

To combine with existing filters instead of replacing them, use KEEPFILTERS() as a wrapper around the filter argument.

Filter Arguments — Three Forms


```
// Form 1: Boolean predicate (simplest – most common)

CALCULATE([Total Sales], Sales[Region] = "East")

CALCULATE([Total Sales], Products[Category] = "Computers")

// Form 2: Table filter (most flexible)

CALCULATE([Total Sales], FILTER(Sales, Sales[Revenue] > 1000))

// FILTER() returns a table of rows satisfying the condition.
// The table replaces the full Sales table in the filter context.
// Use Form 2 when the condition involves multiple columns or a measure.

// Form 3: CALCULATE modifier functions (removes/extends filters)

CALCULATE([Total Sales], ALL(Sales[Region]))

// ALL() removes all filters from the Region column.
// The result is total sales across ALL regions, ignoring any region slicer.

// Combining forms – multiple filter arguments are AND-combined:

CALCULATE(
    [Total Sales],
    Sales[Region] = "East", -- AND
    Sales[Year] = 2024 -- AND
)
```

CALCULATE Modifiers

Modifier	Effect	Typical Use Case
ALL(table)	Removes all filters from the entire table	% of Grand Total denominator
ALL(table[col])	Removes filters from a specific column only	% of Category Total denominator
ALLEXCEPT(table, col1, col2)	Removes all filters EXCEPT the listed columns	Subtotal within a group
ALLSELECTED(table)	Keeps only filters from outside the visual (slicers)	% of Visual Total
KEEPFILTERS(filter)	Adds filter without overriding existing filters	Conditional cumulation
REMOVEFILTERS(col)	Alias for ALL — explicit and readable	Any ALL() scenario
USERELATIONSHIP(col1, col2)	Activates an inactive relationship for this calculation	Role-playing dimensions
CROSSFILTER(col1, col2, dir)	Changes cross-filter direction for this calculation	Bidirectional relationship

```
// % of Grand Total – the most common CALCULATE pattern

[% of Total] :=

DIVIDE(

    [Total Sales],

    CALCULATE([Total Sales], ALL(Sales)), -- remove ALL filters from Sales

    0

)

// % of Category Total – remove only the product filter, keep others

[% of Category] :=

DIVIDE(

    [Total Sales],

    CALCULATE([Total Sales], ALLEXCEPT(Products, Products[Category])),

    0

)

// % of Visual Total – respects slicers but not matrix row/col headers

[% of Visual Total] :=

DIVIDE(

    [Total Sales],

    CALCULATE([Total Sales], ALLSELECTED(Sales)),

    0

)
```

■ PL-300 Exam

CALCULATE is tested on almost every PL-300 exam in some form. Know these patterns cold:

- ALL(table) for Grand Total denominator in % of total.
- ALLEXCEPT for % within a parent group.
- ALLSELECTED for % of what is currently shown (respects slicers).
- KEEPFILTERS when you want to ADD a filter without replacing the existing one.
- CALCULATE with a table filter (FILTER()) when the condition involves a measure.

Know: a simple boolean predicate in CALCULATE (Sales[Region]="East") REPLACES the existing filter on that column. KEEPFILTERS prevents replacement.

■■ Watch Out

Never use a measure reference inside FILTER() as the filter table argument to CALCULATE without care. FILTER(Sales, [Total Sales] > 1000) iterates every row of Sales and calls the measure for each row — creating context transition for every row. This is extremely slow on large tables. Prefer CALCULATE([Total Sales], Sales[Revenue] > 1000) (boolean predicate) when possible, which folds to a simple column filter.

Lesson 4 Filter Functions

ALL · VALUES ·
FILTER ·
SELECTEDVALUE

Filter functions are the toolkit for inspecting and modifying the filter context from inside a DAX expression. They are almost always used as arguments to CALCULATE, or to interrogate what the user has selected in a slicer or visual.

ALL, ALLEXCEPT, ALLSELECTED — Removing Filters

```
// ALL returns a table with NO filters applied – ignores filter context entirely  
  
ALL(Sales) -- entire Sales table, all rows  
  
ALL(Sales[Region]) -- all distinct values in Region column, unfiltered  
  
ALL(Sales[Region], Sales[Year]) -- all region+year combinations, unfiltered  
  
// ALLEXCEPT – remove all filters EXCEPT the ones on listed columns  
  
ALLEXCEPT(Sales, Sales[Year])  
  
-- Returns Sales filtered only by Year. All other column filters removed.  
  
// ALLSELECTED – keep filters from outside the current visual  
  
-- Respects slicers and page-level filters, but not matrix row/col headers  
  
ALLSELECTED(Sales)  
  
ALLSELECTED(Sales[Region]) -- all regions visible given current slicer selection
```

VALUES and DISTINCT — Reading the Filter Context

```
// VALUES – returns the visible values of a column in the current filter context
-- Includes a blank row if there are rows with no relationship match
VALUES(Sales[Region]) -- list of regions currently visible (after all filters)

// DISTINCT – like VALUES but EXCLUDES the blank row
DISTINCT(Sales[Region])

// HASONEVALUE – true if exactly one value is visible in the current filter context
-- Essential guard before using SELECTEDVALUE
HASONEVALUE(Sales[Region])

// SELECTEDVALUE – returns the single selected value, or alternate if multiple/none
[Region Label] :=
SELECTEDVALUE(Sales[Region], "All Regions")
-- Returns "East" if only East is selected.
-- Returns "All Regions" if multiple regions (or none) are selected.

// COUNTROWS(VALUES(...)) – count distinct visible values
[Distinct Regions] :=
COUNTROWS(VALUES(Sales[Region]))
```

FILTER — Row-Level Iteration

```
// FILTER iterates a table and returns rows where condition is TRUE

-- Returns a TABLE – used as a table argument, not a scalar
FILTER(Sales, Sales[Revenue] > 1000)

-- Returns a filtered Sales table (only rows where Revenue > 1000)

// As CALCULATE filter argument:
CALCULATE([Total Sales], FILTER(Sales, Sales[Revenue] > 1000))

// FILTER can reference measures (unlike boolean predicates):
CALCULATE(
    COUNTROWS(Sales),
    FILTER(Sales, [Average Order Value] > 500)
    -- [Average Order Value] is a MEASURE – must use FILTER form, not boolean
)

// FILTER over a related table – find customers above a threshold:
CALCULATE(
    [Total Sales],
    FILTER(Customers, Customers[CreditLimit] > 5000)
)
```

■ How DAX Does It

Boolean predicates (`Sales[Revenue] > 1000`) in `CALCULATE` are faster than `FILTER()` because they use the column index directly. `FILTER()` iterates every row in memory.

Use `FILTER()` only when: (a) the condition involves a measure, (b) you need to filter based on a column not directly in the main table, or (c) you need to preserve row context inside the iteration.

CONTAINS, IN, LOOKUPVALUE

```
// IN operator (syntactic sugar for OR chains)

CALCULATE([Total Sales], Sales[Region] IN {"East", "West", "Central"})

-- Equivalent to: Sales[Region]="East" || Sales[Region]="West" || ...
-- Cleaner and faster for multi-value filters

// CONTAINS – check if a value exists in a table or column

CONTAINS(VALUES(Sales[Region]), Sales[Region], "East")

-- Returns TRUE if "East" is present in the current Region filter

// LOOKUPVALUE – retrieve a value from another table without a relationship
-- Like VLOOKUP in Excel, but for DAX

Products[ManagerName] =

LOOKUPVALUE(

Managers[Name], -- column to return

Managers[ProductCode], -- search column in lookup table

Products[Code] -- search value from current row

)

-- Use when RELATED() is not available (no formal relationship defined)
```

■ PL-300 Exam

Filter function exam traps:

- VALUES vs DISTINCT — VALUES includes a blank row for orphan rows; DISTINCT does not. For counting visually distinct items, DISTINCT is usually correct.
- ALLSELECTED vs ALL — ALLSELECTED respects slicers; ALL ignores everything including slicers.
- FILTER with a measure as the condition is slower than a boolean predicate — know when each is required.
- IN operator: Sales[Region] IN {"East", "West"} — this is tested as the idiomatic pattern for multi-value filters.

Lesson 5 Iterator Functions (X-Functions)

SUMX · AVERAGEX ·
RANKX · When to
iterate

Iterator functions — the X-functions — combine row context and aggregation in a single call. They iterate a table row by row, evaluate an expression for each row in that row's context, and then aggregate the results. This makes them essential whenever you need to compute a row-level value before aggregating, rather than aggregating a single pre-computed column.

The Iterator Mental Model

```
// SUMX – the fundamental iterator

SUMX( , )

// What it does:

// 1. Takes the filtered by the current filter context
// 2. For EACH ROW in that table:
// a. Establishes row context for that row
// b. Evaluates in that row context
// c. Stores the result
// 3. Returns the SUM of all stored results

// Example: revenue calculated per row, then summed
[Total Revenue] :=
SUMX(Sales, Sales[Quantity] * Sales[UnitPrice])

// This is DIFFERENT from:
[Wrong] := SUM(Sales[Quantity]) * SUM(Sales[UnitPrice])
-- This multiplies total quantity by total unit price = wrong!
-- Always compute the row expression first, then aggregate.

// When a Revenue column already exists, both are equivalent:
SUM(Sales[Revenue]) = SUMX(Sales, Sales[Revenue])
-- But SUMX(Sales, Sales[Quantity] * Sales[UnitPrice]) ≠ SUM(Sales[Revenue])
-- only if Sales[Revenue] was not pre-computed (does not exist as a column)
```

The Full X-Function Family

Function	Aggregation	Returns	Typical Use
SUMX(table, expr)	Sum	Scalar number	Revenue = Qty × Price, per-row then sum
AVERAGEX(table, expr)	Average	Scalar number	Average basket value, weighted average
COUNTX(table, expr)	Count non-blank	Integer	Count rows where expr is non-blank
MAXX(table, expr)	Maximum	Scalar	Max of a per-row expression
MINX(table, expr)	Minimum	Scalar	Min of a per-row expression
RANKX(table, expr, [val], [order], [ties])	Rank	Integer	Rank products by sales
CONCATENATEX(table, expr, [delim])	Concatenate	Text	List all products per region
PRODUCTX(table, expr)	Product (×)	Scalar number	Compound growth (rare)

```
// AVERAGEX – weighted average (impossible with AVERAGE alone)
```

```
[Weighted Avg Price] :=
```

```
DIVIDE(
```

```
SUMX(Sales, Sales[Quantity] * Sales[UnitPrice]),
```

```
SUM(Sales[Quantity]),
```

```
0
```

```
)
```

```
// RANKX – rank products by total sales (dense, descending)
```

```
[Product Rank] :=
```

```
RANKX(
```

```
ALL(Products), -- rank across ALL products (ignore filter on Products)
```

```
[Total Sales], -- expression to rank by
```

```
, -- blank = use the current filter context value
```

```
DESC, -- highest sales = rank 1
```

```
Dense -- no gaps in rank (1,2,2,3 not 1,2,2,4)
```

```
)
```

```
// CONCATENATEX – build a text list per group
```

```
[Products in Region] :=
```

```
CONCATENATEX(
```

```
VALUES(Sales[ProductName]),
```

```
Sales[ProductName],
```

```
", " -- delimiter
```

```
)
```


■ How DAX Does It

Iterators create row context. Inside SUMX's second argument, every column reference [Col] resolves to the current row's value — exactly like a calculated column.

Iterators can call CALCULATE or reference measures. When a measure is referenced inside an iterator, context transition happens: the row context is converted to a filter context for that measure evaluation. This is powerful but expensive — use carefully.

■■ Watch Out

Never iterate a large table with a measure reference in the expression without profiling first. SUMX(Sales, [Complex Measure]) triggers context transition for EVERY row — if Sales has 1 million rows and [Complex Measure] runs a CALCULATE, you get 1 million CALCULATE calls. This is the most common cause of slow DAX in production models.

Prefer simple column expressions in SUMX (Sales[Qty] * Sales[Price]) over measure references whenever possible. Reserve measure references for cases where no column expression can achieve the same result.

Lesson 6 Relationships and Navigation

[RELATED](#) ·
[RELATEDTABLE](#) ·
[USERELATIONSHIP](#)

DAX is designed to work with a relational data model. Relationships between tables define which rows from one table correspond to which rows in another. Understanding how DAX navigates these relationships — and what happens when relationships are inactive or go in the wrong direction — is essential for building correct models.

RELATED — Many-to-One Navigation

```
// RELATED follows a relationship from the MANY side to the ONE side

// Requires: active relationship, row context on the many-side table

// Sales is the many side; Products is the one side
// Sales[ProductID] → Products[ProductID] (many-to-one)

// In a calculated column on Sales:
Sales[Category] = RELATED(Products[Category])

-- For each row in Sales, follows the relationship to Products
-- and returns the Category of the matching product.

Sales[UnitCost] = RELATED(Products[StandardCost])

// RELATED also works inside iterators (row context active):
[Revenue at Cost] :=
SUMX(Sales, Sales[Quantity] * RELATED(Products[StandardCost]))

-- For each Sales row: get Qty, follow relationship to get StandardCost, multiply.
```

RELATEDTABLE — One-to-Many Navigation

```
// RELATEDTABLE follows from the ONE side to the MANY side

// Returns a TABLE of all related rows (not a scalar)

// In a calculated column on Products (one side):
Products[OrderCount] =
COUNTROWS(RELATEDTABLE(Sales))

-- For each Product row, returns all Sales rows for that product.
-- COUNTROWS counts them.

// Total revenue per product (calculated column):
Products[TotalRevenue] =
SUMX(RELATEDTABLE(Sales), Sales[Quantity] * Sales[UnitPrice])

// Note: RELATEDTABLE is rarely needed in measures
// because filter context propagates across relationships automatically.
// [Total Sales] = SUM(Sales[Revenue]) already respects the Products filter.
```

USERELATIONSHIP — Activating Inactive Relationships

```
// A table can have only ONE ACTIVE relationship between any two tables.

// If Sales has BOTH OrderDate and ShipDate linked to Calendar[Date],
// only one can be active. The other is inactive.

// Active relationship: Sales[OrderDate] → Calendar[Date] (default)
// Inactive relationship: Sales[ShipDate] → Calendar[Date]

// Measure using the INACTIVE relationship:

[Sales by Ship Date] :=
CALCULATE(
    [Total Sales],
    USERELATIONSHIP(Sales[ShipDate], Calendar[Date])
)

-- USERELATIONSHIP activates the specified relationship for this CALCULATE only.
-- The active relationship is temporarily suspended for this calculation.

// Role-playing dimension pattern:

[YTD Sales by Order Date] :=
CALCULATE([Total Sales], DATESYTD(Calendar[Date]))

-- uses the ACTIVE OrderDate relationship

[YTD Sales by Ship Date] :=
CALCULATE(
    [Total Sales],
    DATESYTD(Calendar[Date]),
    USERELATIONSHIP(Sales[ShipDate], Calendar[Date])
)
```

■ PL-300 Exam

Relationship navigation exam traps:

- RELATED works in row context only (calculated columns, inside iterators). It FAILS in a measure with no row context.
- RELATEDTABLE returns a table — wrap with COUNTROWS, SUMX, etc. to get a scalar.
- USERELATIONSHIP only works inside CALCULATE. It cannot be used standalone.
- Cross-filter direction: in a Star schema with one-to-many relationships, filters flow FROM the dimension TO the fact. If a visual filters by a dimension column, the fact table is automatically filtered. The reverse (fact → dimension) requires CROSSFILTER or a bidirectional relationship — both have performance implications.

Lesson 7 Time Intelligence

Date table · YTD ·
MTD · SAMEPERIOD
LASTYEAR ·
DATEADD

Time intelligence functions are the DAX functions that compare metrics across time periods: year-to-date totals, year-over-year growth, rolling 12-month averages, and more. They all require a properly constructed Date table. Without it, none of them work.

The Date Table — Non-Negotiable Requirement

```
// Requirements for a valid Date table in Power BI:  
  
// 1. One row per calendar day – no gaps  
  
// 2. A Date column of type Date  
  
// 3. Contiguous range covering all dates in your fact tables  
  
// 4. Marked as a Date Table (Table Tools → Mark as Date Table)  
  
// OR the date column must be named "Date" for auto-detection  
  
  
// Minimal Date table – create in Power Query or as a DAX calculated table  
  
Calendar =  
ADDCOLUMNS(  
    CALENDAR(DATE(2020,1,1), DATE(2026,12,31)),  
    "Year", YEAR([Date]),  
    "QuarterNo", QUARTER([Date]),  
    "Quarter", "Q" & FORMAT([Date],"Q"),  
    "MonthNo", MONTH([Date]),  
    "MonthName", FORMAT([Date],"MMMM"),  
    "WeekNo", WEEKNUM([Date],2),  
    "DayOfWeek", WEEKDAY([Date],2),  
    "IsWeekend", IF(WEEKDAY([Date],2) >= 6, TRUE, FALSE)  
)  
  
// After creating: mark as Date Table using Calendar[Date] as the date column
```

YTD, QTD, MTD — Cumulative Totals

```
// Year-to-date total – two equivalent forms

[Sales YTD] :=
TOTALYTD([Total Sales], Calendar[Date])

-- TOTALYTD(expression, dates_column, [filter], [year_end_date])

[Sales YTD (CALCULATE form)] :=
CALCULATE(
    [Total Sales],
    DATESYTD(Calendar[Date]) -- returns all dates from Jan 1 to current date
)

-- These two forms are identical. TOTALYTD is shorthand.

// Fiscal year ending March 31 (Australian fiscal year):
[Fiscal YTD] :=
TOTALYTD([Total Sales], Calendar[Date], "03-31")

// Quarter-to-date and Month-to-date:
[Sales QTD] := TOTALQTD([Total Sales], Calendar[Date])
[Sales MTD] := TOTALMTD([Total Sales], Calendar[Date])
```

SAMEPERIODLASTYEAR and DATEADD

```
// Same period last year – parallel period shifted back exactly 1 year

[Sales LY] :=
CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date]))

// YoY Growth Amount

[YoY Growth] :=
[Total Sales] - [Sales LY]

// YoY Growth %

[YoY Growth %] :=
DIVIDE([YoY Growth], [Sales LY], BLANK())

-- BLANK() as alternate: shows blank (not 0%) when no prior year data

// DATEADD – shift by any interval: DAY, MONTH, QUARTER, YEAR

[Sales Prior Month] :=
CALCULATE([Total Sales], DATEADD(Calendar[Date], -1, MONTH))

[Sales Same Quarter Last Year] :=
CALCULATE([Total Sales], DATEADD(Calendar[Date], -1, YEAR))

-- Same as SAMEPERIODLASTYEAR but using DATEADD form

// Rolling 12 months – last 12 months from the last visible date

[Rolling 12M] :=
CALCULATE(
[Total Sales],
DATESINPERIOD(Calendar[Date], MAX(Calendar[Date]), -12, MONTH)
)
```

Function	What it returns	Common Use
TOTALYTD(expr, dates)	Cumulative from Jan 1 to current date	YTD KPI card
TOTALQTD(expr, dates)	Cumulative from start of current quarter	QTD tracking
TOTALMTD(expr, dates)	Cumulative from start of current month	MTD tracking
DATESYTD(dates)	Table of dates from Jan 1 to current date	CALCULATE argument
SAMEPERIODLASTYEAR(dates)	Same set of dates shifted back 1 year	YoY comparison
DATEADD(dates, n, interval)	Dates shifted by n intervals (±DAY/MONTH/QUARTER/YEAR)	Flexible prior period
DATESINPERIOD(dates, start, n, interval)	n intervals of dates from start	Rolling window
DATESMTD / DATESQTD / DATESYTD	Dates for MTD / QTD / YTD period	CALCULATE argument
PREVIOUSMONTH(dates)	All dates in the prior full month	Month-over-month
PARALLELPERIOD(dates, n, interval)	Full prior period (entire month/quarter/year)	Full prior period comparison

■ PL-300 Exam

Time intelligence exam traps:

- Without a marked Date table, ALL time intelligence functions return BLANK or errors.
- SAMEPERIODLASTYEAR vs DATEADD(-1, YEAR): SAMEPERIODLASTYEAR returns the same set of dates (partial months/quarters preserved). PARALLELPERIOD returns full months/quarters. Know which one gives a partial vs full prior period.
- ALLSELECTED with time intelligence: when a date range slicer is active, DATESYTD respects the slicer upper bound. If no dates are selected, it returns blank.
- Fiscal year end date in TOTALYTD: the format is "MM-DD" as a string, e.g. "06-30" for a June 30 fiscal year end.

Lesson 8 Variables and Advanced Patterns

VAR/RETURN · % of
Total · Running Total ·
Basket analysis

Variables — the VAR/RETURN construct — are the single biggest readability and performance improvement available in DAX. Every professional DAX developer uses them extensively. Beyond variables, five patterns account for the majority of real-world DAX requirements and PL-300 scenario questions.

VAR / RETURN — Variables in DAX

```
// VAR captures a value (or table) in the current evaluation context.

// RETURN evaluates an expression using those captured values.

// Without variables (hard to read, expression repeated three times):

[YoY Growth %] :=
DIVIDE(
    [Total Sales] - CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date])),
    CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date])),
    BLANK()
)

// With variables (readable, prior period calculated ONCE):

[YoY Growth %] :=
VAR CurrentSales = [Total Sales]
VAR PriorYrSales = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date]))
VAR GrowthAmount = CurrentSales - PriorYrSales

RETURN
DIVIDE(GrowthAmount, PriorYrSales, BLANK())

// VAR captures the filter context at the point of definition.
// Inside RETURN, subsequent context changes do not retroactively change the variable.
// This is why VAR replaces EARLIER() in modern DAX – safer and clearer.
```

■ How DAX Does It

VAR evaluates lazily — the expression in a VAR is not computed until RETURN actually uses it. If you define 5 variables but RETURN only references 2, the other 3 are never computed. Use VAR freely for readability without worrying about performance of unused branches.

VAR also works for TABLE variables. VAR SalesSubset = FILTER(Sales, Sales[Year] = 2024) captures a table expression. RETURN COUNTROWS(SalesSubset) counts it once.

Pattern 1 — Percentage of Total

```
// % of Grand Total – remove all filters from the denominator

[% of Grand Total] :=
VAR Numerator = [Total Sales]
VAR Denominator = CALCULATE([Total Sales], ALL(Sales))
RETURN DIVIDE(Numerator, Denominator, 0)

// % of Parent Group – e.g. % of Category within each row of a matrix
// Matrix has Category on rows, Product on sub-rows

[% of Category] :=
VAR Numerator = [Total Sales]
VAR Denominator = CALCULATE([Total Sales], ALLSELECTED(Products[Product]))
RETURN DIVIDE(Numerator, Denominator, 0)

// % of Column Total (respects slicers):

[% of Visual] :=
DIVIDE([Total Sales], CALCULATE([Total Sales], ALLSELECTED()), 0)
```

Pattern 2 — Running Total (Cumulative Sum)

```
// Running total by date – sum of all Sales from the beginning up to current date

[Cumulative Sales] :=
VAR LastDate = MAX(Calendar[Date])
RETURN
CALCULATE(
    [Total Sales],
    Calendar[Date] <= LastDate, -- all dates up to and including this date
    ALL(Calendar) -- remove the date filter (we re-set it above)
)

// Cleaner form using DATESYTD for YTD running total:
[YTD Sales] := TOTALYTD([Total Sales], Calendar[Date])

// Running total respecting ALL existing filters (region, product, etc.):
[Cumulative Sales v2] :=
VAR LastDate = MAX(Calendar[Date])
RETURN
CALCULATE(
    [Total Sales],
    FILTER(ALL(Calendar), Calendar[Date] <= LastDate)
)

-- FILTER(ALL(Calendar), ...) removes the default date filter but applies
-- our custom range – all other filters (Region, Product) are preserved.
```

Pattern 3 — Dynamic ABC Segmentation

```
// Classify products as A (top 70%), B (next 20%), C (bottom 10%) by revenue

[ABC Segment] :=
VAR ProductSales = [Total Sales]
VAR AllProductSales = CALCULATE([Total Sales], ALL(Products))
VAR CumulativeShare =
DIVIDE(
    CALCULATE(
        [Total Sales],
        FILTER(
            ALL(Products),
            [Total Sales] >= ProductSales
        )
    ),
    AllProductSales,
    0
)
RETURN
SWITCH(
    TRUE(),
    CumulativeShare <= 0.70, "A",
    CumulativeShare <= 0.90, "B",
    "C"
)
```

Pattern 4 — SWITCH TRUE() for Conditional Logic

```
// SWITCH(TRUE(), ...) – the DAX equivalent of a nested IF / CASE WHEN
// Evaluates each condition in order, returns the first TRUE result.

[Sales Category] :=
SWITCH(
TRUE(), -- the expression to switch on
[Total Sales] > 100000, "Platinum",
[Total Sales] > 50000, "Gold",
[Total Sales] > 10000, "Silver",
"Bronze" -- default (ELSE) – no condition needed
)

// Column-value switch (when comparing a single value):
[Region Group] :=
SWITCH(
Sales[Region], -- the value to switch on
"East", "Atlantic",
"West", "Pacific",
"Central", "Midwest",
"Other"
)

// SWITCH TRUE() is MORE readable than nested IFs for 3+ conditions.
// SWITCH with a value (second form) is faster than SWITCH TRUE() for simple lookups.
```

■ Pro Tip

Use SWITCH(TRUE(), ...) instead of nested IF() whenever you have 3 or more conditions. It is significantly easier to read, maintain, and debug. A nested IF with 5 levels is nearly impossible to audit; a SWITCH(TRUE()) with 5 conditions is a simple list.

For simple two-way branches, IF() is still the clearest choice. IF(condition, result_true, result_false).

DEMONSTRATION 1 Sales KPI Dashboard — CALCULATE, YoY, and % of Total

Contoso needs a complete set of sales KPIs for a dashboard: total sales, YTD sales, year-over-year growth, and percentage of total by region. This demonstration builds every measure step by step, explaining exactly what each CALCULATE modifier does at each stage.

Source Data Model

Three tables: Sales (TxID, Date, RegionID, ProductID, Quantity, UnitPrice), Region (RegionID, RegionName, Country), Calendar (Date, Year, Quarter, MonthNo, MonthName). Relationships: Sales[Date] → Calendar[Date] (active), Sales[RegionID] → Region[RegionID].

Step 1 — Foundation Measures

```
// Always build foundation measures first.

// Every complex measure calls these – they must be correct.

[Total Sales] :=
SUMX(Sales, Sales[Quantity] * Sales[UnitPrice])

-- Per-row multiplication, then sum. Avoids the SUM*SUM mistake.

[Order Count] :=
COUNTROWS(Sales)

-- Count rows that survive current filter context

[Avg Order Value] :=
DIVIDE([Total Sales], [Order Count], BLANK())

-- BLANK() when no orders (avoids 0/0 showing as 0)
```

Step 2 — Time Intelligence KPIs

```
[Sales YTD] :=
TOTALYTD([Total Sales], Calendar[Date])

[Sales LY] :=
CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date]))

[YoY Growth] :=
VAR Current = [Total Sales]
VAR Prior = [Sales LY]
RETURN [Total Sales] - [Sales LY]

[YoY Growth %] :=
VAR Current = [Total Sales]
VAR Prior = [Sales LY]
RETURN DIVIDE(Current - Prior, Prior, BLANK())

-- Format this measure as % in the Format pane
```

Step 3 — Context-Aware % of Total

```
// % of Grand Total – ignores ALL filters

[% Grand Total] :=
DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Sales)), 0)

// % of Country – remove Region filter, keep Country filter

[% of Country] :=
DIVIDE(
    [Total Sales],
    CALCULATE([Total Sales], ALLEXCEPT(Region, Region[Country])),
    0
)

// % of Visual Total – respects slicers (what the user has filtered to)

[% Visual Total] :=
DIVIDE([Total Sales], CALCULATE([Total Sales], ALLSELECTED(Sales)), 0)
```

■ RESULT

- [Total Sales]: returns correct per-cell values in all matrix configurations.
- [Sales YTD]: accumulates from Jan 1 of each year to the current date in context.
- [YoY Growth %]: BLANK() in cells with no prior year data (first year of data).
- [% Grand Total]: sums to 100% only at the grand total row (as expected).
- [% Visual Total]: sums to 100% within whatever the slicer shows — changes when slicer changes.

DEMONSTRATION 2

Inventory Analysis — Iterators, RANKX, and ABC Classification

A retail model needs: total inventory value (unit cost × quantity on hand), product rank by sales, and ABC classification. This demonstration combines SUMX, RANKX, and the ABC pattern into a production-quality set of measures and calculated columns.

Step 1 — Inventory Value with SUMX

```
// Inventory value cannot use SUM alone – needs per-row multiplication

[Inventory Value] :=

SUMX(
    Inventory,
    Inventory[QtyOnHand] * RELATED(Products[UnitCost])
    -- RELATED follows the relationship from Inventory to Products
    -- for each row of Inventory in the current row context
)

// Average cost weighted by quantity on hand:

[Weighted Avg Cost] :=

DIVIDE(
    SUMX(Inventory, Inventory[QtyOnHand] * RELATED(Products[UnitCost])),
    SUM(Inventory[QtyOnHand]),
    BLANK()
)
```

Step 2 — Product Rank by Sales

```
// Rank EVERY product by total sales – dense ranking, descending

[Product Rank] :=

RANKX(

ALL(Products[ProductName]), -- rank across ALL products

[Total Sales], -- expression to rank by

, -- blank = use current context value

DESC, -- rank 1 = highest sales

Dense -- 1,2,2,3 not 1,2,2,4

)

// Top 10 filter using the rank:

[Top 10 Sales] :=

VAR Rank = [Product Rank]

RETURN IF(Rank <= 10, [Total Sales], BLANK())

-- Blanks for products outside top 10 (removes them from visuals)
```

Step 3 — ABC Classification (Calculated Column)

```
// ABC classification as a calculated column on Products

// Requires: [Total Sales] measure exists, Products table has all products

Products[ABCTier] =
VAR ThisProductSales = [Total Sales]

-- Context transition: CALCULATE converts row context to filter context
-- [Total Sales] is now filtered to THIS product only

VAR AllSales = CALCULATE([Total Sales], ALL(Products))

VAR CumShare =
DIVIDE(
    CALCULATE(
        [Total Sales],
        FILTER(ALL(Products), [Total Sales] >= ThisProductSales)
    ),
    AllSales,
    0
)

RETURN
SWITCH(TRUE(),
    CumShare <= 0.70, "A – Top 70%",
    CumShare <= 0.90, "B – Next 20%",
    "C – Bottom 10%"
)
```

■ RESULT

- [Inventory Value]: correct per-product when used in a table visual filtered to a product.
- [Product Rank]: returns 1 for the best-selling product, N for the Nth, across ALL products regardless of visual filter.
- [Top 10 Sales]: returns revenue for top-10 products, BLANK for all others — cleans visuals automatically.
- Products[ABCTier]: each product row permanently carries its ABC tier. Can be used as a slicer or axis.

Quick Reference — All Module 1 Functions

Core Aggregation

Function	Returns	One-line Use Case
SUM(col)	Scalar	Sum of a numeric column in filter context
AVERAGE(col)	Scalar	Average of a numeric column
COUNT(col)	Integer	Count non-blank values in a column
COUNTA(col)	Integer	Count non-blank including text
COUNTROWS(table)	Integer	Count rows in a table
COUNTBLANK(col)	Integer	Count blank values
DISTINCTCOUNT(col)	Integer	Count distinct non-blank values — uniques
MIN(col) / MAX(col)	Scalar	Min/Max value in filter context
DIVIDE(num, den, alt)	Scalar	Safe division — alt if denominator is zero

Context Modification (CALCULATE arguments)

Function	Effect
ALL(table)	Remove ALL filters from the table
ALL(table[col])	Remove all filters from a specific column
ALLEXCEPT(table, col1, ...)	Remove filters from all columns EXCEPT listed ones
ALLSELECTED(table)	Remove internal (visual) filters; keep external (slicer) filters
KEEPFILTERS(filter)	Add filter without overriding existing filter on that column
REMOVEFILTERS(table)	Alias for ALL — explicit and self-documenting
USERELATIONSHIP(col1, col2)	Activate an inactive relationship for this calculation
CROSSFILTER(col1, col2, dir)	Change cross-filter direction (Both/None/OneWay)

Iterator Functions

Function	Aggregation	Key Use Case
SUMX(table, expr)	Sum of expr per row	Revenue = Qty × Price before summing
AVERAGEX(table, expr)	Average of expr per row	Weighted average price
COUNTX(table, expr)	Count non-blank expr values	Conditional counts
MAXX(table, expr)	Max of expr per row	Max discount applied
MINX(table, expr)	Min of expr per row	Min order size per customer
RANKX(table, expr, [val], [ord], [ties])	Rank of value within table	Product ranking dashboard
CONCATENATEX(table, expr, [del])	Text concatenation over rows	List of products per order

Filter Functions

Function	Returns	Key Use Case
<code>FILTER(table, condition)</code>	Table	Row-level filter for <code>CALCULATE</code> when condition uses a measure
<code>VALUES(col)</code>	Table (1 col)	Visible distinct values in current filter context
<code>DISTINCT(col)</code>	Table (1 col)	Distinct values excluding blank row
<code>HASONEVALUE(col)</code>	Boolean	Guard before <code>SELECTEDVALUE</code> — is exactly one value visible?
<code>SELECTEDVALUE(col, alt)</code>	Scalar	Single selected value, or alt if multiple/none
<code>CONTAINS(table, col, val)</code>	Boolean	True if val exists in col within table
<code>IN {val1, val2}</code>	Boolean	Multi-value filter — readable alternative to OR chains
<code>LOOKUPVALUE(ret, search, val)</code>	Scalar	<code>VLOOKUP</code> equivalent — no relationship needed

Relationship Navigation

Function	Returns	Direction	Context Required
<code>RELATED(col)</code>	Scalar	Many → One	Row context (calc col or inside iterator)
<code>RELATEDTABLE(table)</code>	Table	One → Many	Row context (calc col or inside iterator)
<code>USERELATIONSHIP(c1, c2)</code>	Modifier	Any (inactive)	Inside <code>CALCULATE</code> only
<code>CROSSFILTER(c1, c2, dir)</code>	Modifier	Override direction	Inside <code>CALCULATE</code> only

Time Intelligence

Function	Returns	Use Case
<code>TOTALYTD(expr, dates, [filter], [ye])</code>	Scalar	Year-to-date aggregate, optional fiscal year end
<code>TOTALQTD(expr, dates)</code>	Scalar	Quarter-to-date aggregate
<code>TOTALMTD(expr, dates)</code>	Scalar	Month-to-date aggregate
<code>DATESYTD(dates, [ye])</code>	Table	<code>CALCULATE</code> argument for YTD — Jan 1 to max date
<code>SAMEPERIODLASTYEAR(dates)</code>	Table	Same date range, shifted back 1 year
<code>DATEADD(dates, n, interval)</code>	Table	Shift by n DAY/MONTH/QUARTER/YEAR
<code>DATESINPERIOD(dates, start, n, int)</code>	Table	Rolling window — last 12 months etc.
<code>PARALLELPERIOD(dates, n, interval)</code>	Table	Full prior month/quarter/year
<code>PREVIOUSMONTH(dates)</code>	Table	All dates in the prior complete month
<code>CALENDAR(start, end)</code>	Table	Generate a date table from scratch
<code>CALENDARAUTO()</code>	Table	Auto date table from min/max dates in model

LAB 1 DAX Hands-On Exercises

■ How DAX Does It

No solution code is provided — that is intentional.

The expected outputs let you verify your work. Click each step name in the query editor to inspect intermediate results.

If you are stuck after 2 minutes: re-read the relevant lesson, then try again before reading the hint.

Module 1 is the most heavily tested module in PL-300 — the effort here pays disproportionately.

Exercise 1 Context, Calculated Columns, and Basic Measures

■ 20 minutes

Objective: Understand row context vs filter context by building both calculated columns and measures on the same data, observing the difference in their behaviour.

1 EXPLAIN IT — Before writing any DAX, fill in the table below

Concept	Your plain-English explanation (no DAX)
Row context	
Filter context	
Calculated column	
Measure	
Why SUM(col) in a measure works but [col] alone does not	
What RELATED() does and when you need it	

2 BUILD IT — Create these calculations in Power BI Desktop

Reference Data — Create these tables manually using Enter Data, or paste the following M code into a blank query in the Advanced Editor.

```
// Paste into Power Query Advanced Editor – creates the Sales table

let
Source = #table(
{"TxID", "Date", "Region", "Product", "Qty", "UnitPrice"},
{"T01", #date(2024,1,15), "East", "Laptop", 5, 1200},
{"T02", #date(2024,2,10), "West", "Tablet", 12, 450},
{"T03", #date(2024,2,20), "East", "Phone", 8, 800},
{"T04", #date(2024,3,05), "West", "Laptop", 3, 1200},
{"T05", #date(2024,4,12), "East", "Monitor", 2, 350},
{"T06", #date(2024,5,20), "West", "Laptop", 6, 1200},
{"T07", #date(2024,6,08), "East", "Phone", 4, 800},
{"T08", #date(2024,6,25), "West", "Tablet", 9, 450}})
in
Table.TransformColumnTypes(Source, {
{"Date", type date}, {"Qty", Int64.Type}, {"UnitPrice", type number}})
```

Task 1.1 — Add a *calculated column* Revenue = Qty × UnitPrice. Expected: T01=6000, T02=5400, T03=6400, T04=3600, T05=700, T06=7200, T07=3200, T08=4050.

Task 1.2 — Create a *measure* [Total Sales] = sum of Revenue. Place it in a Card visual. Expected grand total: 40150.

Task 1.3 — Create a *measure* [Order Count] = count of rows. Expected: 8.

Task 1.4 — Create [Avg Order Value] = DIVIDE([Total Sales], [Order Count], BLANK()). Expected: 5018.75. Verify it changes when you filter by Region.

Task 1.5 — Add a calculated column Tier: Gold if Revenue > 5000, Silver if > 2000, Bronze otherwise. Expected: T01=Gold, T02=Silver, T03=Gold, T04=Silver, T05=Bronze, T06=Gold, T07=Silver, T08=Silver.

Task 1.6 — Build a matrix visual: Region on rows, measure [Total Sales] in values. Verify: East = 19350, West = 20800, Grand Total = 40150.

■ Hint

Task 1.1: Calculated columns use = (not :=). Reference columns as Sales[Qty] * Sales[UnitPrice].

Task 1.2: Measures use :=. SUM(Sales[Revenue]) sums the column you created in 1.1.

Task 1.4: DIVIDE(a, b, BLANK()) returns BLANK instead of an error when b is zero. Always use DIVIDE.

Task 1.5: IF(Sales[Revenue] > 5000, "Gold", IF(Sales[Revenue] > 2000, "Silver", "Bronze")).

Task 1.6: Drag Region to matrix Rows, drag [Total Sales] to Values. The matrix creates per-row filter context automatically.

3 BREAK IT — Trigger and diagnose these errors

- Write a calculated column on Sales: Revenue2 = [Total Sales]. Load it. Every row shows 40150 (the grand total). Why? How does context transition explain this? Fix it using CALCULATE.
- In a new measure, write: [Wrong Avg] := Sales[UnitPrice]. Observe the error message. Explain in one sentence why column references fail in measures without an aggregation function.

4 TEACH IT BACK — Answer without consulting the lesson

Reflection: A colleague argues "calculated columns and measures are interchangeable — they both return numbers." Write a paragraph explaining exactly why this is wrong. Use a concrete example of a ratio (like market share %) to show when a measure gives the correct result but a calculated column gives a wrong one.

■ RESULT

- Revenue calculated column: 6000, 5400, 6400, 3600, 700, 7200, 3200, 4050.
- [Total Sales] = 40150 at grand total; 19350 for East, 20800 for West.
- [Avg Order Value] = 5018.75 at grand total; changes correctly per region in the matrix.
- Tier: Gold(T01), Silver(T02), Gold(T03), Silver(T04), Bronze(T05), Gold(T06), Silver(T07), Silver(T08).

Exercise 2 CALCULATE, ALL, and % of Total

■ 20 minutes

Objective: Master filter modification using CALCULATE. Build the three forms of percentage-of-total and observe how each responds to slicer filters.

1 EXPLAIN IT

In plain English, without any DAX: what does CALCULATE do? What happens to the existing filter context when CALCULATE adds a filter argument for a column that is already filtered by a slicer? What is the difference between ALL() and ALLSELECTED()?

2 BUILD IT

Task 2.1 — Build [East Sales] := CALCULATE([Total Sales], Sales[Region] = "East"). Add it to the matrix next to [Total Sales]. Expected: Every row and the grand total show 19350. Verify that a slicer filtering to West does NOT change [East Sales].

Task 2.2 — Build [% Grand Total] := DIVIDE([Total Sales], CALCULATE([Total Sales], ALL(Sales)), 0). Expected in the matrix: East = 48.2%, West = 51.8%, Grand Total = 100%.

Task 2.3 — Build [% Visual Total] using ALLSELECTED. Add a Region slicer. Filter to East only. Verify [% Visual Total] shows 100% for East. Deselect the slicer — verify East shows 48.2% again. Explain the difference.

Task 2.4 — Build [Laptop Sales] := CALCULATE([Total Sales], Sales[Product] = "Laptop"). Add it to a table with Product on rows. Verify every row shows the same value (total Laptop sales). Explain why the Product filter in the visual does not affect this measure.

Task 2.5 (Stretch) — Build [% of Product in Region]: the percentage this product's sales represent within its region. Use ALLEXCEPT on the region dimension. Verify values sum to 100% within each region.

■ Hint

Task 2.1: `CALCULATE([Total Sales], Sales[Region]="East")` OVERRIDES any Region filter from the visual. The slicer is replaced.

Task 2.2: `ALL(Sales)` removes ALL filters from the Sales table including Region, Product, and Date.

Task 2.3: `ALLSELECTED` removes the matrix row/col header filter but keeps the slicer filter. When the slicer is active, the denominator is restricted to what the slicer shows.

Task 2.4: `CALCULATE` adds `Region="Laptop"` — this REPLACES the Product filter from the matrix row. Every row gets the same Laptop-only filter.

Task 2.5: `ALLEXCEPT(Sales, Sales[Region])` removes all column filters except Region. The denominator is regional total.

3 BREAK IT

- Replace `ALL(Sales)` in [% Grand Total] with `ALL(Sales[Region])`. Add a Product slicer and filter to Laptop. Does the denominator change? Why — and is this correct for a "% of all products" use case?
- Use `KEEPFILTERS: CALCULATE([Total Sales], KEEPFILTERS(Sales[Region]="East"))`. Place a Region slicer and filter to West. What does the measure return and why?

4 TEACH IT BACK

Reflection: A report has a YTD Sales measure built with `CALCULATE`. A user reports that the YTD value is the same in every row of a table visual that has a date hierarchy. Diagnose this problem: what is most likely wrong, and what is the fix?

■ RESULT

- `[East Sales]` = 19350 in every row — Region slicer has no effect on it.
- `[% Grand Total]`: East=48.2%, West=51.8%, Grand Total=100%.
- `[% Visual Total]` = 100% for East when only East is in the slicer.
- `[Laptop Sales]` = total Laptop sales (17400) in every row — `CALCULATE` overrides the Product filter.

Exercise 3 Time Intelligence — YTD, YoY, and Rolling Windows

■ 25 minutes

Objective: Build a complete time intelligence suite. Connect a date table, mark it, and build YTD, prior year, YoY growth, and a rolling 3-month average.

1 EXPLAIN IT

For each function below, explain in plain English what set of dates it returns for a filter context of "March 2024":

Function (in context: March 2024)	Dates returned — plain English
DATESYTD(Calendar[Date])	
SAMEPERIODLASTYEAR(Calendar[Date])	
DATEADD(Calendar[Date], -1, MONTH)	
PARALLELPERIOD(Calendar[Date], -1, YEAR)	
DATESINPERIOD(Calendar[Date], MAX(Calendar[Date]), -3, MONTH)	

2 BUILD IT

Setup: Create a Calendar table using CALENDARAUTO() or the CALENDAR() + ADDCOLUMNS pattern from Lesson 7. Mark it as a Date table. Connect Sales[Date] → Calendar[Date].

Task 3.1 — Build [Sales YTD] := TOTALYTD([Total Sales], Calendar[Date]). Place in a line chart with Year/Month on the axis. Verify it accumulates within each year and resets at Jan 1.

Task 3.2 — Build [Sales LY] := CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date])). Expected: Jan 2025 [Sales LY] = whatever Jan 2024 [Total Sales] was.

Task 3.3 — Build [YoY Growth %] using VAR for both current and prior values. Return BLANK() when no prior year exists. Format as percentage.

Task 3.4 — Build [Rolling 3M Avg] := DIVIDE(CALCULATE([Total Sales], DATESINPERIOD(Calendar[Date], MAX(Calendar[Date]), -3, MONTH)), 3, BLANK()). Verify it is a trailing 3-month average, not a 3-month YTD.

Task 3.5 (Stretch) — Build [Fiscal YTD] for a fiscal year ending June 30. Verify that July is the start of the fiscal year.

■ Hint

Setup: CALENDARAUTO() creates a date table spanning all dates in your model. Always follow with ADDCOLUMNS to add Year, Month, Quarter columns. Mark as Date Table before time intelligence functions will work.

Task 3.1: TOTALYTD resets at January 1 each year. If your data spans 2 years, the chart shows 2 separate accumulation curves.

Task 3.3: Use VAR CurrentSales = [Total Sales] and VAR PriorSales = [Sales LY], then DIVIDE(CurrentSales - PriorSales, PriorSales, BLANK()).

Task 3.4: DATESINPERIOD with -3 MONTH from MAX date gives the 3-month window ending on the current date.

Task 3.5: TOTALYTD([Total Sales], Calendar[Date], "06-30") — the year_end_date parameter.

3 BREAK IT

- Delete the "Mark as Date Table" designation. Observe what happens to all time intelligence measures. Restore it. This confirms the non-negotiable requirement.
- Build [YoY Growth %] without using VAR — reference [Sales LY] twice in the same DIVIDE. Does the result change? Now explain why using VAR is still better even if the result is the same.

4 TEACH IT BACK

Reflection: Your colleague asks: "Why do I need a special date table? Can't I just use the date column in my Sales table for TOTALYTD?" Write a complete answer explaining the three requirements for time intelligence to work, what "marking as date table" does internally, and what breaks without it.

■ RESULT

- [Sales YTD]: accumulates within each year, resets January 1 of each new year.
- [Sales LY]: returns the prior year's value for the same period; BLANK for first year.
- [YoY Growth %]: correct percentage with BLANK for periods with no prior year.
- [Rolling 3M Avg]: trailing average, NOT a YTD — it shifts with each month.
- [Fiscal YTD]: July is fiscal month 1 when year_end_date = "06-30".

Exercise 4 Iterators, RANKX, and the Full Pipeline — Capstone

■ 25 minutes

Objective: Integrate everything from Module 1 into a production-quality set of measures. This exercise tests the full PL-300 scenario: context, CALCULATE, time intelligence, and iterators working together.

1 DESIGN BEFORE YOU CODE

Before writing any DAX: Write a numbered list of plain-English steps describing how you would build the following measures: (1) Average Revenue per Order by Region, (2) Product rank by sales that works correctly in a matrix with Region on rows, (3) % of region total for each product within each region. Identify which functions you will need and in what order.

2 BUILD IT

Task 4.1 — Build [Avg Revenue per Order] := DIVIDE([Total Sales], [Order Count], BLANK()). Place in a matrix with Region on rows. Verify it divides correctly per region, not the grand total for every row.

Task 4.2 — Build [Product Rank by Sales] := RANKX(ALL(Products[Product]), [Total Sales], , DESC, Dense). In a table with Product on rows, verify Laptop=1, Tablet=2, Phone=3, Monitor=4.

Task 4.3 — Build [Top 3 Products Flag] := IF([Product Rank by Sales] <= 3, "Top 3", "Other"). Use it to create a slicer. Verify the slicer filters the matrix correctly.

Task 4.4 — Build [% of Region Total]: for each product within a region, what percentage of that region's total does it represent? Use ALLEXCEPT or ALLSELECTED appropriately. Verify each region's product percentages sum to 100%.

Task 4.5 (Stretch) — Build [YoY Growth by Product] using CALCULATE and SAMEPERIODLASTYEAR. Place in a matrix with Product on rows and Year on columns. Verify the first year shows BLANK for YoY growth.

■ Hint

Task 4.2: RANKX(ALL(Products[Product]), ...) ranks across ALL products regardless of visual filter. This is intentional — you want a consistent rank.

Task 4.3: [Top 3 Products Flag] is a MEASURE — it cannot go in a slicer directly. Add a calculated column Products[IsTop3] using the same logic, then use that column as the slicer.

Task 4.4: CALCULATE([Total Sales], ALLEXCEPT(Sales, Sales[Region])) gives regional total while keeping the Region filter active.

Task 4.5: Build [Total Sales LY] = CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date])). Then [YoY Growth %] = DIVIDE([Total Sales] - [Total Sales LY], [Total Sales LY], BLANK()).

3 BREAK IT

- In [Product Rank by Sales], change ALL(Products[Product]) to VALUES(Products[Product]). Observe the rank — every product is rank 1. Explain exactly why this happens using filter context.
- Remove the Date table relationship. Observe what happens to [YoY Growth by Product]. Restore it. This confirms that time intelligence requires an active relationship to a marked Date table.

4 TEACH IT BACK — The Module Capstone

Final Reflection: A new Power BI developer has been asked to build a regional sales performance dashboard showing: total sales, YoY growth, product rank, and % of regional total — all in a single matrix with Region on rows and Year on columns.

Write a design brief — no DAX code — covering:

- Which measures require CALCULATE and what filter arguments each one needs.
- Why [Product Rank] must use ALL() and not VALUES() or FILTERS().
- Why time intelligence requires a marked Date table and how to set it up.
- The difference between % Grand Total, % Visual Total, and % Region Total — and which one the business actually wants.

If you can write this brief fluently, you have mastered Module 1 DAX.

■ RESULT

- [Avg Revenue per Order]: correct per-region divide — not the grand total in every cell.
- [Product Rank]: Laptop=1, Tablet=2, Phone=3, Monitor=4 — consistent across all region filters.
- [Top 3 Products Flag] measure: works as a visual filter when used in a filter pane; calculated column version works as a slicer.
- [% of Region Total]: sums to 100% within each region column of the matrix.
- [YoY Growth by Product]: BLANK for the first year of data; correct % for subsequent years.

Module Review and Takeaways

Review Questions

Q1 — What is the difference between row context and filter context?

Row context is the evaluation context created for a specific row — it knows "which row am I on." It exists in calculated columns (automatically, for every row) and inside iterator functions (for each row being visited). Filter context is the set of active filters on table columns — it defines WHICH ROWS of each table are visible. Filter context is set by slicers, visual headers, and CALCULATE arguments. Measures evaluate in filter context only; they have no implicit row context.

Q2 — What does CALCULATE do, and what is context transition?

CALCULATE takes an expression and one or more filter arguments. It builds a new filter context by modifying the current one according to its filter arguments, then evaluates the expression in that new context. Context transition is the special case where CALCULATE is called inside a row context: it converts every column of the current row into an equivalent filter, effectively converting row context into filter context. This allows measures to be called from calculated columns or iterators.

Q3 — When would you use SUMX instead of SUM?

Use SUMX when the value you want to aggregate does not exist as a pre-computed column — specifically when you need to multiply two columns per row and then sum the results. $\text{SUM}(\text{Sales}[\text{Revenue}])$ and $\text{SUMX}(\text{Sales}, \text{Sales}[\text{Revenue}])$ are identical when a Revenue column exists. But $\text{SUMX}(\text{Sales}, \text{Sales}[\text{Quantity}] * \text{Sales}[\text{UnitPrice}])$ is the correct form when no Revenue column exists — $\text{SUM}(\text{Sales}[\text{Quantity}]) * \text{SUM}(\text{Sales}[\text{UnitPrice}])$ would produce a mathematically incorrect result.

Q4 — What are the requirements for time intelligence functions?

Time intelligence functions require: (1) a contiguous Date table with one row per calendar day and no gaps, (2) a Date column of type Date in that table, (3) the table marked as a Date Table in Power BI, and (4) an active relationship between your fact table's date column and the Date table. Without all four, time intelligence functions return blanks or incorrect results.

Q5 — What is the difference between ALL and ALLSELECTED?

ALL removes ALL filters from the specified table or column — including slicers, page filters, and visual row/column headers. The result is the full, unfiltered table. ALLSELECTED removes internal filters (the ones created by row/column headers in the current visual) but preserves external filters (slicers and page-level filters). Use ALL for "% of Grand Total across everything"; use ALLSELECTED for "% of what the user has chosen to see."

Best Practices

Always use DIVIDE, never /	DIVIDE(a, b, alt) handles division by zero gracefully. The / operator throws an error when the denominator is zero, breaking visuals.
Start with foundation measures	Build [Total Sales], [Order Count], [Avg Order Value] first. Every complex measure composes from these. Never duplicate aggregation logic.
Use VAR liberally	VAR makes measures readable, debuggable, and prevents repeated evaluation of the same sub-expression. Every measure with more than two sub-expressions should use VAR.
Annotate measures with type	Always set the measure's data type and format string in the Format pane immediately after creating it. An unformatted measure is unfinished.
Filter first in CALCULATE	Boolean predicates (col = value) in CALCULATE are faster than FILTER(table, condition) because they use the column index. Use FILTER only when the condition involves a measure.
ALL vs ALLSELECTED: know the user intent	Ask: should this denominator respect the user's slicer selection? Yes → ALLSELECTED. No (absolute grand total) → ALL.
Mark your Date table	Without a marked Date table, time intelligence functions silently fail or return wrong results. Always mark it immediately after creating it.
Use SWITCH(TRUE()) over nested IF	For 3+ conditions, SWITCH(TRUE(), ...) is far more readable and easier to maintain than nested IF statements.
Calculated columns increase model size	Every calculated column is stored in memory for every row. Use measures wherever possible. Only create a calculated column when you need to slice, filter, or group by the result.
Test measures with a matrix	A Card visual hides context errors. Always test a new measure in a matrix with at least one row/column field to verify it responds correctly to different filter contexts.

End of Module 1

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

1

What is DAX?

Foundations of the DAX Language

Understand the Tabular data model, table relationships and cross-filter direction, how DAX differs from Excel formulas and SQL, the distinction between calculated columns and measures, DAX data types and operators, variables, and essential error handling — the complete conceptual foundation for every DAX formula you will write.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 1: What is DAX?

Module Contents

■ Module Overview and Prerequisites

■ Lesson 1: The Tabular Data Model

- What Is a Tabular Model?
- Tables, Columns, and Relationships
- The One-Side and the Many-Side
- Relationship Chains
- Cross-Filter Direction

■ Lesson 2: DAX Compared to Excel, SQL, and MDX

- DAX vs Excel: Cells vs Tables
- Iterators — A New Concept
- DAX vs SQL: A Functional Language
- DAX vs MDX: A Clean Break

■ Lesson 3: Calculated Columns and Measures

- What Is a Calculated Column?
- What Is a Measure?
- When to Use Each — The Definitive Rule
- Introduction to Evaluation Context

■ Lesson 4: DAX Syntax Fundamentals

- Referencing Tables and Columns
- DAX Data Types
- DAX Operators and Table Constructors
- Comments and Code Formatting

■ Lesson 5: Variables in DAX

- Declaring Variables with VAR and RETURN
- Lazy Evaluation and Performance
- Scoping Rules

■ Lesson 6: Error Handling in DAX

- Conversion Errors and Automatic Type Coercion
- Arithmetic Errors — Division and SQRT
- BLANK vs Zero vs NULL
- IFERROR, ISERROR, and ISBLANK
- Generating Intentional Errors

■ Demonstration 1: Exploring the Tabular Model

■ Demonstration 2: Calculated Column vs Measure — The Grand Total Test

■ Lab 1: What is DAX? — Building Your First Formulas

- Exercise 1: Navigate the Data Model
- Exercise 2: Create Calculated Columns
- Exercise 3: Create Measures and Compare Results
- Exercise 4: Variables and Error Handling

■ Module Review and Takeaways

Module Overview

DAX — Data Analysis eXpressions — is the calculation language of Microsoft Power BI, Analysis Services Tabular, and Power Pivot for Excel. It was introduced in 2010 and has grown into one of the most powerful analytical languages in business intelligence. Every KPI, every year-over-year comparison, every dynamic ranking you see in a Power BI report is computed by a DAX formula.

This module establishes the conceptual foundation you need before writing a single formula. Understanding **why** DAX behaves the way it does is more important than memorising function names. DAX beginners who skip this foundation struggle at every advanced topic; those who invest in the fundamentals progress rapidly. The most important idea in this module — and the one that will pay dividends throughout the entire course — is the distinction between a **calculated column** and a **measure**.

EXAM

The PL-300 exam tests DAX fundamentals in every domain. Module 1 concepts — data model design, cross-filter direction, the difference between calculated columns and measures, and BLANK handling — appear directly in exam questions. Do not skip this module even if you have used DAX before.

Before You Begin

- Install Power BI Desktop (free download at powerbi.microsoft.com).
- Download the Contoso sample PBIX file linked in the lab support files — this file is used throughout all exercises.
- No prior DAX knowledge is required. Familiarity with Excel formulas is helpful but not essential.
- Complete this module before proceeding to Module 2 (Introducing DAX). All subsequent modules assume the concepts covered here.

LESSON 1

The Tabular Data Model

Lesson Objectives

After completing this lesson, you will be able to:

- Describe the structure of a Tabular model: tables, columns, rows, and relationships.
- Explain the one-side and many-side of a relationship and identify the key column.
- Trace a relationship chain to explain how a filter propagates across multiple tables.
- Describe the difference between unidirectional and bidirectional cross-filter.
- Explain when bidirectional filtering is appropriate and what risks it introduces.

What Is a Tabular Model?

A Tabular model is a database of **tables** held in memory and optimised for analytical queries. Power BI, Analysis Services Tabular, and Power Pivot all use the same underlying storage engine — **VertiPaq** — which compresses columns individually and answers queries by operating on those compressed column arrays. The entire model lives in RAM, which is why queries return in milliseconds even over hundreds of millions of rows.

DAX is the only language used to program a Tabular model. You write DAX to create calculated columns, measures, and calculated tables. You also write DAX queries to retrieve data from the model. In this course, we focus on Power BI Desktop as our tool, but every formula and concept applies identically to Analysis Services and Power Pivot.

Tables, Columns, and Relationships

A table is a set of rows, each row divided into typed columns. Each column holds one type of information — a date, a number, a text string. When two tables share a common column with matching values, we can create a **relationship** between them. Relationships let DAX use values from one table to filter or enrich another.

TIP

In a well-designed Power BI model you will typically see a **star schema**: a central fact table (Sales, Orders, Transactions) surrounded by dimension tables (Date, Product, Customer, Region). The fact table contains foreign keys and numeric measures. Dimension tables contain descriptive attributes. This is not required by DAX, but it is strongly recommended for performance and clarity.

The One-Side and Many-Side of a Relationship

Every relationship in a Tabular model has two sides, denoted **1** and ***** (or **many**) in the diagram. These are the rules:

Concept	Explanation
One-side (1)	The column used on this side must contain only unique values . No duplicates. No blanks. This is always the dimension table. Example: Product[ProductKey] — every product has exactly one key.
Many-side (*)	The column on this side may repeat the same value many times. This is always the fact table. Example: Sales[ProductKey] — many sales rows refer to the same product.

Key column	The columns used to join the two tables are called the keys . They typically share the same name. The one-side key must be unique; the many-side key is a foreign key that repeats freely.
Inactive relationship	A model can have more than one relationship between the same two tables, but only one can be active at a time. Inactive relationships are shown as dashed lines and are activated in DAX using <code>USERELATIONSHIP()</code> .

Relationship Chains

Relationships can form chains. If Sales is related to Product, and Product is related to ProductCategory, then a filter on ProductCategory automatically propagates to Sales — passing through two relationships. DAX traverses this chain automatically when you write a measure that refers to `Sales[Amount]` in a report sliced by Category.

This is one of the most powerful features of the Tabular model: you declare relationships once in the model, and every DAX measure automatically benefits from them. You never write explicit JOIN syntax in a measure — DAX handles it.

NOTE

In SQL, you must write an explicit JOIN in every query that needs data from related tables. In DAX, relationships are declared once in the model and are automatically used by all measures and calculated columns. This is a fundamental architectural difference.

Cross-Filter Direction

The direction of a relationship determines **which table filters which**. Filters always flow from the one-side to the many-side automatically. The arrow in the relationship diagram shows the direction the filter travels.

Single Direction (→)	Both Directions (↔)
Filter flows from Product (1-side) to Sales (*-side)	Filter flows in both directions simultaneously
A Product slicer filters Sales rows automatically	Sales can also filter Product — and all connected tables
Sales cannot filter Product (single arrow)	Enables slicers on fact-side columns to filter dimensions
Most relationships — the default and safest setting	Introduces risk of ambiguous filter paths in complex models
Predictable behaviour; no ambiguity	Use with care; prefer single direction where possible

Consider a model with Product, Sales, and Date all related. If you place Color (from Product) on rows and Count of Date (from Date) in values, with single-direction relationships, Date is **not** filtered by Color — because the filter from Product cannot cross to Date through Sales. Enabling bidirectional on the Date-Sales relationship fixes this, but creates an ambiguous path if there are multiple relationships between the same tables, causing incorrect results.

EXAM

The PL-300 exam tests cross-filter direction. Key rule: filters flow from the one-side to the many-side by default. Bidirectional relationships are powerful but introduce risk — you will see scenario questions where enabling bidirectional either fixes or breaks a calculation.

LESSON 2

DAX Compared to Excel, SQL, and MDX

Lesson Objectives

After completing this lesson, you will be able to:

- Explain the key differences between DAX and Excel formula behaviour.
- Describe what an iterator is and why it matters.
- Contrast DAX as a functional language with SQL as a declarative language.
- Explain why MDX knowledge does not transfer directly to DAX.

DAX vs Excel: Cells Versus Tables

Excel computes over **cells**, referenced by coordinates (A1, B2, C3). DAX computes over **columns in tables**. There is no concept of a cell address in DAX. Every formula must reference a column, identified by its table name and column name:

```
-- Excel formula (cell-based):  
= E4 * D4  
  
-- DAX equivalent (column-based):  
Sales[SalesAmount] = Sales[ProductQuantity] * Sales[NetPrice]  
  
-- Column reference syntax: TableName[ColumnName]  
-- Both the table name and column name are always required for columns  
-- Measures (discussed in Lesson 3) omit the table name: [MeasureName]
```

This difference has an important implication: in Excel, you can write different formulas in different cells of the same column. In DAX, a calculated column formula applies identically to **every row** of the table — the row context changes automatically for each row. There is no row 4 vs row 5; there is only the current row being evaluated.

Iterators — A New Concept for Excel Users

In Excel, multi-step calculations often require intermediate columns. To compute total revenue, you first create a SalesAmount column (Quantity × Price), then SUM it in a separate cell. DAX can collapse this into a single formula using an **iterator**:

```
-- Excel approach: requires an intermediate calculated column  
-- Step 1 (column): Sales[SalesAmount] = Sales[Quantity] * Sales[Price]  
-- Step 2 (measure): Total Sales := SUM( Sales[SalesAmount] )  
  
-- DAX iterator approach: no intermediate column needed  
Total Sales := SUMX( Sales, Sales[Quantity] * Sales[Price] )  
  
-- SUMX iterates over every row of the Sales table,  
-- computes Quantity * Price for that row,  
-- then sums all the per-row results into a single value.  
-- The calculation happens in memory – no column is created or stored.
```

All DAX iterators follow the same pattern: they take a table as their first argument and an expression to evaluate per row as their second argument. Common iterators include SUMX, AVERAGEX, COUNTX, MAXX, MINX, and RANKX. You will use them extensively throughout this course.

TIP

Prefer iterators over intermediate calculated columns whenever the intermediate value is only needed for one calculation. Calculated columns are stored in RAM — they permanently consume model memory. An iterator performs the same calculation at query time without any storage cost.

DAX vs SQL: A Functional Language

SQL is a **declarative** language: you describe *what* you want and the engine decides how to retrieve it. DAX is a **functional** language: every expression is a function call, and function parameters can themselves be function calls, building up complex nested expressions.

```
-- SQL: declarative -- you declare the result you want
SELECT Customer[Name], SUM(Sales[Amount]) AS Revenue
FROM Sales
JOIN Customer ON Sales.CustomerKey = Customer.CustomerKey
WHERE Customer.Continent = 'Europe'
GROUP BY Customer[Name]

-- DAX query: functional -- you nest function calls
EVALUATE
SUMMARIZECOLUMNS(
    Customer[Name],
    FILTER( Customer, Customer[Continent] = "Europe" ),
    "Revenue", SUM( Sales[Amount] )
)
```

The key practical difference: in SQL, you write explicit JOIN syntax every time you query related tables. In DAX, you declare relationships once in the model and they are **used automatically** by every formula. Writing `SUM(Sales[Amount])` in a measure automatically respects whatever Product or Date filters are active in the report — without a single JOIN being written.

DAX vs MDX: A Clean Break

MDX is the query language for Analysis Services Multidimensional (cube) models. If you have an MDX background, be aware: DAX and MDX share almost nothing. MDX operates in a multidimensional coordinate space defined by dimensions and hierarchies. DAX operates on relational tables.

MDX (Multidimensional)	DAX (Tabular)
Works on a multidimensional cube	Works on relational tables
Coordinates defined by dimension members	Rows identified by column values
Hierarchies are first-class constructs	Hierarchies are optional; calculated in DAX
SCOPE statement controls where formulas apply	IF + ISINSCOPE replaces SCOPE
PrevMember navigates the hierarchy	SAMEPERIODLASTYEAR + date table replaces PrevMember
Leaf-level calculations are very expensive	Leaf-level calculations are inexpensive (VertiPaq)

WARNING

MDX experience often leads to incorrect DAX intuitions. The most dangerous transfer is expecting MDX-style SCOPE behaviour in DAX — it does not exist. Free your mind from MDX concepts and learn DAX on its own terms.

LESSON 3

Calculated Columns and Measures

Lesson Objectives

After completing this lesson, you will be able to:

- Define a calculated column and explain when it is physically stored.
- Define a measure and explain when it is computed.
- Apply the definitive rule for choosing between a calculated column and a measure.
- Explain what evaluation context is at an introductory level.
- Identify the grand-total problem that reveals when a calculated column is wrong.

Calculated Columns

A **calculated column** is a new column you add to an existing table using a DAX formula. It is computed when the data model is refreshed and stored in VertiPaq just like any other column. The formula is evaluated **row by row** — DAX evaluates the formula once for each row of the table, using that row's values.

```
-- Calculated column: computed once at refresh, stored in RAM
-- Syntax: TableName[NewColumn] =

Sales[SalesAmount] = Sales[Quantity] * Sales[Net Price]
Sales[TotalCost] = Sales[Quantity] * Sales[Unit Cost]
Sales[GrossMargin] = Sales[SalesAmount] - Sales[TotalCost]

-- This column exists as a physical column in the Sales table.
-- It consumes RAM proportional to the number of rows.
-- It can be used as an axis in a chart or in a slicer.
-- It is computed in the ROW CONTEXT of each Sales row.
```

WARNING

Calculated columns consume RAM permanently. A model with 10 million rows and 5 unnecessary calculated columns wastes significant memory. Use calculated columns sparingly — only when the column's value must be available as a filter, a slicer axis, or as an input to a relationship.

Measures

A **measure** is a DAX formula that computes a value **on demand**, at query time, based on the current filter context of the report. It is never stored — it is computed fresh every time a visual requests it. Measures always operate on aggregated values, not row-by-row values.

```
-- Measure: computed dynamically, never stored, always aggregates
-- Syntax: [MeasureName] := (note := instead of =)

[Total Sales] := SUM( Sales[SalesAmount] )
[Total Cost] := SUM( Sales[TotalCost] )
[Gross Margin] := [Total Sales] - [Total Cost]

-- This IS correct for a percentage measure:
[Gross Margin %] := DIVIDE( [Gross Margin], [Total Sales] )

-- The measure is computed in the FILTER CONTEXT of the visual.
-- If a slicer selects 'Bikes', [Total Sales] returns only Bike sales.
```



```
-- If the visual has a grand total row, it computes the overall total correctly.
```

The Grand Total Test — The Most Important Rule

The critical difference between calculated columns and measures becomes visible at the **grand total row**. Consider a gross margin percentage:

```
-- WRONG: calculated column for a percentage
Sales[GrossMarginPct] = Sales[GrossMargin] / Sales[SalesAmount]
-- Grand total shows SUM of all per-row percentages -- MEANINGLESS.
-- Changing aggregation to AVERAGE is still wrong.

-- CORRECT: measure for a percentage
[Gross Margin %] := DIVIDE( SUM(Sales[GrossMargin]), SUM(Sales[SalesAmount]) )
-- Grand total correctly computes total margin / total sales.
```

The rule is absolute: any formula that needs to produce a correct result at an aggregated level — in a grand total, a subtotal, or across a slicer selection — **must be a measure**. Percentages, ratios, weighted averages, and rankings can never be correct as calculated columns.

EXAM

This is one of the most frequently tested concepts in PL-300. Questions present a scenario where a report shows incorrect grand totals and ask you to identify the cause. The answer is almost always that a measure should be used instead of a calculated column, or that DIVIDE should be used instead of the / operator.

Introduction to Evaluation Context

The **evaluation context** is the environment in which a DAX formula is computed. There are two types, which you will study in depth in Module 4:

Context	Description	Example
Row context	The context created when DAX iterates over a table row by row. Calculated columns are evaluated in a row context — DAX knows which row it is on.	<code>Sales[GrossMargin] = Sales[SalesAmount] - Sales[TotalCost]</code>
Filter context	The set of filters active at the time a measure is evaluated — determined by slicers, report page filters, chart axes, and visual-level filters. Measures are evaluated in a filter context.	<code>[Total Sales] := SUM(Sales[SalesAmount])</code> — SUM only adds rows passing the filter

NOTE

You do not need to fully understand evaluation contexts yet — that is the subject of Module 4 (Understanding Evaluation Contexts). For now, remember: calculated columns have row context (they know the current row), and measures have filter context (they know the current report selections).

LESSON 4

DAX Syntax Fundamentals

Lesson Objectives

After completing this lesson, you will be able to:

- Write syntactically correct column and measure references.
- Identify the seven DAX data types and their practical implications.
- Use all DAX operators including the IN operator with table constructors.
- Write single-line and multi-line DAX comments.

Referencing Tables and Columns

```
-- Column reference: always include the table name
Sales[SalesAmount] -- Table[Column] syntax
'Product Category'[Name] -- Use single quotes when name contains spaces

-- Measure reference: NEVER include the table name
[Total Sales] -- Just [MeasureName]
[Gross Margin %] -- Square brackets always

-- This convention is enforced by the de facto standard (and this course):
-- Columns always have the table prefix. Measures never do.
-- Why? Context transition (Module 5) makes the distinction critical.

-- Single-line comments: -- or //
Sales[Amount] * 1.25 -- Increase by 25%
Sales[Amount] * 1.25 // Same comment, different style

/* Multi-line comment
spans multiple lines
useful for documenting complex expressions */
```

DAX Data Types

DAX supports seven data types. Understanding them prevents conversion errors and unexpected BLANK propagation:

DAX Type	Power BI Name	Storage	Typical Use
Integer	Whole Number	64-bit integer	Customer keys, counts, year numbers
Decimal	Decimal Number	Double-precision float (64-bit)	Prices, rates — not for exact currency
Currency	Fixed Decimal	64-bit int ÷ 10,000 (4 dp)	Financial amounts requiring exact rounding
DateTime	Date / Time	Days since 30 Dec 1899	Order dates, timestamps — split into Date + Time
Boolean	True/False	1-bit logical	Flag columns, conditions
String	Text	Unicode string	Names, codes, categories
Binary	Binary	Blob	Images — not usable in DAX formulas

WARNING

Never store a DateTime column on a large fact table. A timestamp with microsecond precision is near-unique — VertiPaq cannot compress it. Always split DateTime into a Date column (low cardinality, joins to DimDate) and, if needed, a TimeOfDay or HourOfDay column.

DAX Operators and Table Constructors

```
-- Arithmetic: + - * /
Sales[Margin] = Sales[Revenue] - Sales[Cost]

-- Comparison: = < > >= <=
Sales[IsLargeOrder] = Sales[Quantity] >= 100

-- Logical: && || NOT (also AND() and OR() functions exist)
Sales[IsEuropeLarge] = Customer[Continent]="Europe" && Sales[Quantity]>=100

-- Text concatenation: &
Customer[FullName] = Customer[FirstName] & " " & Customer[LastName]

-- IN operator with table constructor {}
Product[Color] IN { "Red", "Blue", "White" } -- single-column set

-- Multi-column IN
( 'Date'[Year], 'Date'[MonthNumber] ) IN { (2023,12), (2024,1), (2024,2) }

-- Auto type conversion: DAX converts automatically based on operator
5 & 4 -- returns "54" (string) -- & forces string conversion
"5" + "4" -- returns 9 (number) -- + forces numeric conversion
-- Avoid relying on implicit conversion -- always use explicit conversion functions
```

LESSON 5

Variables in DAX

Lesson Objectives

After completing this lesson, you will be able to:

- Declare variables using VAR and provide a result using RETURN.
- Explain lazy evaluation and why it means unused variables have zero cost.
- Use variables to avoid duplicate sub-expressions and improve readability.
- Describe the scoping rules that prevent global variables in DAX.

VAR and RETURN

Variables are one of the most impactful improvements ever made to DAX. Before variables, complex measures had to repeat the same sub-expression multiple times, making them hard to read and potentially slower. Variables solve both problems.

```
-- Without variables: sub-expression SUM(Sales[SalesAmount]) repeated
[Gross Margin %] :=
DIVIDE(
SUM( Sales[SalesAmount] ) - SUM( Sales[TotalCost] ),
SUM( Sales[SalesAmount] ) -- evaluated a SECOND time
)

-- With variables: clean, readable, evaluated only once
[Gross Margin %] :=
VAR TotalSales = SUM( Sales[SalesAmount] )
VAR TotalCost = SUM( Sales[TotalCost] )
VAR GrossMargin = TotalSales - TotalCost
RETURN
DIVIDE( GrossMargin, TotalSales )

-- Rules:
-- 1. VAR defines the variable (uses the CURRENT filter context at definition)
-- 2. RETURN specifies the expression that becomes the measure's result
-- 3. You can define multiple VARs before the single RETURN
-- 4. Variables are local -- they cannot be used outside their expression
-- 5. Variables are lazily evaluated -- unused ones are never computed
```

Lazy Evaluation

DAX evaluates variables using **lazy evaluation**: a variable is only computed if its value is actually used in the RETURN expression. If you define a variable but never reference it, DAX skips its evaluation entirely. This means you can define diagnostic or conditional variables freely without worrying about performance cost when they are not needed.

A variable is also computed **at most once**, regardless of how many times it appears in the RETURN expression. This is the key performance benefit: expensive sub-expressions (like CALCULATE calls or complex aggregations) can be captured in a variable and referenced multiple times at zero additional cost.

Scoping Rules

```
-- Variables are LOCAL to the expression that defines them
-- There are NO global variables in DAX

[Measure A] :=
VAR LocalVar = SUM( Sales[Amount] ) -- only visible inside [Measure A]
RETURN LocalVar

[Measure B] :=
VAR LocalVar = 42 -- a DIFFERENT LocalVar, completely independent
RETURN LocalVar * 2

-- Variables capture the filter context at the point of their DEFINITION
-- This is critical for advanced patterns (covered in Module 6)
-- For now: remember that VAR 'freezes' the context at definition time
```

TIP

Always use variables when writing measures with more than two sub-expressions. Variables make DAX code dramatically more readable and maintainable. They also make debugging easier — you can add a `RETURN variable_name` to inspect the value of any intermediate result.

LESSON 6

Error Handling in DAX

Lesson Objectives

After completing this lesson, you will be able to:

- Identify the three categories of DAX errors: conversion, arithmetic, and BLANK.
- Use IFERROR, ISERROR, and ISBLANK to handle errors gracefully.
- Explain BLANK behaviour in arithmetic and logical expressions.
- Distinguish between BLANK, zero, and empty string.
- Use DIVIDE instead of / to avoid division-by-zero errors.

Conversion Errors

DAX automatically converts between types based on the operator used. The + operator converts both sides to numbers; the & operator converts both sides to strings. Automatic conversion raises an error if the value cannot be converted:

```
-- Safe conversions (constant values, always work):
5 & 4 -- returns "54" (integers become strings)
"5" + "4" -- returns 9 (strings become numbers)

-- Dangerous conversion (depends on column data at runtime):
Sales[VatCode] + 100
-- If VatCode contains "A14", DAX raises:
-- Cannot convert value 'A14' of type Text to type Number

-- Best practice: use explicit conversion functions
VALUE( Sales[VatCode] ) + 100 -- Explicit; fails with clear error
IFERROR( VALUE(Sales[VatCode]) + 100, BLANK() ) -- Safe fallback
```

Arithmetic Errors and DIVIDE

```
-- Division by zero produces an error, not infinity
Sales[Price] / Sales[Quantity] -- Error when Quantity = 0

-- Use DIVIDE() to handle division safely
DIVIDE( Sales[Price], Sales[Quantity] ) -- Returns BLANK when denominator = 0
DIVIDE( Sales[Price], Sales[Quantity], 0 ) -- Returns 0 when denominator = 0
DIVIDE( Sales[Price], Sales[Quantity], "N/A" ) -- Returns "N/A" (text fallback)

-- SQRT of a negative number:
SQRT( -1 ) -- Error: argument has wrong data type

-- Safe pattern: test before computing
IF( Test[Value] >= 0, SQRT(Test[Value]), BLANK() )
```

BLANK — Not Zero, Not Empty String

DAX uses BLANK() to represent missing or unknown values. BLANK is **not** the same as zero or an empty string. However, DAX's implicit comparison rules can make them appear equivalent:

```
-- BLANK comparisons -- these always return TRUE (confusing!)
BLANK() = 0 -- TRUE
```

```

BLANK() = "" -- TRUE

-- Use ISBLANK() to test for true blank
ISBLANK( Sales[DiscountPct] ) -- TRUE only if the cell is truly blank
Sales[DiscountPct] = 0 -- TRUE if blank OR zero

-- BLANK propagation in arithmetic:
BLANK() + 5 = 5 -- BLANK acts as 0 in addition
BLANK() * 5 = BLANK() -- BLANK propagates in multiplication
BLANK() / 5 = BLANK() -- BLANK propagates in division

-- Returning BLANK intentionally (correct pattern for conditional measures)
[Weekend Sales] :=
IF(
WEEKDAY( MAX('Date'[Date]) ) IN {1,7}, -- 1=Sunday, 7=Saturday
SUM( Sales[Amount] ) -- return value on weekends
-- third argument omitted: defaults to BLANK() on weekdays
)

```

IFERROR and ISERROR

```

-- IFERROR: catch any error and return an alternative value
IFERROR( SQRT( Test[Value] ), BLANK() ) -- returns BLANK if SQRT errors
IFERROR( VALUE( Sales[Code] ), 0 ) -- returns 0 if conversion fails

-- Equivalent verbose pattern using ISERROR + IF:
IF( ISERROR( SQRT( Test[Value] ) ), BLANK(), SQRT( Test[Value] ) )
-- IFERROR is preferred: expression is only written once

-- CRITICAL: Variables and IFERROR interact unexpectedly
-- The error happens when the VAR is DEFINED, not when it is used:

-- This always errors -- IFERROR never runs:
VAR Result = SQRT( -1 ) -- error here, execution stops
RETURN IFERROR( Result, 0 )

-- Correct pattern: use IFERROR BEFORE the VAR
VAR Result = IFERROR( SQRT( Test[Value] ), 0 ) -- safe
RETURN Result

```

EXAM

The PL-300 exam tests DIVIDE vs / operator usage. Always use DIVIDE() in measures to avoid division-by-zero errors producing visible error symbols in visuals. The third argument of DIVIDE sets the alternate result (default is BLANK).

DEMONSTRATION 1

Exploring the Tabular Data Model

Scenario: The instructor opens the Contoso PBIX file in Power BI Desktop and walks through the data model, demonstrating relationships, cross-filter direction, and how a filter propagates from a dimension to a fact table.

Demonstration Steps

1	On the ribbon, click Model view (the diagram icon on the left navigation bar). Show the six tables: Sales, Date, Product, Product Subcategory, Product Category, and Customer.
2	Click the relationship line between Sales and Product . In the Edit Relationship dialog, point out the one-side (Product[ProductKey], unique) and many-side (Sales[ProductKey], repeated). Note the single arrow direction.
3	On the ribbon, click Report view . Add a Table visual. Drag Product[Color] to Rows and Count of Date to Values. Show that Count of Date is the same for all colors — the single-direction relationship prevents the Product filter reaching Date.
4	Switch back to Model view. Double-click the Sales-Date relationship. Change Cross filter direction to Both . Click OK.
5	Return to Report view. Show that Count of Date now varies by Color — it counts only dates on which that color was sold. Explain this is the bidirectional effect.
6	Change the relationship back to Single . Explain that bidirectional filtering is rarely needed and introduces risk of ambiguous filter paths in complex models. The default should always be Single.

RESULT

The data model is visible in Model view with all six tables and their relationships. The cross-filter direction change is demonstrated live: Count of Date changes from a constant to a per-color count when bidirectional filtering is enabled. The relationship is restored to single direction.

DEMONSTRATION 2

Calculated Column vs Measure — The Grand Total Test

Scenario: The instructor demonstrates that a gross margin percentage implemented as a calculated column produces a wrong grand total, and then fixes it with a measure.

Demonstration Steps

1	In Power BI Desktop, click Table tools on the ribbon, select the Sales table, and click New column . Enter the formula: Sales[GrossMarginPct] = Sales[GrossMargin] / Sales[SalesAmount] . Press Enter.
2	Add a Matrix visual to the report canvas. Place Product[Category] on Rows. Place Sales[GrossMarginPct] in Values. Change the aggregation to Sum. Show the grand total row.
3	Point out that the grand total shows a large, incorrect number — it is the sum of all per-row margin percentages, which is meaningless. Even changing the aggregation to Average produces the wrong result.
4	Delete the calculated column. Click New measure on the Sales table. Enter: [Gross Margin %] := DIVIDE(SUM(Sales[GrossMargin]), SUM(Sales[SalesAmount])) .
5	Replace Sales[GrossMarginPct] in the visual with the new [Gross Margin %] measure. Show the grand total — it now correctly computes total margin divided by total sales.
6	Add a slicer for Product[Category] . Select Bikes. Show that [Gross Margin %] automatically adjusts to the Bikes-only context. This is the filter context at work.

RESULT

The calculated column version shows an incorrect grand total. The measure version shows the correct grand total and responds correctly to the category slicer. The demonstration confirms the grand total test rule.

LAB 1

What is DAX? — Building Your First Formulas

Lab Scenario

You are a Power BI developer at Contoso Ltd. The sales team has requested a report showing gross margin percentage by product category, with correct subtotals and grand totals. The existing model contains a flat calculated column for GrossMarginPct that is producing incorrect totals. You must fix the model by replacing the calculated column with a measure, add supporting measures, and verify that the data model relationships are correctly configured.

NOTE

Open the file **Contoso_Lab01_Start.pbix** from the lab files folder. All exercises in this lab use this file. Save your work as **Contoso_Lab01_Complete.pbix** when finished.

Exercise 1: Navigate the Data Model

Estimated time: 15 minutes

Task 1: Open and Inspect the Model

1. Open Power BI Desktop. On the **File** menu, click **Open report**, then click **Browse reports**. Navigate to your lab files folder and open **Contoso_Lab01_Start.pbix**.
2. On the left navigation bar, click the **Model view** icon (the diagram icon below the report icon).
3. Examine the diagram. Count the number of tables and record them: Sales, Date, Product, Product Subcategory, Product Category, Customer.
4. Click the relationship line between **Sales** and **Product**. In the **Properties** pane on the right, note the Cross filter direction. Confirm it is set to **Single**.
5. Click the relationship line between **Date** and **Sales**. Note the Cross filter direction. Confirm it is also **Single**.
6. Hover over each table in the diagram to see a preview of its columns. Identify which table is the fact table (contains SalesAmount and Quantity) and which are dimension tables.

Results: You have identified six tables in the model. Sales is the fact table. Date, Product, Product Subcategory, Product Category, and Customer are dimension tables. All relationships are single-direction.

Task 2: Trace a Relationship Chain

1. In Model view, identify the chain: Sales → Product → Product Subcategory → Product Category. Trace the arrows — each arrow points from the one-side to the many-side.
2. On the left navigation bar, click **Report view**. On the ribbon **Insert** tab, click **Table** to add a table visual.
3. In the **Data** pane on the right, expand the **Product Category** table. Drag **Category** to the **Columns** field well of the table visual.
4. Expand the **Sales** table. Drag **SalesAmount** to **Values**. Observe that Sales is correctly filtered by Category, even though there is no direct relationship between Sales and Product Category.
5. Add **Customer[CustomerName]** as a third column. Notice that values are now wrong — the table shows duplicate rows with the same customer across multiple categories. Remove CustomerName from the visual.

Results: SalesAmount is correctly aggregated by Category through the two-hop chain Sales → Product → Product Subcategory → Product Category. Adding CustomerName incorrectly combined categories with individual customers — this is a sign of a mismatched granularity, not a model error.

Exercise 2: Create Calculated Columns

Estimated time: 20 minutes

Task 1: Create Basic Revenue Columns

1. In the **Data** pane, click the **Sales** table to select it. On the **Table tools** ribbon tab, click **New column**.
2. In the formula bar, replace **Column =** with the following formula, then press **Enter**:
`.Sales[SalesAmount] = Sales[Quantity] * Sales[Net Price]`
3. Click **New column** again on the **Table tools** tab. Enter the following formula and press **Enter**:
`.Sales[TotalCost] = Sales[Quantity] * Sales[Unit Cost]`
4. Click **New column** one more time. Enter the following formula and press **Enter**:
`.Sales[GrossMargin] = Sales[SalesAmount] - Sales[TotalCost]`
5. On the left navigation bar, click **Data view** (the table icon). Scroll to the Sales table. Verify that the three new columns appear and contain numerical values.

Results: The Sales table has three new calculated columns: SalesAmount, TotalCost, and GrossMargin. Each row shows the correct per-row calculation. The Data view confirms values are present in all rows.

Task 2: Observe the Grand Total Problem

1. Click **New column** on the **Table tools** tab. Enter the following formula and press **Enter**:
`.Sales[GrossMarginPct] = DIVIDE( Sales[GrossMargin], Sales[SalesAmount] )`
2. Switch to **Report view**. Add a new **Matrix** visual. Place **Product Category[Category]** on **Rows**. Place **Sales[GrossMarginPct]** in **Values**.
3. In the Values field well, click the dropdown arrow next to GrossMarginPct. Verify the aggregation is set to **Sum**. Note the **Total** row at the bottom of the matrix.
4. Record the Total value for GrossMarginPct. It should be a large number — the sum of all per-row percentages. This is the **grand total problem**: the column aggregates incorrectly.
5. Try changing the aggregation to **Average**. The Total value changes but is still incorrect — it is an average of averages, not the true weighted gross margin.

Results: The GrossMarginPct calculated column shows an incorrect Total row regardless of the aggregation method used. This demonstrates why percentages and ratios must be measures, not calculated columns.

Exercise 3: Create Measures and Compare Results

Estimated time: 25 minutes

Task 1: Create Core Measures

1. In the **Data** pane, click the **Sales** table. On the **Table tools** ribbon tab, click **New measure**.
2. Enter the following measure and press **Enter**:
`. [Total Sales] := SUM( Sales[SalesAmount] )`
3. Click **New measure** again. Enter the following measure:
`. [Total Cost] := SUM( Sales[TotalCost] )`

- Click **New measure** again. Enter the following measure:
`[Gross Margin] := [Total Sales] - [Total Cost]`
- Click **New measure** again. Enter the following measure and press **Enter**:
`[Gross Margin %] := DIVIDE( [Gross Margin], [Total Sales] )`
- In the **Data** pane, verify that the four new measures appear in the Sales table with a calculator icon.

Results: Four measures have been created in the Sales table: Total Sales, Total Cost, Gross Margin, and Gross Margin %. Measures appear with a calculator icon in the Data pane.

Task 2: Compare Column vs Measure Grand Totals

- Select the Matrix visual you created in Exercise 2. In the **Values** field well, remove **Sales[GrossMarginPct]** by clicking its X.
- Drag the **[Gross Margin %]** measure from the Data pane into the **Values** field well of the same matrix.
- Examine the **Total** row. The value should now be the correct overall gross margin percentage — total gross margin divided by total sales.
- Add a **Slicer** visual to the canvas. Add **Product Category[Category]** to the slicer. Click **Bikes** in the slicer. Verify that all matrix values update.
- Compare the Bikes [Gross Margin %] value in the matrix with the Total row. Verify that both values now make analytical sense.
- Select **Accessories** in the slicer. Observe that all values update dynamically. This is the **filter context**.

Results: The [Gross Margin %] measure shows a correct Total row for all selections. The slicer demonstrates that the measure responds to the filter context. The calculated column GrossMarginPct would not respond correctly to the grand total.

Exercise 4: Variables and Error Handling

Estimated time: 20 minutes

Task 1: Refactor a Measure Using Variables

- In the **Data** pane, right-click the **[Gross Margin %]** measure and click **Edit measure**.
- Replace the existing formula with the following variable-based version:
`[Gross Margin %] :=
 VAR TotalSales = SUM(Sales[SalesAmount])
 VAR TotalCost = SUM(Sales[TotalCost])
 VAR GrossMargin = TotalSales - TotalCost
 RETURN
 DIVIDE(GrossMargin, TotalSales)`
- Press **Enter**. Verify the measure produces identical results to the previous version in the matrix visual.
- Click **New measure**. Create a diagnostic version that returns the TotalSales variable for inspection:
`[Debug TotalSales] :=
 VAR TotalSales = SUM(Sales[SalesAmount])
 RETURN TotalSales`
- Add **[Debug TotalSales]** to the matrix Values field well. Verify that it matches **[Total Sales]**. Then remove it from the visual (do not delete the measure).

Results: The refactored [Gross Margin %] measure using VAR produces identical results. The debug technique of returning a single VAR confirms the intermediate value. Variables improve readability without changing the calculation result.

Task 2: Safe Division and BLANK Handling

1. Click **New measure**. Create a measure that would fail without DIVIDE:

```
. [Units Per Transaction] :=  
    DIVIDE( SUM(Sales[Quantity]), COUNTROWS(Sales) )
```

2. Add **[Units Per Transaction]** to the matrix visual. Confirm the values are reasonable (typically 1–3 units per transaction).

3. Click **New measure**. Create a measure that uses ISBLANK:

```
. [Sales or Zero] :=  
    IF( ISBLANK( SUM(Sales[SalesAmount]) ), 0, SUM(Sales[SalesAmount]) )
```

4. Add **[Sales or Zero]** to the visual. Verify it shows 0 for any category that has no sales (instead of blank).

5. On the **File** menu, click **Save**. Save the file as **Contoso_Lab01_Complete.pbix**.

Results: The **[Units Per Transaction]** measure uses **DIVIDE** to handle any zero **COUNTROWS** scenario. The **[Sales or Zero]** measure converts **BLANK** to 0 using **IF + ISBLANK**. The completed file is saved as **Contoso_Lab01_Complete.pbix**.

Module Review and Takeaways

Review Questions

What is the role of the one-side of a relationship and what constraint applies to it?

Answer: The one-side (marked with 1 in the diagram) is always the lookup or dimension table. The column on the one-side must contain only unique, non-blank values — it is the key column. Filters propagate automatically from the one-side to the many-side. Typical examples: Product[ProductKey], Customer[CustomerID], Date[Date].

A report shows Sales by Country. The grand total of Margin % is 2,847%. What is the most likely cause and fix?

Answer: The most likely cause is that Margin % is implemented as a calculated column, and the visual is showing the Sum of all per-row margin percentages — which is meaningless. The fix is to delete the calculated column and create a measure: [Margin %] := DIVIDE(SUM(Sales[GrossMargin]), SUM(Sales[SalesAmount])). A measure computes the correct ratio at every aggregation level.

What is the difference between a calculated column and a measure?

Answer: A calculated column is computed row-by-row at model refresh time and stored in VertiPaq — it consumes RAM. It uses row context. A measure is computed on demand at query time based on the filter context of the visual — it uses no storage. Use calculated columns for values needed as filter axes, slicer items, or relationship keys. Use measures for all aggregated or dynamic calculations.

What does DIVIDE(Numerator, Denominator) return when the denominator is zero?

Answer: DIVIDE returns BLANK by default when the denominator is zero or blank. A third optional argument specifies an alternative result: DIVIDE(Num, Den, 0) returns 0 instead of BLANK. DIVIDE should always be used in measures instead of the / operator to prevent error symbols appearing in report visuals.

Why is storing a DateTime column on a fact table a performance anti-pattern?

Answer: VertiPaq compresses columns using dictionary encoding — the fewer distinct values, the better the compression. A DateTime column with microsecond precision has a near-unique value on every row in a transactional table, so VertiPaq cannot compress it. This wastes significant RAM. The correct pattern is to split into a Date column (low cardinality — at most 365 per year — joins to DimDate) and an optional TimeOfDay or HourOfDay integer column.

What does BLANK() = 0 evaluate to in DAX, and why is this a common source of bugs?

Answer: BLANK() = 0 evaluates to TRUE in DAX. BLANK() = " (empty string) also evaluates to TRUE. This means a simple equality test cannot distinguish between a genuine zero and a missing value. Developers who expect BLANK to test as not-equal to zero are surprised when their IF conditions behave incorrectly. The fix is to use ISBLANK() when you need to explicitly test for a missing value, not for zero.

When should you use a bidirectional relationship and when should you avoid it?

Answer: Use bidirectional only when you need a filter from the many-side of a relationship to propagate back through to another table — for example, when slicing by a fact-side attribute and needing it to filter an unrelated dimension. Avoid bidirectional in models with multiple relationship paths between the same tables, as it can create ambiguous filter paths that produce incorrect results. The default should always be single-direction. CROSSFILTER() in CALCULATE is a safer alternative for cases where temporary bidirectional filtering is needed.

What is the purpose of the VAR keyword in a DAX measure?

Answer: VAR declares a named variable that stores the result of a DAX expression. Variables serve three purposes: (1) readability — complex sub-expressions are named and described; (2) performance — a variable is computed at most once regardless of how many times it appears in RETURN; (3) context preservation — a VAR captures the filter context at the point of definition, which enables advanced patterns like comparing current vs previous context.

Best Practices

- Always use the table prefix for column references (Sales[Amount]) and never for measure references ([Total Sales]). This convention is critical for understanding context transition in Module 5.
- Use DIVIDE() instead of the / operator in every measure that performs division. DIVIDE handles zero denominators gracefully, preventing error symbols in published reports.
- Prefer measures over calculated columns whenever a formula could be expressed either way. Measures consume zero RAM; calculated columns consume RAM proportional to the number of rows.
- Use calculated columns only when the value is needed as a slicer axis, a chart axis, a relationship key, or a filter — i.e., when the value must be available before query time.
- Always use variables (VAR/RETURN) in measures with more than two sub-expressions. Variables improve readability, prevent duplicate evaluation, and make debugging easier.
- Never store DateTime as datetime type on a large fact table. Split into Date (joins to DimDate) and HourOfDay integer. This alone can reduce model size by 20–40% in transactional models.
- Design your model as a star schema: one central fact table with integer foreign keys, surrounded by dimension tables. This maximises VertiPaq compression and DAX calculation speed.
- Keep cross-filter direction as Single for all relationships by default. Use CROSSFILTER() in a CALCULATE expression when bidirectional filtering is needed for a specific calculation, rather than enabling it model-wide.
- Test every measure's grand total and subtotals. A measure whose grand total is obviously wrong (too large, equal to the row count, or displaying an error) is a signal that a calculated column approach is being used where a measure is needed.
- Use ISBLANK() when you need to distinguish a truly missing value from zero. Remember: BLANK() = 0 is TRUE in DAX, which means the = operator alone cannot test for blank.

Common Issues and Solutions

Grand total shows a sum of percentages

Cause: Margin %, Share %, or any ratio is implemented as a calculated column. The visual sums the per-row ratios, producing a meaningless total.

Solution: Delete the calculated column. Create a measure using DIVIDE: [Margin %] := DIVIDE(SUM(Sales[Margin]), SUM(Sales[Revenue])). The measure computes the correct ratio at every aggregation level.

Slicer selection does not affect a column in a related table

Cause: The relationship between the sliced table and the target table is single-direction, and the filter is trying to travel against the arrow.

Solution: Check the relationship direction in Model view. If the slicer is on the many-side and needs to filter the one-side, enable bidirectional filtering or use CROSSFILTER() in a CALCULATE expression.

BLANK() values are treated as 0 in a filter

Cause: DAX implicit comparison rules mean BLANK() = 0 returns TRUE. A filter like [Discount] = 0 matches both zero and blank discount rows.

Solution: Use ISBLANK([Discount]) to filter for truly blank rows. Use [Discount] = 0 && NOT ISBLANK([Discount]) to filter for explicit zeros only.

A DateTime column causes the model to be unexpectedly large

Cause: A DateTime column on a large fact table has near-unique values — one distinct timestamp per transaction — preventing VertiPaq compression.

Solution: Add a calculated column `Sales[SaleDate] = DATE(YEAR([CreatedAt]), MONTH([CreatedAt]), DAY([CreatedAt]))` or use Power Query to split at load time. Then hide the original datetime column. In Power Query: `Table.AddColumn(Source, "SaleDate", each Date.From([CreatedAt], type date))`.

IFERROR does not catch an error inside a VAR

Cause: The error occurs when the VAR expression is evaluated — before IFERROR runs. IFERROR wrapping the RETURN expression cannot intercept an error in a VAR definition.

Solution: Move IFERROR inside the VAR definition: `VAR SafeResult = IFERROR(SQRT(Test[Value]), BLANK()) RETURN SafeResult`. The error is caught at the point it occurs, not at RETURN.

Module 1 — Key Functions and Concepts Quick Reference

Function / Concept	Category	Description
SUM(column)	Aggregation	Sum all values in column in current filter context
DIVIDE(num, den [, alt])	Arithmetic	Safe division — returns alt (default BLANK) if den=0
SUMX(table, expression)	Iterator	Iterate table row-by-row, sum expression values
IF(condition, true [, false])	Logic	Conditional — false defaults to BLANK()
IFERROR(expression, alt)	Error	Return alt if expression raises any error
ISERROR(expression)	Error	Returns TRUE if expression raises an error
ISBLANK(value)	Error	Returns TRUE only if value is truly BLANK
BLANK()	Error	Explicit BLANK — missing/unknown value sentinel
VAR name = expression	Variables	Define a named local variable
RETURN expression	Variables	Required after VAR block — specifies result value
COUNTROWS(table)	Aggregation	Count rows of table in current filter context
USERELATIONSHIP(col1, col2)	Relationships	Activate an inactive relationship in CALCULATE

Module Summary

- DAX is the functional calculation language of all Microsoft Tabular tools: Power BI, Analysis Services, and Power Pivot. It operates on tables and columns, not cells. Every DAX expression is a function call.
- A Tabular model is a set of tables held in VertiPaq (in-memory column store) linked by relationships. Filters propagate from the one-side to the many-side automatically. Cross-filter direction should be Single by default.
- A calculated column is computed row-by-row at refresh and stored in RAM. A measure is computed dynamically at query time in the filter context of the visual. Use the grand total test to determine which is correct: any aggregated or ratio calculation must be a measure.
- DAX has seven data types. DateTime on large fact tables causes model bloat — always split to Date + time bucket. The BLANK sentinel is not equal to zero via the = operator — use ISBLANK() for explicit blank tests.
- Variables (VAR / RETURN) eliminate duplicate sub-expressions, improve readability, and capture the current filter context at definition time. Every non-trivial measure should use variables.
- DIVIDE() is always preferred over / for division in measures. IFERROR wraps unsafe expressions and returns a safe alternative. BLANK propagates through multiplication and division but collapses to zero in addition.
- The key conceptual foundation for everything that follows: calculated columns live in row context; measures live in filter context. Modules 4 and 5 develop these concepts to their full depth.

NOTE

Next Module — Module 2: Introducing DAX. Covers the full DAX syntax, all basic aggregation functions (SUM, COUNT, MIN, MAX, AVERAGE), text, date, and math functions, and a complete introduction to calculated tables. Build on Module 1 foundations — every concept here is assumed in Module 2.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

2

Introducing DAX

The Core Functions and Syntax of the Language

Aggregation functions and their iterator counterparts, logical functions including IF and SWITCH, information functions, text manipulation, date and time functions, mathematical and conversion functions, RELATED and RELATEDTABLE for cross-table lookups, and professional DAX code formatting. Every function category essential for PL-300 and daily Power BI work.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 2: Introducing DAX

Module Contents

■ Module Overview and Prerequisites

■ Lesson 1: Aggregation Functions

- SUM, AVERAGE, MIN, MAX, and STDEV
- Counting Functions — COUNT, COUNTA, COUNTROWS, DISTINCTCOUNT
- Aggregators vs Iterators — When to Use Each
- SUMX, AVERAGEX, MINX, MAXX, COUNTX

■ Lesson 2: Logical Functions

- IF — The Foundational Conditional
- AND, OR, NOT — Logical Operators as Functions
- SWITCH — Replacing Nested IFs
- SWITCH(TRUE()) — Multi-Condition Pattern

■ Lesson 3: Information Functions

- ISBLANK, ISERROR, ISNUMBER, ISTEXT, ISNONTEXT, ISLOGICAL
- Practical Usage Patterns

■ Lesson 4: Text Functions

- String Manipulation — LEFT, RIGHT, MID, LEN
- Searching — FIND and SEARCH
- Transformation — UPPER, LOWER, TRIM, SUBSTITUTE, REPLACE
- Combining — CONCATENATE and CONCATENATEX

■ Lesson 5: Date and Time Functions

- Date Construction — DATE, TIME, TODAY, NOW
- Date Extraction — YEAR, MONTH, DAY, HOUR, MINUTE, SECOND
- Date Arithmetic and WEEKDAY, WEEKNUM, EOMONTH
- FORMAT for Date Presentation

■ Lesson 6: Mathematical and Conversion Functions

- Rounding — ROUND, ROUNDUP, ROUNDDOWN, FLOOR, CEILING, INT, TRUNC
- Absolute Value, Power, Square Root — ABS, POWER, SQRT, MOD
- Conversion — VALUE, TEXT/FORMAT, INT, CURRENCY

■ Lesson 7: Relational Functions

- RELATED — Lookup from Many to One
- RELATEDTABLE — Lookup from One to Many
- When RELATED and RELATEDTABLE Are Available

■ Lesson 8: DAX Code Formatting

- Why Formatting Matters
- Official Formatting Rules
- DAXFormatter.com

■ Demonstration 1: Building a Complete Measure Library

■ Demonstration 2: RELATED and RELATEDTABLE in Calculated Columns

■ Lab 2: Introducing DAX — Core Functions

- Exercise 1: Aggregation and Iteration
- Exercise 2: Logical Functions and SWITCH

- Exercise 3: Text and Date Functions
- Exercise 4: RELATED, Conversion, and Code Formatting

■ Module Review and Takeaways

Module Overview

Module 1 established the foundational concepts: the data model, relationships, the difference between calculated columns and measures, and evaluation context at an introductory level. This module is a guided tour of the core DAX function library — the functions you will use in virtually every Power BI model you ever build. By the end of this module you will be able to write measures for aggregation, conditional logic, text manipulation, date calculations, cross-table lookups, and professional code formatting.

This module is intentionally broad: it introduces every major function category without going deep on any single topic. Depth comes in later modules — CALCULATE and filter context (Module 5), time intelligence (Module 8), advanced table functions (Module 3). For now, the goal is fluency with the building blocks. Every function shown here appears regularly in PL-300 exam questions.

EXAM

PL-300 exam questions frequently present incomplete or incorrect DAX formulas and ask you to identify the error or select the correct function. The most tested areas in this module: COUNTROWS vs COUNT, SUMX vs SUM, RELATED vs RELATEDTABLE, SWITCH vs nested IF, and the FORMAT function for date presentation.

Module Prerequisites

- Completed Module 1 (What is DAX?) — specifically the calculated column vs measure distinction.
- Power BI Desktop installed with the Contoso_Lab02_Start.pbix file open.
- Familiarity with the Contoso data model: Sales, Date, Product, Customer tables.

LESSON 1

Aggregation Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Use SUM, AVERAGE, MIN, MAX, and STDEV to aggregate numeric columns.
- Choose the correct counting function from COUNT, COUNTA, COUNTROWS, DISTINCTCOUNT.
- Explain the difference between an aggregator and an iterator, and when to use each.
- Write SUMX, AVERAGEX, MINX, MAXX, and COUNTX to aggregate expressions over tables.

SUM, AVERAGE, MIN, MAX — The Core Aggregators

Aggregation functions are the most fundamental DAX tools. They take a column as their argument and return a single scalar value. In a measure, they automatically respect the current filter context — a SUM in a matrix cell only adds rows that pass the cell's filters.

```
[Total Sales] := SUM( Sales[SalesAmount] ) -- adds all values in the column
[Avg Sale Price] := AVERAGE( Sales[Net Price] ) -- arithmetic mean
[Largest Order] := MAX( Sales[SalesAmount] ) -- maximum value
[Smallest Order] := MIN( Sales[SalesAmount] ) -- minimum value
[Sales StdDev] := STDEV.P( Sales[SalesAmount] ) -- population std deviation
-- STDEV.S for sample

-- MIN and MAX also accept TWO scalar parameters (not columns):
-- Returns the smaller/larger of two values directly
MinOfTwo := MIN( [Measure A], [Measure B] ) -- useful in conditional expressions
MaxOfTwo := MAX( 0, [Some Measure] ) -- clamp to zero minimum

-- Important: DAX aggregators IGNORE blank cells.
-- AVERAGE over 5 rows where one is blank = sum of 4 values / 4 (not / 5)
-- This is DIFFERENT from Excel, which also ignores blanks in AVERAGE
```

Counting Functions

DAX provides several counting functions. Choosing the wrong one is a very common mistake — especially confusing COUNT with COUNTROWS:

Function	Returns	Description / Example
COUNT(column)	Integer	Counts non-blank values in column. Works on any type except Boolean. Returns 0 (not blank) when all values are blank.
COUNTA(column)	Integer	Counts non-blank values in any column type including Boolean. Nearly identical to COUNT in DAX (unlike Excel where they differ).
COUNTBLANK(column)	Integer	Counts blank and empty-string values. Opposite of COUNT. Useful for data quality checks: COUNTBLANK(Sales[CustomerID]).
COUNTROWS(table)	Integer	Counts rows in a TABLE, not a column. Takes a table expression as argument. COUNTROWS(Sales) = total sales rows. Most commonly used counting function.
DISTINCTCOUNT(column)	Integer	Counts distinct non-blank values. DISTINCTCOUNT(Sales[CustomerKey]) = number of unique customers who purchased.

DISTINCTCOUNTNOBLANK(column)	Integer	Same as DISTINCTCOUNT but excludes blank from the count. Use when blank values should not count as a distinct value.
-----------------------------------	---------	--

EXAM

PL-300 distinguishes between COUNTROWS(table) and COUNT(column). COUNTROWS requires a table — COUNTROWS(Sales). COUNT requires a column — COUNT(Sales[OrderID]). Both count non-blank rows/values, but COUNTROWS is generally preferred for counting rows because it is explicit and always unambiguous.

Aggregators vs Iterators

Standard aggregators like SUM work on a single column. They cannot aggregate an *expression*. When you need to compute something row-by-row and then aggregate the results, you need an **iterator**. Every aggregator has an iterator counterpart:

Aggregator (column only)	Iterator (expression per row)
SUM(Sales[SalesAmount])	SUMX(Sales, Sales[Qty] * Sales[Price])
AVERAGE(Sales[DaysToDeliver])	AVERAGEX(Sales, INT(Sales[DelivDate]-Sales[OrdDate]))
MIN(Sales[SalesAmount])	MINX(Sales, Sales[Qty] * Sales[Price])
MAX(Sales[SalesAmount])	MAXX(Sales, Sales[Qty] * Sales[Price])
COUNT(Sales[OrderID])	COUNTX(Sales, IF(Sales[Amount]>0, 1))
Works on a single column only	Works on any expression, evaluated row-by-row
Computed once — column already exists	Computes expression per row at query time
No table scan overhead	Table scan — slight overhead vs simple aggregator

```
-- When to use each:

-- Use SUM when SalesAmount already exists as a column:
[Total Sales] := SUM( Sales[SalesAmount] )

-- Use SUMX when you need to compute before summing
-- (avoids creating an intermediate calculated column):
[Total Sales] := SUMX( Sales, Sales[Quantity] * Sales[Net Price] )

-- Real advantage of SUMX: complex expressions with no stored column
[Discounted Revenue] :=
SUMX(
Sales,
Sales[Quantity] * Sales[Net Price] * ( 1 - Sales[DiscountPct] )
)

-- Average delivery time without a calculated column:
[Avg Delivery Days] :=
AVERAGEX(
Sales,
INT( Sales[Delivery Date] - Sales[Order Date] )
)
```

TIP

SUMX and SUM over a single column are identical in performance — SUM(Sales[Amount]) is internally translated to SUMX(Sales, Sales[Amount]). Use SUM when the column already exists for cleaner code. Use SUMX when you need to compute an expression row-by-row. Never create a calculated column just to enable a SUM — use SUMX instead.

LESSON 2

Logical Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Use IF for conditional logic including the default BLANK third argument.
- Apply AND, OR, and NOT as both operators (&& || !) and functions.
- Replace deeply nested IF chains with SWITCH for readability.
- Use the SWITCH(TRUE()) pattern for multi-condition dispatch.

IF — The Foundational Conditional

IF is the most commonly used conditional function in DAX. It takes up to three arguments: a condition, the value if true, and the optional value if false. When the false argument is omitted it defaults to `BLANK()`:

```
-- Basic IF syntax:
IF( logical_test, value_if_true [, value_if_false] )

-- The third argument (false result) defaults to BLANK() if omitted:
IF( Sales[Quantity] > 1, Sales[Quantity] )
-- is identical to:
IF( Sales[Quantity] > 1, Sales[Quantity], BLANK() )

-- Practical measure example:
[High Value Orders] :=
SUMX(
    Sales,
    IF( Sales[SalesAmount] > 1000, Sales[SalesAmount], 0 )
)

-- IF in a calculated column – classifying orders:
Sales[OrderSize] =
IF(
    Sales[SalesAmount] >= 5000, "Large",
    IF(
        Sales[SalesAmount] >= 1000, "Medium",
        "Small"
    )
)

-- Nested IFs work but become hard to read beyond 3 levels -- use SWITCH instead
```

AND, OR, NOT — Logical Operators as Functions

DAX provides logical operators in two forms — as symbolic operators (&&, ||, NOT) and as functions (`AND()`, `OR()`). The operators are preferred in modern DAX for readability:

```
-- Operator form (preferred in modern DAX):
Sales[IsEuropeLarge] = Customer[Continent] = "Europe" && Sales[Quantity] >= 100
Sales[IsAnyBig] = Sales[Revenue] > 10000 || Sales[Quantity] > 500
Sales[IsNotEurope] = NOT ( Customer[Continent] = "Europe" )

-- Function form (inherited from Excel, valid but verbose):
Sales[IsEuropeLarge] = AND( Customer[Continent]="Europe", Sales[Quantity]>=100 )
```

```

Sales[IsAnyBig] = OR( Sales[Revenue]>10000, Sales[Quantity]>500 )

-- AND() and OR() functions only accept exactly TWO arguments.
-- For three or more conditions, chain them or use &&/|| operators:
[Is Premium Europe] :=
IF(
Customer[Continent] = "Europe"
&& Customer[Segment] = "Premium"
&& Sales[SalesAmount] > 500,
[Total Sales],
BLANK()
)

```

SWITCH — Replacing Nested IF Chains

SWITCH evaluates a single expression and returns different values based on which case matches. It is equivalent to a sequence of nested IFs but dramatically more readable:

```

-- SWITCH syntax:
SWITCH( expression, value1, result1, value2, result2, ..., [else_result] )

-- Nested IF version (hard to read at scale):
Product[SizeDesc] =
IF( Product[Size]="S", "Small",
IF( Product[Size]="M", "Medium",
IF( Product[Size]="L", "Large",
IF( Product[Size]="XL", "Extra Large", "Unknown" )
)
)
)

-- SWITCH version (same logic, far more readable):
Product[SizeDesc] =
SWITCH(
Product[Size],
"S", "Small",
"M", "Medium",
"L", "Large",
"XL", "Extra Large",
"Unknown" -- else clause (no matching value)
)

```

SWITCH(TRUE()) — Multi-Condition Pattern

When you need to test different conditions (not just match a single value), use SWITCH(TRUE(), ...). Passing TRUE() as the first argument means: return the result of the first condition that evaluates to TRUE:

```

-- SWITCH(TRUE()) -- tests conditions in order, returns first match
Sales[OrderBand] =
SWITCH(
TRUE(),
Sales[SalesAmount] >= 10000, "Platinum",
Sales[SalesAmount] >= 5000, "Gold",
Sales[SalesAmount] >= 1000, "Silver",
"Bronze" -- default
)

```

```
)  
  
-- Multi-condition example (replaces complex nested IF):  
Product[Segment] =  
SWITCH(  
TRUE(),  
Product[Size] = "XL" && Product[Color] = "Red", "Red XL Special",  
Product[Size] = "XL" && Product[Color] = "Blue", "Blue XL Special",  
Product[Size] = "L" && Product[Color] = "Green", "Green Large",  
"Standard"  
)  
  
-- Note: SWITCH evaluates conditions in order -- first TRUE wins.  
-- Place the most specific conditions FIRST.
```

LESSON 3

Information Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Use ISBLANK, ISERROR, ISNUMBER, ISTEXT, ISNONTEXT, and ISLOGICAL correctly.
- Explain why ISNUMBER on a text column always returns FALSE.
- Apply information functions to build robust data quality checks.
- Combine information functions with IF for safe conditional expressions.

Information Functions Reference

Information functions interrogate a value's type or state. They all return a Boolean (TRUE / FALSE) and are designed for use inside IF conditions:

Function	Returns	Description / Example
ISBLANK(value)	Boolean	TRUE if value is BLANK. Use instead of = 0 or = "" when you need to test for a genuinely missing value. ISBLANK(Sales[Discount]) ≠ Sales[Discount]=0.
ISERROR(expression)	Boolean	TRUE if expression raises any error. Useful in IF to test before computing. Note: wrapping expensive expressions in ISERROR can slow queries — prefer proactive testing.
ISNUMBER(value)	Boolean	TRUE if value is a number (Integer, Decimal, or Currency). Returns based on the COLUMN TYPE, not row content — a text column containing "123" returns FALSE because the column type is text.
ISTEXT(value)	Boolean	TRUE if value is a string. Like ISNUMBER, based on column type.
ISNONTEXT(value)	Boolean	TRUE if value is not a text string OR is BLANK. ISNONTEXT(BLANK()) = TRUE. More permissive than NOT ISTEXT.
ISLOGICAL(value)	Boolean	TRUE if value is a Boolean (TRUE/FALSE). Rarely needed in practice.

WARNING

ISNUMBER, ISTEXT, and ISLOGICAL test the COLUMN TYPE declared in the model, not the content of individual cells. A column defined as Text that happens to contain numeric strings will always return FALSE for ISNUMBER, regardless of what values are in each row. Fix the column type in Power Query — not in DAX.

```
-- Practical patterns with information functions:

-- Safe shipping cost: use stored value if available, else use default
Sales[ShippingCost] =
IF(
  ISBLANK( Sales[OverrideShipping] ),
  RELATED( Product[DefaultShipping] ), -- default from product
  Sales[OverrideShipping] -- override if explicitly set
)

-- Data quality measure: count rows with missing key
[Missing CustomerKey] := COUNTROWS( FILTER( Sales, ISBLANK( Sales[CustomerKey] ) ) )
```

```
-- Safe margin calculation:
[Safe Margin %] :=
VAR TotalSales = SUM( Sales[SalesAmount] )
RETURN
IF(
ISBLANK( TotalSales ) || TotalSales = 0,
BLANK(),
DIVIDE( SUM(Sales[GrossMargin]), TotalSales )
)
```

LESSON 4

Text Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Extract substrings using LEFT, RIGHT, and MID.
- Locate characters within strings using FIND and SEARCH.
- Transform text using UPPER, LOWER, TRIM, SUBSTITUTE, and REPLACE.
- Concatenate strings using & and CONCATENATEX for multi-row aggregation.

Extraction: LEFT, RIGHT, MID, LEN

```
-- LEFT( text, num_chars ) -- extract from the start
Customer[FirstInitial] = LEFT( Customer[FirstName], 1 ) -- "J"
Product[SKUPrefix] = LEFT( Product[SKU], 3 ) -- "BK-"

-- RIGHT( text, num_chars ) -- extract from the end
Product[SKUSuffix] = RIGHT( Product[SKU], 4 ) -- "0001"

-- MID( text, start_pos, num_chars ) -- extract from any position
-- Position is 1-based (first character = position 1)
Product[SKUCore] = MID( Product[SKU], 4, 5 ) -- chars 4 to 8

-- LEN( text ) -- length of string
Customer[NameLength] = LEN( Customer[FullName] ) -- number of characters

-- Practical: extract year from a text date like '2024-03-15'
Sales[TextYear] = LEFT( Sales[DateString], 4 ) -- "2024"
```

Searching: FIND and SEARCH

```
-- FIND( find_text, within_text [, start_pos] [, not_found] )
-- Case-SENSITIVE search. Returns position of first match (1-based).
-- Returns an error if not found (unless not_found argument is provided)
People[Comma1] = FIND( ",", People[Name], 1, BLANK() ) -- position of first comma

-- SEARCH( find_text, within_text [, start_pos] [, not_found] )
-- Case-INSENSITIVE search. Otherwise identical to FIND.
Product[HasBike] = NOT ISBLANK( SEARCH( "bike", Product[Name], 1, BLANK() ) )

-- Combining FIND with MID to split a name at a delimiter:
-- Full name stored as "Smith, John A."
People[Comma1] = FIND( ",", People[Name], 1, BLANK() )
People[LastName] = IF(
    ISNUMBER( People[Comma1] ),
    LEFT( People[Name], People[Comma1] - 1 ),
    People[Name]
)
People[FirstName] = IF(
    ISNUMBER( People[Comma1] ),
    TRIM( MID( People[Name], People[Comma1] + 1, 100 ) ),
    ""
)
```



```
)
```

Transformation: UPPER, LOWER, TRIM, SUBSTITUTE, REPLACE

```
-- Case transformation
Customer[NameUpper] = UPPER( Customer[Name] ) -- "JOHN SMITH"
Customer[NameLower] = LOWER( Customer[Name] ) -- "john smith"

-- TRIM: removes leading and trailing spaces (and extra internal spaces)
Customer[CleanName] = TRIM( Customer[Name] ) -- removes accidental spaces

-- SUBSTITUTE( text, old_text, new_text [, instance_num] )
-- Replaces ALL occurrences by default
Product[CleanSKU] = SUBSTITUTE( Product[SKU], "-", "" ) -- "BK0001" from "BK-0001"
Product[FirstDash] = SUBSTITUTE( Product[SKU], "-", "", 1 ) -- only first occurrence

-- REPLACE( old_text, start_pos, num_chars, new_text )
-- Replaces by POSITION, not by content
Product[MaskedSKU] = REPLACE( Product[SKU], 1, 2, "XX" ) -- "XX-0001" from "BK-0001"

-- CONCATENATE: joins exactly two strings (use & for more)
Customer[FullName] = CONCATENATE( Customer[FirstName], " " & Customer[LastName] )
-- Same result using & operator (preferred for readability):
Customer[FullName] = Customer[FirstName] & " " & Customer[LastName]
```

CONCATENATEX — Aggregating Text Across Rows

CONCATENATEX is the text-aggregating iterator. It iterates over a table and concatenates a text expression from each row, separated by a delimiter. Unlike CONCATENATE, it works across multiple rows:

```
-- CONCATENATEX( table, expression, delimiter [, order_by, order] )

-- List all colors available for a product category:
[Color List] :=
CONCATENATEX(
VALUES( Product[Color] ), -- distinct colors in current context
Product[Color], -- expression to concatenate per row
", ", -- delimiter
Product[Color], -- sort by color name
ASC -- ascending order
)
-- Result in a Category matrix: "Black, Blue, Red, Silver, White"

-- Note: CONCATENATEX is an iterator -- use it only where needed.
-- Long strings in large tables can slow visuals significantly.
```

LESSON 5

Date and Time Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Construct date and time values using DATE, TIME, TODAY, and NOW.
- Extract date parts using YEAR, MONTH, DAY, HOUR, MINUTE, SECOND.
- Perform date arithmetic using EDATE, EOMONTH, and integer addition.
- Format dates for display using FORMAT.
- Distinguish basic date functions from time intelligence functions (Module 8).

Date Construction and Current Time

```
-- DATE( year, month, day ) -- construct a specific date
'Date'[Date] = DATE( 2024, 3, 15 ) -- March 15, 2024
'Date'[StartOfYear] = DATE( YEAR([Date]), 1, 1 ) -- Jan 1 of same year

-- TODAY() -- current date (no time component)
-- NOW() -- current datetime (with time)
-- Both are volatile -- they recalculate on every query
[Days Since Last Sale] :=
INT( TODAY() - MAX( Sales[SaleDate] ) )

-- TIME( hour, minute, second ) -- construct a time value
Sales[ShiftStart] = TIME( 9, 0, 0 ) -- 09:00:00

-- DateTime arithmetic: internally a float
-- Integer part = days since Dec 30 1899
-- Decimal part = fraction of day
-- Adding 1 to a date increments by exactly one day:
Sales[DueDate] = Sales[OrderDate] + 30 -- 30 days later

-- TIMEVALUE and DATEVALUE convert strings to dates/times:
Sales[ParsedDate] = DATEVALUE( "2024-03-15" ) -- parse ISO date string
```

Date Part Extraction

```
-- Extraction functions -- all return integers
'Date'[Year] = YEAR( 'Date'[Date] ) -- 2024
'Date'[MonthNum] = MONTH( 'Date'[Date] ) -- 1 to 12
'Date'[DayOfMonth] = DAY( 'Date'[Date] ) -- 1 to 31
'Date'[Hour] = HOUR( 'Date'[DateTime] ) -- 0 to 23
'Date'[Minute] = MINUTE( 'Date'[DateTime] ) -- 0 to 59
'Date'[Second] = SECOND( 'Date'[DateTime] ) -- 0 to 59

-- Weekday and week number:
'Date'[DayOfWeek] = WEEKDAY( 'Date'[Date], 2 ) -- 2 = Mon=1, Sun=7 (ISO)
-- 1 = Sun=1, Sat=7 (US)
'Date'[WeekNum] = WEEKNUM( 'Date'[Date], 2 ) -- ISO week number

-- End of month:
'Date'[EOM] = EOMONTH( 'Date'[Date], 0 ) -- last day of same month
```

```
'Date'[EOM_Plus3] = EOMONTH( 'Date'[Date], 3 ) -- last day 3 months ahead

-- Add months (preserves day of month where possible):
Sales[InvoiceDue] = EDATE( Sales[OrderDate], 1 ) -- one month later
```

FORMAT for Date Presentation

```
-- FORMAT( value, format_string ) -- returns a TEXT string
-- Use for display only -- the result is text, not a date

'Date'[MonthYear] = FORMAT( 'Date'[Date], "mmm yyyy" ) -- "Mar 2024"
'Date'[FullDate] = FORMAT( 'Date'[Date], "mmm d, yyyy" ) -- "March 15, 2024"
'Date'[ISODate] = FORMAT( 'Date'[Date], "yyyy-mm-dd" ) -- "2024-03-15"
'Date'[YearMonth] = FORMAT( 'Date'[Date], "yyyy mmm" ) -- "2024 Mar" (sorts correctly)

-- FORMAT with numbers:
[Formatted Revenue] := FORMAT( [Total Sales], "$#,##0.00" ) -- "$1,234,567.89"
[Formatted Pct] := FORMAT( [Gross Margin %], "0.0%" ) -- "34.5%"

-- WARNING: FORMAT returns TEXT. You cannot sort by it correctly if month names
-- are in it. Always keep a separate integer sort column (MonthNumber = MONTH(Date))
-- and set 'Sort by Column' on the MonthName column to sort by MonthNumber.
```

NOTE

The date functions in this lesson (YEAR, MONTH, DAY, EDATE, EOMONTH) are basic date manipulation functions. They are different from time intelligence functions such as SAMEPERIODLASTYEAR, TOTALYTD, and DATEADD, which compare values across time periods. Time intelligence functions require a properly marked Date table and are covered in Module 8.

LESSON 6

Mathematical and Conversion Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Apply rounding functions: ROUND, ROUNDUP, ROUNDDOWN, FLOOR, CEILING, INT, TRUNC.
- Use ABS, POWER, SQRT, MOD, and SIGN for numeric calculations.
- Convert between types using VALUE, INT, CURRENCY, and FORMAT.

Rounding Functions

DAX provides eight rounding functions. The differences are subtle but matter for financial and statistical work:

Function	Returns	Description / Example
ROUND(num, decimals)	Number	Round to nearest. ROUND(2.345, 2) = 2.35. Banker's rounding NOT used.
ROUNDUP(num, decimals)	Number	Always rounds away from zero. ROUNDUP(2.341, 2) = 2.35.
ROUNDDOWN(num, decimals)	Number	Always rounds toward zero (truncates). ROUNDDOWN(2.349, 2) = 2.34.
INT(num)	Integer	Round DOWN to nearest integer. INT(2.9) = 2. INT(-2.1) = -3 (toward -infinity).
TRUNC(num [, decimals])	Number	Truncate toward zero. TRUNC(2.9) = 2. TRUNC(-2.9) = -2. Different from INT for negatives.
FLOOR(num, significance)	Number	Round DOWN to nearest multiple of significance. FLOOR(2.75, 0.5) = 2.5.
CEILING(num, significance)	Number	Round UP to nearest multiple of significance. CEILING(2.25, 0.5) = 2.5.
MROUND(num, multiple)	Number	Round to nearest multiple. MROUND(2.75, 0.5) = 3.0. Rounds to even on tie.

Common Math Functions

```
-- ABS: absolute value (strip the sign)
Sales[AbsVariance] = ABS( Sales[Actual] - Sales[Budget] )

-- POWER(number, exponent) -- same as number^exponent in Excel
Sales[Cubed] = POWER( Sales[SideLength], 3 )

-- SQRT: square root (errors on negative numbers -- use IF to protect)
Stats[StdDev] = SQRT( Stats[Variance] )
Stats[SafeStdDev] = IF( Stats[Variance] >= 0, SQRT(Stats[Variance]), BLANK() )

-- MOD(number, divisor) -- remainder after integer division
'Date'[IsEvenDay] = MOD( 'Date'[DayOfMonth], 2 ) = 0 -- TRUE on even days
'Date'[Quarter] = CEILING( 'Date'[MonthNum] / 3, 1 ) -- Q1=1, Q2=2, Q3=3, Q4=4

-- SIGN(number): returns -1, 0, or 1
Sales[TrendDir] = SIGN( Sales[Revenue] - Sales[PriorRevenue] )
```

Conversion Functions

```
-- VALUE( text ) -- convert string to number
Sales[NumericCode] = VALUE( Sales[CodeString] ) -- "42" -> 42
-- Always protect with IFERROR:
Sales[SafeNum] = IFERROR( VALUE(Sales[CodeString]), BLANK() )

-- INT( value ) -- convert to integer (rounds down)
Sales[DaysToDeliver] = INT( Sales[DeliveryDate] - Sales[OrderDate] )

-- CURRENCY( value ) -- convert to Currency type (4 decimal places)
Sales[PriceFixed] = CURRENCY( Sales[RawPrice] )

-- FORMAT( value, format_string ) -- convert to TEXT for display
-- Covered in Lesson 5 (Date Functions) -- works on numbers too
[Revenue Label] := FORMAT( [Total Sales], "$#,##0" ) -- "$1,234,568"

-- DATE( year, month, day ) -- convert parts to DateTime
Sales[SaleDate] = DATE( YEAR([OrderDateTime]), MONTH([OrderDateTime]), DAY([OrderDateTime]) )
-- Equivalent and simpler:
Sales[SaleDate] = INT( Sales[OrderDateTime] ) -- strips time component
```

LESSON 7

Relational Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Use RELATED to bring a value from a related table into a calculated column.
- Use RELATEDTABLE to access all related rows from the many-side table.
- Explain which table side each function starts from.
- Combine RELATEDTABLE with iterators for complex cross-table calculations.

RELATED — Many to One Lookup

RELATED works in a calculated column on the **many-side** table and looks up a value from the **one-side** (dimension) table. It follows the relationship from the many-side to the one-side automatically, even through a chain of relationships:

```
-- RELATED( column ) -- used in a calculated column on the many-side
-- The column argument must be in a RELATED table (one-side)

-- Simple one-hop: Sales -> Product
Sales[ProductCategory] = RELATED( 'Product Category'[Category] )
-- DAX follows: Sales -> Product -> Product Subcategory -> Product Category
-- through the full chain automatically.

-- Practical example: adjusted cost based on category
Sales[AdjustedCost] =
IF(
    RELATED( 'Product Category'[Category] ) = "Cell Phones",
    Sales[Unit Cost] * 0.85, -- 15% reduction for cell phones
    Sales[Unit Cost]
)

-- RELATED in a measure (inside an iterator):
[Premium Revenue] :=
SUMX(
    Sales,
    IF(
        RELATED( Customer[Segment] ) = "Premium",
        Sales[SalesAmount],
        0
    )
)

-- WARNING: RELATED is only available:
-- 1. In a calculated column on the many-side table
-- 2. Inside an iterator that establishes a row context over the many-side table
```

RELATEDTABLE — One to Many Lookup

RELATEDTABLE is the inverse: it works in a calculated column on the **one-side** table and returns all matching rows from the **many-side** table. The result is a table, so it is typically passed directly to an iterator or aggregator:

```
-- RELATEDTABLE( table ) -- used in a column on the ONE-side
-- Returns a table of all rows in the many-side that match the current row

-- Count products per category:
'Product Category'[ProductCount] = COUNTROWS( RELATEDTABLE( Product ) )

-- Sum sales per category (as a calculated column in Product Category):
'Product Category'[CategorySales] =
SUMX(
    RELATEDTABLE( Sales ), -- all Sales rows for this category
    Sales[Quantity] * Sales[Net Price]
)

-- Count unique customers who bought products in each category:
'Product Category'[UniqueCustomers] =
DISTINCTCOUNT( RELATEDTABLE( Sales )[CustomerKey] )
-- Note: RELATEDTABLE returns a table -- you can reference its columns

-- IMPORTANT: Results from RELATEDTABLE calculated columns are FIXED at refresh.
-- They do not respond to user slicer selections.
-- If you need dynamic results, use a MEASURE with CALCULATE instead.
```

EXAM

A common PL-300 scenario: you need a calculated column in the Customer table showing the number of orders each customer placed. Answer: `Customer[OrderCount] = COUNTROWS(RELATEDTABLE(Sales))`. RELATED cannot be used here (Customer is the one-side). RELATEDTABLE returns the Sales rows matching each customer.

LESSON 8

DAX Code Formatting

Lesson Objectives

After completing this lesson, you will be able to:

- Apply the official DAX formatting rules to any formula.
- Explain why formatting is essential for debugging and team collaboration.
- Use DAXFormatter.com to automatically format complex expressions.
- Format multi-line measures with correct indentation and spacing.

Why Formatting Matters

DAX is a functional language — every expression, no matter how complex, is a single nested function call. Complex measures can span 30 or more lines. Without consistent formatting, a missing parenthesis becomes nearly impossible to find, code reviews are painful, and maintenance is slow. The same formula, unformatted vs formatted, represents the difference between a 10-minute debug and a 2-second scan.

```
-- UNFORMATTED (real example from production code):
IF(CALCULATE(NOT ISEMPY(Balances),ALLEXCEPT(Balances,BalanceDate)),SUMX(ALL(Balances[Account]
),CALCULATE(SUM(Balances[Balance]),LASTNONBLANK(DATESBETWEEN(BalanceDate[Date],BLANK(),MAX(Bal
anceDate[Date])),CALCULATE(COUNTROWS(Balances))))),BLANK())

-- FORMATTED (same formula -- suddenly readable):
IF(
  CALCULATE(
    NOT ISEMPY( Balances ),
    ALLEXCEPT( Balances, BalanceDate )
  ),
  SUMX(
    ALL( Balances[Account] ),
    CALCULATE(
      SUM( Balances[Balance] ),
      LASTNONBLANK(
        DATESBETWEEN( BalanceDate[Date], BLANK(), MAX(BalanceDate[Date]) ),
        CALCULATE( COUNTROWS( Balances ) )
      )
    )
  ),
  BLANK()
)
```

The Official Formatting Rules

- **Function names in UPPERCASE.** SUM, CALCULATE, IF — always uppercase. Column and measure names use their original casing.
- **One argument per line** for functions with multiple arguments. Single-argument functions may stay on one line: SUM(Sales[Amount]).
- **Closing parenthesis on its own line**, aligned with the opening function name.
- **Indent by 4 spaces** for each level of nesting. Consistent indentation shows the call structure instantly.
- **Column references:** always Tab leName[Co lumnName] with no space before [.

- **Measure references:** always `[MeasureName]` with no table prefix.
- **Space between function name and opening parenthesis:** `SUM ( Sales[Amount] )` — some developers omit this; be consistent.
- **Operators on the right** when wrapping long conditions: `Sales[Amount] > 0` on one line, not split across lines.

TIP

Use [DAXFormatter.com](https://daxformatter.com) (free, browser-based) to auto-format any DAX expression. Paste your unformatted code, click Format, and copy the result back. In Power BI Desktop, press Shift+Enter to add a new line in the formula bar, and Ctrl+scroll to increase font size for easier editing.

DEMONSTRATION 1

Building a Complete Measure Library

Scenario: The instructor builds a complete set of reusable measures for the Contoso model, demonstrating aggregation, logic, text, and date functions in a structured measure table.

Demonstration Steps

1	In Power BI Desktop, click Enter data on the Home tab. Create an empty table named _Measures with a single text column. Delete the column after creation. This empty table will hold all measures as a best-practice pattern — keeping measures separate from data tables.
2	Select the _Measures table. Click New measure . Enter [Total Sales] := SUM(Sales[SalesAmount]) . Add [Total Cost] , [Gross Margin] , and [Gross Margin %] := DIVIDE([Gross Margin],[Total Sales]) in sequence.
3	Create an order count measure: [Order Count] := COUNTROWS(Sales) . Create [Avg Order Value] := DIVIDE([Total Sales], [Order Count]) .
4	Create a text measure showing the top category: [Sales Summary] := "Total: " & FORMAT([Total Sales],"\$#,##0") & " across " & [Order Count] & " orders" . Add this to a card visual to show dynamic text.
5	Create a date-based measure: [Days Since Latest Sale] := INT(TODAY() - MAX(Sales[SaleDate])) . Show it on a card. Explain it recalculates dynamically.
6	Add all measures to a Matrix visual (rows = Product Category). Show that all measures respond correctly to the category filter. Format the matrix: Currency for revenue, Percentage for Gross Margin %, whole number for Order Count.

RESULT

A **_Measures** table holds six organised measures. All measures respond correctly to the Product Category row context in the matrix. The Sales Summary text measure shows a dynamic string that updates with every filter.

DEMONSTRATION 2

RELATED and RELATEDTABLE in Calculated Columns

Scenario: The instructor demonstrates **RELATED** to bring product category into the Sales table, and **RELATEDTABLE** to count orders per customer, then discusses why these values are static and when measures should be used instead.

Demonstration Steps

1	Select the Sales table. Click New column . Enter: Sales[ProductCategory] = RELATED('Product Category'[Category]) . Switch to Data view and show the column — each Sales row now displays the category of the product sold, fetched through the two-hop relationship chain.
2	Create a second column: Sales[AdjCost] = IF(RELATED('Product Category'[Category]) = "Bikes", Sales[Unit Cost] * 0.9, Sales[Unit Cost]) . Show that bike rows have a 10% lower cost.
3	Select the Customer table. Click New column . Enter: Customer[TotalOrders] = COUNTROWS(RELATEDTABLE(Sales)) . Show the column — each customer now displays their total order count.
4	Create another column: Customer[TotalRevenue] = SUMX(RELATEDTABLE(Sales), Sales[Quantity]*Sales[Net Price]) . Show the per-customer total revenue as a static calculated column.
5	Add a slicer for Year. Select 2023. Show that Customer[TotalOrders] and Customer[TotalRevenue] do not change — they are calculated columns and are fixed at refresh time. This is why dynamic aggregations require measures, not calculated columns.
6	Create a measure to contrast: [Orders in Context] := COUNTROWS(Sales) . Add it to the Customer matrix visual. Show that it changes with the Year slicer while the calculated column stays fixed.

RESULT

RELATED successfully brings Product Category into Sales. **RELATEDTABLE** counts orders and sums revenue per customer. The demonstration confirms that calculated columns are static — they do not respond to user filter selections. Dynamic aggregations must be measures.

LAB 2

Introducing DAX — Core Functions

Lab Scenario

The Contoso BI team needs a set of standard measures and calculated columns to support quarterly reporting. You will build the complete measure library from scratch, add classification columns using SWITCH and IF, create date-based calculated columns, build cross-table lookups with RELATED, and format all code to the official DAX standard.

NOTE

Open Contoso_Lab02_Start.pbix. Save your completed work as Contoso_Lab02_Complete.pbix. All exercises build on each other — complete them in order.

Exercise 1: Aggregation and Iteration

Estimated time: 20 minutes

Task 1: Create an Empty Measures Table

1. On the **Home** tab in Power BI Desktop, click **Enter data**.
2. Name the table **_Measures**. Leave Column1 empty. Click **Load**.
3. In the **Data** pane, right-click the Column1 in **_Measures** and click **Delete**. Click **OK** to confirm. The table is now empty and will hold only measures.

Results: An empty _Measures table exists in the Data pane with no columns. This is the destination for all measures you create in this lab.

Task 2: Create Core Aggregation Measures

1. Click the **_Measures** table in the Data pane to select it. On the **Table tools** tab, click **New measure**.
2. Enter the following measure and press **Enter**:

```
[Total Sales] := SUM( Sales[SalesAmount] )
```

3. Create three more measures one at a time, pressing Enter after each:

```
a. [Total Cost] := SUM( Sales[TotalCost] )
```

```
b. [Gross Margin] := [Total Sales] - [Total Cost]
```

```
c. [Gross Margin %] := DIVIDE( [Gross Margin], [Total Sales] )
```

4. Create counting measures:

```
a. [Order Count] := COUNTROWS( Sales )
```

```
b. [Unique Customers] := DISTINCTCOUNT( Sales[CustomerKey] )
```

5. Create an iterator measure that avoids an intermediate calculated column:

```
[Discounted Revenue] :=  
SUMX(  
    Sales,  
    Sales[Quantity] * Sales[Net Price] * ( 1 - Sales[DiscountPct] )  
)
```

6. Add a **Matrix** visual to the report canvas. Place **Product Category[Category]** on Rows. Add all six measures to Values. Verify the grand total row makes sense for each.

Results: Six measures in `_Measures`. Matrix shows correct values per category with correct grand totals. `[Gross Margin %]` grand total is the overall percentage, not a sum of rows. `[Discounted Revenue]` calculates without any intermediate calculated column.

Exercise 2: Logical Functions and SWITCH

Estimated time: 20 minutes

Task 1: Classify Orders with SWITCH

1. Select the **Sales** table. Click **New column**. Enter the following calculated column:

```
Sales[OrderBand] =
    SWITCH(
        TRUE(),
        Sales[SalesAmount] >= 5000, "Large",
        Sales[SalesAmount] >= 1000, "Medium",
        Sales[SalesAmount] >= 100, "Small",
        "Micro"
    )
```

2. Switch to **Data view**. Verify that each Sales row has an OrderBand value. Sort by SalesAmount descending and confirm Large rows have the highest amounts.
3. Return to **Report view**. Add a **Bar chart** visual. Place **Sales[OrderBand]** on the X axis and **[Order Count]** on the Y axis. Observe the distribution across bands.

Results: `Sales[OrderBand]` column correctly classifies each row. The bar chart shows order count distribution across Large, Medium, Small, and Micro bands.

Task 2: Conditional Measure with IF

1. In the `_Measures` table, create a new measure:

```
[Large Order Revenue] :=
    SUMX(
        Sales,
        IF( Sales[SalesAmount] >= 5000, Sales[SalesAmount], 0 )
    )
```

2. Create a companion measure for percentage of revenue from large orders:

```
[Large Order %] :=
    DIVIDE( [Large Order Revenue], [Total Sales] )
```

3. Add `[Large Order Revenue]` and `[Large Order %]` to the Category matrix. Format `[Large Order %]` as a percentage. Verify that categories with more expensive products show a higher Large Order %.

Results: `[Large Order Revenue]` sums only orders over \$5,000. `[Large Order %]` shows the correct proportion for each category and the grand total.

Exercise 3: Text and Date Functions

Estimated time: 25 minutes

Task 1: Build Text Columns in the Customer Table

1. Select the **Customer** table. Click **New column**. Build a full name column:

```
Customer[FullName] =
    Customer[FirstName] & " " & Customer[LastName]
```

2. Create a display name with last name first:

```
Customer[DisplayName] =
    Customer[LastName] & ", " & Customer[FirstName]
```

3. Create a masked email showing only the domain:

```
Customer[EmailDomain] =
    IF(
        ISNUMBER( FIND("@", Customer[EmailAddress], 1, BLANK()) ),
        UPPER( MID( Customer[EmailAddress],
            FIND("@", Customer[EmailAddress]) + 1,
            100 ) ),
        "UNKNOWN"
    )
```

4. Switch to **Data view**. Verify **FullName**, **DisplayName**, and **EmailDomain** are populated correctly for several rows.

Results: Customer table has three new text columns. FullName concatenates first and last. DisplayName reverses the order. EmailDomain extracts and uppercases the domain part of the email address.

Task 2: Build Date Columns in the Date Table

1. Select the **'Date'** table. Click **New column**. Add a month-year display label:

```
'Date'[MonthLabel] = FORMAT( 'Date'[Date], "mmm yyyy" )
```

2. Add a sort key for the MonthLabel column (so charts sort chronologically, not alphabetically):

```
'Date'[MonthSortKey] = YEAR( 'Date'[Date] ) * 100 + MONTH( 'Date'[Date] )
```

3. In Model view, select the **MonthLabel** column in the Date table. In the **Column tools** tab, click **Sort by column** and select **MonthSortKey**.

4. Add an **IsWeekend** column:

```
'Date'[IsWeekend] = WEEKDAY( 'Date'[Date], 2 ) >= 6
```

5. Add a line chart to the report. Use **'Date'[MonthLabel]** on the X axis and **[Total Sales]** on Y. Verify months sort in chronological order.

Results: Date table has MonthLabel (display), MonthSortKey (sort order), and IsWeekend columns. The line chart's X axis shows months in correct chronological order, not alphabetical.

Exercise 4: RELATED, Conversion, and Code Formatting

Estimated time: 25 minutes

Task 1: RELATED Lookups in Sales

1. Select the **Sales** table. Click **New column**. Bring the product category into the Sales table:

```
Sales[Category] = RELATED( 'Product Category'[Category] )
```

2. Create a column that applies a category-specific discount factor:

```
Sales[CategoryRevenue] =
    SWITCH(
        RELATED( 'Product Category'[Category] ),
        "Bikes", Sales[SalesAmount] * 0.95,
        "Accessories", Sales[SalesAmount] * 0.90,
        "Clothing", Sales[SalesAmount] * 0.92,
        Sales[SalesAmount]
    )
```

3. Switch to Data view. Sort by Category. Verify Bikes rows show 95% of SalesAmount in CategoryRevenue.

Results: Sales[Category] pulls the category through the relationship chain. Sales[CategoryRevenue] applies category-specific factors using SWITCH + RELATED.

Task 2: RELATEDTABLE in the Customer Table

1. Select the **Customer** table. Click **New column**:

```
Customer[LifetimeOrders] = COUNTROWS( RELATEDTABLE( Sales ) )
```

2. Create a customer value tier column using SWITCH:

```
Customer[CustomerTier] =  
    VAR LTO = Customer[LifetimeOrders]  
    RETURN  
    SWITCH(  
        TRUE(),  
        LTO >= 50, "Platinum",  
        LTO >= 20, "Gold",  
        LTO >= 5, "Silver",  
        "Bronze"  
    )
```

3. Add a **Pie chart** to the report. Use **Customer[CustomerTier]** as the legend and **[Unique Customers]** as values. Observe the tier distribution.
4. Add a Year slicer. Select different years. Observe that the CustomerTier slices **do not change** — confirm this is expected (calculated columns are static) — but the [Unique Customers] measure does change.
5. **Format all measures** created in this lab using DAXFormatter.com. Open a browser and go to www.daxformatter.com. Copy the **[Discounted Revenue]** measure formula, paste it into the site, click Format, and verify it matches the indented format shown in Lesson 8.
6. On the **File** menu, click **Save As**. Save the file as **Contoso_Lab02_Complete.pbix**.

Results: Customer[LifetimeOrders] and Customer[CustomerTier] are populated. The pie chart shows tier distribution. Year slicer confirms columns are static. All measures are formatted according to official DAX rules. File saved.

Module Review and Takeaways

Review Questions

What is the difference between SUM and SUMX, and when should you use each?

Answer: SUM(column) aggregates all values in a single column — it is simple and fast. SUMX(table, expression) iterates over a table row-by-row, evaluates an expression per row, then sums the results. Use SUM when the column already exists. Use SUMX when you need to compute a value (like Quantity * Price) before summing, avoiding the need for an intermediate calculated column. Internally, SUM(Sales[Amount]) is translated to SUMX(Sales, Sales[Amount]) — performance is identical for single-column cases.

A report needs to count the distinct number of customers who made a purchase. Which function should you use?

Answer: DISTINCTCOUNT(Sales[CustomerKey]) — this counts the number of unique values in the CustomerKey column, excluding blank. It automatically respects the filter context, so in a year matrix it counts distinct customers per year. COUNTROWS(Sales) would count total sales rows — very different. COUNT(Sales[CustomerKey]) would count non-blank customer keys but not distinct ones.

A calculated column in the Customer table should count how many orders each customer placed. Which function do you use and why?

Answer: RELATEDTABLE — because Customer is on the one-side of the relationship. RELATEDTABLE(Sales) returns all Sales rows related to the current customer row. The formula is: Customer[OrderCount] = COUNTROWS(RELATEDTABLE(Sales)). RELATED cannot be used here because RELATED traverses from many to one; RELATEDTABLE traverses from one to many.

What does SWITCH(TRUE(), ...) do and why is it useful?

Answer: SWITCH normally matches a single expression against a list of values. By passing TRUE() as the first argument, each 'value' in the list becomes a condition (a Boolean expression). SWITCH returns the result corresponding to the first condition that evaluates to TRUE. This pattern replaces deeply nested IF chains with a flat, readable list of conditions and results. Conditions are evaluated in order — place the most specific conditions first.

What is the difference between FIND and SEARCH?

Answer: FIND is case-sensitive — FIND("bike", [Name]) will not match 'Bike'. SEARCH is case-insensitive — SEARCH("bike", [Name]) matches 'bike', 'Bike', 'BIKE'. Both return the 1-based position of the first match, or an error if not found. Both accept a not_found argument (the fourth parameter) to return a value instead of raising an error — always use BLANK() or 0 as not_found in production.

Why does FORMAT return text and why is this a problem for sorting?

Answer: FORMAT always returns a string — it is a conversion function that produces a human-readable representation. When you use FORMAT(Date, 'mmm yyyy') to create a MonthYear column, the result is alphabetically sorted by default: Apr 2024 comes before Mar 2024 alphabetically. The fix is to add a separate integer MonthSortKey = YEAR([Date])*100 + MONTH([Date]) column and use Sort by Column to make MonthYear sort by MonthSortKey.

What is the RELATED function and what constraint applies to its use?

Answer: RELATED(column) retrieves the value of a column from a related table, traversing the relationship from the many-side to the one-side. It is only available in a row context — either in a calculated column on the

many-side table, or inside an iterator (like SUMX) that scans the many-side table. It cannot be used directly in a measure body without an enclosing iterator, because measures do not have a row context.

You want to classify each customer into Bronze/Silver/Gold/Platinum based on their lifetime order count. Should you use a calculated column or a measure, and why?

Answer: A calculated column — `Customer[CustomerTier] = SWITCH(TRUE(), ...)`. Customer tier is a property of the customer that does not change based on user filters. It is computed once at refresh from the full lifetime order history. A measure would be appropriate if the tier needed to be dynamic — for example, if the tier should reflect orders only in the selected time period. Since tier is a fixed attribute, a calculated column is correct here.

Best Practices

- Store all measures in a dedicated empty table named `_Measures` (or similar). This prevents measures from being scattered across data tables and makes the model easier to navigate and audit.
- Always use `SUMX` over an intermediate calculated column when computing a row-level expression before aggregating. `SUMX` has no storage cost; calculated columns consume RAM permanently.
- Use `SWITCH` instead of nested `IF` for any conditional logic with three or more cases. `SWITCH` is more readable, easier to maintain, and performs identically to nested `IFs`.
- Always provide a `not_found` argument to `FIND` and `SEARCH`. Without it, a single row without the search string raises an error that propagates to the entire column. Use `BLANK()` as the fourth argument in production code.
- Never sort text columns that contain formatted month names (Jan, Feb, Mar) alphabetically. Always create a companion integer sort key and use `Sort by Column`.
- Use `RELATED` inside `SUMX` to access dimension attributes row-by-row without creating an intermediate calculated column. This is a very common and efficient pattern.
- Format every measure to the official DAX standard. Use [DAXFormatter.com](https://daxformatter.com) for complex expressions. Unformatted DAX is the number one source of bugs in production models.
- Prefer `&&` and `||` over `AND()` and `OR()` in modern DAX — they are more concise and readable. Reserve `AND()` and `OR()` for compatibility scenarios or when matching existing code conventions.
- When building date display columns (MonthYear, QuarterLabel), always create a separate integer sort key and configure `Sort by Column`. Never rely on alphabetical sorting for dates.
- Validate `ISNUMBER` and `ISTEXT` against column type, not cell content. Fix type mismatches in Power Query (change column type to Whole Number, Decimal, etc.) rather than trying to work around them in DAX.

Common Issues and Solutions

COUNTROWS returns total rows, not matching rows

Cause: `COUNTROWS(Sales)` always returns the total row count of the Sales table in the current filter context. If no filter is active, it returns all rows. Developers sometimes use it where they want `DISTINCTCOUNT` or a filtered count.

Solution: Use `DISTINCTCOUNT(Sales[CustomerKey])` to count unique customers. Use `CALCULATE(COUNTROWS(Sales), )` to count rows meeting a condition. Use `COUNTX(FILTER(Sales, ), Sales[OrderID])` for conditional row counts.

RELATED returns an error in a measure

Cause: `RELATED` requires a row context. A measure does not have a row context — it only has a filter context. Writing `RELATED(Product[Category])` directly in a measure body raises an error.

Solution: Wrap the measure logic in an iterator: `SUMX(Sales, IF(RELATED(Product[Category])="Bikes", Sales[Amount], 0))`. Inside `SUMX`, each row of Sales provides the row context `RELATED` needs.

FORMAT sorts months alphabetically instead of chronologically

Cause: `FORMAT(Date, "mmm yyyy")` returns "Apr 2024", "Aug 2024", "Dec 2024" — which sorts alphabetically: Apr, Aug, Dec, Feb...

Solution: Add `Date[MonthSortKey] = YEAR([Date])*100 + MONTH([Date])`. In Model view, select the MonthLabel column, click `Sort by Column`, select MonthSortKey. Now April 2024 (202404) correctly sorts

before August 2024 (202408).

SWITCH does not match the expected case

Cause: SWITCH comparison is strict type equality. SWITCH(Product[Size], "S", "Small") will not match if Product[Size] contains a space (" S") or is mixed case ("s").

Solution: Apply TRIM(Product[Size]) and UPPER(Product[Size]) before SWITCH:
SWITCH(UPPER(TRIM(Product[Size])), "S", "Small", ...). Or use SWITCH(TRUE(), Product[Size]="S" || Product[Size]=" S", "Small", ...).

AVERAGEX returns wrong result after removing the intermediate column

Cause: The developer replaced Sales[DaysToDeliver] calculated column with AVERAGEX(Sales, INT(Sales[DelivDate] - Sales[OrdDate])) but the result differs. Likely cause: the original column was stored with specific rounding; the expression uses INT which truncates rather than rounds.

Solution: Verify the original column formula matches the AVERAGEX expression exactly. If the original used ROUND(Sales[DelDate]-Sales[OrdDate], 0), use AVERAGEX(Sales, ROUND(Sales[DelDate]-Sales[OrdDate], 0)).

Module 2 — Core Functions Quick Reference

Function / Category	Group	Key Notes
SUM / AVERAGE / MIN / MAX	Aggregation	Aggregate a column. Ignore blanks. Auto-respects filter context.
COUNTROWS(table)	Count	Count rows in a table (not column). Most common counting function.
DISTINCTCOUNT(column)	Count	Count distinct non-blank values in column.
COUNTBLANK(column)	Count	Count blank and empty-string cells — data quality checks.
SUMX / AVERAGEX / MINX / MAXX	Iterator	Iterate table, compute expression per row, aggregate results.
IF(cond, true [, false])	Logic	Conditional. False defaults to BLANK() if omitted.
SWITCH(expr, val, res [,...])	Logic	Multi-case conditional. Cleaner than nested IFs.
SWITCH(TRUE(), cond, res [,...])	Logic	Multi-condition dispatch — first TRUE condition wins.
ISBLANK / ISERROR / ISNUMBER	Info	Test value state or type. All return Boolean.
LEFT / RIGHT / MID	Text	Extract substrings by position.
FIND(find, in [,start] [,notfound])	Text	Case-sensitive position search. Return notfound if missing.
SEARCH(find, in [,start] [,notfound])	Text	Case-insensitive position search.
UPPER / LOWER / TRIM	Text	Case transform and whitespace removal.
SUBSTITUTE(text, old, new [,n])	Text	Replace text content. n = which occurrence (default all).
CONCATENATEX(table, expr, delim)	Text	Iterator — join text values from multiple rows.
DATE / TIME / TODAY / NOW	Date	Construct or retrieve date/time values.
YEAR / MONTH / DAY / WEEKDAY	Date	Extract integer date parts.
EOMONTH / EDATE	Date	Navigate to end of month or add months.
FORMAT(value, format_string)	Display	Convert to text for display. Cannot be used for calculation.
ROUND / ROUNDUP / ROUNDDOWN	Math	Round to decimal places.
FLOOR / CEILING	Math	Round to nearest multiple of significance.
VALUE / INT / CURRENCY	Convert	Convert between text, integer, and currency types.
RELATED(column)	Relational	Many-to-one lookup. Requires row context.
RELATEDTABLE(table)	Relational	One-to-many — returns all related rows as a table.

Module Summary

- DAX aggregation functions (SUM, AVERAGE, MIN, MAX) work on single columns. Iterator counterparts (SUMX, AVERAGEX, MINX, MAXX) work on expressions evaluated row-by-row. Use iterators to avoid intermediate calculated columns. COUNTROWS(table) counts rows; DISTINCTCOUNT(column) counts unique values.
- IF is the foundation of conditional logic. SWITCH replaces nested IFs cleanly. SWITCH(TRUE(), ...) tests multiple different conditions in priority order. AND/OR can be written as && and || operators — preferred in modern DAX.
- Information functions (ISBLANK, ISERROR, ISNUMBER, ISTEXT) test value state and type. ISNUMBER tests the column type declared in the model, not the cell content — fix type mismatches in Power Query, not in DAX.
- Text functions cover extraction (LEFT, RIGHT, MID), search (FIND case-sensitive, SEARCH case-insensitive), transformation (UPPER, LOWER, TRIM, SUBSTITUTE), and multi-row aggregation (CONCATENATEX). Always provide a not_found fallback to FIND and SEARCH in production code.
- Date functions construct (DATE, TIME), extract (YEAR, MONTH, DAY, WEEKDAY), and navigate (EDATE, EOMONTH). FORMAT converts dates and numbers to display text — always pair text month/year columns with an integer sort key.
- RELATED(column) looks up from the many-side to the one-side — requires row context. RELATEDTABLE(table) returns all related rows from the many-side — used on the one-side. Calculated columns using these functions are static and do not respond to user filters.
- Format all DAX code. Use one function argument per line, indent consistently, and use DAXFormatter.com for complex expressions. Unformatted DAX is the leading cause of bugs and maintenance problems in production models.

NOTE

Next Module — Module 3: Using Basic Table Functions. Introduces table functions — DAX functions that return tables rather than scalars: FILTER, ALL, VALUES, DISTINCT, ADDCOLUMNS, SUMMARIZE, and CROSSJOIN. These are the building blocks of CALCULATE, which is the most important function in DAX and is covered in depth in Module 5.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

3

Using Basic Table Functions

FILTER · ALL · VALUES · ALLSELECTED

Understand the shift from scalar to table expressions.

Master FILTER, ALL, ALLEXCEPT, VALUES, DISTINCT, ALLSELECTED, HASONEVALUE, and SELECTEDVALUE. Build calculated tables, write DAX queries with EVALUATE, compose nested table functions, and handle the blank row correctly.

Prerequisite: Modules 1 and 2.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 3: Using Basic Table Functions

Module Contents

■ Module Overview and Prerequisites

■ Lesson 1: Scalar vs Table Expressions

- What Is a Table Expression?
- Calculated Tables
- Variables That Store Tables
- DAX Queries and EVALUATE

■ Lesson 2: FILTER — Restricting Rows

- FILTER Syntax and Mechanics
- FILTER as an Iterator
- Nested FILTER and Performance
- FILTER with RELATED
- COUNTROWS + FILTER — The Preferred Pattern

■ Lesson 3: ALL and ALLEXCEPT — Removing Filters

- ALL on a Table
- ALL on Columns — Distinct Value Lists
- The Grand Total Percentage Pattern
- ALLEXCEPT
- ALLNOBLANKROW

■ Lesson 4: VALUES, DISTINCT, and the Blank Row

- VALUES vs ALL — Filter-Aware vs Filter-Ignoring
- DISTINCT vs VALUES — The Blank Row Difference
- The Blank Row Explained
- VALUES as a Scalar — HASONEVALUE and SELECTEDVALUE
- CONCATENATEX with VALUES

■ Lesson 5: ALLSELECTED — Respecting Outer Filters

- The Problem ALL Cannot Solve
- ALLSELECTED Syntax and Semantics
- ALL vs VALUES vs ALLSELECTED Comparison

■ Lesson 6: Composing Table Functions

- Nesting Table Functions
- The Top-N by Average Sales Pattern

■ Demonstration 1: Percentage of Total Pattern

■ Demonstration 2: VALUES, HASONEVALUE, SELECTEDVALUE in Action

■ Lab 3: Using Basic Table Functions

- Exercise 1: FILTER — Conditional Aggregations
- Exercise 2: ALL — Grand Total Percentages
- Exercise 3: VALUES and SELECTEDVALUE
- Exercise 4: ALLSELECTED and Composed Table Functions

■ Module Review and Takeaways

Module Overview

Modules 1 and 2 introduced DAX syntax, evaluation context, and the core function library. Every formula returned a **scalar value** — a single number, date, or text string. This module introduces a fundamentally different kind of DAX expression: the **table expression**. A table expression returns an entire table rather than a single value, and it is the building block of all advanced DAX.

The functions covered here — **FILTER**, **ALL**, **ALLEXCEPT**, **VALUES**, **DISTINCT**, **ALLSELECTED**, **HASONEVALUE**, and **SELECTEDVALUE** — appear in almost every non-trivial DAX measure. They are indispensable for percentage-of-total calculations, conditional aggregations, slicer-relative comparisons, and dynamic labelling. You will use all of them every day.

A critical concept introduced in this module is the **blank row**: an automatic row that DAX adds to dimension tables when there are orphaned rows in the fact table. Understanding the blank row is essential for writing correct measures in real-world models with imperfect data.

EXAM

The PL-300 exam tests table functions extensively. High-frequency scenarios: choosing between **ALL** and **VALUES** for a percentage denominator, the difference between **ALLSELECTED** and **ALL** in slicer contexts, and the correct use of **SELECTEDVALUE** for single-selection labels.

LESSON 1

Scalar Expressions vs Table Expressions

Lesson Objectives

After completing this lesson, you will be able to:

- Distinguish a scalar expression from a table expression.
- Create a calculated table using a table expression.
- Assign a table expression to a variable and use it in subsequent calculations.
- Write and execute a basic DAX query using EVALUATE.

What Is a Table Expression?

Until now every DAX formula has returned a **scalar** — a single value. A **table expression** returns an entire table: a set of rows and columns. Table expressions cannot be the final value of a measure or calculated column — those always require a scalar. But table expressions are used everywhere *inside* formulas, as arguments to iterators and other functions.

```
-- Scalar expressions (return a single value):
4 + 3
SUM ( Sales[Quantity] )

-- Table expressions (return a table):
Sales -- the entire Sales table
FILTER ( Sales, Sales[Quantity] > 1 ) -- a subset of rows
ALL ( 'Product' ) -- all rows, ignoring filters
VALUES ( 'Product'[Color] ) -- distinct visible colors

-- Table expressions appear INSIDE scalar expressions:
[Multi-Item Sales] :=
SUMX (
    FILTER ( Sales, Sales[Quantity] > 1 ), -- table expression as argument
    Sales[Quantity] * Sales[Net Price] -- per-row scalar
)
-- SUMX receives a table from FILTER; returns a single scalar.
```

Calculated Tables

A **calculated table** is a table whose rows and columns are defined by a DAX table expression rather than loaded from a data source. Created in Model view via **Modeling > New table**:

```
-- Products priced above 3,000
ExpensiveProducts =
FILTER (
    'Product',
    'Product'[Unit Price] > 3000
)

-- All distinct colors (a single-column lookup table)
Colors = ALL ( 'Product'[Color] )

-- All existing Category + Subcategory combinations
CategorySubcategory =
ALL ( 'Product'[Category], 'Product'[Subcategory] )
```

```
-- Key facts:
-- * Computed at model refresh, stored in VertiPaq
-- * Can have relationships defined against them
-- * Primary use cases: Date tables, parameter tables, bridge tables
```

TIP

The most important use of calculated tables in professional Power BI is creating a Date table using `CALENDAR()` or `CALENDARAUTO()`. Date tables are covered in Module 8 (Time Intelligence). For now, understand that any table expression can become a physical calculated table.

Variables That Store Tables

Variables can store table expressions, not just scalars. Storing a table in a variable avoids re-evaluating the same expression multiple times:

```
-- Simple: table stored in a variable, then passed to SUMX
[Multi-Item Sales] :=
VAR MultiItems = FILTER ( Sales, Sales[Quantity] > 1 )
RETURN
SUMX ( MultiItems, Sales[Quantity] * Sales[Net Price] )

-- Complex: three chained table variables
BestCategories =
VAR AllSubcats =
ALL ( 'Product'[Category], 'Product'[Subcategory] )
VAR AvgSales =
AVERAGEX (
AllSubcats,
SUMX ( RELATEDTABLE ( Sales ), Sales[Quantity] * Sales[Net Price] )
)
VAR TopSubcats =
FILTER (
AllSubcats,
SUMX ( RELATEDTABLE ( Sales ), Sales[Quantity] * Sales[Net Price] )
>= AvgSales * 2
)
RETURN TopSubcats
```

DAX Queries and EVALUATE

A DAX **query** uses the `EVALUATE` statement to return a table. Run in **DAX Studio** (free at daxstudio.org) for debugging, testing, and profiling:

```
-- Simplest query: all rows from a table
EVALUATE 'Product'

-- Filtered query with sort order
EVALUATE
FILTER ( 'Product', 'Product'[Unit Price] > 3000 )
ORDER BY 'Product'[Color], 'Product'[Unit Price] DESC

-- Query with local measure for debugging
DEFINE
MEASURE Sales[Test Sales] =
```

```
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )  
EVALUATE  
SUMMARIZECOLUMNS (  
    'Product'[Category],  
    "Sales", [Test Sales]  
)  
  
-- Rules:  
-- The expression after EVALUATE must return a TABLE  
-- ORDER BY columns must appear in the result  
-- DEFINE MEASURE lets you test measures without touching the model
```

TIP

Install DAX Studio (daxstudio.org) and connect it to Power BI Desktop. You can run EVALUATE queries, inspect execution plans, and profile measure performance. DAX Studio is the essential tool for any professional DAX developer.

LESSON 2

FILTER — Restricting Rows

Lesson Objectives

After completing this lesson, you will be able to:

- Apply FILTER to produce a subset of rows from any table.
- Use FILTER as the table argument to an iterator like SUMX.
- Nest multiple FILTER calls and explain the performance implications.
- Use FILTER with RELATED to filter on columns from related tables.
- Use COUNTROWS(FILTER(...)) as the standard conditional counting pattern.

FILTER Syntax and Mechanics

FILTER (table, condition) is simultaneously a table function and an iterator. It scans every row of the table, evaluates the condition in the row context of each row, and returns only the rows where the condition is TRUE:

```
-- Calculated table: Fabrikam products
FabrikamProducts =
FILTER (
    'Product',
    'Product'[Brand] = "Fabrikam"
)

-- FILTER as the table argument of SUMX:
[Multi-Item Revenue] :=
SUMX (
    FILTER ( Sales, Sales[Quantity] > 1 ),
    Sales[Quantity] * Sales[Net Price]
)

-- IMPORTANT: FILTER adds to the existing filter context; it does not replace it.
-- [Red Sales] in a matrix sliced by Category shows sales that are
-- BOTH red AND in that category.
[Red Sales] :=
SUMX (
    FILTER ( Sales, RELATED ( 'Product'[Color] ) = "Red" ),
    Sales[Quantity] * Sales[Net Price]
)
```

FILTER as an Iterator — Using RELATED Inside FILTER

Because FILTER iterates rows it creates a **row context**. This is why RELATED can be used inside the condition — RELATED requires a row context to look up the current row's value in the one-side table:

```
-- RELATED inside FILTER: filter Sales on a Product column
[Blue or Red Sales] :=
SUMX (
    FILTER (
        Sales,
        RELATED ( 'Product'[Color] ) IN { "Blue", "Red" }
    ),
    Sales[Quantity] * Sales[Net Price]
)
```

```

)

-- Nested FILTER = AND condition:
FabrikamHighMargin =
FILTER (
    FILTER (
        'Product',
        'Product'[Unit Price] > 'Product'[Unit Cost] * 3 -- most selective first
    ),
    'Product'[Brand] = "Fabrikam"
)

-- Equivalent single FILTER with AND:
FabrikamHighMargin =
FILTER (
    'Product',
    'Product'[Brand] = "Fabrikam"
    && 'Product'[Unit Price] > 'Product'[Unit Cost] * 3
)

```

TIP

When nesting two FILTER calls, put the most selective condition in the innermost FILTER. It runs first, reducing the table size before the outer filter scans it. On large tables this ordering makes a measurable performance difference.

COUNTROWS + FILTER — The Preferred Counting Pattern

To count rows meeting a condition, COUNTROWS with FILTER is clearer and better-optimised by DAX than SUMX with IF:

```

-- Approach 1: SUMX + IF (works but less clear)
[Red Products v1] := SUMX ( 'Product', IF ( 'Product'[Color] = "Red", 1, 0 ) )

-- Approach 2: COUNTROWS + FILTER (preferred)
[Red Products v2] := COUNTROWS ( FILTER ( 'Product', 'Product'[Color] = "Red" ) )

-- Practical examples:
[Large Orders] := COUNTROWS ( FILTER ( Sales, Sales[Quantity] * Sales[Net Price] > 1000 ) )
[Large Order %] := DIVIDE ( [Large Orders], COUNTROWS ( Sales ) )

-- Note: CALCULATE is preferred in production (Module 5):
[Red Products v3] := CALCULATE ( COUNTROWS ( 'Product' ), 'Product'[Color] = "Red" )

```

NOTE

The examples in this module use FILTER to teach how table expressions compose. In production DAX, CALCULATE is the preferred and more efficient way to apply filters to aggregations. Module 5 covers CALCULATE in full.

LESSON 3

ALL and ALLEXCEPT — Removing Filters**Lesson Objectives**

After completing this lesson, you will be able to:

- Use ALL on a table to return all rows, ignoring any active filter context.
- Use ALL on columns to return all distinct values.
- Build a grand total percentage measure using ALL in the denominator.
- Describe what ALLEXCEPT does and when to prefer it over ALL.
- Explain ALLNOBLANKROW.

ALL on a Table

ALL (table) returns every row of the table, completely ignoring any active filter context. It is the opposite of the filter context: where filters restrict visible rows, ALL restores all of them:

```
-- ALL: always the full table, slicer has no effect
[All Sales] :=
SUMX (
  ALL ( Sales ), -- ignore filter context
  Sales[Quantity] * Sales[Net Price]
)
-- In a matrix sliced by Category, [All Sales] shows the SAME grand total
-- in every row -- the filter context has no effect.

-- ALL cannot accept a table expression -- only a table name:
ALL ( Sales ) -- OK
ALL ( FILTER ( Sales, ... ) ) -- ERROR: expression not allowed as argument
```

ALL on Columns — Distinct Value Lists

When called with column names, ALL returns all distinct values of those columns ignoring filters. Multiple columns yield all existing combinations:

```
-- Single column: all distinct colors (ignores filters)
AllColors = ALL ( 'Product'[Color] )

-- Two columns: all Category + Subcategory combinations
AllCombos = ALL ( 'Product'[Category], 'Product'[Subcategory] )

-- In a measure:
[Total Colors] := COUNTROWS ( ALL ( 'Product'[Color] ) ) -- always constant
[Visible Colors] := COUNTROWS ( VALUES ( 'Product'[Color] ) ) -- changes with slicer
```

The Grand Total Percentage Pattern

The most important practical use of ALL is the **percentage of grand total** pattern. The numerator uses the current filter context; the denominator uses ALL to always get the grand total:

```
[Sales Amount] :=
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )

[All Sales Amount] :=
```

```
SUMX ( ALL ( Sales ), Sales[Quantity] * Sales[Net Price] )

[Sales %] := DIVIDE ( [Sales Amount], [All Sales Amount] )

-- In a matrix with Category on rows:
-- Category | Sales Amount | All Sales | Sales %
-- Audio | 85,000 | 500,000 | 17.0%
-- Bikes | 210,000 | 500,000 | 42.0%
-- Clothing | 65,000 | 500,000 | 13.0%
-- TV & Video | 140,000 | 500,000 | 28.0%
-- Total | 500,000 | 500,000 | 100.0%

-- NOTE: ALL ignores slicers too. See ALLSELECTED (Lesson 5)
-- if you need the percentage to be relative to the slicer selection.
```

EXAM

The percentage-of-total pattern with ALL is one of the most tested DAX patterns in PL-300. Memorise: `DIVIDE( [measure], SUMX(ALL(table), ...) )`. The `CALCULATE`-based version — `DIVIDE( [measure], CALCULATE([measure], ALL(table)) )` — is preferred in production and is covered in Module 5.

ALLEXCEPT and ALLNOBLANKROW

`ALLEXCEPT ( table, col1, col2, ... )` returns all distinct combinations of columns in the table, *excluding* the listed columns from the result. Instead of specifying which columns you want, you specify which ones to leave out:

```
-- These two are EQUIVALENT for a Product table with:
-- ProductKey, ProductName, Brand, Class, Color

ALL ( 'Product'[ProductName], 'Product'[Brand], 'Product'[Class] )
ALLEXCEPT ( 'Product', 'Product'[ProductKey], 'Product'[Color] )

-- BUT if UnitCost and UnitPrice columns are later added:
-- ALL version: still returns only ProductName, Brand, Class
-- ALLEXCEPT version: automatically includes the new columns too

-- ALLNOBLANKROW: same as ALL but excludes the automatic blank row
-- Use when iterating dimension tables with potential invalid relationships
COUNTROWS ( ALLNOBLANKROW ( 'Product' ) )
```

LESSON 4

VALUES, DISTINCT, and the Blank Row

Lesson Objectives

After completing this lesson, you will be able to:

- Explain the difference between VALUES, DISTINCT, and ALL.
- Describe what the blank row is and when DAX creates it automatically.
- Explain why measures return unexpected results with invalid relationships.
- Apply HASONEVALUE and SELECTEDVALUE for dynamic labels.
- Build a comma-separated list using CONCATENATEX with VALUES.

VALUES vs ALL — Filter-Aware vs Filter-Ignoring

Function	Returns	Filter Awareness	Typical Use
ALL(col)	All distinct values of col	Ignores filter context	Grand total denominator
VALUES(col)	Distinct visible values	Respects all active filters	Iterating visible items
DISTINCT(col)	Distinct visible values	Respects filters; no blank row	When blank row must be excluded

```
[Total Colors] := COUNTROWS ( ALL ( 'Product'[Color] ) ) -- always same number
[Visible Colors] := COUNTROWS ( VALUES ( 'Product'[Color] ) ) -- changes with slicer

-- With a Bikes category slicer active:
-- [Total Colors] = 15 (all colors in the entire Product table)
-- [Visible Colors] = 4 (only colors used by Bikes products)
```

The Blank Row — What It Is and Why It Exists

The **blank row** is a synthetic row that DAX adds to the **one-side** table of a relationship when fact rows have foreign keys with no matching dimension row. All such orphaned fact rows are grouped under a single blank row:

```
-- Scenario: Sales[ProductKey] = 9999 but no Product row has ProductKey = 9999
-- DAX adds ONE blank row to Product with all columns blank.
-- All orphaned Sales rows point to this single blank row.

-- Effect on counting functions:
COUNTROWS ( ALL ( 'Product'[Color] ) ) -- includes blank row -> N+1
COUNTROWS ( VALUES ( 'Product'[Color] ) ) -- includes blank row -> N+1
COUNTROWS ( DISTINCT ( 'Product'[Color] ) ) -- EXCLUDES blank row -> N

-- The blank row is INVISIBLE in Data view and Table visuals.
-- It appears as a blank category in matrix visuals.

-- Best practice: fix the data in Power Query.
-- Guard against it: use ALLNOBLANKROW when iterating dimensions.
```


WARNING

In a model with invalid relationships, `DIVIDE( sales, COUNTROWS(VALUES(Product[Code])) )` shows a huge incorrect number for the blank category row — the blank row counts as 1 in the denominator but accumulates many orphaned sales in the numerator. Always fix bad data in Power Query before it reaches the model.

VALUES as a Scalar — HASONEVALUE and SELECTEDVALUE

When exactly one value is visible, `VALUES(col)` returns a single-row table that DAX auto-converts to a scalar. Use `HASONEVALUE` to guard the call, or `SELECTEDVALUE` for a cleaner one-liner:

```
-- Guard pattern:
[Brand Label] :=
IF (
    COUNTROWS ( VALUES ( 'Product'[Brand] ) ) = 1,
    VALUES ( 'Product'[Brand] )
)

-- HASONEVALUE shortcut:
[Brand Label v2] :=
IF (
    HASONEVALUE ( 'Product'[Brand] ),
    VALUES ( 'Product'[Brand] )
)

-- SELECTEDVALUE -- cleanest approach:
[Brand Label v3] := SELECTEDVALUE ( 'Product'[Brand] ) -- BLANK if multi
[Brand Label v4] := SELECTEDVALUE ( 'Product'[Brand], "All Brands" ) -- else value

-- CONCATENATEX + VALUES: list all visible brands
[Brand Names] :=
CONCATENATEX (
    VALUES ( 'Product'[Brand] ),
    'Product'[Brand],
    ", ",
    'Product'[Brand], ASC
)

-- With Fabrikam + Contoso selected: "Contoso, Fabrikam"
```

LESSON 5

ALLSELECTED — Respecting Outer Filters**Lesson Objectives**

After completing this lesson, you will be able to:

- Explain the problem with ALL when a slicer is active.
- Describe what ALLSELECTED returns and how it differs from ALL and VALUES.
- Build a slicer-relative percentage measure using ALLSELECTED.
- Identify when ALLSELECTED produces unexpected results.

The Problem ALL Cannot Solve

With the grand total percentage measure using ALL in the denominator, activating a category slicer breaks the 'sums to 100%' expectation. If only Bikes and Clothing are selected, the two category rows might show 42% and 13% for a total of 55% — because the denominator (ALL) still uses the grand total of *all* categories. The Total row never reaches 100%.

What we want is a percentage relative to the *slicer selection*: Bikes + Clothing should together equal 100%. This is exactly what ALLSELECTED provides.

ALLSELECTED — Syntax and Semantics

ALLSELECTED (table) returns rows visible considering only the filters **outside** of the current visual — slicers, page filters, report filters — but *not* the matrix row/column context:

```
-- ALLSELECTED version:
[Sales % of Selection] :=
DIVIDE (
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ),
SUMX ( ALLSELECTED ( Sales ), Sales[Quantity] * Sales[Net Price] )
)

-- With Bikes + Clothing slicer active:
-- Category | Sales Amount | Sales % (ALL) | Sales % (ALLSELECTED)
-- Bikes | 210,000 | 42.0% | 76.4%
-- Clothing | 65,000 | 13.0% | 23.6%
-- Total | 275,000 | 55.0% | 100.0%

-- ALL denominator = 500,000 (full grand total, ignores slicer)
-- ALLSELECTED denominator = 275,000 (only Bikes + Clothing)
```

WARNING

ALLSELECTED is described by the authors of this book as probably the most complex table function in DAX. In simple slicer-relative percentage measures it works predictably. In nested CALCULATE calls it can produce results that are extremely hard to reason about even for experienced developers. Use it only in the straightforward pattern shown here.

ALL vs VALUES vs ALLSELECTED — Comparison

Function	What It Returns	Slicer-Aware?	Use Case
ALL(table)	All rows; ignores every filter	No — ignores slicers	Absolute grand total

VALUES(col)	Visible values; full filter context	Yes — respects everything	Iterating current selection
ALLSELECTED(table)	Rows visible to outer filters only	Yes — respects slicers	Slicer-relative %; always 100%

LESSON 6

Composing Table Functions

Lesson Objectives

After completing this lesson, you will be able to:

- Nest table functions to combine their effects in a single expression.
- Build a top-N calculated table using nested FILTER, AVERAGEX, and RELATEDTABLE.
- Use table variables to break complex expressions into readable steps.

Nesting Table Functions

Table functions compose freely — the result of any table function can be passed as the table argument to another table function. DAX evaluates from innermost outward:

```
-- RELATEDTABLE inside FILTER (calculated column in Product):
'Product'[MultiItemSales] =
SUMX (
  FILTER (
    RELATEDTABLE ( Sales ), -- 1st: related Sales rows
    Sales[Quantity] > 1 -- 2nd: filter those rows
  ),
  Sales[Quantity] * Sales[Net Price] -- 3rd: sum
)

-- FILTER inside SUMX inside DIVIDE:
[Premium Revenue Share] :=
DIVIDE (
  SUMX (
    FILTER ( Sales, Sales[Net Price] > 500 ),
    Sales[Quantity] * Sales[Net Price]
  ),
  SUMX ( ALL ( Sales ), Sales[Quantity] * Sales[Net Price] )
)
```

The Top-N by Average Sales Pattern

This calculated table finds subcategories whose sales exceed twice the average — a classic composition of ALL, AVERAGEX, RELATEDTABLE, and FILTER using table variables:

```
-- Calculated table: subcategories with sales >= 2x the subcategory average
BestCategories =

VAR AllSubcats =
  ALL ( 'Product'[Category], 'Product'[Subcategory] )

VAR AvgSales =
  AVERAGEX (
    AllSubcats, -- iterate all subcategory combinations
    SUMX ( -- compute sales for each
      RELATEDTABLE ( Sales ),
      Sales[Quantity] * Sales[Net Price]
    )
  )
```

```
VAR TopSubcats =  
    FILTER (  
        AllSubcats,  
        VAR SubcatSales =  
            SUMX ( RELATEDTABLE ( Sales ), Sales[Quantity] * Sales[Net Price] )  
        RETURN  
        SubcatSales >= AvgSales * 2  
    )  
  
RETURN TopSubcats  
  
-- Execution order:  
-- 1. ALL produces all (Category, Subcategory) pairs  
-- 2. AVERAGEX iterates AllSubcats, calling SUMX for each row  
-- 3. FILTER iterates AllSubcats again, retaining rows >= 2x average  
-- 4. TopSubcats is the final calculated table
```

CONCEPT

Variables capture the filter context at the point of their definition. AvgSales is computed from ALL (no filter context), representing the true average across all subcategories. Inside FILTER, each row's SUMX uses RELATEDTABLE which operates in the row context created by FILTER's iteration — the Sales rows related to that subcategory.

DEMONSTRATION 1

Building the Percentage of Total Pattern

Scenario: The instructor builds three versions of a percentage measure — using ALL, then ALLSELECTED — and demonstrates the difference live with a category slicer.

Demonstration Steps

1	Open Contoso_Demo03_Start.pbix. Add the base measure: <code>[Sales Amount] := SUMX(Sales, Sales[Quantity] * Sales[Net Price])</code> .
2	Add a Matrix visual. Place Product Category[Category] on Rows. Add [Sales Amount] to Values.
3	Click New measure. Create: <code>[All Sales] := SUMX(ALL(Sales), Sales[Quantity] * Sales[Net Price])</code> . Add to the matrix. Show it is the same in every row.
4	Create: <code>[Sales % of Total] := DIVIDE([Sales Amount], [All Sales])</code> . Add to matrix. Verify all rows sum to 100% in the Total row.
5	Add a Category Slicer. Select only Bikes and Clothing. Show [Sales % of Total] Total row is now ~55% — ALL ignores the slicer.
6	Create: <code>[Sales % of Selection] := DIVIDE([Sales Amount], SUMX(ALLSELECTED(Sales), Sales[Quantity]*Sales[Net Price]))</code> . Add to matrix. With Bikes+Clothing selected, show this version sums to 100%.
7	Clear the slicer (all categories). Show both percentage measures are now identical. Explain: without a slicer, ALL and ALLSELECTED return the same rows.
RESULT	Three measures demonstrated. With no slicer: both percentage measures identical. With a category slicer: ALL-based measure does not sum to 100%; ALLSELECTED-based measure always sums to 100% within the selection.

DEMONSTRATION 2

VALUES, HASONEVALUE, and SELECTEDVALUE in Action

Scenario: The instructor demonstrates VALUES, HASONEVALUE, and SELECTEDVALUE for dynamic labels and conditional measures that respond to slicer selections.

Demonstration Steps

1	Create: [Selected Brand] := SELECTEDVALUE(Product[Brand], "All Brands"). Add a Card visual. Add a Brand slicer. Test 0, 1, and 2 brands selected.
2	Create: [Brand Context] := IF(HASONEVALUE(Product[Brand]), "Brand: " & VALUES(Product[Brand]), COUNTROWS(VALUES(Product[Brand])) & " brands"). Add to a second Card and test with the slicer.
3	Create [Visible Colors] := COUNTROWS(VALUES(Product[Color])) and [All Colors] := COUNTROWS(ALL(Product[Color])). Apply a Category slicer — show [Visible Colors] changes, [All Colors] stays constant.
4	Create [Brand List] := CONCATENATEX(VALUES(Product[Brand]), Product[Brand], ", ", Product[Brand], ASC). Select two brands — Card shows comma-separated list.
5	Create [Orders > 1000] := COUNTROWS(FILTER(Sales, Sales[Quantity]*Sales[Net Price]>1000)). Add a Year slicer — show the count responds to the year filter.
6	Create [Large Order %] := DIVIDE([Orders > 1000], COUNTROWS(Sales)). Demonstrate the measure updating with both category and year slicers simultaneously.
RESULT	Six measures demonstrated: SELECTEDVALUE with else, HASONEVALUE conditional label, filter-aware vs ALL color counts, CONCATENATEX list, and COUNTROWS + FILTER conditional counting.

LAB 3

Using Basic Table Functions

Lab Scenario

The Contoso analytics team needs a suite of measures for an executive dashboard: percentage of total revenue by category, percentage relative to slicer selection, count of large orders, dynamic brand labels, and a calculated table of top subcategories. You will build all of these using the table functions covered in this module.

NOTE

Open Contoso_Lab03_Start.pbix. This file extends the Module 2 completed lab. Save your completed work as Contoso_Lab03_Complete.pbix.

Exercise 1: FILTER — Conditional Aggregations

Estimated time: 20 minutes

Task 1: Build Conditional Revenue Measures

1. In the Data pane, select the **Sales** table. Click **New measure**. Confirm the base sales measure exists (or create it):

```
[Sales Amount] := SUMX( Sales, Sales[Quantity] * Sales[Net Price] )
```

2. Create a large-order revenue measure (line total > 500):

```
[Large Order Revenue] :=
SUMX (
    FILTER ( Sales, Sales[Quantity] * Sales[Net Price] > 500 ),
    Sales[Quantity] * Sales[Net Price]
)
```

3. Create a count of large orders:

```
[Large Order Count] :=
COUNTROWS ( FILTER ( Sales, Sales[Quantity] * Sales[Net Price] > 500 ) )
```

4. Create red product sales using FILTER + RELATED:

```
[Red Sales] :=
SUMX (
    FILTER ( Sales, RELATED ( 'Product'[Color] ) = "Red" ),
    Sales[Quantity] * Sales[Net Price]
)
```

5. Add a **Matrix visual** with Product Category on Rows and all four measures as Values. Verify [Large Order Revenue] is a subset of [Sales Amount] at every row.

Results: Four measures visible in the matrix. [Large Order Revenue] is always less than or equal to [Sales Amount]. [Red Sales] correctly shows revenue for red items only, within each category.

Task 2: Percentage of Large Orders

1. Click New measure. Create:

```
[Large Order %] := DIVIDE ( [Large Order Count], COUNTROWS ( Sales ) )
```

2. Add a **Clustered Bar chart** with Product Category on Y-axis and [Large Order %] on X-axis. Format as percentage.
3. Add a **Year slicer**. Select a single year. Verify the bar chart and matrix both update.

Results: [Large Order %] shows the fraction of orders exceeding 500 in line total, by category. Both visuals respond correctly to the year slicer.

Exercise 2: ALL — Grand Total Percentages

Estimated time: 20 minutes

Task 1: Build ALL-Based and ALLSELECTED-Based Percentage Measures

1. Click New measure. Create the ALL-based denominator:

```
. [All Sales] :=
    SUMX ( ALL ( Sales ), Sales[Quantity] * Sales[Net Price] )
```

2. Create the grand total percentage:

```
. [Sales % of Grand Total] := DIVIDE ( [Sales Amount], [All Sales] )
```

3. Add a Matrix with Product Category on Rows. Add [Sales Amount] and [Sales % of Grand Total]. Verify all rows sum to 100% in the Total row.

4. Add a **Category slicer**. Select two categories. Observe the Total row of [Sales % of Grand Total] is less than 100%. Record the Total row value.

5. Create the ALLSELECTED version:

```
. [Sales % of Selection] :=
    DIVIDE (
        [Sales Amount],
        SUMX ( ALLSELECTED ( Sales ), Sales[Quantity] * Sales[Net Price] )
    )
```

6. Add [Sales % of Selection] to the matrix. With the slicer active, verify: [Sales % of Selection] Total row = 100%; [Sales % of Grand Total] Total row = the fraction from step 4.

Results: Both percentage measures are visible. Without a slicer they are identical. With a slicer: ALL-based reflects share of everything; ALLSELECTED-based always sums to 100%.

Exercise 3: VALUES and SELECTEDVALUE

Estimated time: 20 minutes

Task 1: Dynamic Labels and Visible Counts

1. Click New measure on the **Product** table. Create:

```
. [Selected Brand] := SELECTEDVALUE ( 'Product'[Brand], "All Brands" )
```

2. Add a **Card visual** with [Selected Brand]. Add a **Brand slicer**. Test 0, 1, and 2 brands selected. Verify correct text.

3. Create a context label:

```
. [Context Label] :=
    IF (
        HASONEVALUE ( 'Product'[Brand] ),
        "Brand: " & VALUES ( 'Product'[Brand] ),
        COUNTROWS ( VALUES ( 'Product'[Brand] ) ) & " brands selected"
    )
```

4. Add a second Card with [Context Label]. Test with 0, 1, and 2 brands selected.

5. Create visible and all color count measures:

```
a. [Visible Colors] := COUNTROWS ( VALUES ( 'Product'[Color] ) )
b. [All Colors] := COUNTROWS ( ALL ( 'Product'[Color] ) )
```

6. Add both to Cards. Add a Category slicer. Select Bikes. Verify [Visible Colors] drops to the number of colors available for Bikes; [All Colors] stays constant.

Results: [Selected Brand] shows correct text for all selection states. [Context Label] switches between single-brand name and count label. [Visible Colors] is filter-aware; [All Colors] is always constant.

Exercise 4: ALLSELECTED and Composed Table Functions

Estimated time: 25 minutes

Task 1: FILTER + ALL vs FILTER + ALLSELECTED

1. Click New measure. Create:

```
. [High Value Rate (All)] :=
    DIVIDE (
        COUNTROWS ( FILTER ( Sales, Sales[Quantity] * Sales[Net Price] > 1000 ) ),
        COUNTROWS ( ALL ( Sales ) )
    )
```

2. Add to a Card. Apply a Year slicer. Show the measure does NOT change — ALL ignores the year filter.

3. Create the ALLSELECTED version:

```
. [High Value Rate (Selected)] :=
    DIVIDE (
        COUNTROWS ( FILTER ( Sales, Sales[Quantity] * Sales[Net Price] > 1000 ) ),
        COUNTROWS ( ALLSELECTED ( Sales ) )
    )
```

4. Add alongside step 1. With the Year slicer, show the second measure changes. Explain: ALLSELECTED respects the year slicer; ALL does not.

Task 2: Build a Top-Subcategory Calculated Table

1. On the **Modeling** tab, click **New table**. Enter:

```
. TopSubcategories =
    VAR AllSubcats = ALL ( 'Product'[Category], 'Product'[Subcategory] )
    VAR AvgSales =
        AVERAGEX (
            AllSubcats,
            SUMX ( RELATEDTABLE ( Sales ), Sales[Quantity] * Sales[Net Price] )
        )
    VAR TopSubcats =
        FILTER (
            AllSubcats,
            SUMX ( RELATEDTABLE ( Sales ), Sales[Quantity] * Sales[Net Price] )
            >= AvgSales * 1.5
        )
    RETURN TopSubcats
```

2. Switch to **Data view**. Click TopSubcategories. Verify it contains Category and Subcategory columns with qualifying rows only.
3. Add a **Table visual**. Add both columns from TopSubcategories and [Sales Amount]. Verify only qualifying subcategories appear.
4. Save as **Contoso_Lab03_Complete.pbix**.

Results: TopSubcategories table contains subcategories with sales at least 1.5x the average. Table visual shows those subcategories with sales amounts. File saved.

Module Review and Takeaways

Review Questions

What is a table expression and how does it differ from a scalar expression?

Answer: A scalar expression returns a single value (number, text, date, boolean). A table expression returns a table — a set of rows and columns. Measures and calculated columns must be scalar at their outermost level. Table expressions appear inside formulas as arguments to iterators or as calculated table bodies. Variables can store either scalars or tables.

What is the difference between FILTER and CALCULATE for applying conditions?

Answer: FILTER is a table function and iterator — it scans every row and returns a subset. CALCULATE modifies the filter context and evaluates an expression in that new context. CALCULATE is more powerful and efficient for filter modifications; it uses the query engine's optimisation to apply filters using compressed column indexes. In production DAX, CALCULATE is almost always preferred. FILTER is used when you genuinely need a filtered table as input to a non-trivial iterator.

What does ALL(table) return and how does it interact with the filter context?

Answer: ALL(table) returns every row of the table completely ignoring any active filter context — slicers, report filters, page filters, and matrix row/column filters are all bypassed. ALL cannot accept a table expression as its argument — only a table name or column list.

When should you use ALL vs ALLSELECTED in a percentage denominator?

Answer: Use ALL when the percentage should be computed relative to the absolute grand total, regardless of any slicer. The Total row will not sum to 100% when a slicer narrows the selection. Use ALLSELECTED when the percentage should be relative to the current slicer selection — the Total row always sums to 100% within the selection. Without a slicer, both produce identical results.

What is the blank row and when does DAX create it?

Answer: The blank row is a synthetic row that DAX adds to the one-side dimension table when fact rows have foreign keys with no matching primary key in the dimension. All orphaned fact rows are grouped under one blank row. VALUES and ALL include the blank row; DISTINCT excludes it. ALLNOBLANKROW explicitly removes it. Fix invalid relationships in Power Query.

What is the difference between VALUES and DISTINCT?

Answer: Both return distinct visible values of a column in the current filter context. The only difference: VALUES includes the blank row if one exists; DISTINCT always excludes it. In a valid model with no orphaned keys, they return identical results. Use VALUES as the default; use DISTINCT only when you must exclude the blank row.

When does VALUES return a scalar? What are HASONEVALUE and SELECTEDVALUE?

Answer: VALUES returns a scalar when exactly one row is visible — DAX auto-converts a single-row, single-column table to a scalar. HASONEVALUE(column) returns TRUE if exactly one value is visible. SELECTEDVALUE(column [, else]) combines the check and return: it gives the single value or else (default BLANK) when multiple values exist. SELECTEDVALUE is used for dynamic labels and parameter-driven calculations.

A SUMX formula using FILTER(Sales, RELATED(Product[Color])='Red') is very slow on 50M rows. Why and how to fix?

Answer: FILTER iterates all 50M rows calling RELATED on each — a full table scan with a cross-table lookup that VertiPaq cannot optimise using column indexes. The fix: replace with CALCULATE([Sales Amount], Product[Color] = 'Red'). CALCULATE applies the filter via the relationship at engine level, enabling segment-level filtering which can be 10-100x faster on large fact tables.

Best Practices

- Use `COUNTROWS(FILTER(table, condition))` for conditional row counts — cleaner than `SUMX` with `IF`.
- In production, prefer `CALCULATE` over `FILTER` for applying conditions to aggregations. Use `FILTER` only when you genuinely need a filtered table as input to a non-trivial iterator.
- When nesting two `FILTER` calls, put the most selective condition in the innermost `FILTER` to reduce rows before the outer filter scans.
- Fix invalid relationships in Power Query rather than working around the blank row in DAX. Add missing dimension rows or filter orphaned fact rows before data loads.
- Use `ALLNOBLANKROW` instead of `ALL` when iterating a dimension table in a model with possible invalid relationships — it explicitly excludes the blank row.
- Use `SELECTEDVALUE` as the default for dynamic labels. It is more concise than `IF(HASONEVALUE(...), VALUES(...), else)` and communicates intent clearly.
- Use `DISTINCT` instead of `VALUES` when computing averages per dimension member and the blank row would distort the result.
- When building a percentage, decide upfront whether the denominator is `ALL` (absolute grand total) or `ALLSELECTED` (selection-relative, always 100% within slicer). The choice must match the business question.
- Store complex table expressions in variables. `VAR AllSubcats = ALL(...)` evaluated once is always preferable to repeating `ALL(...)` in multiple places.
- Test calculated table expressions as `EVALUATE` queries in DAX Studio before converting to model tables. This verifies output without requiring a full model refresh.

Common Issues and Solutions

Percentage rows don't sum to 100% when a slicer is active

Cause: The denominator uses `ALL`, which ignores slicer filters. The numerator respects the slicer, the denominator does not.

Solution: Replace `ALL(Sales)` with `ALLSELECTED(Sales)` in the denominator. `ALLSELECTED` makes the denominator match the slicer-filtered total.

VALUES causes 'Cannot convert table to scalar' error

Cause: `VALUES` returned multiple rows and DAX tried to use it as a scalar.

Solution: Use `SELECTEDVALUE(column, else)` instead. It returns the single value or else gracefully.

Matrix shows unexpected blank row with large numbers

Cause: The model has an invalid relationship — orphaned fact rows. DAX added a blank row to the dimension and all orphaned rows accumulate under it.

Solution: Fix the root cause in Power Query. As a workaround, add a visual-level filter: `Product Category[Category] is not blank`.

FILTER with RELATED is extremely slow on a large fact table

Cause: `FILTER` iterates every fact row calling `RELATED` on each — a full table scan with a cross-table lookup that VertiPaq cannot optimise.

Solution: Replace `FILTER(Sales, RELATED(Product[Color]) = 'Red')` with `CALCULATE([Sales Amount], Product[Color] = 'Red')`. `CALCULATE` applies the filter at engine level.

`ALL(FILTER(Sales, ...))` raises an error

Cause: `ALL` cannot accept a table expression as its argument — only a table name or column list.

Solution: Use a `VAR`: `VAR FilteredSales = FILTER(Sales, ...)` `RETURN SUMX(FilteredSales, ...)`. Apply `ALL` to a base table name, not to the result of `FILTER`.

Module 3 — Table Function Quick Reference

Function	Description	Example
<code>FILTER(table, condition)</code>	Returns rows where condition is TRUE. Iterator — creates row context.	<code>FILTER(Sales, Sales[Qty] > 1)</code>
<code>ALL(table)</code>	All rows ignoring every active filter. Arg must be table name.	<code>ALL(Sales)</code>
<code>ALL(col1 [, col2 ...])</code>	All distinct values/combinations ignoring filters.	<code>ALL(Product[Color])</code>
<code>ALLEXCEPT(table, col1 ...)</code>	All distinct values of columns NOT listed, ignoring filters.	<code>ALLEXCEPT(Product, Product[Key])</code>
<code>ALLNOBLANKROW(table)</code>	Like ALL but excludes the automatic blank row.	<code>ALLNOBLANKROW(Product)</code>
<code>VALUES(col)</code>	Distinct visible values in filter context. Includes blank row.	<code>VALUES(Product[Brand])</code>
<code>DISTINCT(col)</code>	Distinct visible values in filter context. Excludes blank row.	<code>DISTINCT(Product[Brand])</code>
<code>ALLSELECTED(table)</code>	Rows visible to outer filters (slicers) only; ignores matrix context.	<code>ALLSELECTED(Sales)</code>
<code>HASONEVALUE(col)</code>	TRUE if exactly one distinct value of col is visible.	<code>HASONEVALUE(Product[Brand])</code>
<code>SELECTEDVALUE(col [, else])</code>	Returns single visible value or else (BLANK by default).	<code>SELECTEDVALUE(Product[Brand], "All")</code>
<code>EVALUATE table</code>	DAX query statement — runs in DAX Studio, SSMS, Excel.	<code>EVALUATE 'Product'</code>

Module Summary

- A table expression returns a table rather than a scalar. Measures and calculated columns must be scalar at their outermost level, but table expressions appear as arguments to iterators, as calculated table bodies, and in EVALUATE queries. Variables can store tables.
- FILTER(table, condition) is both a table function and an iterator. It scans rows, evaluates the condition in row context, and returns matching rows. COUNTROWS(FILTER(...)) is the preferred conditional counting pattern. In production, CALCULATE is more efficient than FILTER for filter modifications.
- ALL(table) returns all rows ignoring the filter context. ALL(col) returns all distinct values. ALLEXCEPT specifies columns to exclude rather than include. ALLNOBLANKROW explicitly removes the blank row added for invalid relationship orphans.
- VALUES returns filter-aware distinct column values and includes the blank row. DISTINCT returns the same but excludes the blank row. ALL ignores filters entirely. The blank row exists when fact rows reference missing dimension keys — fix it in Power Query.
- HASONEVALUE tests whether exactly one value is visible. SELECTEDVALUE returns the single visible value or an else argument. Both are used for dynamic labels and parameter measures.
- ALLSELECTED returns rows visible to outer filters (slicers) but not to the matrix row/column context. Use it in a percentage denominator when the measure should always sum to 100% within the slicer selection. Do not use ALLSELECTED in complex nested CALCULATE expressions without thorough testing.
- Table functions compose freely — FILTER inside SUMX, RELATEDTABLE inside FILTER, ALL inside AVERAGEX. Variables break complex compositions into named steps and prevent repeated evaluation.

NOTE

Next — Module 4: Understanding Evaluation Contexts. The most important conceptual module in this course. Covers filter context and row context in full depth: how they are created, how they interact, and why SUM in a calculated column returns the entire table total. This module is the key to understanding every advanced DAX pattern.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

4

Understanding Evaluation Contexts

Filter Context · Row Context · RELATED · Context Interaction

Master the two evaluation contexts at the heart of every DAX formula.

Understand why SUM in a calculated column returns the grand total, why RELATED is needed in row context, and how filter context propagates automatically through relationships.

The most important conceptual module in this course.

Prerequisite: Modules 1, 2, and 3.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 4: Understanding Evaluation Contexts

Module Contents

■ Module Overview and the Evaluation Context Mantra

■ Lesson 1: The Filter Context

- What Is the Filter Context?
- How Reports Create Filter Contexts
- Multiple Fields — Compound Filter Contexts
- Slicers, Page Filters, and Other Visuals Contribute Too
- Formal Definition: A Filter Context as a Set of Filters

■ Lesson 2: The Row Context

- What Is the Row Context?
- Calculated Columns Have an Automatic Row Context
- Iterators Create Row Contexts
- SUM in a Calculated Column — The Classic Trap
- Measures Have No Automatic Row Context

■ Lesson 3: Nested Row Contexts and Variables

- Two Row Contexts on the Same Table
- The Inner Context Hides the Outer Context
- Using Variables to Capture the Outer Row Context
- EARLIER — Pre-variable Technique
- The UnitPriceRank Worked Example

■ Lesson 4: Row Context and Relationships

- The Row Context Does NOT Use Relationships
- RELATED — Looking Up Values from the One-side Table
- RELATEDTABLE — Retrieving the Many-side Table
- Chaining Relationships with RELATED and RELATEDTABLE

■ Lesson 5: Filter Context and Relationships

- The Complete Mantra
- Filter Context Propagates Through Relationships Automatically
- One-to-Many and the Direction of Filter Propagation
- Bidirectional Cross-filter — Power and Danger
- Filter Context vs Row Context: Comparison Table

■ Lesson 6: Context Interaction — Filter Context Inside an Iterator

- Both Contexts Exist Simultaneously
- Aggregators Use Filter Context; Column References Use Row Context
- FILTER Respects the Outer Filter Context
- ALL Overrides the Filter Context
- Context Interaction Summary

■ Demonstration 1: The Two Contexts Side-by-Side

■ Demonstration 2: RELATED and RELATEDTABLE in Action

■ Lab 4: Understanding Evaluation Contexts

- Exercise 1: Filter Context Exploration
- Exercise 2: Row Context — Calculated Columns and Iterators

- Exercise 3: Nested Row Contexts and UnitPriceRank
- Exercise 4: RELATED, RELATEDTABLE, and Context with Relationships

■ Module Review and Takeaways

Module Overview

Modules 1 through 3 gave you the vocabulary of DAX: syntax, function categories, scalar and table expressions. This module gives you the **grammar** — the rules that govern how every single DAX formula is evaluated. Without a deep understanding of evaluation contexts, you can write DAX formulas that appear to work but produce subtly wrong results; you can spend hours debugging measures whose behaviour you cannot predict; and you cannot progress to CALCULATE, time intelligence, or any advanced DAX.

There are exactly **two** evaluation contexts in DAX. Everything that DAX computes — every measure, every calculated column, every iterator — is evaluated inside one or both of them. Understanding them completely is the single most important investment you can make in your DAX education.

CONCEPT

The evaluation context mantra — memorise this and repeat it whenever you are confused by a DAX result: "The filter context **FILTERS** the model. The row context **ITERATES** one table. The filter context does not iterate. The row context does not filter."

EXAM

PL-300 tests evaluation contexts extensively. High-frequency scenarios: why SUM in a calculated column returns the grand total, why RELATED is needed to look up a column from a related table in a calculated column, and how changing the cross-filter direction of a relationship affects a measure. Every advanced DAX topic — CALCULATE, time intelligence, row context transition — builds directly on this module.

LESSON 1

The Filter Context

Lesson Objectives

After completing this lesson, you will be able to:

- Define the filter context and state what it filters.
- Explain how a Power BI matrix visual creates a filter context for each cell.
- Describe how slicers, page filters, and other visuals all contribute to one unified filter context.
- Read a formal definition of a filter context as a set of column filters.

What Is the Filter Context?

The **filter context** is the set of filters that are active when DAX evaluates a formula. It defines *which rows in each table are visible* at the moment of evaluation. Every DAX formula — every measure in every cell of every visual — is evaluated inside a filter context.

The filter context does not execute the formula differently for each row. It makes some rows invisible and some rows visible, then lets the formula run on the visible rows.

```
-- The same measure formula:
[Sales Amount] := SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )

-- Evaluated in a DIFFERENT filter context in each cell of a matrix:
-- Cell: Audio category -> filter context: Product[Category] = "Audio"
-- Sales rows for Audio products are visible. Formula runs on those.
-- Result: sales of Audio products only.

-- Cell: Bikes category -> filter context: Product[Category] = "Bikes"
-- Sales rows for Bikes products are visible. Formula runs on those.
-- Result: sales of Bikes products only.

-- Cell: Total row -> filter context: empty (no category filter)
-- All Sales rows are visible.
-- Result: grand total of all sales.

-- The formula is IDENTICAL in all three cells. Only the filter context changes.
```

How Reports Create Filter Contexts

When you place a field on the rows of a matrix, Power BI evaluates each cell's measure in a filter context that restricts data to the row's value. For each row, Power BI says to DAX: 'evaluate this measure as if only rows matching this field value exist.'

```
-- Matrix: Product[Category] on rows, [Sales Amount] as value
-- For the 'Audio' row, DAX evaluates [Sales Amount] in a filter context where:
-- Product[Category] = "Audio"
-- This filter propagates through the Category -> Product -> Sales relationship chain.
-- Result: only Audio product sales rows are visible to SUMX.

-- Add Product[Brand] to columns as well:
-- For the 'Audio / Fabrikam' cell, DAX evaluates [Sales Amount] in a filter context:
-- Product[Category] = "Audio" AND Product[Brand] = "Fabrikam"
-- Both row and column contribute to a SINGLE combined filter context.
```

```
-- Result: only Audio+Fabrikam sales rows are visible.
```

All Visual Filters Contribute to One Filter Context

Fields on rows, fields on columns, slicers, page filters, report filters, visual-level filters — they ALL contribute to a single unified filter context. DAX sees one combined filter context, regardless of how many different visuals contributed to it.

```
-- A report page has:
-- Category slicer: 'Audio' selected
-- Year slicer: 2023 selected
-- Matrix row: Product[Brand] = 'Fabrikam'

-- The filter context for the [Sales Amount] cell in that matrix row is:
-- Product[Category] = 'Audio' (from category slicer)
-- Date[Year] = 2023 (from year slicer)
-- Product[Brand] = 'Fabrikam' (from matrix row)

-- All three combine into ONE filter context.
-- [Sales Amount] sees only 2023 Audio Fabrikam sales rows.

-- Key point: there is no hierarchy between slicers, page filters, and visuals.
-- They all equally contribute to the single filter context evaluated by DAX.
```

Formal Definition — A Filter Context as a Set of Filters

More precisely, a filter context is a collection of **filters**, where each filter specifies the set of allowed values for a column. These filters restrict which rows in each table are visible to DAX formulas.

Filter source	Column filtered	Effect on data
Matrix row: Category = Audio	Product[Category]	Only Audio products visible
Matrix column: Year = 2023	Date[Year]	Only 2023 dates visible
Slicer: Brand = Fabrikam	Product[Brand]	Only Fabrikam products visible
Page filter: Country = Germany	Customer[Country]	Only German customers visible

CONCEPT

The correct way to read SUMX(Sales, Sales[Quantity]*Sales[Net Price]) is: 'Sum the product of Quantity and Net Price for all rows in the Sales table that are visible in the CURRENT FILTER CONTEXT.' The filter context is always part of how a formula is read.

In a measure, there is no automatic row context. If you want a row context in a measure, you create it with an **iterator** (SUMX, AVERAGEX, FILTER, MAXX, etc.). The iterator creates a new row context for each row it scans:

```
-- SUMX creates a row context for each row of Sales:
[Gross Margin] :=
SUMX (
    Sales, -- table to iterate
    Sales[Quantity] * (Sales[Net Price] - Sales[Unit Cost]) -- per-row expression
)
-- For row 1: row context has Quantity=3, Net Price=100, Unit Cost=60 -> 120
-- For row 2: row context has Quantity=1, Net Price=250, Unit Cost=140 -> 110
-- ...
-- SUMX sums all per-row results: 120 + 110 + ... = [Gross Margin]

-- All iterators follow the same four-step process:
-- 1. Evaluate the table argument in the existing (filter) context
-- 2. Create a new row context for the first row
-- 3. Evaluate the expression in the current row context
-- 4. Move to the next row, repeat; aggregate all results
```

SUM in a Calculated Column — The Classic Trap

This is the most important test of understanding. What does this calculated column return?

```
Sales[SumQuantity] = SUM ( Sales[Quantity] )
```

Instinct says: 'SUM of quantity for the current row.' But that is wrong. The correct answer is: **the grand total of all quantities in the entire Sales table**, repeated on every row. Here is why:

```
-- Sales[SumQuantity] = SUM ( Sales[Quantity] )
--
-- SUM is an AGGREGATOR, not a column reference.
-- Aggregators like SUM, MIN, MAX, COUNT only respond to the FILTER CONTEXT.
-- They IGNORE the row context entirely.
--
-- In a calculated column:
-- Row context: Sales row 1 (Quantity=3)
-- Filter context: EMPTY (no report filter is active during model refresh)
--
-- SUM(Sales[Quantity]) sees the empty filter context -> ALL Sales rows visible
-- SUM returns the total of all Quantity values in the entire table
-- That same total is written to EVERY row of the calculated column.
--
-- Row 1: SumQuantity = 47,532 (total of all Sales)
-- Row 2: SumQuantity = 47,532 (same total -- row context had no effect on SUM)
-- Row 3: SumQuantity = 47,532 ...
--
-- This is NOT a bug. It is the correct and intentional behavior.
-- Aggregators use filter context. Column references use row context.
-- The two evaluation contexts exist simultaneously but independently.
```

CONCEPT

Aggregators (SUM, MIN, MAX, COUNT, AVERAGE) only respond to the filter context. They ignore the row context completely. Column references (Sales[Quantity]) only work when there IS a row context. This is not a limitation — it is the fundamental design of DAX.

Measures Have No Automatic Row Context

A measure is evaluated in a filter context created by the report. There is no automatic row context in a measure. This is why bare column references inside a measure without an iterator are invalid:

```
-- INVALID measure: no row context means DAX cannot resolve which row's Net Price
[Bad Margin] := Sales[Net Price] - Sales[Unit Cost] -- ERROR

-- VALID: wrap in an iterator to create a row context
[Gross Margin] :=
SUMX ( Sales, Sales[Net Price] - Sales[Unit Cost] ) -- OK: SUMX creates row context

-- The difference between a calculated column and a measure:
--
-- Calculated column = evaluated once per row (automatic row context)
-- in an EMPTY filter context (during refresh)
--
-- Measure = evaluated once per visual cell (NO automatic row context)
-- in the FILTER CONTEXT created by the report
```

LESSON 3

Nested Row Contexts and Variables

Lesson Objectives

After completing this lesson, you will be able to:

- Describe what happens when two row contexts iterate the same table simultaneously.
- Explain why the inner row context hides the outer row context for the same table.
- Use a variable to capture the outer row context before entering an inner iterator.
- Describe the EARLIER function and when it was used before variables existed.
- Build a UnitPriceRank calculated column using variables.

Two Row Contexts on the Same Table

It is possible to have **two row contexts active simultaneously on the same table**. This happens when a calculated column (which creates an outer row context) contains an iterator that creates an inner row context on the same table:

```
-- Calculated column in Product:
-- Outer row context: iterates Product (one row per product, from the calculated column)
-- Inner row context: FILTER iterates Product again

'Product'[MoreExpensiveCount] =
COUNTROWS (
FILTER (
'Product',
'Product'[Unit Price] > ??? -- which Product[Unit Price]?
)
)

-- Inside the FILTER condition, Product[Unit Price] refers to the INNER row context
-- (the product being scanned by FILTER), NOT the outer row context
-- (the product whose column is being computed).
-- The outer context is HIDDEN by the inner context on the same table.
```

Using Variables to Capture the Outer Row Context

Variables are evaluated in the row context that is active *at the point the variable is defined*. This makes them the clean, modern solution for accessing an outer row context value before entering an inner iterator:

```
-- Correct implementation using a variable:
'Product'[UnitPriceRankDense] =
VAR PriceOfCurrentProduct = 'Product'[Unit Price]
-- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
-- This is evaluated in the OUTER row context (the calculated column's row).
-- It captures the current product's price as a scalar into a variable.

VAR MoreExpensivePrices =
FILTER (
VALUES ( 'Product'[Unit Price] ),
'Product'[Unit Price] > PriceOfCurrentProduct
-- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
-- PriceOfCurrentProduct is a scalar captured BEFORE the inner iteration.
-- It is no longer ambiguous: it is always the outer product's price.
-- 'Product'[Unit Price] here refers to the INNER row context (each price
```

```
-- being scanned by FILTER).
)

RETURN
COUNTROWS ( MoreExpensivePrices ) + 1
-- Result: rank 1 = most expensive, rank N = cheapest.
-- Products with the same price share the same rank (dense rank).
```

CONCEPT

Variables are not just for readability — they are the fundamental tool for working with nested row contexts. A variable captures its value in the evaluation context active at the point of its VAR declaration, and retains that value regardless of what context changes happen inside the RETURN block. This is why VAR is preferred over EARLIER for all modern DAX.

EARLIER — Pre-variable Technique

Before variables were added to DAX (in 2015), the **EARLIER** function was used to access the outer row context. It is still valid DAX, and you will encounter it in older code. Understanding it is important even if you always use variables in new code:

```
-- EARLIER accesses the PREVIOUS (outer) row context for a column:
'Product'[UnitPriceRankDense] =
COUNTROWS (
FILTER (
VALUES ( 'Product'[Unit Price] ),
'Product'[Unit Price] > EARLIER ( 'Product'[Unit Price] )
-- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
-- EARLIER('Product'[Unit Price]) = the outer row context's Unit Price
-- 'Product'[Unit Price] (left side) = the inner row context's Unit Price
)
) + 1

-- EARLIER notes:
-- Accepts an optional second parameter: number of context levels to skip
-- EARLIEST accesses the outermost row context for a table
-- In modern DAX: always prefer VAR over EARLIER (clearer, safer, faster)
```

TIP

Always use VAR to capture outer context values instead of EARLIER in new code. EARLIER is harder to read, only works in calculated columns (not measures), and is not supported in some contexts. If you see EARLIER in existing code, translate it to VAR to understand what it does.

LESSON 4

Row Context and Relationships

Lesson Objectives

After completing this lesson, you will be able to:

- Explain why the row context does NOT automatically use relationships.
- Use RELATED to look up a column value from a one-side related table.
- Use RELATEDTABLE to retrieve all related rows from a many-side table.
- Traverse a chain of relationships using RELATED and RELATEDTABLE.

The Row Context Does NOT Use Relationships

This surprises most DAX beginners. When a calculated column creates a row context that iterates the Sales table, that row context covers Sales columns only. Columns from related tables (Product, Customer, Date) are **not** automatically accessible:

```
-- FAILS: row context is on Sales, but Product[Unit Price] is in a different table
Sales[UnitPriceVariance] = Sales[Net Price] - 'Product'[Unit Price] -- ERROR
-- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
-- No row context on Product table.
-- DAX does not know which Product row to use.

-- The mantra: 'The row context iterates ONE table.'
-- It does not follow relationships to other tables automatically.
```

RELATED — Looking Up Values from the One-side Table

RELATED (column) follows the relationship from the many-side table (where the row context is active) to the one-side table and returns the column value for the related row. It requires a row context and a many-to-one relationship:

```
-- RELATED works because Sales is on the many-side of its relationship to Product:
Sales[UnitPriceVariance] = Sales[Net Price] - RELATED ( 'Product'[Unit Price] )
-- ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

-- For the current Sales row, RELATED looks up the related Product row
-- (via Sales[ProductKey] -> Product[ProductKey]) and returns Product[Unit Price].

-- More examples of RELATED in calculated columns:
Sales[ProductColor] = RELATED ( 'Product'[Color] )
Sales[CustomerCountry] = RELATED ( Customer[Country] )
Sales[Category] = RELATED ( 'Product'[Category] ) -- traverses two hops

-- RELATED can traverse a CHAIN of relationships:
-- Sales -> Product -> Product Subcategory -> Product Category
Sales[CategoryName] = RELATED ( 'Product Category'[Category] ) -- three hops

-- RELATED also works inside FILTER in a measure:
[Red Sales] :=
SUMX (
    FILTER ( Sales, RELATED ( 'Product'[Color] ) = "Red" ),
    Sales[Quantity] * Sales[Net Price]
)
```

RELATEDTABLE — Retrieving the Many-side Table

RELATEDTABLE (table) is the reverse direction: when the row context is on the **one-side** table, RELATEDTABLE returns all related rows from the **many-side** table:

```
-- Calculated column in Product (one-side):
-- RELATEDTABLE returns all Sales rows related to the current product
'Product'[NumberOfSales] =
COUNTROWS ( RELATEDTABLE ( Sales ) )
-- For Product row 'Contoso Phone X720': returns count of Sales rows for that product

-- Another example: total sales amount per product (in Product table):
'Product'[TotalSalesAmount] =
SUMX (
    RELATEDTABLE ( Sales ), -- Sales rows for the current product
    Sales[Quantity] * Sales[Net Price] -- per-row calculation
)

-- RELATEDTABLE traverses chains too -- from Product Category (top) to Sales (bottom):
'Product Category'[NumberOfSales] =
VAR SalesOfCategory = RELATEDTABLE ( Sales ) -- traverses Category->Subcategory->Product->Sales
RETURN COUNTROWS ( SalesOfCategory )

-- RELATED and RELATEDTABLE summary:
-- RELATED: many-side -> one-side (go UP the relationship)
-- RELATEDTABLE: one-side -> many-side (go DOWN the relationship)
```

EXAM

RELATED requires: (1) an active row context, and (2) a many-to-one relationship from the iterated table to the target table. RELATEDTABLE requires: (1) an active row context, and (2) a one-to-many relationship from the iterated table to the target table. Neither function works inside a measure without an enclosing iterator because measures have no automatic row context.

LESSON 5

Filter Context and Relationships

Lesson Objectives

After completing this lesson, you will be able to:

- State the complete evaluation context mantra.
- Explain how a filter context on a dimension table propagates to the fact table through a relationship.
- Describe how filter propagation direction is controlled by the relationship's cross-filter direction.
- Explain why enabling bidirectional cross-filter on all relationships is dangerous.

The Complete Mantra

Now we can extend the mantra to its full form, which covers the key difference between how each context interacts with relationships:

```
-- The complete evaluation context mantra:
--
-- The FILTER CONTEXT filters the MODEL.
-- The ROW CONTEXT iterates ONE TABLE.
--
-- Consequences:
-- Filter context -> propagates through relationships AUTOMATICALLY
-- Row context -> does NOT use relationships; needs RELATED / RELATEDTABLE
--
-- The filter context does not iterate.
-- The row context does not filter.
```

Filter Context Propagates Through Relationships Automatically

When a filter context restricts a column in a dimension table, that filter automatically propagates through relationships to the fact table. No function is needed — the relationship does the work:

```
-- [Sales Amount] := SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
--
-- Matrix row: Product[Category] = 'Audio'
-- Filter context: Product[Category] = 'Audio'
--
-- The filter is placed on the Product table.
-- The relationship Product -> Sales (one-to-many) propagates the filter.
-- Only Sales rows whose related Product has Category='Audio' are visible.
-- SUMX iterates ONLY those Sales rows.
-- No RELATED, no FILTER, no function needed -- the relationship handles it.

-- Another example: filter from Customer propagates to Sales
-- Slicer: Customer[Country] = 'Germany'
-- Filter propagates: Customer -> Sales (one-to-many, standard direction)
-- SUMX sees only German customers' sales rows automatically.
```

Cross-filter Direction — Which Way Does the Filter Flow?

A relationship has a cross-filter direction that controls which way the filter propagates. The direction is set on the relationship in Power BI Model view or in Tabular Editor:

Direction	Behaviour	Typical Use
Single (One-to-Many)	Filter flows from one-side (dimension) to many-side (fact). Standard for star schema.	Product->Sales, Customer->Sales, Date->Sales
Both (Bidirectional)	Filter flows both ways. Allows filtering facts from the dimension AND dimensions from the fact.	Use sparingly — only when required

```
-- Standard (single) direction: Category slicer -> filters Products -> filters Sales
[NumOfSales] := COUNTROWS ( Sales ) -- responds to Category slicer
[NumOfProducts] := COUNTROWS ( 'Product' ) -- responds to Category slicer
[NumOfCustomers] := COUNTROWS ( Customer ) -- does NOT respond to Category slicer
-- Customer is on a separate branch. No relationship links Category to Customer.

-- Bidirectional on Sales<->Product:
-- Now Customer[Education] slicer can filter Product table (via Sales bridge)
-- because filter can travel: Customer->Sales->Product
-- WARNING: bidirectional filters can cause DAX to produce ambiguous results
-- and can dramatically slow down queries on large models.
-- Enable bidirectional only when you understand the full impact.
```

WARNING

The authors of The Definitive Guide to DAX explicitly state: enabling bidirectional cross-filter on all relationships is NOT a good option, even though it might seem like it enables more flexible analysis. Bidirectional relationships can create circular ambiguity and performance problems. Enable them only when necessary and with full understanding of the model's DAX implications.

Filter Context vs Row Context — Comparison

Property	Filter Context	Row Context
What it does	Filters the model — makes rows visible or invisible	Iterates one table — points at the current row
Who creates it	The report (visuals, slicers, page/report filters)	Calculated columns (automatically) or iterators (SUMX, FILTER, etc.)
Uses relationships?	YES — propagates through relationships automatically	NO — requires RELATED or RELATEDTABLE explicitly
Scope	Entire model (all tables affected via relationships)	One table only (only the iterated table's columns are directly accessible)
Works for aggregators?	YES — SUM, MIN, MAX see the filtered rows	NO — SUM ignores row context; it uses filter context only
Works for column references?	NOT directly — need row context for column values	YES — Sales[Quantity] returns the current row's value
Can be nested?	Yes — CALCULATE modifies it (Module 5)	Yes — iterators can nest; inner hides outer on same table

LESSON 6

Context Interaction — Both Contexts at Once

Lesson Objectives

After completing this lesson, you will be able to:

- Explain how both contexts are active simultaneously during iteration.
- State which parts of a formula respond to the filter context and which to the row context.
- Explain why FILTER respects the outer filter context.
- Explain why ALL ignores the filter context.
- Trace a complete formula evaluation through both contexts.

Both Contexts Are Active Simultaneously

Inside an iterator, BOTH contexts are active at the same time. The filter context restricts which rows the iterator scans. The row context makes column values accessible for each scanned row. They are independent — each affects different parts of the formula:

```
-- Measure evaluated in a cell where Category = 'Audio' (filter context)
[Red Audio Sales] :=
SUMX (
  FILTER (
    Sales, -- (1) FILTER evaluates Sales...
    RELATED ( 'Product'[Color] ) = "Red" -- ...in the existing filter context
  ),
  Sales[Quantity] * Sales[Net Price] -- (2) SUMX evaluates this per row
)

-- Trace:
-- Filter context: Product[Category] = 'Audio' (from report)
-- FILTER iterates Sales -- only Audio-related rows visible (filter context)
-- For each row, RELATED uses row context to get Product[Color]
-- FILTER keeps rows where Color = 'Red'
-- SUMX sums Quantity*Net Price for those rows
-- Result: revenue of red Audio products in the current filter context
```

Aggregators Use Filter Context; Column References Use Row Context

This single rule resolves most DAX confusion. Remember it whenever a formula behaves unexpectedly:

```
-- Inside SUMX, two kinds of expressions behave differently:
[Example] :=
SUMX (
  Sales,
  Sales[Quantity] -- column ref -> uses ROW context (current row's qty)
  * Sales[Net Price] -- column ref -> uses ROW context (current row's price)
  - SUM ( Sales[Unit Cost] ) -- aggregator -> uses FILTER context (total cost!)
)

-- SUM(Sales[Unit Cost]) inside SUMX is computed ONCE using the filter context,
-- not once per row. This means [Example] subtracts the TOTAL cost from each row's
-- revenue -- almost certainly NOT what was intended.
-- The correct formula uses column references for row-level values:
```



```
[Gross Margin] :=
SUMX (
Sales,
Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) -- all column refs
)
```

FILTER Respects the Outer Filter Context; ALL Overrides It

```
-- Scenario: measure in a matrix with Product[Brand] on rows
-- Filter context: Brand = 'Fabrikam'

-- FILTER: respects the filter context -- iterates the ALREADY-FILTERED table
[Red Fabrikam Products] :=
COUNTROWS ( FILTER ( 'Product', 'Product'[Color] = "Red" ) )
-- Product is already filtered to Fabrikam rows (by filter context)
-- FILTER further restricts to Red within Fabrikam
-- Result: count of red Fabrikam products (NOT all red products)

-- ALL: overrides the filter context -- ignores Brand filter
[All Red Products] :=
COUNTROWS ( FILTER ( ALL ( 'Product' ), 'Product'[Color] = "Red" ) )
-- ALL('Product') returns ALL products, ignoring the Fabrikam filter context
-- FILTER then keeps Red ones from the full table
-- Result: total red products across ALL brands (same in every Brand row)
```

CONCEPT

FILTER is not a context modifier — it is an iterator. It scans the table as it exists in the current filter context, then returns a subset. If the filter context has already restricted the table, FILTER works on that restricted version. This is why FILTER and filter context compound: a Brand filter in context AND a Color filter in FILTER means Brand AND Color.

DEMONSTRATION 1

The Two Contexts Side-by-Side

Scenario: The instructor builds three calculated columns and three measures that demonstrate the difference between filter context and row context, including the SUM-in-calculated-column trap.

Demonstration Steps

1	Open Contoso_Demo04_Start.pbix. In the Sales table, add calculated column: Sales[Line Total] = Sales[Quantity] * Sales[Net Price]. Show the column values — each row has a different value (row context at work).
2	Add calculated column: Sales[SumQty] = SUM(Sales[Quantity]). Show the result — every row has the SAME value (grand total). Explain: SUM is an aggregator, uses filter context only (which is empty at refresh). This is the most important 'gotcha' in DAX.
3	Add calculated column: Sales[PriceRank] using the VAR pattern from Lesson 3. Show that rank 1 = most expensive. Highlight how VAR captures the outer row context.
4	Add a Matrix visual. Rows = Product[Category], Values = [Sales Amount]. Show that selecting 'Audio' in a slicer changes the measure result — filter context at work.
5	Add a second measure: [Wrong Margin] := Sales[Net Price] - Sales[Unit Cost]. Show the error — no row context in a measure means bare column references fail.
6	Fix it: [Gross Margin] := SUMX(Sales, Sales[Net Price] - Sales[Unit Cost]). Show it works. Explain the iterator creates the required row context.
RESULT	Six formulas demonstrated. Calculated column with column refs = correct per-row values. SUM in a calculated column = grand total on every row. Bare column refs in a measure = error. Iterator + column refs in a measure = correct.

DEMONSTRATION 2

RELATED and RELATEDTABLE in Action

Scenario: The instructor demonstrates RELATED (many-to-one) and RELATEDTABLE (one-to-many) in calculated columns, then shows how the filter context propagates through relationships without any explicit function.

Demonstration Steps

1	In the Sales table, add: Sales[ProductColor] = RELATED(Product[Color]). Show each Sales row now has the color of its related product. Remove RELATED and show the error: 'Product[Color] is not in the Sales row context.'
2	In the Sales table, add: Sales[Category] = RELATED('Product'[Category]). Show the chain traversal: Sales -> Product -> Product Subcategory -> Product Category. Confirm the column is populated correctly.
3	In the Product table, add: Product[SalesCount] = COUNTROWS(RELATEDTABLE(Sales)). Show each product row now has its total sales transaction count.
4	Add a Matrix. Rows = Customer[Country], Values = [Sales Amount]. No RELATED, no FILTER — yet filtering by Customer[Country] correctly restricts Sales. Explain: filter context on Customer propagates to Sales via the relationship automatically.
5	Change the matrix to show [NumOfProducts] := COUNTROWS(Product). Apply a Category slicer. Show Product count changes. Then disable bidirectional cross-filter on Sales<->Product. Now show that a Customer Education slicer no longer filters Product count (unidirectional).
6	Re-enable bidirectional. Explain the performance warning. Close the demo.
RESULT	RELATED and RELATEDTABLE demonstrated in calculated columns. Filter context propagation through relationships shown without any explicit function. Effect of relationship cross-filter direction on filter propagation demonstrated.

LAB 4

Understanding Evaluation Contexts

Lab Scenario

The Contoso analytics team is reviewing several formulas that produce unexpected results. Your task is to diagnose the issues using your understanding of evaluation contexts, fix each formula, and add new calculated columns and measures that correctly leverage both the filter context and the row context.

NOTE

Open Contoso_Lab04_Start.pbix. This file extends the Module 3 completed lab. All tables and relationships from the DAX series are present. Save your work as Contoso_Lab04_Complete.pbix.

Exercise 1: Filter Context Exploration

Estimated time: 20 minutes

Task 1: Observe how the filter context changes measure results

1. Add a **Matrix visual**. Rows = Product[Category], Columns = Date[Year], Values = [Sales Amount]. Observe each cell shows the correct sales for that category AND year combination.
2. Add a **Country slicer** using Customer[Country]. Select 'Germany'. Observe ALL cells update — the slicer adds a third filter to every cell's filter context.
3. Click any single cell in the matrix. Note the values in all other visuals on the page also update — cross-filter creates additional filter context.
4. In the Filters pane, add a **report-level filter**: Product[Brand] = 'Fabrikam'. Verify the matrix now only shows Fabrikam products, confirming report-level filters contribute to the filter context evaluated by all measures on all pages.

Results: Each matrix cell correctly shows sales for Category + Year + Country + Brand. All four filter sources contribute to one unified filter context per cell.

Task 2: Diagnose and fix a broken measure

1. The model contains a broken measure: `[Avg Line Revenue] := Sales[Quantity] * Sales[Net Price]`. When placed on a Card visual, it shows an error. Explain why.
2. Fix the measure using AVERAGEX:


```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Avg Line Revenue] :=  
  AVERAGEX ( Sales, Sales[Quantity] * Sales[Net Price] ) 'frags': [ParaFrag(__tag__='para', bold=0,  
  fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Avg Line Revenue] :=',  
  textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,  
  fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",  
  textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,  
  fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='AVERAGEX ( Sales,  
  Sales[Quantity] * Sales[Net Price] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style':  
  'bulletText': None 'debug': 0 ) #Paragraph
```
3. Place on a Card. Verify it shows the average revenue per sales transaction. Verify it responds to the Country slicer — the average changes because the filter context restricts which rows AVERAGEX iterates.

Results: [Avg Line Revenue] fixed. The error was caused by bare column references in a measure (no row context). AVERAGEX creates the required row context.

Exercise 2: Row Context — Calculated Columns and Iterators

Estimated time: 20 minutes

Task 1: Add calculated columns that use the row context

1. In the **Sales** table, add a calculated column:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': 'Sales[Gross Margin Per Unit] = Sales[Net Price] - Sales[Unit Cost]' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Sales[Gross Margin Per Unit] = Sales[Net Price] - Sales[Unit Cost]', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. In the **Sales** table, add another calculated column:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': 'Sales[Line Gross Margin] = Sales[Quantity] * Sales[Gross Margin Per Unit]' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Sales[Line Gross Margin] = Sales[Quantity] * Sales[Gross Margin Per Unit]', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 ) #Paragraph
```

3. Switch to Data view. Select the Sales table. Verify each row of [Gross Margin Per Unit] has a different value. Verify [Line Gross Margin] = [Quantity] * [Gross Margin Per Unit] for each row.

Task 2: Add a calculated column that demonstrates the SUM trap

1. Add a calculated column to verify you understand the SUM behavior:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': 'Sales[TotalSalesTest] = SUM ( Sales[Net Price] )' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Sales[TotalSalesTest] = SUM ( Sales[Net Price] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Observe in Data view: every row shows the SAME value — the grand total of Net Price across all rows. This confirms SUM ignores the row context. Delete this column after verifying.

3. Now add a measure that correctly uses both contexts:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Total Gross Margin] := SUMX ( Sales, Sales[Quantity] * (Sales[Net Price] - Sales[Unit Cost]) )' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Total Gross Margin] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='SUMX ( Sales, Sales[Quantity] * (Sales[Net Price] - Sales[Unit Cost]) )', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 ) #Paragraph
```

4. Add to a Card. Verify it matches the sum of the [Line Gross Margin] column. Apply the Country slicer — the measure updates (filter context), but the calculated column values in Data view do not change (calculated columns are computed once at refresh with empty filter context).

Results: Row context behavior confirmed. Calculated columns: per-row values, empty filter context. SUM in a calculated column: grand total on every row. Measure with SUMX: correct aggregation responding to filter context.

Exercise 3: Nested Row Contexts and UnitPriceRank

Estimated time: 20 minutes

Task 1: Build UnitPriceRankDense using a variable

1. In the **Product** table, add a calculated column:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': ""Product'[UnitPriceRankDense] =
    VAR CurrentPrice = 'Product'[Unit Price]
    VAR HigherPrices =
    FILTER (
    VALUES ( 'Product'[Unit Price] ),
    'Product'[Unit Price] > CurrentPrice
    )
    RETURN
    COUNTROWS ( HigherPrices ) + 1" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier',
    fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=""Product'[UnitPriceRankDense] =",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR CurrentPrice =
    'Product'[Unit Price]", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
    bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text=",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR HigherPrices =",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" FILTER (",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" VALUES ( 'Product'[Unit Price]
    ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" 'Product'[Unit Price] >
    CurrentPrice", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=")",
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" COUNTROWS ( HigherPrices )
    + 1", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Switch to Data view, sort by [UnitPriceRankDense]. Verify rank 1 = most expensive product price tier. Verify products with identical prices share the same rank (dense ranking).

3. Try building the same column WITHOUT a variable, using EARLIER:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': ""Product'[UnitPriceRankEarlier] =
    COUNTROWS (
```

```

FILTER (
VALUES ( 'Product'[Unit Price] ),
'Product'[Unit Price] > EARLIER ( 'Product'[Unit Price] )
)
) + 1" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
link=[], rise=0, text=""Product'[UnitPriceRankEarlier] =", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='COUNTROWS (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' FILTER (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="" VALUES ( 'Product'[Unit Price]
)", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="" 'Product'[Unit Price] > EARLIER
( 'Product'[Unit Price] )", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=')', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='') + 1',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

4. Confirm both columns produce identical results. Note that the VAR version is substantially easier to read.

Task 2: Add a count of more expensive products (not dense)

1. Add another calculated column counting how many individual products (not price tiers) are more expensive:

```

. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': ""Product'[MoreExpensiveProductCount] =
VAR CurrentPrice = 'Product'[Unit Price]
RETURN
COUNTROWS (
FILTER ( 'Product', 'Product'[Unit Price] > CurrentPrice )
)" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
rise=0, text=""Product'[MoreExpensiveProductCount] =", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=""VAR CurrentPrice = 'Product'[Unit Price]", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="

```



```

textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' COUNTROWS (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" FILTER ( 'Product',
'Product'[Unit Price] > CurrentPrice )", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' )', textColor=Color(.101961,.227451,.360784,1), us_lines=[])) 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Compare [UnitPriceRankDense] with [MoreExpensiveProductCount]+1. Explain the difference: dense rank counts distinct price tiers; MoreExpensiveProductCount counts individual products.

Results: Both VAR and EARLIER approaches produce identical dense price rankings. The VAR approach is confirmed as more readable. The distinction between counting price tiers vs individual products is clear.

Exercise 4: RELATED, RELATEDTABLE, and Context with Relationships

Estimated time: 25 minutes

Task 1: Add RELATED calculated columns to Sales

1. In the **Sales** table, add:
 - a. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "Sales[Product Color] = RELATED ('Product'[Color])" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="Sales[Product Color] = RELATED ('Product'[Color])", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
 - b. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "Sales[Product Category] = RELATED ('Product'[Category])" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="Sales[Product Category] = RELATED ('Product'[Category])", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
 - c. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "Sales[Customer Segment] = RELATED (Customer[Segment])" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="Sales[Customer Segment] = RELATED (Customer[Segment])", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
2. Switch to Data view. Verify each Sales row now shows the color, category, and segment of its related product and customer respectively.
3. Attempt to add this column WITHOUT RELATED:


```

. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "Sales[Bad Color] = 'Product'[Color] -- will ERROR"
'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="Sales[Bad Color] = 'Product'[Color] -- will ERROR",
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```
4. Observe the error. Explain: the row context is on Sales; Product[Color] requires a row context on Product, which does not exist. Delete this column attempt.

Task 2: Add a RELATEDTABLE calculated column to Product

1. In the **Product** table, add:

- a. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ""Product'[TotalOrders] = COUNTROWS (RELATEDTABLE (Sales))" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=""Product'[TotalOrders] = COUNTROWS (RELATEDTABLE (Sales))", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
- b. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': ""Product'[ProductRevenue] = SUMX (RELATEDTABLE (Sales), Sales[Quantity] * Sales[Net Price])" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=""Product'[ProductRevenue] =", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='SUMX (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' RELATEDTABLE (Sales)', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' Sales[Quantity] * Sales[Net Price]', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=')', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph

2. Verify in Data view each product shows its own order count and revenue.

Task 3: Observe automatic filter context propagation

1. Create a measure: `[Count Products] := COUNTROWS( 'Product' )`. Place on a Card. Add a Category slicer. Observe [Count Products] changes when a category is selected — the filter context on Category propagates to Product automatically.
2. Create: `[Count Customers] := COUNTROWS( Customer )`. Place on a Card. Apply the Category slicer. Observe [Count Customers] does NOT change — the category filter does not propagate from Product to Customer because there is no direct relationship (they connect only through Sales).
3. Now apply a Customer[Country] slicer. Observe [Count Products] DOES change — because there IS a relationship Customer -> Sales -> Product, and Sales<->Product has bidirectional cross-filtering in the Contoso model.
4. Save as **Contoso_Lab04_Complete.pbix**.

Results: RELATED columns populated correctly in Sales. RELATEDTABLE columns correct in Product. Filter context propagation: Category slicer filters Product (direct) but not Customer (no path). Country slicer filters Product (via bidirectional Sales<->Product). All behaviours explained by the complete evaluation context mantra.

Module Review and Takeaways

Review Questions

What is the filter context? How is it created?

Answer: The filter context is the set of active filters that determines which rows are visible to DAX formulas at the moment of evaluation. It is created by the report — by fields placed on rows/columns of a matrix, slicers, page filters, report filters, and cross-filtering from other visuals. All of these contribute to one unified filter context for each cell.

What is the row context? How is it created?

Answer: The row context is the context that makes DAX aware of a specific current row in a table, allowing column references like Sales[Quantity] to return the value for that row. It is created automatically for calculated columns (DAX iterates every row during model refresh) and programmatically by iterators (SUMX, AVERAGEX, FILTER, etc.) in measures.

Why does SUM(Sales[Quantity]) in a calculated column return the same value on every row?

Answer: SUM is an aggregator. Aggregators respond only to the filter context, not the row context. In a calculated column, there is no active filter context (the model is being refreshed outside of any report). So SUM sees all rows of Sales and returns the grand total. That same grand total is written to every row. The row context has no effect on SUM.

Why do bare column references fail inside a measure?

Answer: A measure has no automatic row context. Column references like Sales[Quantity] require a row context to know which row's value to return. Without a row context, DAX cannot resolve the reference. The fix is to wrap the expression in an iterator (SUMX, AVERAGEX, etc.) which creates a row context for each row it scans.

What does RELATED do? When is it needed?

Answer: RELATED(column) follows a many-to-one relationship from the currently iterated table to a related table and returns the column value for the matching row. It is needed because the row context only covers the columns of the one table being iterated — it does not automatically access related tables. RELATED requires an active row context and a many-to-one relationship to the target table.

What is the difference between how the filter context and the row context interact with relationships?

Answer: The filter context filters the entire model — it propagates through relationships automatically according to the cross-filter direction. No function is needed. The row context iterates only one table — it does not use relationships at all. To access a related table's column in a row context, RELATED or RELATEDTABLE must be used explicitly.

Two row contexts are active on the Product table. A formula references Product[Unit Price]. Which row context does DAX use?

Answer: The inner (most recently created) row context. When two row contexts exist on the same table, the inner row context hides the outer one. To access the outer row context's value, use a VAR to capture it before entering the inner iterator. The pre-variable alternative is EARLIER(column), which returns the column value from the next outer row context.

What is the complete evaluation context mantra?

Answer: The filter context filters the MODEL. The row context iterates ONE TABLE. The filter context propagates through relationships automatically. The row context does not use relationships (requires RELATED/RELATEDTABLE). The filter context does not iterate. The row context does not filter.

Best Practices

- Memorise and repeat the mantra: 'The filter context filters the model. The row context iterates one table.' When a formula surprises you, return to the mantra and trace which context is active where.
- Always use VAR to capture an outer row context value before entering an inner iterator. Never use EARLIER in new code — VAR is clearer, faster, and works in more situations.
- Never use bare column references (Sales[Quantity]) in a measure without an enclosing iterator. Always use aggregators (SUM, MIN, MAX) or iterators (SUMX, AVERAGEX) for measure formulas.
- Avoid SUM or other aggregators inside iterators when you intend per-row values. SUM(Sales[Unit Cost]) inside SUMX uses the filter context and returns the total, not the current row's cost. Use column references (Sales[Unit Cost]) for per-row access.
- Use RELATED in calculated columns and inside iterator expressions to look up dimension attributes. Use RELATEDTABLE to retrieve the set of fact rows for a dimension row.
- Prefer unidirectional (one-to-many) relationships in star schemas. Enable bidirectional cross-filtering only when the business requirement genuinely needs it and you understand the performance and DAX ambiguity implications.
- When debugging an unexpected measure result, ask two questions: (1) What is the filter context at the cell where the wrong value appears? (2) Is there a row context active, and if so, which table is it on?
- Calculated columns are evaluated at model refresh time with an empty filter context. Do not rely on report-level filters to affect calculated column values — they will not.

Common Issues and Solutions

SUM in calculated column returns same value on every row

Cause: SUM is an aggregator — it uses filter context only, which is empty during calculated column evaluation. Row context has no effect on SUM.

Solution: Use a column reference (Sales[Quantity]) instead of SUM(Sales[Quantity]) inside a calculated column. If you need a running aggregation per row, use a different approach.

Bare column reference in a measure causes an error

Cause: Measures have no automatic row context. Column references require a row context to resolve.

Solution: Wrap the expression in an iterator: SUMX(Sales, Sales[Quantity]*Sales[Net Price]) instead of Sales[Quantity]*Sales[Net Price].

RELATED returns an error in a measure

Cause: RELATED requires an active row context. In a measure with no enclosing iterator, there is no row context.

Solution: Wrap RELATED inside an iterator: SUMX(FILTER(Sales, RELATED(Product[Color])='Red'), ...).

Two columns from the same table in a nested row context are ambiguous

Cause: When two row contexts exist on the same table, column references resolve to the inner context.

Solution: Capture the outer row context value in a VAR before entering the inner iterator. The variable retains the outer value regardless of inner context changes.

A slicer on Customer[Country] unexpectedly changes [Count Products]

Cause: The model has a bidirectional cross-filter relationship between Sales and Product. The filter propagates: Customer -> Sales -> Product.

Solution: This is correct model behaviour. If it is undesired, set the Sales<->Product relationship to single (one-to-many) cross-filter direction in Model view.

Module 4 — Quick Reference

Concept / Function	Description	Use Where
Filter context	Set of filters making rows visible. Created by visuals, slicers, page/report filters.	Every measure cell
Row context	Points at the current row. Enables column references.	Calculated columns; iterators in measures
Both active at once	Filter restricts what rows exist; row context says which row is current.	Inside any iterator
RELATED(col)	Follow many-to-one relationship to retrieve a column value from a related table.	Calculated columns; inside iterators
RELATEDTABLE(tbl)	Follow one-to-many relationship to retrieve all related rows from another table.	Calculated columns; inside iterators
VAR x = col	Captures column value in current row context. Retains value inside RETURN regardless of inner contexts.	Nested row contexts
EARLIER(col)	Returns column value from the next outer row context on the same table. Pre-variable technique.	Legacy code only
Aggregators (SUM etc.)	Respond to filter context only. Ignore row context.	Everywhere
Column references	Respond to row context only. Require an active row context.	Calculated columns; inside iterators
Bidirectional filter	Filter propagates in both directions of a relationship.	Use with caution

Module Summary

- There are exactly two evaluation contexts in DAX: the filter context and the row context. They are completely different concepts — not variations of the same thing. Every DAX formula is evaluated in one or both of them simultaneously.
- The filter context filters the model. It is created by the report and propagates through relationships automatically. Aggregators (SUM, MIN, MAX) respond to the filter context only.
- The row context iterates one table. It is created automatically for calculated columns and programmatically by iterators (SUMX, FILTER, etc.) in measures. Column references (Sales[Quantity]) respond to the row context.
- SUM in a calculated column returns the grand total on every row because SUM ignores the row context. This is correct DAX behaviour, not a bug.
- The row context does not use relationships. RELATED (many-to-one) and RELATEDTABLE (one-to-many) are needed to access related tables from within a row context.
- When two row contexts exist on the same table, the inner hides the outer. Use VAR to capture the outer context's value before entering the inner iterator. EARLIER is the legacy alternative.
- The filter context filters the whole model through relationships automatically. Bidirectional cross-filter allows filter propagation in both directions but should be used carefully due to performance and DAX ambiguity risks.

NOTE

Next — Module 5: Understanding CALCULATE and CALCULATETABLE. CALCULATE is the most important function in DAX — it modifies the filter context. Everything in this module is the prerequisite for understanding CALCULATE. Module 5 is where the theoretical foundation built here becomes practical power.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

5

Understanding CALCULATE

and CALCULATETABLE

The most important function in DAX — master it completely.

Learn how CALCULATE modifies the filter context, the four-step evaluation model, filter overwriting vs merging, KEEPFILTERS, context transition, USERELATIONSHIP, and CALCULATE modifiers.

Prerequisite: Modules 1–4 (especially Module 4: Evaluation Contexts).

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 5: Understanding CALCULATE and CALCULATETABLE

Module Contents

■ Module Overview — Why CALCULATE Is Everything

■ Lesson 1: Introducing CALCULATE

- The Problem CALCULATE Solves
- CALCULATE Syntax and the Four-Step Evaluation Model
- Filter Arguments as Tables — Boolean Shorthand Is Syntax Sugar
- CALCULATE Overwrites Existing Filters on the Same Column
- CALCULATE Merges Filters on Different Columns

■ Lesson 2: CALCULATE and ALL — Removing Filters

- Using ALL as a CALCULATE Filter Argument
- ALL on a Column vs ALL on a Table
- ALL on Multiple Tables
- ALL as REMOVEFILTER — The CALCULATE Modifier Distinction

■ Lesson 3: Building Percentage Measures with CALCULATE

- Percentage of Grand Total — Category Level
- When the Denominator Is Wrong — Diagnosing the Filter
- Combining ALL and VALUES
- The ALLSELECTED Pattern — Percentage of Visible Total

■ Lesson 4: KEEPFILTERS — Intersecting Instead of Overwriting

- Default CALCULATE Behaviour: Overwrite
- KEEPFILTERS: Add the New Filter to the Existing One
- KEEPFILTERS with Multiple Values
- When to Use KEEPFILTERS

■ Lesson 5: Filter Arguments — Multi-column Filters and Table Filters

- Single-column Boolean Predicates — When They Work
- Multi-column Conditions — Must Use Explicit FILTER
- Table Filters — Powerful but Dangerous
- Evaluation Order of CALCULATE Filter Arguments
- Nested CALCULATE — Inner Filter Wins

■ Lesson 6: Context Transition

- What Is Context Transition?
- CALCULATE Invalidates the Row Context
- CALCULATE Adds All Row Context Columns as Filter Arguments
- Measure References Always Trigger Context Transition
- Context Transition in Calculated Columns
- Circular Dependencies and the Row Identifier Solution

■ Lesson 7: CALCULATETABLE

- CALCULATETABLE Syntax and Purpose
- When to Use CALCULATETABLE vs CALCULATE

■ Lesson 8: CALCULATE Modifiers

- ALL as REMOVEFILTER — Modifier vs Table Function

- USERRELATIONSHIP — Activating an Inactive Relationship
- CROSSFILTER — Changing Relationship Direction Temporarily
- Modifier Precedence Rules

■ **Demonstration 1: CALCULATE in Action — Contoso Brand Comparisons**

■ **Demonstration 2: Context Transition — The Hidden Danger**

■ **Lab 5: Understanding CALCULATE and CALCULATETABLE**

- Exercise 1: Core CALCULATE — Overwrite, Merge, and Boolean Syntax
- Exercise 2: Percentage Measures with ALL and VALUES
- Exercise 3: KEEPFILTERS and Multi-column Filters
- Exercise 4: Context Transition in Calculated Columns and Measures
- Exercise 5: CALCULATETABLE and USERRELATIONSHIP

■ **Module Review and Takeaways**

Module Overview — Why CALCULATE Is Everything

Every DAX concept covered so far has been building toward this module. CALCULATE is the most important, most powerful, and most complex function in DAX. It is the **only** function that can modify the filter context. Without it, you can only read the filter context the report creates — you cannot change it. With it, you can compute any comparison, ratio, running total, year-over-year change, or segment-versus-whole analysis that business intelligence requires.

The function itself has a simple description: it evaluates an expression in a modified filter context. The complexity comes from understanding exactly *how* it modifies the filter context, what happens when it interacts with existing filters, and the critically important side effect called **context transition**.

CONCEPT

CALCULATE is the only function in DAX that can: (1) replace an existing filter on a column, (2) add a new filter to the filter context, (3) remove a filter from the filter context (via ALL), (4) activate an inactive relationship (via USERELATIONSHIP), and (5) perform context transition — converting a row context into a filter context. All other DAX functions either read the current context or create a row context.

EXAM

CALCULATE is tested extensively in PL-300. Key scenarios: computing a measure for a fixed subset of data regardless of slicers, percentage-of-total and percentage-of-parent calculations, context transition behaviour when calling a measure from a calculated column, and the difference between ALL as a table function vs ALL as a CALCULATE modifier.

LESSON 1

Introducing CALCULATE

Lesson Objectives

After completing this lesson, you will be able to:

- Explain the problem CALCULATE solves that cannot be solved with FILTER alone.
- Write a CALCULATE expression and describe each component.
- Explain the four-step CALCULATE evaluation model.
- Explain why Boolean filter arguments are syntax sugar for FILTER(ALL(column), condition).
- Predict whether CALCULATE will overwrite or merge with an existing filter.

The Problem CALCULATE Solves

Suppose you need a measure that always computes the Contoso brand's gross margin, regardless of what brand the user has selected in a slicer. Without CALCULATE, the best you can do is use FILTER to restrict the rows SUMX scans:

```
-- Works, but verbose and misses an important capability:
[Contoso GM via FILTER] :=
VAR ContosoSales =
FILTER ( Sales, RELATED ( 'Product'[Brand] ) = "Contoso" )
RETURN
SUMX ( ContosoSales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )

-- Problem 1: Verbose – you must rewrite the full aggregation formula.
-- Problem 2: It cannot reuse existing measures like [Gross Margin] or [GM %].
-- Problem 3: [GM %] = DIVIDE([Gross Margin],[Sales Amount]) depends on BOTH measures
-- being filtered to Contoso. FILTER on Sales alone does not achieve this cleanly.

-- CALCULATE solves all three problems:
[Contoso GM] := CALCULATE ( [Gross Margin], 'Product'[Brand] = "Contoso" )
[Contoso GM%]:= CALCULATE ( [GM %], 'Product'[Brand] = "Contoso" )
-- Reuses existing measures. Concise. The filter context change applies to everything
-- inside the measure, including nested measure references.
```

CALCULATE Syntax

```
CALCULATE (
, -- The expression to evaluate in the new filter context
[], -- One or more filter arguments (optional)
[], -- Each filter argument is a TABLE or a Boolean predicate
...
)

-- CALCULATETABLE has identical syntax but returns a table instead of a scalar:
CALCULATETABLE (
,
[], [], ...
)
```

The Four-Step CALCULATE Evaluation Model

Every time CALCULATE executes, it follows exactly these four steps in order:

Step	Action	Notes
	Copy the current filter context.	CALCULATE starts with a copy of the existing filter context — it never discards it entirely.
2	Evaluate each filter argument outside the new context.	Filter arguments are evaluated in the ORIGINAL filter context, not the modified one. This is the 'outside-in' evaluation order.
	Apply filter modifiers (ALL, USERELATIONSHIP, CROSSFILTER) first.	CALCULATE modifiers take precedence over filter arguments. They modify the copied context before filter arguments are applied.
4	Apply filter arguments — overwrite or merge, then evaluate the expression.	Each filter argument either replaces an existing filter on the same column or adds a new filter. The expression is then evaluated in the final context.

Boolean Filter Arguments Are Syntax Sugar

When you write a Boolean condition as a CALCULATE filter argument, DAX automatically expands it into a FILTER(ALL(column), condition) expression. The two forms are completely equivalent:

```
-- Short form (Boolean predicate -- syntax sugar):
[Contoso Sales] := CALCULATE ( [Sales Amount], 'Product'[Brand] = "Contoso" )

-- Exactly equivalent expanded form:
[Contoso Sales] :=
CALCULATE (
[Sales Amount],
FILTER (
ALL ( 'Product'[Brand] ), -- All brand values (ignores existing brand filter)
'Product'[Brand] = "Contoso"
)
)

-- The ALL() around the column is critical: it means the filter argument
-- provides the FULL list of valid values for Brand = {Contoso},
-- regardless of what Brand filter was in place before.
-- This is why CALCULATE overwrites existing filters on the same column.

-- IMPORTANT LIMITATION of Boolean syntax:
-- You can only reference ONE column in a Boolean predicate.
-- This works: 'Product'[Brand] = "Contoso"
-- This FAILS: Sales[Quantity] * Sales[Net Price] >= 1000 (two columns)
-- For multi-column conditions, use explicit FILTER (see Lesson 5).
```

CALCULATE Overwrites vs Merges Filters

```
-- SCENARIO: Report has a slicer with Brand = 'Litware' selected.
-- Filter context entering CALCULATE: Product[Brand] = 'Litware'

-- Case 1: CALCULATE filter on the SAME column -> OVERWRITE
[Contoso Sales] := CALCULATE ( [Sales Amount], 'Product'[Brand] = "Contoso" )
-- CALCULATE filter: Brand = {Contoso}
-- Existing filter: Brand = {Litware}
-- Result context: Brand = {Contoso} <- Litware is replaced
-- [Contoso Sales] always shows Contoso sales, regardless of Brand slicer.

-- Case 2: CALCULATE filter on a DIFFERENT column -> MERGE (AND)
```

```
[Audio Sales] := CALCULATE ( [Sales Amount], 'Product'[Category] = "Audio" )  
-- CALCULATE filter: Category = {Audio}  
-- Existing filter: Brand = {Litware}  
-- Result context: Brand = {Litware} AND Category = {Audio}  
-- [Audio Sales] shows Audio sales for the Litware brand only.  
  
-- The rule: CALCULATE replaces filters on columns it touches,  
-- and leaves all other existing filters intact.
```

CONCEPT

CALCULATE does NOT replace the entire filter context. It only replaces existing filters on the specific columns targeted by its filter arguments. All other filters in the context remain active. This is why a Brand CALCULATE filter still respects a Year slicer — it only overwrites the Brand filter, not the Year filter.

LESSON 2

CALCULATE and ALL — Removing Filters**Lesson Objectives**

After completing this lesson, you will be able to:

- Use ALL as a CALCULATE filter argument to remove filters.
- Distinguish between ALL on a column, ALL on a table, and ALL on multiple tables.
- Explain the ALL as REMOVEFILTER distinction and why it matters.
- Use ALL to compute a percentage of grand total.

Using ALL as a CALCULATE Filter Argument

ALL used inside CALCULATE removes the filter on the specified column or table from the filter context. This is how you make a measure ignore a slicer:

```
-- Remove the Category filter -> denominator for % of total:
[All Category Sales] :=
CALCULATE ( [Sales Amount], ALL ( 'Product'[Category] ) )

-- In a matrix with Category on rows:
-- Each row's filter context has Product[Category] = 'Audio', 'Bikes', etc.
-- ALL('Product'[Category]) removes that filter entirely.
-- Result: always the grand total of ALL categories, on every row.

-- Now build the percentage:
[Sales Pct of Total] :=
VAR CurrentSales = [Sales Amount] -- uses current filter context
VAR TotalSales = CALCULATE ( [Sales Amount], ALL ( 'Product'[Category] ) )
RETURN DIVIDE ( CurrentSales, TotalSales )

-- Audio row: CurrentSales = Audio sales. TotalSales = ALL sales.
-- Percentage = Audio / ALL -> e.g. 28.4%
```

ALL on a Column vs ALL on a Table vs ALL on Multiple Tables

```
-- ALL on ONE column: removes filter on that column only
CALCULATE ( [Sales Amount], ALL ( 'Product'[Category] ) )
-- Product[Brand] filter is still active if present.

-- ALL on a TABLE: removes filters on ALL columns of that table
CALCULATE ( [Sales Amount], ALL ( 'Product' ) )
-- Removes Category, Brand, Color, Subcategory -- every Product column filter.
-- Useful when you want grand total ignoring ALL product dimensions.

-- ALL on MULTIPLE TABLES: removes filters from both tables simultaneously
CALCULATE ( [Sales Amount], ALL ( 'Product' ), ALL ( Customer ) )
-- Removes all Product AND all Customer filters.
-- Useful for denominators that should ignore both product and customer slicers.

-- ALL on the FACT TABLE: powerful but nuanced (uses expanded tables)
CALCULATE ( [Sales Amount], ALL ( Sales ) )
-- Removes filters that originated from Sales AND all tables related to Sales.
-- This is the broadest possible ALL -- removes virtually all filters.
-- Use with care; covered in depth in advanced modules.
```

ALL as REMOVEFILTER — The CALCULATE Modifier Distinction

When ALL appears as a top-level argument of CALCULATE (not nested inside FILTER), it does NOT behave as a table function returning all rows. Instead, it acts as a **CALCULATE modifier** — it removes filters. This distinction matters in advanced scenarios:

```
-- ALL as a CALCULATE MODIFIER (top-level CALCULATE argument):
CALCULATE ( [Sales Amount], ALL ( 'Product'[Category] ) )
-- This is equivalent to REMOVEFILTER('Product'[Category])
-- It does not 'return all categories' -- it removes the category filter.

-- ALL as a TABLE FUNCTION (nested inside FILTER inside CALCULATE):
CALCULATE (
[Sales Amount],
FILTER ( ALL ( 'Product'[Category] ), 'Product'[Category] = "Audio" )
)
-- Here ALL IS a table function: it returns all category values.
-- FILTER then restricts them to Audio.
-- This is the expanded form of the Boolean predicate shorthand.

-- Practical implication in nested CALCULATE:
CALCULATE (
CALCULATE ( [Sales Amount], ALL ( 'Date'[Year] ) ), -- MODIFIER: removes Year filter
'Date'[Year] = 2023 -- FILTER: sets Year = 2023
)
-- Step 1 (outer): copies context, adds Year=2023 filter.
-- Step 2 (inner): modifier ALL(Year) acts first -> removes Year filter.
-- Step 3 (inner): no filter arguments remain -> ALL years visible.
-- Final result: ALL years, regardless of outer Year=2023.
```

EXAM

When ALL is a top-level argument of CALCULATE it is a CALCULATE modifier (REMOVEFILTER), evaluated before filter arguments. When ALL is nested inside FILTER inside CALCULATE it is a table function. The distinction matters when CALCULATE is nested: the modifier runs before filter arguments, so it can undo filters that an outer CALCULATE applied.

LESSON 3

Building Percentage Measures with CALCULATE**Lesson Objectives**

After completing this lesson, you will be able to:

- Build a percentage of grand total measure using CALCULATE and ALL.
- Diagnose an incorrect percentage by tracing the filter context.
- Use VALUES inside CALCULATE to preserve specific dimension filters.
- Implement the ALLSELECTED pattern for percentage of visible total.

Percentage of Grand Total — Category Level

```
-- Correct percentage of grand total:
[Sales Pct of Total] :=
VAR CurrentSales = [Sales Amount]
VAR AllSales = CALCULATE ( [Sales Amount], ALL ( 'Product' ) )
RETURN DIVIDE ( CurrentSales, AllSales )

-- In a matrix with Product[Category] on rows, Product[Color] on columns:
-- CurrentSales for 'Audio / Black' = audio+black sales
-- AllSales = total of ALL products (ALL removes category AND color AND brand etc.)
-- Pct = audio+black / all -> correct proportion of grand total
--
-- BUT: if the denominator should only remove Category (not Color):
[Sales Pct of Category] :=
VAR CurrentSales = [Sales Amount]
VAR AllInCategory = CALCULATE ( [Sales Amount], ALL ( 'Product'[Category] ) )
RETURN DIVIDE ( CurrentSales, AllInCategory )
-- For 'Audio / Black': denominator = all categories but only Black color.
-- Shows: what % of Black-product sales does Audio represent?
```

Combining ALL and VALUES — Anchoring One Dimension

Sometimes you need to remove filters on some columns but keep others. The ALL / VALUES combination is the precise tool for this:

```
-- Goal: % of total, but KEEP the Year filter active in the denominator.
-- i.e. denominator = total of ALL categories within the CURRENT year.
[Sales Pct of Year Total] :=
VAR CurrentSales = [Sales Amount]
VAR AllCategoriesInYear =
CALCULATE (
[Sales Amount],
ALL ( 'Product'[Category] ), -- REMOVE category filter
VALUES ( 'Date'[Calendar Year] ) -- KEEP year filter (re-apply)
)
RETURN DIVIDE ( CurrentSales, AllCategoriesInYear )

-- How VALUES works here:
-- VALUES('Date'[Calendar Year]) is evaluated in the ORIGINAL filter context (outside CALCULATE).
-- It returns a single-row table: {2023} (if Year slicer = 2023).
-- CALCULATE then uses this as a filter argument: Year must be in {2023}.
```

```
-- Combined effect: ALL removes Category filter; VALUES re-applies Year filter.
-- Result: denominator = total of all categories for the current year only.
```

The ALLSELECTED Pattern — Percentage of Visible Total

ALLSELECTED removes filters applied by the current visual (rows/columns) but keeps filters from external sources (slicers, page filters, other visuals). This produces a percentage relative to what the user currently sees:

```
-- ALLSELECTED: denominator = total of what's currently visible
[Sales Pct of Visible Total] :=
VAR CurrentSales = [Sales Amount]
VAR VisibleTotal = CALCULATE ( [Sales Amount], ALLSELECTED ( 'Product'[Category] ) )
RETURN DIVIDE ( CurrentSales, VisibleTotal )

-- Scenario A: No slicer active, all categories visible in the matrix.
-- VisibleTotal = grand total. Percentages add to 100%.

-- Scenario B: Slicer selects only 'Audio' and 'Computers'.
-- Those are the only rows visible in the matrix.
-- ALLSELECTED removes the row-level Category filter but keeps the slicer filter.
-- VisibleTotal = Audio + Computers total (not ALL categories).
-- Percentages within the visible rows still add to 100%.

-- Compare with ALL('Product'[Category]):
-- ALL always returns grand total of everything, ignoring even slicers.
-- ALLSELECTED respects slicer context but ignores the visual's own row filter.
--
-- ALLSELECTED without arguments is also valid as a CALCULATE modifier:
CALCULATE ( [Sales Amount], ALLSELECTED() )
-- Restores the filter context that was active OUTSIDE the current visual.
-- Advanced; full semantics covered in Module 10.
```

TIP

Use `ALLSELECTED` when your percentage should stay meaningful after a user filters a slicer. Use `ALL` when the percentage should always be anchored to the absolute grand total regardless of any user selection. These are the two most common denominators for percentage measures in production reports.

LESSON 4

KEEPFILTERS — Intersecting Instead of Overwriting**Lesson Objectives**

After completing this lesson, you will be able to:

- Explain the default CALCULATE overwrite behaviour on same-column filters.
- Use KEEPFILTERS to add a filter rather than replace an existing one.
- Predict the result of KEEPFILTERS when multiple values are in the existing context.
- Choose between default CALCULATE and KEEPFILTERS for a given business requirement.

Default CALCULATE Behaviour: Overwrite

```
-- Default: CALCULATE replaces existing filter on same column.
[Audio Sales] := CALCULATE ( [Sales Amount], 'Product'[Category] = "Audio" )

-- In a matrix where Product[Category] = 'Computers' on a row:
-- Existing filter: Category = {Computers}
-- CALCULATE filter: Category = {Audio}
-- Result context: Category = {Audio} <- Computers replaced by Audio
-- [Audio Sales] shows Audio sales on EVERY category row.
-- This is correct when you want 'always show Audio sales regardless of context.'
```

KEEPFILTERS: Add Without Replacing

```
-- KEEPFILTERS: intersects the new filter with the existing one.
[Audio Sales KeepFilters] :=
CALCULATE ( [Sales Amount], KEEPFILTERS ( 'Product'[Category] = "Audio" ) )

-- In the same matrix where Category = 'Computers':
-- Existing filter: Category = {Computers}
-- KEEPFILTERS filter: Category = {Audio}
-- Result context: Category = {Computers} AND {Audio} = EMPTY SET
-- [Audio Sales KeepFilters] returns BLANK on the Computers row.

-- On the Audio row:
-- Existing filter: Category = {Audio}
-- KEEPFILTERS filter: Category = {Audio}
-- Intersection = {Audio} -> returns Audio sales.

-- Practical use case: show Audio sales only WHERE Audio is already in context.
-- If the user has NOT filtered to Audio, the measure returns blank.
-- If the user HAS filtered to Audio, it returns Audio sales.
```

KEEPFILTERS with Multiple Values

```
-- Example: CALCULATE filter covers Audio AND Computers.
[Audio-Computers Sales] :=
CALCULATE ( [Sales Amount], 'Product'[Category] IN { "Audio", "Computers" } )
-- Default: replaces existing Category filter with {Audio, Computers}.
-- All category rows show the combined Audio+Computers total.

[Audio-Computers KeepFilters] :=
CALCULATE ( [Sales Amount], KEEPFILTERS ( 'Product'[Category] IN { "Audio", "Computers" } ) )
```

```
-- KEEPFILTERS: intersects {Audio, Computers} with whatever is in context.  
-- Audio row: {Audio} AND {Audio,Computers} = {Audio} -> Audio sales  
-- Computers row: {Computers} AND {Audio,Computers} = {Computers} -> Computers sales  
-- Bikes row: {Bikes} AND {Audio,Computers} = EMPTY -> Blank  
-- Total row: all categories AND {Audio,Computers} = {Audio,Computers} -> combined
```

CONCEPT

KEEPFILTERS changes CALCULATE's behaviour from OVERWRITE to INTERSECT for the column it wraps. Think of it as: without KEEPFILTERS, the new filter wins. With KEEPFILTERS, both the old filter and the new filter must be satisfied simultaneously. If they are mutually exclusive, the result is blank.

LESSON 5

Filter Arguments — Multi-column Filters and Evaluation Order

Lesson Objectives

After completing this lesson, you will be able to:

- Explain when Boolean predicates work and when explicit FILTER is required.
- Write a multi-column FILTER condition as a CALCULATE filter argument.
- Explain why table filters (FILTER on the fact table) are dangerous.
- Describe the evaluation order of CALCULATE filter arguments.
- Predict the result of nested CALCULATE with conflicting filters.

Single-column vs Multi-column Conditions

```
-- Single-column Boolean: works as syntax sugar (one column only)
[Red Sales] := CALCULATE ( [Sales Amount], 'Product'[Color] = "Red" )

-- Multi-column condition: MUST use explicit FILTER
-- This FAILS (two columns in one Boolean predicate):
[Large Transactions] :=
CALCULATE ( [Sales Amount], Sales[Quantity] * Sales[Net Price] >= 1000 ) -- ERROR

-- CORRECT multi-column filter using explicit FILTER:
[Large Transactions] :=
CALCULATE (
[Sales Amount],
FILTER (
ALL ( Sales[Quantity], Sales[Net Price] ), -- returns all QTY/Price combinations
Sales[Quantity] * Sales[Net Price] >= 1000 -- keep combinations >= 1000
)
)
-- The filter context now contains a multi-column filter:
-- a set of (Quantity, Net Price) pairs where their product >= 1000.
-- Only Sales rows matching those exact combinations are visible.
```

Table Filters — Powerful but Dangerous

```
-- Table filter: FILTER on the fact table directly (AVOID in most cases)
[Large Transactions Dangerous] :=
CALCULATE (
[Sales Amount],
FILTER ( Sales, Sales[Quantity] * Sales[Net Price] >= 1000 )
)

-- Why it's dangerous:
-- FILTER(Sales, ...) iterates ALL of Sales (no ALL() wrapper).
-- It respects the current filter context -- meaning it's already filtered.
-- Then CALCULATE receives this pre-filtered table as its filter argument.
-- The result is a filter on Sales that interacts with existing filters
-- in complex and often unexpected ways.
-- Additionally, iterating Sales (millions of rows) inside CALCULATE
-- forces the query engine into row-by-row evaluation -- very slow.

-- Recommended: always use ALL() in the FILTER table argument:
```

```
FILTER ( ALL ( Sales[Quantity], Sales[Net Price] ), ... ) -- clean column filter
-- This produces a column-level filter (efficient) rather than a row-level filter.
```

Evaluation Order of CALCULATE Filter Arguments

CALCULATE evaluates its filter arguments in the ORIGINAL (outer) filter context, not in the modified context. This is the 'outside-in' rule and is critical for nested CALCULATE:

```
-- Filter arguments are evaluated BEFORE CALCULATE modifies the context.
-- Consider:
[Example] :=
CALCULATE (
[Sales Amount],
ColorRed, -- VAR: evaluated in the ORIGINAL context
BrandContoso -- VAR: evaluated in the ORIGINAL context
)
-- Both ColorRed and BrandContoso are evaluated outside CALCULATE,
-- before the new filter context takes effect.

-- NESTED CALCULATE: inner filter overwrites outer filter on same column.
[Sales Yellow and Black] :=
CALCULATE (
CALCULATE (
[Sales Amount],
'Product'[Color] IN { "Yellow", "Black" } -- inner: sets Color = {Yellow,Black}
),
'Product'[Color] IN { "Black", "Blue" } -- outer: sets Color = {Black,Blue}
)
-- Outer CALCULATE runs first: Color = {Black, Blue}
-- Inner CALCULATE runs second: Color = {Yellow, Black} -> OVERWRITES outer
-- Final color filter: {Yellow, Black}
-- Result: sales of Yellow + Black products

-- With KEEPFILTERS on the inner:
-- Inner KEEPFILTERS: intersect {Yellow,Black} with outer {Black,Blue} = {Black} only
-- Result: Black sales only
```

LESSON 6

Context Transition

Lesson Objectives

After completing this lesson, you will be able to:

- Define context transition and state which function causes it.
- Explain what 'CALCULATE invalidates the row context' means.
- Describe what filter arguments CALCULATE adds automatically during context transition.
- Explain why every measure reference in a row context triggers context transition.
- Identify and avoid the circular dependency trap in calculated columns using CALCULATE.

What Is Context Transition?

Context transition is a specific behaviour triggered by CALCULATE whenever it is called while a row context is active. It is automatic, invisible, and one of the most powerful — and most dangerous — features of DAX:

CONCEPT

Context transition definition: When CALCULATE executes inside a row context, it (1) invalidates the row context (column references inside CALCULATE can no longer use it), and (2) automatically adds filter arguments for ALL columns of the iterated table, each filtered to the value of that column in the current row. The combined effect converts the row context into an equivalent filter context. The filter then propagates through relationships like any other filter context.

CALCULATE Adds All Row Context Columns as Hidden Filter Arguments

```
-- SUMX creates a row context. Inside it, CALCULATE triggers context transition.
[Sum Num Of Sales] := SUMX ( Sales, COUNTROWS ( Sales ) )
-- For each Sales row, row context has: Product='A', Quantity=1, Net Price=11, ...
-- COUNTROWS(Sales) uses the FILTER CONTEXT (empty outside the iteration)
-- -> returns the total count of all Sales rows, on EVERY row.
-- Result: NumOfSales squared (e.g. 37984 * 37984).

-- Wrap COUNTROWS in CALCULATE to trigger context transition:
[Sum Num Of Sales CT] := SUMX ( Sales, CALCULATE ( COUNTROWS ( Sales ) ) )
-- Now CALCULATE triggers context transition.
-- For each row (e.g. Product='A', Quantity=1, Net Price=11.00):
-- CALCULATE auto-adds hidden filters:
-- Sales[Product] = 'A'
-- Sales[Quantity] = 1
-- Sales[Net Price] = 11.00
-- ... (all Sales columns)
-- COUNTROWS(Sales) now sees only rows matching ALL those values.
-- Result: 1 row per unique combination -> sums to total rows = 37,984.
```

Measure References Always Trigger Context Transition

This is the most important implication of context transition for everyday DAX. Every time you reference a measure inside a row context (a calculated column or an iterator), DAX implicitly wraps the measure in CALCULATE, triggering context transition automatically:

```
-- A measure reference in a calculated column:
'Product'[Performance] =
```

```

VAR TotalSales = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- no CT
VAR CurrentSales = [Sales Amount] -- measure reference -> implicit CALCULATE -> CT!
RETURN IF ( CurrentSales >= TotalSales * 0.01, "High", "Regular" )

-- [Sales Amount] := SUMX(Sales, Sales[Quantity]*Sales[Net Price])
-- When called from a row context in Product:
-- Context transition fires: adds Product[ProductKey] = as filter.
-- The filter propagates through Product->Sales relationship.
-- SUMX inside [Sales Amount] only iterates Sales rows for THIS product.
-- CurrentSales = sales of the current product. Correct!

-- TotalSales (no CALCULATE, no measure ref) = iterates ALL of Sales.
-- No context transition -> TotalSales = grand total. Also correct!

-- DANGER: using [Sales Amount] for BOTH variables:
'Product'[Performance Wrong] =
VAR TotalSales = [Sales Amount] -- CT fires -> this product's sales, not grand total!
VAR CurrentSales = [Sales Amount] -- CT fires -> same product's sales
RETURN IF ( CurrentSales >= TotalSales * 0.01, "High", "Regular" )
-- TotalSales and CurrentSales are IDENTICAL. Every product appears 'High Performance'.
-- The fix: use explicit SUMX (no CT) for TotalSales.

```

Context Transition — The Performance Rule

Context transition in calculated columns on large tables is expensive. When CALCULATE is called in a row context, it filters ALL columns of the iterated table. On a table with many columns, this generates a large, complex filter context per row. Always be aware of where context transition is happening:

```

-- SAFE: context transition on Product (small table, unique ProductKey)
'Product'[ProductSales] = CALCULATE ( [Sales Amount] )
-- ProductKey is the primary key -> context transition filters to exactly one product.
-- Performance: acceptable (small table, unique key).

-- RISKY: context transition on Sales (large fact table, no unique key)
Sales[SomeMeasure] = CALCULATE ( [SomeExpensiveMeasure] )
-- Sales has NO primary key (multiple columns needed to identify a row).
-- Context transition adds ALL Sales columns as filter arguments.
-- Creates a massive, complex filter context for each of potentially millions of rows.
-- Performance: can be catastrophically slow.
-- Also triggers circular dependency warnings (see next section).

```

Circular Dependencies and the Row Identifier Solution

When CALCULATE is used in a calculated column of a table without a unique key column, DAX detects that the column being calculated would be included in its own filter arguments (since context transition filters ALL columns). This creates a circular dependency:

```

-- CIRCULAR DEPENDENCY example:
Sales[AllSalesQty] = CALCULATE ( SUM ( Sales[Quantity] ) )
-- Context transition would auto-add: Sales[AllSalesQty] =
-- But AllSalesQty is what we're computing -- DAX cannot compute it if it
-- depends on its own current value. ERROR: circular dependency detected.

-- SOLUTIONS:
-- (1) Mark a column as the row identifier (unique key) in the table settings.
-- Context transition then only uses the key column, not all columns.

```



```
-- Available in Analysis Services; limited in Power BI.

-- (2) Use a table with a natural unique key (like Product with ProductKey).
-- The DAX engine auto-detects columns on the one-side of relationships
-- as unique identifiers. Context transition only uses those key columns.
-- 'Product'[ProductSales] = CALCULATE([Sales Amount]) -- works cleanly.

-- (3) Avoid CALCULATE / measure references in calculated columns on large tables.
-- Use SUMX with explicit RELATED/RELATEDTABLE instead.
```

LESSON 7

CALCULATETABLE

Lesson Objectives

After completing this lesson, you will be able to:

- State the difference between CALCULATE and CALCULATETABLE.
- Write a CALCULATETABLE expression to produce a filtered table.
- Identify scenarios where CALCULATETABLE is needed instead of CALCULATE.

CALCULATETABLE — Same Semantics, Table Result

```
-- CALCULATE returns a scalar (number, text, date, etc.)
[Contoso Sales] := CALCULATE ( [Sales Amount], 'Product'[Brand] = "Contoso" )

-- CALCULATETABLE returns a TABLE (can be used where a table is expected)
ContosoProducts = CALCULATETABLE ( 'Product', 'Product'[Brand] = "Contoso" )
-- Returns the Product table filtered to Contoso brand.
-- Same filter mechanics as CALCULATE: overwrites, merges, KEEPFILTERS, etc.

-- CALCULATETABLE as a filter argument:
[Sales of Contoso Products] :=
CALCULATE (
[Sales Amount],
CALCULATETABLE ( 'Product', 'Product'[Brand] = "Contoso" )
)
-- Filters Sales to only rows whose related Product is in the Contoso subset.

-- Common use: provide a filtered table to SUMX or other iterators:
[Red Product Count] :=
COUNTROWS (
CALCULATETABLE ( 'Product', 'Product'[Color] = "Red" )
)
-- Counts red products in the current filter context.

-- Key distinction from FILTER:
-- FILTER(table, condition) iterates the table row by row (uses row context)
-- CALCULATETABLE(table, condition) modifies the filter context (uses CALCULATE semantics)
-- Both return tables, but their behaviour differs with relationships and context.
```

CONCEPT

Use CALCULATETABLE when you need a table expression that uses CALCULATE semantics — filter context modification, KEEPFILTERS, USERELATIONSHIP, etc. Use FILTER when you need to iterate a table with a condition that requires a row context (column references, RELATED). In many simple cases both produce the same result, but they differ when the condition involves relationships or when performance matters.

LESSON 8

CALCULATE Modifiers**Lesson Objectives**

After completing this lesson, you will be able to:

- List all CALCULATE modifiers and describe what each does.
- Use USERELATIONSHIP to activate an inactive relationship in a measure.
- Use CROSSFILTER to change a relationship's cross-filter direction temporarily.
- State the precedence rule: modifiers are applied before filter arguments.

CALCULATE Modifiers — Overview

CALCULATE modifiers are special functions that can only appear as top-level arguments of CALCULATE. They modify the model structure (relationships, filter direction) rather than adding column filters. They are always applied before any filter arguments:

Modifier	What It Does	Typical Use Case
ALL (table/column)	Removes all filters on the specified table or column (REMOVEFILTER semantics).	Denominators for percentages; grand total calculations.
ALLSELECTED (table/column)	Removes visual-level filters but keeps external filters (slicers, page filters).	Percentage of visible total; respects user slicer context.
KEEPFILTERS (filter)	Wraps a filter argument to intersect rather than overwrite existing filters.	Show values only when a specific filter is already active.
USERELATIONSHIP (col1, col2)	Temporarily activates an inactive relationship, deactivating the current active one.	Role-playing dimensions (e.g., Order Date vs Delivery Date on the same Date table).
CROSSFILTER (col1, col2, direction)	Changes the cross-filter direction of a relationship temporarily. Direction: BOTH, ONE, or NONE.	Force bidirectional filtering for one specific calculation without changing the model.

USERELATIONSHIP — Role-playing Dimensions

```
-- Model: Sales has OrderDate and DeliveryDate columns.
-- Date table connected to Sales[OrderDate] (ACTIVE relationship).
-- Date table also connected to Sales[DeliveryDate] (INACTIVE relationship).

-- Default measure uses the ACTIVE relationship (Order Date):
[Sales Amount] := SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
-- Slicing by Date[Year] filters by order year.

-- Measure using INACTIVE relationship (Delivery Date):
[Delivered Amount] :=
CALCULATE (
[Sales Amount],
USERELATIONSHIP ( Sales[DeliveryDate], 'Date'[Date] )
)
-- USERELATIONSHIP is a CALCULATE modifier -- applied before filter arguments.
-- It activates Sales[DeliveryDate] <-> Date[Date] and deactivates the OrderDate one.
-- [Sales Amount] inside now responds to Date[Year] by DELIVERY year.
```

```
-- IMPORTANT: USERELATIONSHIP is processed before filter arguments.
-- The CALCULATE inner expression is evaluated with the new relationship active.
-- Filter arguments (e.g., a year filter) are applied to the modified model.
```

CROSSFILTER — Temporary Bidirectional Filtering

```
-- Force bidirectional filtering for one calculation only:
[Customers Buying Red] :=
CALCULATE (
    DISTINCTCOUNT ( Sales[CustomerKey] ),
    'Product'[Color] = "Red",
    CROSSFILTER ( Sales[CustomerKey], Customer[CustomerKey], BOTH )
)
-- Without CROSSFILTER: Product filter does not propagate to Customer table
-- (if Customer->Sales is single-direction).
-- With CROSSFILTER BOTH: filter can travel Product->Sales->Customer.
-- Result: count of distinct customers who bought red products.

-- CROSSFILTER directions:
-- BOTH -> bidirectional (filters flow both ways)
-- ONE -> single direction (from column2 to column1, or left to right)
-- NONE -> disables the relationship entirely for this calculation
```

EXAM

Modifier precedence: CALCULATE modifiers (ALL, USERELATIONSHIP, CROSSFILTER, ALLSELECTED) are always applied FIRST, before any filter arguments. This means USERELATIONSHIP changes the relationship architecture before filters are applied, and ALL removes filters before new filter arguments add them back. Understanding this order is critical for nested CALCULATE analysis.

DEMONSTRATION 1

CALCULATE in Action — Contoso Brand Comparisons

Scenario: The instructor builds a brand comparison report for Contoso Retail, demonstrating filter overwrite, merge, KEEPFILTERS, and the percentage patterns using the Contoso model from the lab series.

Demonstration Steps

1	Open Contoso_Demo05_Start.pbix. Add measure: [Contoso Sales] := CALCULATE([Sales Amount], 'Product'[Brand]="Contoso"). Place on a matrix with Brand on rows. Show it returns Contoso sales on EVERY row — CALCULATE overwrites the Brand filter from the report.
2	Add a Category slicer. Select 'Audio'. Show [Contoso Sales] now shows Contoso Audio sales — Category filter merges (different column). Confirm with [Contoso Sales KeepFilters] := CALCULATE([Sales Amount], KEEPFILTERS('Product'[Brand]="Contoso")). Compare: KeepFilters returns Blank on non-Contoso brand rows.
3	Build the percentage trio: [Sales Pct ALL] using ALL('Product'), [Sales Pct Category] using ALL('Product'[Category]), [Sales Pct Visible] using ALLSELECTED('Product'[Category]). Place all three on a matrix. Apply a Category slicer and show how each denominator behaves differently.
4	Demonstrate [Sales Amount] vs [Contoso Sales] in a matrix with both Category and Year. Trace one cell: Audio/2023. Filter context entering CALCULATE = {Category=Audio, Year=2023}. CALCULATE adds Brand=Contoso -> replaces existing Brand filter (none) -> adds Brand filter. Year and Category remain: result = Contoso Audio 2023 sales.
5	Add [Delivered Amount] using USERELATIONSHIP(Sales[DeliveryDate], 'Date'[Date]). Compare with [Sales Amount] on a matrix with Date[Year] on rows. Show that the years differ: delivered totals may differ from ordered totals.
RESULT	Five demonstrations complete. Overwrite vs merge behaviour confirmed. KEEPFILTERS intersection demonstrated. Three percentage patterns compared. USERELATIONSHIP behaviour shown.

DEMONSTRATION 2

Context Transition — The Hidden Danger

Scenario: The instructor demonstrates context transition in calculated columns and measures, including the correct and incorrect use of measure references inside a row context, and how to diagnose context transition issues.

Demonstration Steps

1	In the Product table, add calculated column 'Product'[PerformanceCorrect] using TotalSales=SUMX and CurrentSales=[Sales Amount] (measure ref). Show correct results — context transition on the measure ref filters to current product via ProductKey.
2	Add 'Product'[PerformanceWrong] using BOTH TotalSales AND CurrentSales as measure references. Show every product is classified 'High Performance' — context transition fires on both, both return the same per-product value. TotalSales is not the grand total.
3	Show the fix: replace TotalSales with SUMX(Sales, Sales[Quantity]*Sales[Net Price]) — no CALCULATE, no CT, truly iterates all Sales. Verify results are now correct.
4	Demonstrate Sum Num Of Sales := SUMX(Sales, COUNTROWS(Sales)). Explain why result is NumOfSales^2 (no CT, COUNTROWS sees entire table). Fix with CALCULATE inside: SUMX(Sales, CALCULATE(COUNTROWS(Sales))). CT converts each row context to a filter -> COUNTROWS sees only matching rows -> sums to total.
5	Open Performance Analyzer. Compare the two versions of the calculated column — the measure reference version vs the explicit SUMX version. Show that the explicit SUMX is faster on large models because it avoids context transition overhead.
RESULT	Context transition demonstrated: measure reference in row context triggers CT. Wrong use of measure refs for 'grand total' identified and fixed. SUMX inside CALCULATE CT mechanics verified. Performance impact noted.

LAB 5

Understanding CALCULATE and CALCULATETABLE

Lab Scenario

The Contoso analytics team needs a suite of comparison measures for the executive dashboard. You will implement brand comparisons, percentage measures, a context-transition-safe calculated column, and a delivery-date analysis using USERELATIONSHIP. You will also diagnose and fix two broken measures that misuse context transition.

NOTE

Open Contoso_Lab05_Start.pbix. This file contains the completed data model from Labs 1–4, with all tables, relationships, and base measures. The model includes an inactive relationship: Sales[DeliveryDate] <-> Date[Date]. Save your work as Contoso_Lab05_Complete.pbix.

Exercise 1: Core CALCULATE — Overwrite, Merge, and Boolean Syntax

Estimated time: 20 minutes

Task 1: Build brand comparison measures

1. Add the following three measures to the Sales table:

- a. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[Contoso Sales] :=
CALCULATE ([Sales Amount], \Product\[Brand] = "Contoso") 'frags': [ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Contoso Sales] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='CALCULATE ([Sales Amount], \Product\[Brand] = "Contoso")', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
- b. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[Contoso GM%] :=
CALCULATE ([Gross Margin %], \Product\[Brand] = "Contoso") 'frags': [ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Contoso GM%] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='CALCULATE ([Gross Margin %], \Product\[Brand] = "Contoso")', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0) #Paragraph
- c. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': '[Non-Contoso Sales] :=
CALCULATE ([Sales Amount], \Product\[Brand] <> "Contoso") 'frags': [ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Non-Contoso Sales] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="CALCULATE ([Sales Amount], 'Product\[Brand] ", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='<', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0, fontFamily='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='> "Contoso")',

```
textColor=Color(.101961,.227451,.360784,1), us_lines=[])) 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Add a Matrix visual. Rows = Product[Category], Values = [Sales Amount], [Contoso Sales], [Non-Contoso Sales]. Verify [Contoso Sales] returns Contoso sales on every category row. Verify [Sales Amount] + [Non-Contoso Sales] is NOT equal to [Contoso Sales] + [Non-Contoso Sales] because [Sales Amount] uses the full category filter while the others override Brand.
3. Add a Brand slicer. Select 'Fabrikam'. Observe [Contoso Sales] still shows Contoso sales — CALCULATE overwrites the slicer. Observe [Sales Amount] shows only Fabrikam sales — the report filter is respected.

Task 2: Verify filter merge on a different column

1. Add measure: [Audio Contoso] := CALCULATE([Sales Amount], 'Product'[Category]="Audio", 'Product'[Brand]="Contoso"). Place on a Card. With no slicer active, verify it shows Contoso Audio sales. Apply a Year slicer. Verify it updates — Year is on a different table and merges.

Results: Brand comparison measures confirmed. CALCULATE overwrites the Brand slicer. Category and Year filters merge because they are on different columns from the Brand filter.

Exercise 2: Percentage Measures with ALL and VALUES

Estimated time: 25 minutes

Task 1: Build three percentage measures

1. Add the following measures:

- a. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales % of Grand Total] :=
VAR Current = [Sales Amount]
VAR Total = CALCULATE ([Sales Amount], ALL ('Product'))
RETURN DIVIDE (Current, Total)" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier',
fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Sales % of Grand Total] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Current = [Sales Amount]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Total = CALCULATE (
[Sales Amount], ALL ('Product'))', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN DIVIDE (Current, Total)', textColor=Color(.101961,.227451,.360784,1), us_lines=[])]
'style': 'bulletText': None 'debug': 0) #Paragraph
- b. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales % of Category Total] :=
VAR Current = [Sales Amount]
VAR CategoryTotal = CALCULATE ([Sales Amount], ALL ('Product'[Category]))
RETURN DIVIDE (Current, CategoryTotal)" 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Sales % of Category Total] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,


```

fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Current = [Sales Amount]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR CategoryTotal =
CALCULATE ( [Sales Amount], ALL ( 'Product'[Category] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN DIVIDE ( Current,
CategoryTotal )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None
'debug': 0 ) #Paragraph

```

c. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales % of Visible Total] :=

```

VAR Current = [Sales Amount]
VAR VisibleTotal = CALCULATE ( [Sales Amount], ALLSELECTED ( 'Product'[Category] ) )
RETURN DIVIDE ( Current, VisibleTotal )" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier',
fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Sales % of Visible Total] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Current = [Sales Amount]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR VisibleTotal =
CALCULATE ( [Sales Amount], ALLSELECTED ( 'Product'[Category] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN DIVIDE ( Current,
VisibleTotal )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None
'debug': 0 ) #Paragraph

```

- Place all three on a Matrix with Product[Category] on rows.
- With no slicer: verify all three sum to 100% at the Grand Total row.
- Apply a Category slicer: select 'Audio' and 'Computers' only. Observe: (a) [Sales % of Grand Total] percentages drop — denominator is still all sales. (b) [Sales % of Category Total] and (c) [Sales % of Visible Total] percentages add to 100% within the visible rows.
- Add measure using ALL+VALUES:

. Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales % within Year] :=

```

VAR Current = [Sales Amount]
VAR YearTotal = CALCULATE (
[Sales Amount],
ALL ( 'Product'[Category] ),
VALUES ( 'Date'[Year] )
)
RETURN DIVIDE ( Current, YearTotal )" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier',
fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Sales % within Year] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,

```

```

fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Current = [Sales Amount]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR YearTotal = CALCULATE
(', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Sales Amount]',',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" ALL ( 'Product'[Category] ),",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" VALUES ( 'Date'[Year] )",',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=')',',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN DIVIDE ( Current,
YearTotal )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug':
0 ) #Paragraph

```

6. Add a Year slicer. Select 2023. Verify [Sales % within Year] percentages sum to 100% within 2023 only and the denominator ignores Category but respects the Year.

Results: Three percentage patterns demonstrated and compared. ALL vs ALLSELECTED vs ALL+VALUES behaviour confirmed with slicer interaction.

Exercise 3: KEEPFILTERS and Multi-column Filters

Estimated time: 20 minutes

Task 1: Implement KEEPFILTERS

1. Add measure:

```

. Paragraph ( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Audio Sales KeepFilters] :=
CALCULATE ( [Sales Amount], KEEPFILTERS ( \'Product\[Category] = "Audio" ) ) ' 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='[Audio Sales KeepFilters] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CALCULATE ( [Sales Amount], KEEPFILTERS ( \'Product\[Category] = "Audio" ) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Compare on a matrix with Product[Category] on rows: [Audio Sales] (without KEEPFILTERS) vs [Audio Sales KeepFilters]. Confirm: [Audio Sales] returns same value on every row. [Audio Sales KeepFilters] returns Audio value on Audio row, Blank on all other rows.

Task 2: Build a multi-column CALCULATE filter**1. Add measure to count high-value transactions:**

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[High Value Orders] :=
    CALCULATE (
    [Order Count],
    FILTER (
    ALL ( Sales[Quantity], Sales[Net Price] ),
    Sales[Quantity] * Sales[Net Price] >= 500
    )
    ) 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
    rise=0, text='[High Value Orders] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
    ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
    link=[], rise=0, text='', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
    ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
    text='CALCULATE (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
    bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Order Count],',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' FILTER (',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ALL ( Sales[Quantity], Sales[Net
    Price] ),', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' Sales[Quantity] * Sales[Net
    Price] >= 500', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
    bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' )',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text='',
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=''),
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]) ] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Place on a Card. Verify it counts transactions where quantity * price >= 500. Verify it respects a Category slicer — Category is on a different table and merges.**3. Attempt the single-column Boolean shorthand: `CALCULATE( [Order Count], Sales[Quantity]*Sales[Net Price]>=500)`. Observe the error: multi-column condition cannot use Boolean shorthand. Delete the broken attempt.**

Results: KEEPFILTERS intersection confirmed. Multi-column filter with explicit FILTER works correctly. Boolean shorthand fails for multi-column conditions as expected.

Exercise 4: Context Transition in Calculated Columns and Measures

Estimated time: 20 minutes

Task 1: Build a correct performance classification column

1. In the **Product** table, add a calculated column:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '\Product\[PerformanceTier] =
VAR GrandTotalSales =
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- NO context transition
VAR ThisProductSales =
CALCULATE ( SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) ) -- CT applies
RETURN
SWITCH (
TRUE (),
ThisProductSales >= GrandTotalSales * 0.05, "Tier 1 — Top 5%",
ThisProductSales >= GrandTotalSales * 0.01, "Tier 2 — Top 1%",
"Tier 3 — Standard"
)' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
rise=0, text=""Product\[PerformanceTier] =", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR GrandTotalSales =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- NO context transition',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR ThisProductSales =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' CALCULATE ( SUMX ( Sales,
Sales[Quantity] * Sales[Net Price] ) ) -- CT applies', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' SWITCH (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' TRUE (),',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ThisProductSales >=
GrandTotalSales * 0.05, "Tier 1 — Top 5%",', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
```

```

ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' ThisProductSales >= GrandTotalSales * 0.01, "Tier 2 — Top 1%"',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' "Tier 3 — Standard"',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' '),
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Switch to Data view. Sort by [PerformanceTier]. Verify Tier 1 products genuinely have the highest individual product sales.

Task 2: Diagnose a broken measure reference

1. Add a WRONG version to observe the problem:

```

. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '\Product\[PerformanceTierWrong] =
VAR GrandTotalSales = [Sales Amount] -- measure ref -> CT fires -> per-product sales!
VAR ThisProductSales= [Sales Amount] -- same CT -> same result
RETURN IF ( ThisProductSales >= GrandTotalSales * 0.01, "High", "Regular" )' 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=""Product\[PerformanceTierWrong] =", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR GrandTotalSales = [Sales Amount] -- measure ref -> CT fires -> per-product sales!',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR ThisProductSales= [Sales
Amount] -- same CT -> same result', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN IF ( ThisProductSales >= GrandTotalSales * 0.01, "High", "Regular" )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Observe: every single product is classified 'High'. This is because both variables use the same measure reference which triggers CT, making both variables return the same per-product value. GrandTotalSales is NOT the grand total in this context.
3. Delete the wrong column after confirming the difference.

Results: PerformanceTier correctly classifies products using explicit SUMX for grand total and CALCULATE for current product (context transition). PerformanceTierWrong demonstrates the classic CT trap.

Exercise 5: CALCULATETABLE and USERELATIONSHIP

Estimated time: 20 minutes

Task 1: Use CALCULATETABLE as a variable

1. Add a measure that uses CALCULATETABLE to get Contoso products:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Contoso Product Count] :=
    VAR ContosoProducts =
        CALCULATETABLE ( \Product\, \Product\[Brand] = "Contoso" )
    RETURN COUNTROWS ( ContosoProducts )' 'frags': [ParaFrag(__tag__='para', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Contoso Product Count] :=',
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR ContosoProducts =',
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' CALCULATETABLE (
        \Product\, \Product\[Brand] = "Contoso" )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
        ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
        link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
        ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
        text='RETURN COUNTROWS ( ContosoProducts )', textColor=Color(.101961,.227451,.360784,1),
        us_lines=[])] 'style': 'bulletText': None 'debug': 0 ) #Paragraph
```

2. Place on a Card. Apply a Category slicer. Verify the count updates — the CALCULATETABLE inherits the filter context including Category.

Task 2: Build a Delivery Date measure with USERELATIONSHIP

1. The model has an inactive relationship: Sales[DeliveryDate] <-> Date[Date]. Verify this in Model view — it appears as a dashed line.

2. Add a measure:

```
. Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Delivered Amount] :=
    CALCULATE (
        [Sales Amount],
        USERELATIONSHIP ( Sales[DeliveryDate], 'Date'[Date] )
    )" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
        rise=0, text='[Delivered Amount] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
        ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
        link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
        ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
        text='CALCULATE (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
        bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Sales Amount],',
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
        textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
        fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" USERELATIONSHIP (
        Sales[DeliveryDate], 'Date'[Date] )", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
        ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
```

```
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),  
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,  
text=')', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )  
#Paragraph
```

3. Add a Matrix. Rows = Date[Year], Values = [Sales Amount], [Delivered Amount]. Observe the differences — some orders placed in year Y are delivered in year Y+1. The two measures may differ significantly for the first and last years.
4. Save as **Contoso_Lab05_Complete.pbix**.

Results: CALCULATETABLE correctly returns a filtered table in the current context. USERELATIONSHIP activates the inactive DeliveryDate relationship. Ordered vs Delivered amounts differ as expected across year boundaries.

Module Review and Takeaways

Review Questions

What are the four steps CALCULATE performs when it evaluates?

Answer: 1. Copy the current filter context. 2. Evaluate filter arguments in the original (outer) filter context. 3. Apply CALCULATE modifiers (ALL, USERELATIONSHIP, CROSSFILTER) — these run first. 4. Apply filter arguments: replace existing filters on the same columns, add new filters on new columns. Evaluate the expression in the resulting context.

CALCULATE has a Boolean filter argument 'Product[Brand] = "Contoso"'. The report already has a Brand slicer with 'Fabrikam' selected. What is the result?

Answer: CALCULATE overwrites the existing Brand filter. The filter argument expands to `FILTER(ALL(Product[Brand]), Product[Brand]="Contoso")`, which provides the full list of valid brands as {Contoso}, replacing the slicer's {Fabrikam}. The measure evaluates as if Brand = Contoso, ignoring the slicer selection.

What does KEEPFILTERS do, and when should you use it?

Answer: KEEPFILTERS wraps a CALCULATE filter argument to change its behaviour from overwrite to intersect. Instead of replacing an existing filter on the same column, CALCULATE merges both filters using AND (intersection). If the two filters are mutually exclusive, the result is blank. Use KEEPFILTERS when you want the measure to return a value only when the requested filter is already present in the context — for example, showing Audio sales only when Audio is selected.

Explain context transition. What triggers it, and what does it do?

Answer: Context transition is triggered by CALCULATE whenever a row context is active. It has two effects: (1) it invalidates the current row context inside CALCULATE (column references can no longer use it directly), and (2) it automatically adds all columns of the iterated table as filter arguments, each set to the current row's value. The combined effect converts the row context into an equivalent filter context, which then propagates through relationships like any other filter context.

Why is it dangerous to use a measure reference as the 'grand total' variable in a calculated column?

Answer: Every measure reference in a row context implicitly wraps the measure in CALCULATE, triggering context transition. This means the measure is evaluated in a filter context derived from the current row — not in an empty context. If you use [Sales Amount] as a 'grand total', context transition filters it to the current row's product, returning that product's sales — the same value as the 'current product' variable. The fix: use explicit SUMX without CALCULATE for the variable that should return the true grand total.

What is CALCULATETABLE? How does it differ from CALCULATE?

Answer: CALCULATETABLE has identical semantics to CALCULATE — same four-step evaluation, same filter argument rules, same modifiers. The only difference is that CALCULATETABLE returns a table rather than a scalar value. Use CALCULATETABLE when you need a table expression in a context where CALCULATE semantics (filter modification, KEEPFILTERS, USERELATIONSHIP) are required.

What does USERELATIONSHIP do, and when is it needed?

Answer: USERELATIONSHIP is a CALCULATE modifier that temporarily activates an inactive relationship in the model and deactivates the currently active relationship between the same two tables. It is needed when a table plays multiple roles (role-playing dimension) — for example, a Date table connected to Sales via both

OrderDate and DeliveryDate. Only one relationship can be active at a time; USERELATIONSHIP switches which one is active for the duration of the CALCULATE evaluation.

What is the precedence rule for CALCULATE modifiers vs filter arguments?

Answer: CALCULATE modifiers (ALL, ALLSELECTED, USERELATIONSHIP, CROSSFILTER, KEEPFILTERS) are always applied before filter arguments. This means USERELATIONSHIP changes the relationship architecture before filters are applied, and ALL removes filters before new filter arguments add them back. In nested CALCULATE, outer filter arguments are applied first; inner filter arguments override them (inner wins on same-column filters, unless KEEPFILTERS is used).

Best Practices

- Always read CALCULATE filter arguments as tables, even when using Boolean shorthand. The Boolean form 'Product[Brand]="Contoso"' is equivalent to FILTER(ALL(Product[Brand]), ...). Understanding the expanded form prevents confusion about overwrite vs merge behaviour.
- Use Boolean shorthand for single-column conditions. Use explicit FILTER(ALL(cols), condition) for multi-column conditions. Never use FILTER(FactTable, condition) as a CALCULATE argument — it is slow and semantically complex.
- When computing a percentage denominator, always choose the correct ALL scope: ALL(column) removes one column's filter; ALL(table) removes all columns; ALLSELECTED respects external slicers. Mismatching scope is the most common percentage error.
- In calculated columns, never use a measure reference for a variable that should return the grand total. Context transition will filter it to the current row. Use explicit SUMX/COUNTROWS with no CALCULATE wrapper for true grand totals in calculated columns.
- Prefer CALCULATETABLE over FILTER when the condition involves relationships or when CALCULATE modifiers (KEEPFILTERS, USERELATIONSHIP) are needed on the table expression.
- Use USERELATIONSHIP for role-playing dimensions. Keep the most frequently queried relationship active in the model; activate the others via USERELATIONSHIP only in the measures that need them.
- Be explicit about context transition. When calling a measure inside an iterator, always ask: 'Should context transition fire here? Do I want the measure filtered to the current row?' If not, replace the measure reference with the explicit expression it wraps.
- Test CALCULATE measures by tracing one specific cell: write down the filter context entering CALCULATE, apply the four-step evaluation model, and confirm the predicted result matches the actual result.

Common Issues and Solutions

CALCULATE measure ignores the Brand slicer

Cause: CALCULATE's filter argument overwrites the existing Brand filter from the slicer. This is intentional for 'always Contoso' measures but may be unintended for 'filter-aware' measures.

Solution: If the measure should intersect with the slicer rather than override it, wrap the filter argument in `KEEPFILTERS`.

Percentage denominator returns the wrong total

Cause: ALL scope is incorrect: `ALL(column)` removes only one column's filter. Other filters (from different columns or tables) remain active.

Solution: Expand the ALL scope: use `ALL(table)` to remove all product filters, or add ALL for each additional table that should not filter the denominator.

Calculated column classifies every product as 'High Performance'

Cause: The 'grand total' variable uses a measure reference, triggering context transition. The measure is filtered to the current product — it returns that product's sales, the same value as the 'current product' variable.

Solution: Replace the grand total variable with an explicit `SUMX(Sales, ...)` without CALCULATE. This avoids context transition and truly returns the grand total.

Circular dependency error on a calculated column using CALCULATE

Cause: CALCULATE in a calculated column triggers context transition, which auto-adds ALL columns as filter arguments — including the column being calculated, creating a circular reference.

Solution: Ensure the table has a unique key column that the DAX engine recognises as the row identifier. This limits context transition to the key column only. For tables without a natural key, avoid CALCULATE in calculated columns.

USERRELATIONSHIP measure returns the same result as the default measure

Cause: The inactive relationship referenced in USERRELATIONSHIP may not match the column names correctly, or the Date table may be filtered by the active relationship's year column.

Solution: Verify the relationship exists and is inactive in Model view. Ensure the USERRELATIONSHIP column arguments exactly match the relationship definition. Use `CROSSFILTER(col1, col2, NONE)` to disable the active relationship if needed.

Module 5 — Quick Reference

Syntax / Concept	Description	Primary Use
<code>CALCULATE(expr, f1, f2, ...)</code>	Evaluate <code>expr</code> in a modified filter context. Replace filters on <code>f1</code> , <code>f2</code> columns; merge on others.	Every advanced DAX measure
Boolean predicate	Syntax sugar for <code>FILTER(ALL(col), condition)</code> . Single column only.	Simple column filters
<code>FILTER(ALL(col), condition)</code>	Explicit expanded form. Required for multi-column conditions.	Multi-column filter arguments
<code>ALL(col/table)</code>	As <code>CALCULATE</code> top-level arg: <code>REMOVEFILTER</code> . As nested table function: return all values.	Percentage denominators
<code>ALLSELECTED(col)</code>	Remove visual-level filters; keep external slicer/page filters.	% of visible total
<code>KEEPFILTERS(filter)</code>	Intersect new filter with existing filter instead of replacing it.	Conditional visibility
<code>CALCULATETABLE(tbl, ...)</code>	Same as <code>CALCULATE</code> but returns a table. All same semantics and modifiers apply.	Filtered table expressions
<code>USERELATIONSHIP(c1, c2)</code>	<code>CALCULATE</code> modifier: activate an inactive relationship, deactivate the current active one.	Role-playing dimensions
<code>CROSSFILTER(c1, c2, dir)</code>	<code>CALCULATE</code> modifier: change a relationship's cross-filter direction temporarily.	Temporary bidirectional
Context transition	When <code>CALCULATE</code> runs in a row context: invalidates row context, adds all iterated columns as filters.	Calculated columns; measure refs in iterators
Modifier precedence	Modifiers (<code>ALL</code> , <code>USERELATIONSHIP</code> , <code>CROSSFILTER</code>) applied before filter arguments.	Nested <code>CALCULATE</code> analysis

Module Summary

- `CALCULATE` is the only DAX function that modifies the filter context. It is the single most important function in the language and is used in virtually every non-trivial DAX measure.
- `CALCULATE` follows a four-step evaluation model: copy the context, evaluate filter arguments in the original context, apply modifiers first, then apply filter arguments.
- A `CALCULATE` filter argument replaces existing filters on the same column (overwrite), but merges with filters on different columns (AND). `KEEPFILTERS` changes overwrite to intersect for a specific filter argument.
- Boolean filter predicates are syntax sugar for `FILTER(ALL(column), condition)`. Multi-column conditions require explicit `FILTER` with `ALL` wrapping the column list.
- `ALL` used as a top-level `CALCULATE` argument is a modifier (`REMOVEFILTER`), not a table function. Modifiers are applied before filter arguments — this governs the behaviour of nested `CALCULATE`.
- Context transition is triggered by `CALCULATE` inside a row context. It invalidates the row context and converts it into an equivalent filter context by adding all iterated table columns as filter arguments. Every measure reference in a row context triggers context transition implicitly.

■ Use explicit SUMX/COUNTROWS (without CALCULATE) in calculated columns when you need a true grand total that is not affected by context transition.

■ CALCULATETABLE has identical semantics to CALCULATE but returns a table. USERELATIONSHIP and CROSSFILTER are CALCULATE modifiers that temporarily modify the model's relationship architecture for the duration of the evaluation.

NOTE

Next — Module 6: Variables. With CALCULATE mastered, Module 6 covers the VAR keyword in depth: variable scoping rules, why variables are evaluated exactly once, how they interact with context transition, and the VAR patterns that make complex DAX formulas readable and maintainable.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

6

Variables

VAR · RETURN · Scope · Context Capture · Performance

Write cleaner, faster, and safer DAX with variables.

Learn VAR/RETURN syntax, how variables capture the evaluation context at declaration time, scalar vs table variables, variable scope rules, lazy evaluation, and the critical difference between a variable and a measure reference.

Prerequisite: Modules 1–5.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 6: Variables

Module Contents

■ Module Overview — Two Reasons Variables Exist

■ Lesson 1: VAR and RETURN — Syntax and Basics

- VAR/RETURN Syntax
- Scalar Variables
- Variables Are Evaluated Once
- RETURN Is the Exit Point
- Variables for Debugging — Swapping the RETURN Value

■ Lesson 2: Variable Scope

- Variables Are Available After Their Declaration
- Nested VAR/RETURN Blocks
- Variables Defined Inside an Iterator
- No Circular Dependencies Between Variables
- Naming Conflicts with Table Names

■ Lesson 3: Table Variables

- Storing a Table in a Variable
- Reusing a Table Variable Multiple Times
- Table Variables and CALCULATETABLE
- ADDCOLUMNS and SELECTCOLUMNS with Table Variables

■ Lesson 4: The Critical Rule — Variables Capture Context at Declaration

- A Variable Stores a Value, Not a Formula
- Variables vs Measure References — The Filter Context Trap
- The Average-of-Customers Problem
- Solving Context-Sensitive Calculations with Variables

■ Lesson 5: Lazy Evaluation

- Variables Are Only Evaluated If Used
- Skipping Expensive Sub-expressions
- Using Lazy Evaluation for Conditional Logic

■ Lesson 6: Variables and Performance

- Evaluating a Sub-expression Once vs Many Times
- Variables vs Repeated Measure References
- Table Variables for Pre-computed Lookups
- The RANKX and Moving Average Patterns with Variables

■ Lesson 7: VAR Best Practices and Patterns

- One Variable per Concept
- Always Use a Single Variable After RETURN
- Naming Conventions
- When NOT to Use a Variable

■ Demonstration 1: Variables Transform a Complex Measure

■ Demonstration 2: The Filter Context Trap — Variable vs Measure Reference

■ Lab 6: Variables

- Exercise 1: Rewriting Measures with Variables

- Exercise 2: Scalar and Table Variables — Gross Margin Analysis
- Exercise 3: The Context Capture Rule — Average per Customer
- Exercise 4: Performance — Max Daily Sales and Date of Max

■ Module Review and Takeaways

Module Overview — Two Reasons Variables Exist

Variables were added to DAX in 2015 and immediately became essential to every professional DAX developer's toolkit. They exist for exactly two reasons: **readability** and **performance**.

Without variables, a complex DAX expression that uses the same sub-expression in three places must repeat that sub-expression verbatim three times — making the code fragile, verbose, and error-prone. With variables, you name a sub-expression once and reference its name everywhere. The formula becomes a readable sequence of named steps with a single **RETURN** statement that is trivially easy to modify.

The performance benefit is equally significant: DAX evaluates a variable's expression **exactly once**, regardless of how many times the variable is referenced in the **RETURN** block. Without variables, each reference to a sub-expression forces a fresh evaluation — sometimes with full context transition overhead.

CONCEPT

The most important rule about variables in DAX: a variable captures and freezes the evaluation context at the point of its `VAR` declaration. It stores a **VALUE**, not a formula. Every subsequent reference to that variable returns the same pre-computed value, regardless of what the context is at the point of use. This is completely different from a measure reference, which is re-evaluated in whatever context is active at the point of the call.

EXAM

PL-300 tests variables primarily through: understanding why a variable does not respond to context changes after its declaration, the difference between a variable and a measure reference in a row context, and using variables to capture outer row context values in nested iterations (the **EARLIER** replacement pattern from Module 4).

LESSON 1

VAR and RETURN — Syntax and Basics

Lesson Objectives

After completing this lesson, you will be able to:

- Write a DAX expression using VAR and RETURN.
- Explain that a variable is evaluated exactly once, at the point of its declaration.
- Use the RETURN swap technique to debug a multi-variable expression.
- Identify the three reasons to prefer variables over repeated sub-expressions.

VAR / RETURN Syntax

The VAR keyword defines a named sub-expression. The RETURN keyword marks the value the enclosing function or measure returns. Every VAR block must have exactly one RETURN:

```
-- Minimal syntax:
VAR =
RETURN

-- Multiple variables before a single RETURN:
[Gross Margin %] :=
VAR GrossMargin = SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )
VAR SalesAmount = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
RETURN
DIVIDE ( GrossMargin, SalesAmount )

-- The same measure WITHOUT variables (harder to read, expressions repeated):
[Gross Margin % NoVar] :=
DIVIDE (
SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) ),
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
)

-- With variables: each SUMX expression is written once, named clearly,
-- and the RETURN line reads like plain English: GrossMargin / SalesAmount.
```

Variables Are Evaluated Exactly Once

DAX evaluates each variable's expression a single time, when DAX first needs the variable's value. If you reference the variable ten times in the RETURN block, the underlying expression is still only computed once. This is both the readability and the performance benefit in one guarantee:

```
[Complex Margin Analysis] :=
VAR TotalCost =
SUMX ( Sales, Sales[Quantity] * Sales[Unit Cost] ) -- evaluated ONCE
VAR SalesAmount =
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- evaluated ONCE
VAR GrossMargin = SalesAmount - TotalCost -- uses cached SalesAmount
VAR MarginPct = DIVIDE ( GrossMargin, SalesAmount ) -- uses cached SalesAmount again
VAR MarginLabel =
IF ( MarginPct >= 0.3, "High", IF ( MarginPct >= 0.15, "Medium", "Low" ) )
RETURN
MarginLabel
```

```
-- SalesAmount is referenced in GrossMargin and MarginPct.  
-- Despite two references, the SUMX over Sales is only executed once.  
-- Without variables: two separate SUMX calls = double the computation.
```

Debugging with the RETURN Swap

Variables make debugging effortless. To inspect an intermediate value, simply change RETURN to point at any variable in the chain. No code rewriting needed:

```
[Gross Margin %] :=  
VAR GrossMargin = SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )  
VAR SalesAmount = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )  
VAR MarginPct = DIVIDE ( GrossMargin, SalesAmount )  
RETURN  
MarginPct -- normal: returns the percentage  
  
-- To debug: is GrossMargin computing correctly?  
-- Just change RETURN temporarily:  
RETURN  
GrossMargin -- debug: returns the raw gross margin value  
  
-- To check SalesAmount:  
RETURN  
SalesAmount -- debug: returns total sales  
  
-- Switch back to RETURN MarginPct when done.  
-- This technique is far easier than restructuring the whole formula each time.
```

TIP

Always write the `RETURN` expression as a single variable reference — never write a complex expression directly after RETURN. This costs nothing and makes every intermediate step instantly inspectable by simply swapping which variable follows RETURN.

LESSON 2

Variable Scope

Lesson Objectives

After completing this lesson, you will be able to:

- State the rule that a variable is only accessible after its declaration.
- Explain how variables defined inside an iterator are scoped to that iterator.
- Describe nested VAR/RETURN blocks and when they are used.
- Explain why circular dependencies between variables are impossible.
- Identify and avoid naming conflicts between variable names and table names.

Variables Are Available After Their Declaration — Not Before

A variable can only be referenced at or after the point in the code where it is declared. You cannot reference a variable before its VAR line — this is why circular dependencies between variables are structurally impossible:

```
-- Valid: B references A, which is declared before B.
VAR A = [Sales Amount]
VAR B = A * 1.1 -- A is available here
RETURN B

-- INVALID: A references B, but B is not yet declared.
VAR A = B * 0.9 -- ERROR: B is not defined yet
VAR B = [Sales Amount]
RETURN A

-- IMPOSSIBLE: circular dependency
VAR A = B + 1 -- A needs B
VAR B = A + 1 -- B needs A -> DAX prevents this at parse time
RETURN A
```

Variables Defined Inside an Iterator Are Scoped to That Iterator

When variables are defined inside the expression executed for each row of an iterator, they are re-evaluated and exist only within that one row's execution. They cannot be accessed outside the iterator:

```
-- Variables scoped to each SUMX iteration:
[Gross Margin] :=
SUMX (
    Sales,
    VAR Qty = Sales[Quantity] -- scoped to this row of Sales
    VAR Price = Sales[Net Price] -- scoped to this row
    VAR Cost = Sales[Unit Cost] -- scoped to this row
    VAR LineMargin = Qty * ( Price - Cost ) -- uses row-scoped variables
    RETURN
    LineMargin
)

-- Qty, Price, Cost, LineMargin only exist while SUMX processes one row.
-- They are reassigned for each new row.
-- They are NOT accessible outside of the SUMX expression.

-- Compare with outer-scope variables (accessible everywhere in RETURN):
[Complex Measure] :=
```

```

VAR AllSales = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- outer scope
VAR TopCategories =
FILTER (
VALUES ( 'Product'[Category] ),
VAR CatSales = [Sales Amount] -- scoped to each FILTER row iteration
RETURN CatSales > AllSales * 0.1 -- AllSales is accessible (outer scope)
)
RETURN COUNTROWS ( TopCategories )

```

Nested VAR/RETURN Blocks

It is possible to write VAR/RETURN blocks nested inside each other. An inner variable defined within the expression of an outer variable is only accessible within that inner scope:

```

-- Two non-nested VAR/RETURN blocks in the same SUMX (separate scopes):
[Margin] :=
SUMX (
Sales,
( VAR LineAmount = Sales[Quantity] * Sales[Net Price]
RETURN LineAmount
)
-
( VAR LineCost = Sales[Quantity] * Sales[Unit Cost]
RETURN LineCost
)
)
-- LineAmount and LineCost are in separate, non-nested scopes.
-- LineAmount cannot reference LineCost and vice versa.

-- NESTED: inner variable defined within the RETURN expression of an outer variable
[Margin Nested] :=
SUMX (
Sales,
VAR LineAmount = Sales[Quantity] * Sales[Net Price]
RETURN (
LineAmount -- LineAmount accessible here
- VAR LineCost = Sales[Quantity] * Sales[Unit Cost]
RETURN (
LineCost -- LineCost accessible here
-- LineAmount is ALSO still accessible here (outer scope)
)
)
)
-- In the nested version, LineAmount remains in scope inside the inner block.
-- The outer variable's scope extends into any nested block.

```

Naming Conflicts with Table Names

In DAX, a name can refer to either a table or a variable — but not both at the same time. If you name a variable with the same name as an existing table, the variable shadows the table within its scope:

```

-- DANGEROUS: variable named 'Sales' shadows the Sales table
VAR Sales = FILTER ( Sales, Sales[Net Price] > 100 ) -- first 'Sales' = variable name
-- second 'Sales' = the table
RETURN COUNTROWS ( Sales ) -- 'Sales' here = the variable, not the table

```

```
-- This actually WORKS in DAX (the variable shadows the table),  
-- but it is extremely confusing. AVOID naming variables after tables.  
  
-- Best practice: use descriptive names that could never be table names:  
VAR HighValueSales = FILTER ( Sales, Sales[Net Price] > 100 )  
VAR LargeCustomers = FILTER ( Customer, [Sales Amount] > 10000 )  
VAR FilteredDateRange = CALCULATETABLE ( 'Date', 'Date'[Year] = 2023 )
```

WARNING

Choose variable names that are clearly distinct from any table name in your model. If you later add a table with the same name as a variable used in a measure, the variable will shadow the new table throughout that measure, potentially causing silent incorrect results with no error message. Prefix variable names with a context hint: `<font face="Courier" color="#1A3A5C">FilteredSales</font>`, `<font face="Courier" color="#1A3A5C">TopCustomers</font>`, `<font face="Courier" color="#1A3A5C">CurrentYearDates</font>`.

LESSON 3

Table Variables

Lesson Objectives

After completing this lesson, you will be able to:

- Store a table expression in a variable and explain when to do so.
- Reuse a table variable in multiple downstream expressions for performance.
- Use ADDCOLUMNS to enrich a table variable with computed columns.
- Use SELECTCOLUMNS to project only required columns from a table variable.

Storing a Table in a Variable

A variable can hold a table just as easily as a scalar value. The type is inferred automatically from the expression — if the expression returns a table, the variable is a table variable:

```
-- Scalar variable (expression returns a number):
VAR TotalSales = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- scalar

-- Table variable (expression returns a table):
VAR HighValueSales =
FILTER ( Sales, Sales[Net Price] * Sales[Quantity] >= 500 ) -- table

VAR ContosoProducts =
CALCULATETABLE ( 'Product', 'Product'[Brand] = "Contoso" ) -- table

VAR TopCategoryNames =
TOPN ( 5, VALUES ( 'Product'[Category] ), [Sales Amount], DESC ) -- table

-- Table variables can be used anywhere a table is expected:
RETURN COUNTROWS ( HighValueSales ) -- count rows in the filtered table
-- or:
RETURN SUMX ( HighValueSales, Sales[Quantity] ) -- iterate the filtered table
-- or:
RETURN CALCULATE ( [Sales Amount], ContosoProducts ) -- use as CALCULATE filter
```

Reusing a Table Variable for Performance

The performance benefit of table variables is significant when the same filtered table is needed in multiple places. Without a variable, the filter would be re-executed for each reference:

```
-- WITHOUT table variable: FILTER runs three times
[Multi Ref Without Var] :=
VAR Count = COUNTROWS ( FILTER ( Sales, Sales[Net Price] > 500 ) ) -- FILTER #1
VAR Amount = SUMX ( FILTER ( Sales, Sales[Net Price] > 500 ), -- FILTER #2
Sales[Quantity] * Sales[Net Price] )
VAR Avg = AVERAGEX ( FILTER ( Sales, Sales[Net Price] > 500 ), -- FILTER #3
Sales[Net Price] )
RETURN Amount

-- WITH table variable: FILTER runs once, result reused three times
[Multi Ref With Var] :=
VAR HighPriceSales = FILTER ( Sales, Sales[Net Price] > 500 ) -- FILTER once
VAR Count = COUNTROWS ( HighPriceSales ) -- reuse
```

```
VAR Amount = SUMX ( HighPriceSales, Sales[Quantity] * Sales[Net Price] )
VAR Avg = AVERAGEX ( HighPriceSales, Sales[Net Price] ) -- reuse again
RETURN Amount
-- Identical results. The version with a table variable is faster
-- because FILTER only runs once. On large tables this is a significant gain.
```

ADDCOLUMNS and SELECTCOLUMNS with Table Variables

ADDCOLUMNS enriches a table variable with computed columns. SELECTCOLUMNS projects a subset of columns. Together they enable powerful pre-computation patterns:

```
-- ADDCOLUMNS: append computed columns to an existing table
VAR DailySales =
ADDCOLUMNS (
VALUES ( 'Date'[Date] ), -- base table: one row per date
"DailyRevenue", [Sales Amount], -- add computed column
"DailyOrders", [Order Count] -- add another
)
-- DailySales is now a table with three columns:
-- Date[Date], DailyRevenue, DailyOrders

-- Use DailySales to find the max daily revenue:
VAR MaxDailyRevenue = MAXX ( DailySales, [DailyRevenue] )

-- Find the date(s) that achieved the maximum:
VAR DatesWithMax =
SELECTCOLUMNS ( -- keep only Date column
FILTER ( DailySales, [DailyRevenue] = MaxDailyRevenue ),
"MaxDate", 'Date'[Date]
)

RETURN
IF ( COUNTROWS ( DatesWithMax ) = 1, DatesWithMax, BLANK() )

-- Performance insight:
-- DailySales is built once and scanned twice (MAXX + FILTER).
-- Without the variable, [Sales Amount] would be computed inside MAXX AND FILTER
-- separately -- context transition fires for each date in each scan.
```

TIP

The `ADDCOLUMNS` + variable pattern is the correct way to compute a measure at a different granularity and then aggregate it. Build the granular table once, compute the measure via `ADDCOLUMNS`, store it in a variable, and then scan the variable with `MAXX`, `MINX`, `AVERAGEX`, or `FILTER`. This avoids double-computation and makes the intent of the formula immediately clear.

LESSON 4

The Critical Rule — Variables Capture Context at Declaration

Lesson Objectives

After completing this lesson, you will be able to:

- State the critical rule: a variable stores a value, not a formula.
- Predict the result of a variable reference regardless of the context at the point of use.
- Explain why a variable cannot be used to replace a measure reference in a percentage formula.
- Describe the Average-of-Customers problem and explain why a variable solves it.
- Contrast the behaviour of a variable with the behaviour of a measure reference.

A Variable Stores a Value, Not a Formula

This is the most important concept in this module and the source of the most common variable misuse. When DAX evaluates a VAR declaration, it computes the value in the evaluation context that is active at that exact moment and stores the result. The stored result never changes — it is frozen:

```
-- A variable is a SNAPSHOT of a value, not a live formula.
[Sales % of Total] :=
VAR CurrentSales = [Sales Amount] -- evaluated NOW in current filter context
VAR TotalSales = CALCULATE ( [Sales Amount], ALL ( 'Product' ) ) -- also NOW
RETURN DIVIDE ( CurrentSales, TotalSales )

-- CurrentSales is computed in the filter context of the cell (e.g. Category=Audio).
-- Its value = Audio sales. That value is frozen.
-- TotalSales is computed with ALL(Product) active. Its value = grand total. Also frozen.
-- DIVIDE uses these frozen values. Result: Audio / Grand Total. Correct.

-- The value of CurrentSales does NOT change inside the RETURN block,
-- even if you nest CALCULATE calls that modify the filter context.
-- CurrentSales is a number at this point, not a DAX expression.
```

The Filter Context Trap — Variable vs Measure Reference

The most critical practical consequence of this rule: a variable cannot substitute for a measure reference when the measure *needs to be re-evaluated in a different filter context*:

```
-- WRONG: using a variable where a measure reference is needed
[Sales % of Total WRONG] :=
VAR SalesAmt = [Sales Amount] -- frozen at current filter context (Audio)
RETURN
DIVIDE (
SalesAmt, -- numerator: Audio sales
CALCULATE ( SalesAmt, ALL('Product') ) -- denominator: STILL Audio sales!
) -- CALCULATE cannot change a frozen scalar

-- CALCULATE(SalesAmt, ALL('Product')) does NOT re-evaluate [Sales Amount] in a new
-- context. SalesAmt is already a number (e.g. 1,234,567). CALCULATE of a literal
-- number returns that same number regardless of any filters.
-- Result: DIVIDE(1234567, 1234567) = 1.0 on every single row. Wrong!

-- CORRECT: use a measure reference as the CALCULATE expression
[Sales % of Total CORRECT] :=
```

```

VAR CurrentSales = [Sales Amount] -- frozen: current category sales
VAR TotalSales = CALCULATE ( [Sales Amount], ALL ('Product') ) -- re-evaluated in new context
RETURN DIVIDE ( CurrentSales, TotalSales )
-- [Sales Amount] inside CALCULATE is a MEASURE REFERENCE, not a variable.
-- CALCULATE wraps it, changes the filter context to ALL products,
-- and the measure is re-evaluated in that new context.
-- TotalSales = true grand total. Correct.

```

CONCEPT

The golden rule: use a variable when you want to freeze a value computed in the current context. Use a measure reference when you want DAX to re-evaluate the measure in a potentially different context. These are two fundamentally different operations. A variable is a number (or table). A measure reference is a live expression. CALCULATE can change the context for a measure reference; it cannot change the value of a variable.

The Average-of-Customers Problem

This is a classic scenario where a variable is essential. The goal: count customers who bought more than the average customer. The naive approach fails because context transition fires on the measure reference inside the FILTER iteration:

```

-- BROKEN: average recomputed per customer due to context transition
[Customers Above Avg BROKEN] :=
COUNTROWS (
  FILTER (
    Customer,
    [Sales Amount] > [AverageSalesPerCustomer] -- CT fires on both measure refs!
  )
)
-- Inside the FILTER iteration over Customer:
-- [Sales Amount] triggers CT -> filtered to current customer -> that customer's sales
-- [AverageSalesPerCustomer] = AVERAGEX(Customer,[Sales Amount]) also triggers CT
-- -> filters to current customer -> returns that customer's own sales (not the avg)
-- Comparison: current_customer_sales > current_customer_sales -> always FALSE
-- Result: 0 customers. Always wrong.

-- CORRECT: compute the average ONCE, outside the iteration, as a variable
[Customers Above Avg] :=
VAR AvgSales =
  AVERAGEX ( Customer, [Sales Amount] ) -- evaluated ONCE, in original context
  -- no iteration, no CT, true average
RETURN
COUNTROWS (
  FILTER (
    Customer,
    [Sales Amount] > AvgSales -- AvgSales is a frozen scalar
  ) -- FILTER only applies CT to [Sales Amount]
) -- AvgSales does not change per customer

```

CONCEPT

Variables are the correct tool whenever you need to compute a value **once** in the outer context and then compare it row-by-row inside an iteration. The variable freezes the outer value before the iteration begins. The iteration's context transition has no effect on a frozen scalar. This pattern appears in: above-average filters, above-median filters, target vs actual comparisons, and any threshold calculated outside the current row.

LESSON 5

Lazy Evaluation

Lesson Objectives

After completing this lesson, you will be able to:

- Explain what lazy evaluation means for DAX variables.
- Describe how lazy evaluation avoids evaluating unused variables.
- Use lazy evaluation to skip expensive sub-expressions in conditional logic.
- Explain why lazy evaluation is an optimisation detail, not a semantic feature to rely on.

Variables Are Only Evaluated If Used

DAX uses lazy evaluation for variables: a variable's expression is only evaluated if the variable is actually referenced in the RETURN block. If you define a variable but never use it, DAX skips its evaluation entirely:

```
-- Unused variable: DummyError is never referenced in RETURN
[Safe Measure] :=
VAR DummyError = 1 / 0 -- would cause a division-by-zero error
VAR RealResult = [Sales Amount]
RETURN
RealResult -- DummyError never used -> never evaluated -> no error

-- This is intentional DAX behaviour:
-- If a variable is unused, its expression is never evaluated.
-- The formula executes without any error.

-- Practical consequence: do not write complex variables 'just in case'.
-- Variables defined but unused add no value and may mask errors during development.
-- However, you CAN rely on this behaviour for conditional logic (see next section).
```

Using Lazy Evaluation for Conditional Logic

Lazy evaluation enables a powerful pattern: define expensive variables only when needed, inside the branch that uses them, using IF statements:

```
-- Without lazy evaluation awareness (both branches always evaluated):
[Conditional Wrong] :=
VAR ExpensiveResult = -- always runs
VAR CheapResult = -- always runs
RETURN
IF ( HASONEVALUE ( 'Product'[Category] ), ExpensiveResult, CheapResult )

-- With lazy evaluation: use nested VAR inside the IF branch
[Conditional Right] :=
IF (
HASONEVALUE ( 'Product'[Category] ),
VAR ExpensiveResult = -- only runs when condition TRUE
RETURN ExpensiveResult,
VAR CheapResult = -- only runs when condition FALSE
RETURN CheapResult
)

-- The expensive calculation only runs when a single category is selected.
-- When multiple categories are visible (e.g. the Total row),
```

```
-- the cheap path executes and the expensive variable is never evaluated.

-- Real example: expensive RANKX only on detail rows, not total rows
[Category Rank] :=
IF (
    HASONEVALUE ( 'Product'[Category] ),
    VAR Rank = RANKX ( ALL ( 'Product'[Category] ), [Sales Amount] )
    RETURN Rank,
    BLANK() -- total row: return blank immediately, RANKX never runs
)
```

NOTE

Lazy evaluation is an engine optimisation. It means you should structure your code so that expensive variables are inside the branch that actually needs them. However, do not write code that relies on the absence of an error in an unused variable as a correctness mechanism — that makes the formula fragile and hard to understand.

LESSON 6

Variables and Performance

Lesson Objectives

After completing this lesson, you will be able to:

- Explain why variables improve performance when a sub-expression is used multiple times.
- Apply the ADDCOLUMNS pre-computation pattern to avoid redundant context transitions.
- Refactor a RANKX measure to correctly use variables for the ranking lookup.
- Build a 30-day moving average using a table variable for the date range.

Evaluating a Sub-expression Once vs Many Times

Every time a measure is referenced without being stored in a variable, it is re-evaluated in its current context. When the same measure appears in two places in a formula, it runs twice. Variables eliminate this:

```
-- Measure referenced three times -> evaluated three times (with possible CT each time)
[Stats No Var] :=
VAR AboveAvg = COUNTROWS ( FILTER ( Customer, [Sales Amount] > AVERAGEX(Customer,[Sales Amount]) ) )
VAR BelowAvg = COUNTROWS ( FILTER ( Customer, [Sales Amount] < AVERAGEX(Customer,[Sales Amount]) ) )
VAR AtAvg = COUNTROWS ( Customer ) - AboveAvg - BelowAvg
RETURN AboveAvg & " above, " & AtAvg & " at, " & BelowAvg & " below"
-- AVERAGEX(Customer,[Sales Amount]) runs TWICE -- full customer iteration each time.

-- Variable caches the average -> evaluated once
[Stats With Var] :=
VAR AvgSales = AVERAGEX ( Customer, [Sales Amount] ) -- computed ONCE
VAR AboveAvg = COUNTROWS ( FILTER ( Customer, [Sales Amount] > AvgSales ) )
VAR BelowAvg = COUNTROWS ( FILTER ( Customer, [Sales Amount] < AvgSales ) )
VAR AtAvg = COUNTROWS ( Customer ) - AboveAvg - BelowAvg
VAR Result = AboveAvg & " above, " & AtAvg & " at, " & BelowAvg & " below"
RETURN Result
-- AvgSales is computed once. The two FILTER calls reuse the scalar.
```

ADDCOLUMNS Pre-computation — Max Daily Sales and Date of Max

This pattern combines a table variable with ADDCOLUMNS to pre-compute a measure at a fine granularity and then query the result multiple times without recomputing:

```
-- Goal: find the date on which sales were at their maximum.

-- STEP 1: compute sales for every date, store in a table variable.
VAR DailySalesTable =
ADDCOLUMNS (
VALUES ( 'Date'[Date] ), -- one row per date in current filter context
"Revenue", [Sales Amount] -- add computed Revenue column
)
-- DailySalesTable has columns: Date[Date], Revenue
-- [Sales Amount] is evaluated once per date row (context transition on Date).

-- STEP 2: find the maximum revenue value.
VAR MaxRevenue = MAXX ( DailySalesTable, [Revenue] )
```

```

-- Scans DailySalesTable (already computed) -- no re-evaluation of [Sales Amount].

-- STEP 3: find the date(s) that match the maximum.
VAR DateOfMax =
SELECTCOLUMNS (
FILTER ( DailySalesTable, [Revenue] = MaxRevenue ),
"TheDate", 'Date'[Date]
)
-- FILTER scans DailySalesTable (already computed) -- no re-evaluation.
-- SELECTCOLUMNS drops the Revenue column, keeping only the date.

-- STEP 4: return the date only if unique.
RETURN
IF ( COUNTROWS ( DateOfMax ) = 1, DateOfMax, BLANK () )

-- WITHOUT variables: [Sales Amount] would be evaluated inside MAXX and FILTER
-- separately -- context transition fires for each date in each scan.
-- With variables: [Sales Amount] fires once per date, in ADDCOLUMNS only.

```

30-Day Moving Average with a Table Variable

```

[30-Day Moving Avg Sales] :=
VAR LastVisibleDate = MAX ( 'Date'[Date] )
VAR NumberOfDays = 30
VAR DateWindow =
FILTER (
ALL ( 'Date' ), -- all dates, ignoring filter context
'Date'[Date] > LastVisibleDate - NumberOfDays
&& 'Date'[Date] <= LastVisibleDate
)
VAR Result =
CALCULATE (
AVERAGEX ( 'Date', [Sales Amount] ), -- average daily sales in window
DateWindow -- apply the date window as filter
)
RETURN Result

-- How the variables decompose the logic:
-- LastVisibleDate: the anchor date from the current visual cell (e.g. chart point)
-- NumberOfDays: readable constant -- easy to change to 7, 90, etc.
-- DateWindow: the 30-day date range as a table
-- Result: average daily sales within that window
-- RETURN Result: single variable, instantly swappable for debugging

-- Note: days with zero sales are excluded from AVERAGEX's average.
-- If you want to include zero-sales days in the denominator:
VAR TotalRevenue = CALCULATE ( [Sales Amount], DateWindow )
VAR Result = DIVIDE ( TotalRevenue, NumberOfDays )
-- This divides by 30 always, counting non-sales days as zero.

```

LESSON 7

VAR Best Practices and Patterns

Lesson Objectives

After completing this lesson, you will be able to:

- Apply the one-variable-per-concept naming discipline.
- Write every RETURN statement as a single variable reference.
- Identify scenarios where a variable is the wrong tool.
- Apply a consistent naming convention for DAX variables.

Best Practice Rules for Variables

Rule	Why It Matters	Example
One variable per concept.	Each variable should represent one clearly named business concept. Never combine two unrelated calculations into one variable.	VAR GrossMargin = ... not VAR GMandCount = ...
RETURN a single variable.	Complex expressions after RETURN cannot be debugged by swapping. Always assign the final result to a variable and RETURN that variable.	VAR Result = ... RETURN Result not RETURN DIVIDE(A,B)+C
Name variables clearly.	Variable names should be self-documenting. A reader should understand what the variable holds without reading its definition.	CurrentYearSales, PreviousYearSales, YoYGrowth
Never name a variable after an existing table.	The variable will shadow the table within its scope, causing silent errors if the formula is later modified.	Use FilteredSales not Sales
Use a variable for any sub-expression used more than once.	Repeated sub-expressions are evaluated multiple times. A variable guarantees single evaluation and prevents the sub-expressions from diverging.	Store AVERAGEX, FILTER results, CALCULATETABLE results in variables.
Use a variable to capture the outer context before an inner iteration.	The EARLIER-replacement pattern. Inside FILTER or SUMX, a variable declared outside captures the pre-iteration value.	VAR CurrentPrice = 'Product'[UnitPrice] FILTER(...,> CurrentPrice)
Keep variables close to their point of use.	Defining a variable 20 lines before it is first used makes the formula hard to read. Define variables just before the step that uses them.	Group related VAR lines together near their RETURN sub-expression.

When NOT to Use a Variable

Variables are not always the right tool. Understand these scenarios where a variable would produce incorrect results:

```
-- WRONG: using a variable where the measure needs to be re-evaluated per cell
-- Goal: show each product's % contribution to its category total.

[Pct of Category WRONG] :=
VAR CategorySales = CALCULATE ( [Sales Amount], ALL ( 'Product'[Product Name] ) )
RETURN DIVIDE ( [Sales Amount], CategorySales )
-- CategorySales is frozen in the current filter context (e.g. Category=Audio, Brand=Contoso).
-- ALL only removes Product Name filter -- Brand filter remains.
```



```
-- If category total should be ALL brands of that category, this is wrong.

-- The correct version uses a measure reference inside CALCULATE (not a variable):
[Pct of Category CORRECT] :=
DIVIDE (
[Sales Amount],
CALCULATE ( [Sales Amount], ALL ( 'Product'[Product Name] ) )
)
-- Here [Sales Amount] inside CALCULATE is a LIVE measure reference.
-- CALCULATE changes the filter context AND re-evaluates [Sales Amount] in that context.

-- RULE: if you need CALCULATE to re-evaluate something in a new context,
-- that something must be a measure reference (live expression),
-- NOT a variable (frozen value).
```

DEMONSTRATION 1

Variables Transform a Complex Measure

Scenario: The instructor refactors a multi-step measure from a dense, unreadable expression into a clean, debuggable, variable-driven formula — demonstrating all three benefits: readability, debuggability, and performance.

Demonstration Steps

1	Open Contoso_Demo06_Start.pbix. Show the existing measure [Gross Margin %] written without variables. Point out the repeated SUMX expressions. Count how many times Sales is scanned.
2	Refactor live: add VAR GrossMargin, VAR SalesAmount. Change RETURN to DIVIDE(GrossMargin, SalesAmount). Show that the result is identical. Highlight that each SUMX now appears exactly once.
3	Demonstrate the debug technique: change RETURN to GrossMargin. Show the Card shows the raw gross margin value. Change to SalesAmount. Show sales amount. Change back to DIVIDE expression. Confirm the final result.
4	Add a third variable: VAR MarginPct = DIVIDE(GrossMargin, SalesAmount). Change RETURN to MarginPct. Identical result. Now the RETURN line is a single named concept.
5	Open Performance Analyzer. Record a slicer click on the original measure and the variable version. Show that the variable version has a lower DAX query time because each SUMX runs once.
6	Add the [Customers Above Average] broken measure without a variable. Show the result is always 0. Fix it with VAR AvgSales = AVERAGEX(Customer,[Sales Amount]). Show the correct count of above-average customers.
RESULT	Variable refactoring demonstrated live. Performance improvement visible in analyzer. The average-customers fix confirms the context-capture rule.

DEMONSTRATION 2

The Filter Context Trap — Variable vs Measure Reference

Scenario: The instructor demonstrates side-by-side the difference between a variable and a measure reference inside CALCULATE, showing when each is correct and why the wrong choice produces silently incorrect results.

Demonstration Steps

1	Add measure [Sales % WRONG] := VAR S=[Sales Amount] RETURN DIVIDE(S, CALCULATE(S, ALL('Product'))). Place on a matrix with Category on rows. Show every row returns exactly 1.0 (100%). Explain: CALCULATE(S, ALL) where S is already a number does nothing — the ALL filter has no effect on a literal scalar.
2	Add measure [Sales % CORRECT] with VAR CurrentSales = [Sales Amount] and CALCULATE([Sales Amount], ALL('Product')) using a live measure reference. Show the correct percentages. Confirm they add to 100% at the Grand Total.
3	Trace one cell together: Audio row. Filter context = {Category=Audio}. CurrentSales is evaluated here, frozen = Audio sales. CALCULATE([Sales Amount], ALL) changes context to no-product-filter, [Sales Amount] re-evaluates = grand total. DIVIDE = correct percentage.
4	Repeat with the broken version, tracing: CurrentSales = Audio sales (frozen). CALCULATE(frozen_number, ALL) = frozen_number unchanged = Audio sales again. DIVIDE = 1.0. The ALL did nothing because there was nothing to re-evaluate.
5	Ask attendees: can we use CALCULATE(CurrentSales * 1.1, ALL('Product'))? Explain: yes, you can use a variable in an expression inside CALCULATE, but the variable's value does not change. Only a measure reference gets re-evaluated.
RESULT	The frozen-value nature of variables demonstrated with a side-by-side comparison. The distinction between 'variable used in CALCULATE' vs 'measure reference in CALCULATE' is clear.

LAB 6

Variables

Lab Scenario

The Contoso BI team has a backlog of measures that are difficult to maintain: some are dense nested expressions without variables, some have incorrect use of measure references vs variables, and one advanced measure requires the ADDCOLUMNS pre-computation pattern. Your task is to refactor, fix, and build these measures using best-practice variable patterns.

NOTE

Open Contoso_Lab06_Start.pbix. This file contains the completed model and measures from Labs 1–5. Several placeholder measures are included with comments indicating what each should compute. Save your work as Contoso_Lab06_Complete.pbix.

Exercise 1: Rewriting Measures with Variables

Estimated time: 20 minutes

Task 1: Refactor [Gross Margin %] and [Sales % of Total]

1. Locate the existing measure [Gross Margin %]. It is currently written without variables. Rewrite it using variables:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Gross Margin %] :=
VAR GrossMargin = SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )
VAR SalesAmount = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
VAR Result = DIVIDE ( GrossMargin, SalesAmount )
RETURN Result' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Gross Margin %] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR GrossMargin = SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR SalesAmount = SUMX (
Sales, Sales[Quantity] * Sales[Net Price] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR Result = DIVIDE ( GrossMargin, SalesAmount )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN Result',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Verify the result is identical to the original on a matrix with Category on rows.
3. Use the debug technique: change RETURN to GrossMargin. Note the raw value. Change to SalesAmount. Note it. Change back to Result. Confirm the percentage is correct.

4. Refactor [Sales % of Total] (currently written without variables):

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales % of Total] :=
VAR CurrentSales = [Sales Amount]
VAR TotalSales = CALCULATE ( [Sales Amount], ALL ( 'Product' ) )
VAR Result = DIVIDE ( CurrentSales, TotalSales )
RETURN Result" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Sales % of Total] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR CurrentSales = [Sales Amount]', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR TotalSales = CALCULATE ( [Sales Amount], ALL ( 'Product' ) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Result = DIVIDE (
CurrentSales, TotalSales )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph
```

5. Verify percentages sum to 100% at the Grand Total row.

Results: Both measures refactored with variables. Debug technique applied. Results identical to original. Each SUMX expression appears exactly once.

Exercise 2: Scalar and Table Variables — Gross Margin Analysis

Estimated time: 20 minutes

Task 1: Build a margin classification measure

1. Add the following measure to the Sales table:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Margin Tier] :=
VAR GrossMargin = SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )
VAR SalesAmount = SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
VAR MarginPct = DIVIDE ( GrossMargin, SalesAmount )
VAR Tier =
SWITCH (
TRUE (),
MarginPct >= 0.40, "Tier 1 — Premium (40%+)",
MarginPct >= 0.25, "Tier 2 — Standard (25-40%)",
MarginPct >= 0.10, "Tier 3 — Low (10-25%)",
"Tier 4 — At Risk (<10%)"
)
RETURN Tier" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Margin Tier] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
```

```

ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR GrossMargin = SUMX ( Sales, Sales[Quantity] * ( Sales[Net Price] - Sales[Unit Cost] ) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR SalesAmount = SUMX (
Sales, Sales[Quantity] * Sales[Net Price] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR MarginPct = DIVIDE ( GrossMargin, SalesAmount )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Tier =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' SWITCH (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' TRUE ()',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' MarginPct >= 0.40, "Tier 1 —
Premium (40%+)",', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' MarginPct >= 0.25, "Tier 2 —
Standard (25-40%)",', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' MarginPct >= 0.10, "Tier 3 —
Low (10-25%)",', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' "Tier 4 — At Risk (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='<',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='10%)"',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN Tier',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )

```

#Paragraph

2. Place on a matrix with Product[Subcategory] on rows. Verify each subcategory is correctly classified.

Task 2: Build a table variable for high-margin products

1. Add the following measure:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[High Margin Product Count] :=
VAR HighMarginProducts =
FILTER (
VALUES ( 'Product'[Product Name] ),
[Gross Margin %] >= 0.35
)
VAR Result = COUNTROWS ( HighMarginProducts )
RETURN Result" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[High Margin Product Count] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR HighMarginProducts =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' FILTER (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" VALUES ( 'Product'[Product
Name] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Gross Margin %] >= 0.35',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Result = COUNTROWS (
HighMarginProducts )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph
```

2. Add a second measure that reuses the same table variable pattern but computes total sales of high-margin products:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[High Margin Sales] :=
VAR HighMarginProducts =
CALCULATETABLE (
VALUES ( 'Product'[Product Name] ),
[Gross Margin %] >= 0.35
```



```

)
VAR Result = CALCULATE ( [Sales Amount], HighMarginProducts )
RETURN Result" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[High Margin Sales] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR HighMarginProducts =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' CALCULATE(TABLE (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=" VALUES ( 'Product'[Product Name] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' [Gross Margin %] >= 0.35', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Result = CALCULATE (
[Sales Amount], HighMarginProducts )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph

```

- Place both on a matrix. Apply a Category slicer. Verify both measures respond to the slicer.

Results: Margin classification measure with SWITCH and VAR chain. Table variables used for both counting and summing high-margin products. Both measures respond correctly to slicers.

Exercise 3: The Context Capture Rule — Average per Customer

Estimated time: 20 minutes

Task 1: Build the broken measure and confirm it fails

- Add the broken measure to observe the problem:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Customers Above Avg BROKEN] :=
COUNTROWS (
FILTER (
Customer,
[Sales Amount] > AVERAGEX ( Customer, [Sales Amount] )
)
) 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],

```



```

rise=0, text=[Customers Above Avg BROKEN] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='COUNTROWS (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' FILTER (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' Customer,',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Sales Amount] > AVERAGEX (
Customer, [Sales Amount] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=')',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Place on a Card. Observe the result is 0 (or blank). Explain why: AVERAGEX(Customer,[Sales Amount]) inside FILTER triggers context transition per customer — it returns that customer's own sales, not the average. No customer is greater than their own sales.

Task 2: Fix using a variable

1. Add the correct measure:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Customers Above Average] :=
VAR AvgSalesPerCustomer =
AVERAGEX ( Customer, [Sales Amount] ) -- computed once, in outer filter context
VAR AboveAvgCustomers =
FILTER (
Customer,
[Sales Amount] > AvgSalesPerCustomer -- AvgSalesPerCustomer is a frozen scalar
)
VAR Result = COUNTROWS ( AboveAvgCustomers )
RETURN Result' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Customers Above Average] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR AvgSalesPerCustomer =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' AVERAGEX ( Customer, [Sales
Amount] ) -- computed once, in outer filter context', textColor=Color(.101961,.227451,.360784,1),

```

```

us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR AboveAvgCustomers =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' FILTER (' , textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' Customer,',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Sales Amount] >
AvgSalesPerCustomer -- AvgSalesPerCustomer is a frozen scalar',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' '),
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Result = COUNTROWS (
AboveAvgCustomers )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph

```

- Place on a Card next to the broken version. Confirm the correct version returns a meaningful number (greater than zero). Apply a Country slicer — confirm the correct version updates.

Task 3: Extend — what % of customers are above average?

- Add a measure for the percentage of customers above average:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[% Customers Above Average] :=
VAR AvgSalesPerCustomer = AVERAGEX ( Customer, [Sales Amount] )
VAR TotalCustomers = COUNTROWS ( Customer )
VAR AboveAvgCount =
COUNTROWS ( FILTER ( Customer, [Sales Amount] > AvgSalesPerCustomer ) )
VAR Result = DIVIDE ( AboveAvgCount, TotalCustomers )
RETURN Result' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[% Customers Above Average] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR AvgSalesPerCustomer =
AVERAGEX ( Customer, [Sales Amount] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR TotalCustomers = COUNTROWS ( Customer )', textColor=Color(.101961,.227451,.360784,1),

```

```
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR AboveAvgCount =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' COUNTROWS ( FILTER ( Customer, [Sales Amount] > AvgSalesPerCustomer ) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Result = DIVIDE (
AboveAvgCount, TotalCustomers )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph
```

- Note that AvgSalesPerCustomer is reused across both COUNTROWS calls — it is evaluated once and the frozen scalar is compared in FILTER. TotalCustomers is also a simple COUNTROWS that does not need to be inside an iterator.

Results: Broken measure confirmed: always returns 0 due to context transition. Correct measure returns accurate count of above-average customers. Percentage extension reuses the average variable efficiently.

Exercise 4: Performance — Max Daily Sales and Date of Max

Estimated time: 25 minutes

Task 1: Build Max Daily Sales using MAXX

- Add a measure that finds the maximum daily sales in the current filter context:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Max Daily Sales] :=
VAR Result = MAXX ( 'Date', [Sales Amount] )
RETURN Result" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Max Daily Sales] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="VAR Result = MAXX ( 'Date', [Sales Amount] )", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph
```

- Place on a matrix with Date[Year] on rows. Verify each year shows the highest single-day sales amount for that year.

Task 2: Build Date of Max using ADDCOLUMNS

- Add the full pre-computation pattern:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Date of Max Sales] :=
VAR DailySalesTable =
```

```

ADDCOLUMNS (
VALUES ( 'Date'[Date] ),
"DailyRevenue", [Sales Amount]
)
VAR MaxRevenue = MAXX ( DailySalesTable, [DailyRevenue] )
VAR DateOfMax =
SELECTCOLUMNS (
FILTER ( DailySalesTable, [DailyRevenue] = MaxRevenue ),
"TheDate", 'Date'[Date]
)
VAR Result =
IF ( COUNTROWS ( DateOfMax ) = 1, DateOfMax, BLANK () )
RETURN Result 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Date of Max Sales] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR DailySalesTable =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' ADDCOLUMNS (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=" VALUES ( 'Date'[Date] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=" \"DailyRevenue\", [Sales Amount]\", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR MaxRevenue = MAXX (
DailySalesTable, [DailyRevenue] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR DateOfMax =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' SELECTCOLUMNS (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' FILTER ( DailySalesTable, [DailyRevenue] = MaxRevenue ),',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,

```

```

fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="TheDate", \Date\[Date]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' '),
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Result =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' IF ( COUNTROWS (
DateOfMax ) = 1, DateOfMax, BLANK () )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='RETURN Result', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph

```

- Place on a matrix with Date[Year] on rows. Verify each year shows the exact date that had the maximum sales. If blank, multiple dates tied for the maximum — the IF correctly returns blank.
- Open **Performance Analyzer**. Compare [Max Daily Sales] and [Date of Max Sales]. Note [Date of Max Sales] may show a higher DAX query time — ADDCOLUMNS builds a complete daily sales table. This is the cost of pre-computation. The benefit is that MaxRevenue and DateOfMax both scan the pre-built variable rather than recomputing [Sales Amount] twice.

Task 3: Add a 30-day moving average

- Add the moving average measure:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[30-Day Moving Avg Sales] :=
VAR LastDate = MAX ( 'Date'[Date] )
VAR Days = 30
VAR DateWindow =
FILTER (
ALL ( 'Date' ),
'Date'[Date] > LastDate - Days
&& 'Date'[Date] <= LastDate
)
VAR Result =
CALCULATE (
AVERAGEX ( 'Date', [Sales Amount] ),
DateWindow
)
RETURN Result" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[30-Day Moving Avg Sales] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR LastDate = MAX (
'Date'[Date] )", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',

```



```

bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Days = 30',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR DateWindow =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' FILTER (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" ALL ( 'Date' ),",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" 'Date'[Date] > LastDate - Days",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" 'Date'[Date] ",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='<',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='= LastDate',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' CALCULATE (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" AVERAGEX ( 'Date', [Sales
Amount] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,

```

```

fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' DateWindow',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' '),
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN Result',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Place on a line chart with Date[Date] on the X-axis. Verify the line is smoother than the raw daily sales line — it averages the trailing 30 days.
3. Save as **Contoso_Lab06_Complete.pbix**.

Results: Max Daily Sales using MAXX. Date of Max using ADDCOLUMNS pre-computation. 30-day moving average using a date-window table variable. Performance Analyzer shows DailySalesTable computed once for both MAXX and FILTER.

Module Review and Takeaways

Review Questions

What are the two primary reasons variables exist in DAX?

Answer: Readability and performance. Readability: variables allow you to name sub-expressions and build formulas as a sequence of clearly labelled steps with a single, readable RETURN line. Performance: DAX evaluates a variable's expression exactly once, regardless of how many times the variable is referenced. Without variables, a sub-expression referenced multiple times is evaluated multiple times.

A variable is declared as `VAR X = [Sales Amount]`. Later in the measure, the measure is placed in a cell where the filter context is `Category=Audio`. Inside the RETURN block, `CALCULATE(X, ALL('Product'))` is called. What does this expression return?

Answer: It returns the value of X — the sales amount frozen at the time X was declared. X was already evaluated in the current filter context (`Category=Audio`) and stored as a scalar. `CALCULATE` cannot change the value of a frozen scalar. `CALCULATE(scalar, filter) = scalar`, regardless of the filter argument. To get the grand total, `[Sales Amount]` (a live measure reference) must be passed to `CALCULATE`, not a variable.

Why does using a measure reference (instead of a variable) for the average inside a FILTER iteration cause the wrong result?

Answer: Every measure reference in a row context triggers context transition. `FILTER` iterates Customer row by row. On each row, the measure reference `AVERAGEX(Customer,[Sales Amount])` has context transition applied — it is filtered to the current customer. The result is that customer's own sales, not the average across all customers. The fix: compute the average once as a variable before the `FILTER` iteration. The variable freezes the average in the outer context, and `FILTER`'s row context has no effect on the frozen scalar.

What is the scope of a variable defined inside a SUMX expression?

Answer: Variables defined inside the expression argument of an iterator (`SUMX`, `FILTER`, etc.) are scoped to the individual row evaluation of that iterator. They are re-assigned for each new row and are not accessible outside the iterator's expression. Variables defined before the iterator (in the outer scope) remain accessible inside the iterator.

What does lazy evaluation mean for variables, and what practical benefit does it provide?

Answer: DAX uses lazy evaluation: a variable's expression is only evaluated if the variable is actually referenced in the RETURN block. If a variable is never used, its expression is skipped entirely. The practical benefit: you can define expensive sub-expressions inside IF branches, and they only execute when that branch is taken. For example, a `RANKX` calculation inside an `IF(HASONEVALUE(...))` branch only runs when a single value is visible — not on Total rows.

Describe the ADDCOLUMNS pre-computation pattern and explain when to use it.

Answer: The pattern: build a table variable using `ADDCOLUMNS` to compute a measure at a specific granularity (e.g. one row per date). Then query that table variable multiple times — with `MAXX`, `FILTER`, `AVERAGEX`, etc. Each subsequent query scans the pre-computed variable rather than re-triggering context transition on the measure. Use it when the same measure must be evaluated at a fine granularity and then aggregated or filtered in multiple ways — avoiding redundant re-evaluation.

What is the recommended style rule for the RETURN statement?

Answer: The RETURN statement should always consist of a single variable reference — never a complex expression. Assign the final result to a variable (e.g. VAR Result = ...) and write RETURN Result. This makes every intermediate step instantly inspectable by simply changing which variable follows RETURN, with no other code changes needed.

Module 6 — Quick Reference

Concept	Key Behaviour	Use / Caution
<code>VAR name = expr</code>	Declares a named variable. Expression evaluated once, in the current context at declaration time.	Any DAX measure or calculated column
<code>RETURN expr</code>	Returns the value of <code>expr</code> from the enclosing <code>VAR</code> block. Should always be a single variable reference.	Required closing of every <code>VAR</code> block
<code>Variable = frozen value</code>	Stores a value, not a formula. <code>CALCULATE</code> cannot change it. Evaluated once at declaration.	Avoid in <code>CALCULATE</code> expressions needing re-evaluation
<code>Measure reference = live expr</code>	Re-evaluated in whatever context is active at the point of call. <code>CALCULATE</code> modifies the context and the measure re-evaluates.	Use inside <code>CALCULATE</code> for true denominators
Scope: outer <code>VAR</code>	Accessible from declaration to end of <code>RETURN</code> block, including inside nested iterators.	Cross-iterator value capture
Scope: inner <code>VAR</code> (inside iterator)	Accessible only within that iterator's per-row expression. Re-assigned each row.	Per-row intermediate values
Lazy evaluation	Variable only evaluated if referenced in <code>RETURN</code> . Unused variables are skipped.	Conditional expensive calculations
<code>ADDCOLUMNS</code> pattern	Enrich a table with computed measure columns. Store in a variable. Query the variable multiple times without re-evaluating the measure.	Max/min/date-of-max patterns
Naming conflict	Variable name shadows a table name in its scope. Avoid naming variables after tables.	Always use descriptive prefixed names

Module Summary

- Variables exist for two reasons: readability and performance. They allow you to name sub-expressions, build formulas as readable sequences of steps, and guarantee that each sub-expression is evaluated exactly once.
- A variable stores a value, not a formula. It is evaluated once at the point of its `VAR` declaration, in the evaluation context active at that moment. After that, the variable is a frozen scalar or table — it cannot change.
- `CALCULATE` cannot re-evaluate a variable. `CALCULATE(variable, filter)` returns the variable's frozen value regardless of the filter. To get `CALCULATE` to re-evaluate something in a new context, pass a measure reference — not a variable.
- Variables are scoped from their declaration to the end of the enclosing `RETURN` block. Variables defined inside an iterator are scoped to that iterator's per-row evaluation. Outer-scope variables are accessible inside nested iterators.
- DAX uses lazy evaluation: variables that are never referenced are never evaluated. Use this to place expensive calculations inside `IF` branches so they only run when needed.
- Table variables can hold complete table expressions. Storing a `FILTER` or `CALCULATETABLE` result in a variable and reusing it avoids re-executing the filter multiple times. The `ADDCOLUMNS` pre-computation pattern uses table variables to compute a measure at granularity once and then query the result multiple times efficiently.

- The critical best practice: always write RETURN as a single variable reference. Never name variables after table names. Use a variable for any sub-expression referenced more than once. Use a variable (not a measure reference) to capture an outer-context value before entering an inner iteration.

NOTE

Next — Module 7: Working with Iterators and CALCULATE. With variables fully mastered, Module 7 explores the full power of iterators: SUMX, AVERAGEX, MAXX, MINX, CONCATENATEX, RANKX, ADDCOLUMNS, SELECTCOLUMNS, and the granularity-changing patterns used in daily and moving calculations. All the patterns introduced at the end of this module will be developed fully in Module 7.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

7

Working with Iterators

SUMX · AVERAGEX · MAXX · RANKX · CONCATENATEX · ADDCOLUMNS · Granularity

Master the full power of DAX iterators combined with context transition.

Learn the two kinds of iteration, the granularity-first design rule, nested iterators and cardinality, CONCATENATEX string building, ADDCOLUMNS/SELECTCOLUMNS, the RANKX three-step algorithm, and the working-day granularity pattern.

Prerequisite: Modules 1–6.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 7: Working with Iterators

Module Contents

■ Module Overview — Two Kinds of Iteration

■ Lesson 1: Iterator Anatomy — What Makes a Function an Iterator

- The Iterator Contract: Table + Expression + Aggregation
- Row Context Inside an Iterator
- The Granularity-First Design Rule

■ Lesson 2: Simple Iterators — SUMX, AVERAGEX, MINX, MAXX, COUNTX

- SUMX — Row-by-Row Summation
- AVERAGEX — Ignoring Blanks
- MINX and MAXX — Row-by-Row Min/Max
- COUNTX and COUNTAX

■ Lesson 3: Iterators with Context Transition

- The Measure Reference Pattern
- Max Daily Sales — MAXX with a Measure
- Average Sales per Customer
- Performance and Cardinality Warning

■ Lesson 4: Nested Iterators — Cardinality and Performance

- What Is a Nested Iterator?
- Cardinality of Nested Iterations
- When to Avoid Nesting

■ Lesson 5: CONCATENATEX — String Aggregation

- Basic CONCATENATEX
- Sorting with CONCATENATEX
- The Selected Colors Label Pattern

■ Lesson 6: Table-Returning Iterators — ADDCOLUMNS and SELECTCOLUMNS

- ADDCOLUMNS — Enriching a Table with Computed Columns
- SELECTCOLUMNS — Projecting a Column Subset
- Using ADDCOLUMNS in Measures for Pre-computation

■ Lesson 7: RANKX — The Three-Step Algorithm

- How RANKX Works (Steps 1, 2, 3)
- RANKX Parameters — Value, Order, Ties
- Protecting RANKX with HASONEVALUE
- ALL vs ALLSELECTED in RANKX

■ Lesson 8: Changing Calculation Granularity

- Why Simple Division Fails at the Total Level
- The Granularity Pattern — Iterate at Month Level
- Choosing the Correct Granularity Column

■ Demonstration 1: MAXX, AVERAGEX, and Context Transition

■ Demonstration 2: RANKX Deep Dive — Ties, ALLSELECTED, and Protection

■ Lab 7: Working with Iterators

- Exercise 1: Simple Iterators and Sales at List Price
- Exercise 2: Context Transition Iterators

- Exercise 3: CONCATENATEX and RANKX
- Exercise 4: Granularity — Sales per Working Day

■ Module Review and Takeaways

Module Overview — Two Kinds of Iteration

Every DAX iterator follows the same contract: it takes a table as its first argument, evaluates an expression in the row context of each row of that table, and aggregates the results. What makes iterators powerful is that the expression argument can include a measure reference — which triggers context transition and allows the iterator to compute any measure at the granularity of each row, regardless of the surrounding filter context.

There are **two kinds of iteration** in DAX:

- **Row-by-row calculation:** the expression uses column references directly (e.g. `Sales[Quantity] * Sales[Net Price]`). No context transition. Fast. Used for simple line-level computations.
- **Context-transition iteration:** the expression includes a measure reference (or an explicit `CALCULATE`), triggering context transition on each row. The measure is evaluated in a filter context equivalent to that row. Powerful but more expensive — always check the cardinality of the outer table.

CONCEPT

The three-part design rule for iterators: (1) choose the granularity — what table to iterate over; (2) choose the expression — what to compute per row; (3) choose the aggregation — how to combine the per-row results (SUM, MAX, AVERAGE, etc.). Always decide the granularity first. The wrong granularity produces wrong totals regardless of how correct the per-row expression is.

LESSON 1

Iterator Anatomy — What Makes a Function an Iterator

Lesson Objectives

After completing this lesson, you will be able to:

- Define what makes a DAX function an iterator.
- Describe the row context that exists inside any iterator.
- State the three-part granularity-first design rule.
- Identify iterator functions by their X suffix or ROW CONTEXT marker.

The Iterator Contract

An iterator is any DAX function whose second argument is an expression evaluated in a row context. The iterator creates a row context for each row of the first argument (table), evaluates the second argument in that context, and then aggregates. Functions with this property are marked ROW CONTEXT in the official DAX.guide documentation:

```
-- Every iterator follows this pattern:
ITERATORX ( , )

-- Examples:
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] ) -- sum of row values
AVERAGEX ( Customer, [Sales Amount] ) -- avg measure per customer
MAXX ( 'Date', [Sales Amount] ) -- max measure per date
MINX ( Sales, Sales[Net Price] ) -- min column value per row
COUNTX ( Sales, Sales[Net Price] ) -- count non-blank values
RANKX ( ALL('Product'[Category]), [Sales Amount] ) -- rank by measure value
CONCATENATEX ( VALUES('Product'[Color]), 'Product'[Color], ", " ) -- string join
```

Row Context Inside an Iterator

While an iterator processes each row of its table, a row context exists for that table. Column references in the expression resolve to the current row's values. Crucially, this row context does **not** automatically become a filter context — for that, CALCULATE (explicit or implicit via measure reference) is needed:

```
-- Row context: Sales[Quantity] and Sales[Net Price] refer to the CURRENT ROW
SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
-- On each Sales row, DAX reads that row's Quantity and Net Price, multiplies them.
-- No context transition. Row context only. Fast.

-- Context transition: [Sales Amount] is a measure reference.
-- AVERAGEX triggers CT for each Customer row.
AVERAGEX ( Customer, [Sales Amount] )
-- For each Customer row, CT fires: Customer table filtered to current customer.
-- [Sales Amount] evaluates in that filter context -> that customer's total sales.
-- AVERAGEX averages all those per-customer totals. Slower than row-by-row.
```


EXAM

PL-300 tests the distinction between iterators that use column references (no context transition) vs measure references (context transition fires). Know that `SUMX(Sales, Sales[Quantity]*Sales[Net Price])` has no CT, while `AVERAGEX(Customer, [Sales Amount])` does. The performance cost of CT scales with the cardinality of the iterator's table.

LESSON 2

Simple Iterators — SUMX, AVERAGEX, MINX, MAXX, COUNTX**Lesson Objectives**

After completing this lesson, you will be able to:

- Write SUMX for line-level arithmetic.
- Explain why AVERAGEX ignores blank values and how to force zero-counting.
- Use MAXX and MINX to find extreme values at a chosen granularity.
- Distinguish COUNTX (non-blank numeric) from COUNTAX (non-blank any type).

SUMX — Row-by-Row Summation

```
-- Sales Amount using SUMX (required: column arithmetic is row-level)
[Sales Amount] := SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
-- Cannot use SUM here because SUM cannot multiply two columns.

-- Sales at list price (Product[Unit Price] is in a related table)
[Sales at List Price] :=
SUMX (
Sales,
Sales[Quantity] * RELATED ( 'Product'[Unit Price] ) -- RELATED in row context
)
-- RELATED works because Sales row context is active during the iteration.
-- For each Sales row, RELATED follows the relationship to get the related Product row.

-- Discount amount
[Discount Amount] :=
SUMX ( Sales, Sales[Quantity] * ( RELATED('Product'[Unit Price]) - Sales[Net Price] ) )
```

AVERAGEX — Ignoring Blanks

```
-- Average sales per customer (all customers with any sales)
[Avg Sales per Customer] := AVERAGEX ( Customer, [Sales Amount] )
-- AVERAGEX ignores customers where [Sales Amount] returns BLANK.
-- Denominator = count of customers with non-blank sales.

-- If you want zero-sales customers counted in the denominator:
[Avg Sales per Customer Incl Zero] :=
DIVIDE ( [Sales Amount], DISTINCTCOUNT ( Sales[CustomerKey] ) )
-- Divides total sales by count of distinct customers who appear in Sales.
-- Does not count customers with NO sales rows at all (they are not in Sales).

-- Average line amount per order (ignoring lines with zero price)
[Avg Line Amount] := AVERAGEX ( Sales, Sales[Quantity] * Sales[Net Price] )
```

MAXX and MINX

```
-- Maximum sale price in current filter context
[Max Net Price] := MAXX ( Sales, Sales[Net Price] )

-- Maximum daily sales (using measure reference -> context transition)
[Max Daily Sales] := MAXX ( 'Date', [Sales Amount] )
-- Iterates every date in the filter context.
```

```
-- For each date, CT filters to that date -> [Sales Amount] = that day's sales.  
-- MAXX returns the highest single-day value.  
  
-- Minimum non-zero price  
[Min Net Price Nonzero] :=  
MINX ( FILTER ( Sales, Sales[Net Price] > 0 ), Sales[Net Price] )
```

COUNTX and COUNTAX

COUNTX counts rows where the expression evaluates to a non-blank number. COUNTAX counts non-blank values of any data type. Both are less commonly needed than COUNTROWS:

```
-- Count rows where Net Price is not blank (same as COUNTROWS for most numeric columns)  
[Count Priced Lines] := COUNTX ( Sales, Sales[Net Price] )  
  
-- Count rows where a text value is present  
[Count Named Products] := COUNTAX ( 'Product', 'Product'[Product Name] )  
  
-- Prefer COUNTROWS for clarity:  
[Order Count] := COUNTROWS ( Sales ) -- counts ALL rows, including blank-expression rows  
-- COUNTX(Sales, expr) < COUNTROWS(Sales) when expr evaluates to blank on some rows.
```

LESSON 3

Iterators with Context Transition

Lesson Objectives

After completing this lesson, you will be able to:

- Explain how a measure reference inside an iterator triggers context transition.
- Build Max Daily Sales using MAXX with a measure reference.
- Build Average Sales per Customer using AVERAGEX with context transition.
- State when context transition inside an iterator is expensive and how to avoid it.

The Measure Reference Pattern

Any measure reference inside an iterator triggers context transition for each row. The iterator creates a row context; the measure reference wraps itself in an implicit CALCULATE, converting the row context into an equivalent filter context. The measure then evaluates in that filter context — receiving only the data relevant to the current row:

```
-- Pattern: Granularity Table → Measure Reference → CT fires per row

-- Max daily sales: granularity = date, expression = [Sales Amount] (measure ref)
[Max Daily Sales] := MAXX ( 'Date', [Sales Amount] )
-- Row: Date = Jan 15 2023. CT fires -> Date table filtered to Jan 15 2023.
-- [Sales Amount] = total sales on Jan 15. MAXX finds the maximum across all dates.

-- Average sales per customer: granularity = customer, expression = [Sales Amount]
[Avg Sales per Customer] := AVERAGEX ( Customer, [Sales Amount] )
-- Row: CustomerKey = 1234. CT fires -> Customer filtered to CustomerKey=1234.
-- [Sales Amount] propagates to Sales via relationship -> that customer's sales.
-- AVERAGEX averages all per-customer values.

-- Both patterns rely on the relationship from the granularity table to Sales.
-- The relationship must be either active or enabled via USERELATIONSHIP.
```

Date of Max Sales — Combining MAXX with ADDCOLUMNS

```
-- Goal: find which specific date had the maximum sales.

-- Version 1: simple but [Sales Amount] evaluated twice
[Date of Max V1] :=
VAR MaxSales = MAXX ( 'Date', [Sales Amount] ) -- CT fires for each date
VAR DateOfMax =
FILTER (
VALUES ( 'Date'[Date] ),
[Sales Amount] = MaxSales -- CT fires AGAIN for each date
)
RETURN IF ( COUNTROWS(DateOfMax) = 1, DateOfMax, BLANK() )

-- Version 2: ADDCOLUMNS pre-computes daily sales once
[Date of Max V2] :=
VAR DailySales =
ADDCOLUMNS (
VALUES ( 'Date'[Date] ),
"Revenue", [Sales Amount] -- CT fires once per date, result stored
```

```
)  
VAR MaxSales = MAXX ( DailySales, [Revenue] ) -- scans stored table  
VAR DateOfMax =  
SELECTCOLUMNS (  
    FILTER ( DailySales, [Revenue] = MaxSales ), -- scans stored table  
    "TheDate", 'Date'[Date]  
)  
RETURN IF ( COUNTROWS(DateOfMax) = 1, DateOfMax, BLANK() )  
-- [Sales Amount] evaluates once per date (in ADDCOLUMNS).  
-- MAXX and FILTER both scan the pre-computed DailySales table -- no extra CT.
```

Performance and Cardinality Warning

WARNING

Context transition inside an iterator is expensive because it forces the engine to create a filter context per row. The cost scales with the cardinality of the iterator's table. `AVERAGEX(Customer,[Sales Amount])` over 100,000 customers = 100,000 context transitions. Always check the cardinality. If the outer table has millions of rows, consider whether the same result can be achieved by aggregating at a coarser level, pre-computing with `ADDCOLUMNS`, or restructuring the formula to avoid CT.

LESSON 4

Nested Iterators — Cardinality and Performance

Lesson Objectives

After completing this lesson, you will be able to:

- Identify when iterators are nested.
- Calculate the effective cardinality of a nested iteration.
- Explain why nested iterators require temporary in-memory tables.
- State the design alternative to avoid nested iterators in most cases.

Nested Iterator Cardinality

A nested iterator occurs when the expression of an outer iterator itself contains another iterator. The total cardinality is the product of the two table sizes. Only the innermost iterator can use the optimised (VertiPaq) query plan; outer iterators force in-memory temporary tables:

```
-- Sales at list price using RELATED (no nesting, no CT):
[Sales at List Price] :=
SUMX ( Sales, Sales[Quantity] * RELATED('Product'[Unit Price]) )
-- Cardinality: Sales rows only. Fast.

-- WRONG pattern: nested SUMX (cardinality = Products x Sales)
[Sales at List Price SLOW] :=
SUMX (
VALUES ( 'Product' ), -- outer: iterate Products (e.g. 2,500 rows)
SUMX (
Sales, -- inner: iterate ALL Sales per product (37,984 rows)
IF ( Sales[ProductKey] = 'Product'[ProductKey],
'Product'[Unit Price] * Sales[Quantity], 0 )
)
)
-- Cardinality: 2,500 Products x 37,984 Sales = ~95 million operations.
-- The inner SUMX iterates the full Sales table for EACH product.
-- Result is correct but extremely slow. Avoid this pattern.

-- CORRECT pattern with CT (cardinality = Products, inner uses CT+relationship):
[Sales at List Price CT] :=
SUMX (
VALUES ( 'Product' ),
'Product'[Unit Price] * CALCULATE ( SUM ( Sales[Quantity] ) )
)
-- CALCULATE triggers CT -> Sales filtered to current product.
-- SUM(Sales[Quantity]) = quantity for that product. Multiply by list price.
-- Cardinality: Products only (~2,500). Inner CALCULATE uses relationship filter.
```

TIP

When you have a nested iterator and the inner table is large, ask: can the inner aggregation be replaced by a measure reference with context transition? CT uses the relationship to push a filter to the inner table efficiently, rather than physically iterating every inner row per outer row.

LESSON 5

CONCATENATEX — String Aggregation**Lesson Objectives**

After completing this lesson, you will be able to:

- Write CONCATENATEX to produce a comma-separated list from a table.
- Add sorting and a separator to CONCATENATEX.
- Build a smart 'selected filters' label using CONCATENATEX and HASONEVALUE.

CONCATENATEX Syntax and Sorting

```
-- Basic: join all visible color values with a comma
[Selected Colors Basic] :=
CONCATENATEX ( VALUES ( 'Product'[Color] ), 'Product'[Color], ", " )
-- Syntax: CONCATENATEX ( table, expression, [separator], [order_by], [direction] )

-- With sorting (ASC alphabetical):
[Selected Colors Sorted] :=
CONCATENATEX ( VALUES ( 'Product'[Color] ), 'Product'[Color], ", ",
'Product'[Color], ASC )

-- Smart label: show 'All colors', 'N colors', or the actual list
[Color Filter Label] :=
VAR Colors = VALUES ( 'Product'[Color] )
VAR TotalColors = COUNTROWS ( ALL ( 'Product'[Color] ) )
VAR SelectedCount = COUNTROWS ( Colors )
VAR ColorList = CONCATENATEX ( Colors, 'Product'[Color], ", ", 'Product'[Color], ASC )
VAR Result =
SWITCH (
TRUE(),
SelectedCount = TotalColors, "All colors.",
SelectedCount > 5, SelectedCount & " colors selected.",
ColorList & "."
)
RETURN Result
-- Label changes dynamically with slicer selections.
-- 5 or fewer colors: shows the names. More than 5: shows count. All: generic message.
```

LESSON 6

Table-Returning Iterators — ADDCOLUMNS and SELECTCOLUMNS**Lesson Objectives**

After completing this lesson, you will be able to:

- Use ADDCOLUMNS to append computed columns to a table.
- Use SELECTCOLUMNS to project a column subset.
- Combine ADDCOLUMNS with variable patterns for performance.

ADDCOLUMNS and SELECTCOLUMNS

```
-- ADDCOLUMNS: returns original table columns PLUS new computed columns
-- Syntax: ADDCOLUMNS ( table, "ColName", expression [, "ColName2", expr2 ...] )

-- Example: add sales and product count per color
Colors =
ADDCOLUMNS (
VALUES ( 'Product'[Color] ), -- base table: one row per color
"SalesAmount", [Sales Amount], -- CT fires per color: that color's sales
"ProductCount", CALCULATE ( COUNTROWS ( 'Product' ) ) -- CT fires per color
)
-- Result: 3 columns: Color, SalesAmount, ProductCount

-- SELECTCOLUMNS: returns ONLY the specified columns (discards original columns)
-- Syntax: SELECTCOLUMNS ( table, "ColName", expression [, ...] )

Colors2 =
SELECTCOLUMNS (
VALUES ( 'Product'[Color] ),
"Color", 'Product'[Color], -- must explicitly include Color
"SalesAmount", [Sales Amount],
"ProductCount", CALCULATE ( COUNTROWS ( 'Product' ) )
)
-- Result identical, but Color column must be explicitly listed.

-- Performance pattern in measures:
[Date of Max V2] :=
VAR DailySales = ADDCOLUMNS ( VALUES ( 'Date'[Date] ), "Rev", [Sales Amount] )
VAR MaxRev = MAXX ( DailySales, [Rev] ) -- scans pre-built table
VAR MaxDates = SELECTCOLUMNS ( FILTER ( DailySales, [Rev] = MaxRev ),
"D", 'Date'[Date] )
RETURN IF ( COUNTROWS(MaxDates)=1, MaxDates, BLANK() )
```


LESSON 7

RANKX — The Three-Step Algorithm

Lesson Objectives

After completing this lesson, you will be able to:

- Explain RANKX's three steps: build lookup, evaluate current value, find position.
- Use HASONEVALUE to protect RANKX from producing meaningless totals.
- Explain the difference between SKIP (default) and DENSE tie-breaking.
- Explain why ALL vs ALLSELECTED produces different ranking gaps.
- Identify when a mismatched slicing granularity causes RANKX to always return 1.

How RANKX Works — Three Steps

RANKX is an iterator that operates in three distinct steps. Understanding these steps is essential for correct usage:

```
-- Full RANKX syntax:
RANKX ( , [ , [ , [ , ] ] ] )

-- Example: rank product categories by sales amount
[Rank Cat on Sales] :=
RANKX (
ALL ( 'Product'[Category] ), -- Step 1 table: all categories
[Sales Amount] -- Step 1 + Step 2 expression
)

-- Step 1: Build a lookup table.
-- Iterate ALL('Product'[Category]). For each category row, CT fires.
-- [Sales Amount] evaluates in that category's filter context.
-- Result: a lookup table of (Sales Amount) values, one per category, sorted DESC.
-- E.g.: [4,500,000 | 3,200,000 | 2,800,000 | 1,100,000 | ...]

-- Step 2: Evaluate expression in the ORIGINAL (outer) filter context.
-- If current cell = Cameras and Camcorders:
-- Filter context = {Category = Cameras and Camcorders}.
-- [Sales Amount] = 3,200,000 (that category's total).

-- Step 3: Find position of 3,200,000 in the sorted lookup table.
-- [4,500,000 | 3,200,000 | ...] -> position 2. Result = 2.

-- Grand total problem: at the Total row, filter context = all categories.
-- Step 2: [Sales Amount] = grand total (sum of all) = e.g. 12,000,000.
-- Step 3: 12,000,000 > any single-category value -> position 1. Meaningless!

-- Fix: protect with HASONEVALUE
[Rank Cat Protected] :=
IF (
HASONEVALUE ( 'Product'[Category] ),
RANKX ( ALL ( 'Product'[Category] ), [Sales Amount] ),
BLANK()
)
-- Returns BLANK on total rows. Clean output.
```

RANKX Parameters — Ties (SKIP vs DENSE) and Sort Order

```
-- Default: DESC order (highest value = rank 1), SKIP ties.
[Brand Rank Default] := RANKX ( ALL('Product'[Brand]), [Sales Amount] )

-- ASC order: lowest value = rank 1
[Brand Rank ASC] := RANKX ( ALL('Product'[Brand]), [Sales Amount], , ASC )

-- DENSE ties: when two brands tie, both get the same rank,
-- and the next rank is +1 (no gaps).
[Brand Rank Dense] := RANKX ( ALL('Product'[Brand]), [Sales Amount], , , DENSE )

-- SKIP ties (default): when two brands tie at rank 3,
-- both get rank 3, and the next brand gets rank 5 (skips 4).
[Brand Rank Skip] := RANKX ( ALL('Product'[Brand]), [Sales Amount], , , SKIP )

-- ALLSELECTED: rank only within what the user has selected
-- (removes gaps that ALL would create when some brands are filtered out)
[Brand Rank Selected] :=
IF (
HASONEVALUE ( 'Product'[Brand] ),
RANKX ( ALLSELECTED ( 'Product'[Brand] ), [Sales Amount] ),
BLANK()
)
```

The Cross-Slicing Trap

WARNING

If you slice a RANKX measure by a field that is finer than the ranking dimension, the result is almost always 1 (or meaningless). Example: ranking brands by sales amount, then adding a Color slicer. The lookup table (Step 1) is built per brand AND per color (CT fires). The current value (Step 2) is also per brand AND per color. Every brand's value in the current cell is its highest possible slice of itself — always $\geq$ all its subsets in the lookup table — resulting in rank 1 on every row. Protect with `HASONEVALUE` on the ranking column.

LESSON 8

Changing Calculation Granularity

Lesson Objectives

After completing this lesson, you will be able to:

- Explain why simple division fails when computing per-working-day at aggregated levels.
- Apply the granularity pattern: iterate the correct granularity column.
- Explain why iterating a Month column fails at the Grand Total and how to fix it.

Why Simple Division Fails at the Total Level

A common requirement: show sales per working day for each month. A naive DIVIDE fails at the year and grand total levels because the denominator (working days) is computed over the full visible date range, including months with no sales:

```
-- Working day helpers (Date table has IsWorkingDay = 1 or 0)
[NumOfWorkingDays] := SUM ( 'Date'[IsWorkingDay] )
[NumOfDays] := COUNTROWS ( 'Date' )

-- WRONG: simple division
[SalesPerWorkingDay Wrong] := DIVIDE ( [Sales Amount], [NumOfWorkingDays] )
-- At month level: correct (e.g. Jan 2007 = 47,000 / 23 working days = 2,043)
-- At year level: WRONG (e.g. 2007 = total annual sales / 261 working days)
-- 261 includes months before Contoso's first sale – inflated denominator.

-- CORRECT: iterate months, accumulate working days only in months WITH sales
[SalesPerWorkingDay V1] :=
VAR WorkingDays =
SUMX (
VALUES ( 'Date'[Month] ), -- iterate distinct month values
IF ( [Sales Amount] > 0, -- only count months with sales
[NumOfWorkingDays] ) -- CT fires: working days in that month
)
VAR Result = DIVIDE ( [Sales Amount], WorkingDays )
RETURN Result
-- Year level is now correct. But Grand Total still fails!
-- Reason: VALUES('Date'[Month]) at Grand Total returns month names across ALL years.
-- Jan 2007 and Jan 2008 are both named 'January' -> collapsed to one row!
-- The denominator under-counts because distinct month names < actual month+year pairs.

-- FIX: iterate the year-month combination column
[SalesPerWorkingDay V2] :=
VAR WorkingDays =
SUMX (
VALUES ( 'Date'[Year Month Number] ), -- unique per year+month (e.g. 200701)
IF ( [Sales Amount] > 0, [NumOfWorkingDays] )
)
VAR Result = DIVIDE ( [Sales Amount], WorkingDays )
RETURN Result
-- Year Month Number (integer like 200701) is unique per year+month pair.
-- VALUES over this column returns distinct year+months in filter context.
-- At Grand Total: all year+months that have sales data -> correct denominator.
```

CONCEPT

The granularity rule: when changing calculation granularity with an iterator, the column you iterate must be unique at the level you want. If you iterate a Month Name column, two different years' Januaries collapse into one row. Use a column that is unique per year+month (Year Month Number, or EOMONTH). The granularity failure always manifests at the aggregate level — individual row values look correct but totals are wrong.

DEMONSTRATION 1

MAXX, AVERAGEX, and Context Transition

Scenario: The instructor builds three measures that leverage context transition inside iterators — Max Daily Sales, Date of Max, and Average Sales per Customer — demonstrating the full CT mechanics and the ADDCOLUMNS pre-computation optimisation.

1	Start with [Max Daily Sales] = MAXX('Date',[Sales Amount]). Show on matrix by Year. Trace one cell: 'Date' iteration -> Date=Jan 15 -> CT -> Sales filtered to Jan 15 -> measure evaluates -> that day's sales. MAXX = highest.
2	Add [Date of Max V1] using MAXX + FILTER with measure reference twice. Show result. Open Performance Analyzer, note DAX query time.
3	Refactor to [Date of Max V2] using ADDCOLUMNS(VALUES('Date'[Date]),"Rev",[Sales Amount]). Show that [Sales Amount] is computed once per date inside ADDCOLUMNS. MAXX and FILTER both scan the pre-built DailySales variable. Re-run Performance Analyzer. Show improvement.
4	Add [Avg Sales per Customer] = AVERAGEX(Customer,[Sales Amount]). Show on Card. Apply Country slicer. Show the average updates — CT restricts the Sales data to the filtered customers via the Customer->Sales relationship.
5	Discuss the cardinality cost: if Customer has 18,000 rows, CT fires 18,000 times. Compare with a simple DIVIDE([Sales Amount], DISTINCTCOUNT(Sales[CustomerKey])) — much faster, but counts only customers who appear in Sales, not all customers.
RESULT	All three CT iterator patterns demonstrated with Performance Analyzer comparison. ADDCOLUMNS optimisation validated.

DEMONSTRATION 2

RANKX Deep Dive — Ties, ALLSELECTED, and Protection

Scenario: The instructor builds the RANKX measure from scratch, demonstrates the meaningless total row problem, adds protection, and shows the difference between ALL and ALLSELECTED in rankings.

1	Add [Rank Cat on Sales] = RANKX(ALL('Product'[Category]),[Sales Amount]). Place on matrix with Category. Show correct rankings. Show Grand Total = 1 (meaningless).
2	Trace one row live: Audio category. Step 1: ALL(Category) -> iterate all 8 categories -> build sorted lookup of their sales amounts. Step 2: outer context = Audio -> [Sales Amount] = Audio total. Step 3: find Audio's value in lookup -> position = rank.
3	Add HASONEVALUE protection. Show Grand Total becomes blank. Add a Brand slicer -> observe ranking still works (lookup and value both filtered by Brand via CT).
4	Demonstrate SKIP vs DENSE: add a [Rounded Sales] measure that rounds to nearest million. Show that two brands with the same rounded sales both get rank 3 (SKIP shows gap to 5). Add DENSE version — no gap.
5	Demonstrate the cross-slicing trap: remove protection, add Color slicer. Show rank = 1 on almost every row. Explain the Step 1 / Step 2 mismatch. Add protection back.
6	Switch ALL to ALLSELECTED. Filter Category slicer to 4 categories. Show ALL version has gaps (ranks 1,3,5,7). Show ALLSELECTED version is consecutive (1,2,3,4).
RESULT	Full RANKX algorithm traced. All five parameters demonstrated. ALL vs ALLSELECTED gap behaviour confirmed visually.

LAB 7

Working with Iterators

Lab Scenario

The Contoso analytics team needs advanced aggregation measures: sales at list price, maximum/average at custom granularities, a product rank leaderboard, a color filter label, and a corrected sales-per-working-day measure. You will build all of these using the iterator patterns from this module.

NOTE

Open Contoso_Lab07_Start.pbix (extends Lab 6). The Date table contains IsWorkingDay, Year Month Number, Month, and Year columns. Save completed work as Contoso_Lab07_Complete.pbix.

Exercise 1: Simple Iterators and Sales at List Price

Estimated time: 15 minutes

Task 1: Sales at List Price using RELATED

1. Add the following measure:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales at List Price] :=
SUMX ( Sales, Sales[Quantity] * RELATED ( 'Product'[Unit Price] ) )" 'frags': [ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Sales at List Price] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="SUMX ( Sales, Sales[Quantity] *
RELATED ( 'Product'[Unit Price] ) )", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style':
'bulletText': None 'debug': 0 ) #Paragraph
```

2. Place on a matrix with Product[Category] on rows, alongside [Sales Amount]. Show the difference — List Price is higher. The discount = [Sales at List Price] - [Sales Amount].

3. Add the discount measure:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Discount Amount] :=
SUMX ( Sales, Sales[Quantity] * ( RELATED('Product'[Unit Price]) - Sales[Net Price] ) )" 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='[Discount Amount] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="SUMX ( Sales, Sales[Quantity] * ( RELATED('Product'[Unit Price]) - Sales[Net Price] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="SUMX ( Sales, Sales[Quantity] * ( RELATED('Product'[Unit Price]) - Sales[Net Price] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

Task 2: MINX and MAXX on columns

1. Add measures for the minimum and maximum net price in any context:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Min Net Price] := MINX ( Sales, Sales[Net Price] )
[Max Net Price] := MAXX ( Sales, Sales[Net Price] )' 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Min Net Price] := MINX ( Sales,
Sales[Net Price] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="SUMX ( Sales, Sales[Quantity] * ( RELATED('Product'[Unit Price]) - Sales[Net Price] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
```

```
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Max Net Price] := MAXX (
Sales, Sales[Net Price] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText':
None 'debug': 0 ) #Paragraph
```

Results: Sales at List Price vs Sales Amount comparison. Discount Amount. Min/Max net price varying by Category slicer.

Exercise 2: Context Transition Iterators

Estimated time: 25 minutes

Task 1: Max Daily Sales and Date of Max

1. Add [Max Daily Sales] := MAXX('Date',[Sales Amount]).
2. Add the optimised Date of Max measure using ADDCOLUMNS:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Date of Max Sales] :=
VAR DailySales =
ADDCOLUMNS ( VALUES ( \'Date\'[Date] ), "Rev", [Sales Amount] )
VAR MaxRev = MAXX ( DailySales, [Rev] )
VAR MaxDate =
SELECTCOLUMNS (
FILTER ( DailySales, [Rev] = MaxRev ),
"TheDate", \'Date\'[Date]
)
RETURN IF ( COUNTROWS(MaxDate)=1, MaxDate, BLANK() ) 'frags': [ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Date of Max Sales] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR DailySales =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ADDCOLUMNS ( VALUES (
\'Date\'[Date] ), "Rev", [Sales Amount] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR MaxRev = MAXX ( DailySales, [Rev] )', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR MaxDate =', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' SELECTCOLUMNS (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' FILTER ( DailySales, [Rev] = MaxRev )', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
```



```

text= "TheDate", \Date\[Date], textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=')', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN IF (
COUNTROWS(MaxDate)=1, MaxDate, BLANK() )', textColor=Color(.101961,.227451,.360784,1),
us_lines=[])] 'style': 'bulletText': None 'debug': 0 ) #Paragraph

```

3. Place on a matrix with Year on rows. Verify each year shows its peak sales date.

Task 2: Average Sales per Customer

1. Add [Avg Sales per Customer] := AVERAGEX(Customer,[Sales Amount]).
2. Place on a Card. Apply Country and Year slicers. Confirm the average updates correctly.
3. Open Performance Analyzer. Record a slicer interaction. Note the query time.

Results: Date of Max shows correct peak dates per year. Avg Sales per Customer updates with slicers. Performance Analyzer baseline recorded.

Exercise 3: CONCATENATEX and RANKX

Estimated time: 25 minutes

Task 1: Color Filter Label

1. Add the smart color label measure:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Color Filter Label] :=
VAR Colors = VALUES ( \Product\[Color] )
VAR TotalColors = COUNTROWS ( ALL ( \Product\[Color] ) )
VAR SelectedCount = COUNTROWS ( Colors )
VAR ColorList = CONCATENATEX ( Colors, \Product\[Color], ", ",
\Product\[Color], ASC )
RETURN SWITCH ( TRUE(),
SelectedCount = TotalColors, "All colors.",
SelectedCount > 5, SelectedCount & " colors selected.",
ColorList & "." ) 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0,
italic=0, link=[], rise=0, text='[Color Filter Label] :=', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="VAR Colors = VALUES ( 'Product'[Color] )", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="VAR TotalColors = COUNTROWS ( ALL ( 'Product'[Color] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR SelectedCount =
COUNTROWS ( Colors )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),

```

```

ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR ColorList = CONCATENATEX ( Colors, \'Product\'[Color], ", "',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" 'Product'[Color], ASC )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN SWITCH ( TRUE(),',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' SelectedCount = TotalColors,
"All colors."', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' SelectedCount > 5,
SelectedCount ', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' " colors selected."',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ColorList ',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' " " '),
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Add a Card visual with [Color Filter Label]. Add a Color slicer. Select 1, 3, 6, and all colors. Verify the label changes appropriately.

Task 2: Category Rank by Sales

1. Add the protected RANKX measure:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Category Rank] :=
IF (
HASONEVALUE ( 'Product'[Category] ),
RANKX ( ALL ( 'Product'[Category] ), [Sales Amount] ),
BLANK()
)" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
rise=0, text='[Category Rank] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='IF (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" HASONEVALUE (
'Product'[Category] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),

```

```

ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=" RANKX ( ALL ( 'Product'[Category] ), [Sales Amount] ),",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' BLANK()',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=')',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

- Place on a matrix with Category on rows alongside [Sales Amount]. Verify the Total row is blank. Verify rank 1 = highest-selling category.
- Switch ALL to ALLSELECTED. Filter Category slicer to 4 categories. Confirm rankings are 1–4 (no gaps).

Results: Color label dynamic with slicer. Category Rank correct with blank total. ALLSELECTED version shows gap-free rankings within selection.

Exercise 4: Granularity — Sales per Working Day

Estimated time: 20 minutes

Task 1: Naive version and confirm the total is wrong

- Add [NumOfWorkingDays] := SUM('Date'[IsWorkingDay]).
- Add [SalesPerWorkingDay Wrong] := DIVIDE([Sales Amount],[NumOfWorkingDays]).
- Place on matrix with Year > Month hierarchy. Note monthly values look plausible. Note the Year and Grand Total values are surprisingly low.

Task 2: Fix using Year Month Number granularity

- Add the corrected measure:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales per Working Day] :=
VAR WorkingDays =
SUMX (
VALUES ( 'Date'[Year Month Number] ),
IF ( [Sales Amount] > 0, [NumOfWorkingDays] )
)
RETURN DIVIDE ( [Sales Amount], WorkingDays )" 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[Sales per Working Day] :=",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR WorkingDays =',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' SUMX (',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,

```

```

fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" VALUES ( 'Date'[Year Month
Number] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' IF ( [Sales Amount] > 0,
[NumOfWorkingDays] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=' )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN DIVIDE ( [Sales
Amount], WorkingDays )', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText':
None 'debug': 0 ) #Paragraph

```

2. Add to the matrix. Compare with [SalesPerWorkingDay Wrong]. Verify Year and Grand Total are now reasonable (comparable to monthly values). The fixed version's year total should be close to the average of its months.
3. Save as **Contoso_Lab07_Complete.pbix**.

Results: Naive version confirmed wrong at totals. Fixed version uses Year Month Number granularity to count only months with sales. Year and Grand Total now correct.

Module Review and Takeaways

Review Questions

What are the two kinds of iteration in DAX and when is each appropriate?

Answer: Row-by-row calculation: the expression uses direct column references (e.g. `Sales[Quantity] * Sales[Net Price]`). No context transition, no relationship traversal. Used for simple line-level arithmetic. Fast. Context-transition iteration: the expression contains a measure reference (or explicit `CALCULATE`), triggering CT per row. Allows the iterator to evaluate any measure at the granularity of each row. Powerful but expensive — cost scales with cardinality.

What is the three-part granularity-first design rule for iterators?

Answer: (1) Granularity: choose the table to iterate — this determines the level of detail. (2) Expression: define what to compute per row of that table. (3) Aggregation: decide how to combine the per-row results (`SUM`, `MAX`, `AVERAGE`, etc.). Always decide the granularity first. Wrong granularity = wrong totals, even if the per-row expression is correct.

Explain RANKX's three-step algorithm.

Answer: Step 1: `RANKX` iterates the first argument table. For each row, CT fires, the second argument (expression) is evaluated, and the results form a sorted lookup table. Step 2: `RANKX` evaluates the second argument in the original (outer) filter context, producing the value for the current cell. Step 3: `RANKX` finds the position of the Step 2 value in the sorted lookup table. That position is the rank. The Grand Total always shows 1 because the total value exceeds all individual subtotals — protect with `HASONEVALUE`.

Why does iterating 'Date'[Month] fail at the Grand Total in the SalesPerWorkingDay pattern?

Answer: `VALUES('Date'[Month])` at the Grand Total returns distinct month names across all years. January 2007 and January 2008 are both named 'January' — they collapse into a single row in the `VALUES` result. The iterator therefore sees fewer rows than actual year+month combinations, causing the denominator (working days) to be under-counted. Fix: iterate a column that is unique per year+month combination, such as Year Month Number (an integer like 200701).

What is the difference between ALL and ALLSELECTED in a RANKX lookup table?

Answer: `ALL` returns all values in the column regardless of any slicer or filter. If the user has filtered to 4 of 8 brands, `ALL` still builds a lookup with all 8 brands. The 4 visible brands may get ranks 1, 3, 6, 8 (gaps because 4 hidden brands hold those positions). `ALLSELECTED` returns only the values visible in the current slicer context. The 4 visible brands get ranks 1, 2, 3, 4 (no gaps). Use `ALLSELECTED` when the user expects rankings within their current selection.

Module Summary

- Every iterator follows the contract: iterate a table, evaluate an expression per row in row context, aggregate results. Functions with a `ROW CONTEXT` argument are iterators. Most have an X suffix (`SUMX`, `AVERAGEX`, `MAXX`, `RANKX`...).
- Two kinds of iteration: (1) row-by-row column arithmetic — no CT, fast; (2) measure reference — CT fires per row, powerful, scales in cost with cardinality. Always check the cardinality of the outer table before using CT iteration.
- Nested iterators have cardinality = product of outer and inner table sizes. Only the innermost can use the optimised engine; outer iterations require temporary in-memory tables. Replace inner iterations with measure

references + CT wherever possible.

■ **CONCATENATEX** aggregates string expressions. Three optional parameters: separator, order column, direction. Use **COUNTROWS(ALL(...))** vs **COUNTROWS(VALUES(...))** to build smart dynamic labels.

■ **ADDCOLUMNS** appends computed columns to a table. **SELECTCOLUMNS** projects a subset. Both are iterators. Use them to pre-compute measure values at a granularity and store in a table variable, avoiding repeated CT in subsequent **MAXX**, **FILTER**, and **AVERAGEX** calls.

■ **RANKX** three steps: build sorted lookup (iteration + CT), evaluate current value (original context), find position. Always protect with **HASONEVALUE**. Use **DENSE** for gap-free ties. Use **ALLSELECTED** for rankings within selections.

■ Granularity pattern: iterate a uniquely-identified column for the desired granularity. Month names are not unique across years — use Year Month Number (or equivalent) for year+month granularity.

NOTE

Next — Module 8: Time Intelligence Calculations. With iterators mastered, Module 8 covers date tables, **DATESYTD**, **SAMEPERIODLASTYEAR**, **DATEADD**, moving annual totals, and semi-additive measures — the most commonly tested advanced DAX topics in the PL-300 exam.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

8

Time Intelligence

YTD · PY · YOY · MAT · LASTNONBLANK · Semi-Additive

Master DAX time intelligence from the ground up.

Build a proper date table, understand the Auto-ALL mechanism, write YTD/QTD/MTD, period-over-period comparisons, YOY growth, moving annual totals with DATESINPERIOD and DATESBETWEEN, and semi-additive measures with LASTNONBLANK.

Prerequisite: Modules 1–7.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 8: Time Intelligence Calculations

Module Contents

■ Module Overview — Time Intelligence and the Date Table

■ Lesson 1: Building a Proper Date Table

- Requirements for a Valid Date Table
- CALENDAR and CALENDARAUTO
- ADDCOLUMNS Date Table Pattern
- Marking the Date Table
- Auto Date/Time — Why to Disable It

■ Lesson 2: Multiple Dates and Role-Playing Dimensions

- Multiple Relationships to the Date Table
- Multiple Date Tables — One per Role

■ Lesson 3: Understanding the Auto-ALL Mechanism

- How Time Intelligence Functions Filter the Date Table
- The Automatic ALL on DateTime Columns
- Why the Relationship Column Must Be DateTime
- What Happens with an Integer Relationship Key

■ Lesson 4: Year-to-Date, Quarter-to-Date, Month-to-Date

- DATESYTD, DATESQTD, DATESMTD
- TOTALYTD, TOTALQTD, TOTALMTD
- Custom Fiscal Year End with DATESYTD

■ Lesson 5: Period-over-Period Comparisons

- SAMEPERIODLASTYEAR
- DATEADD — Generic Period Shifting
- PARALLELPERIOD — Full Period vs Partial Period
- PREVIOUSYEAR, PREVIOUSQUARTER, PREVIOUSMONTH
- Combining Time Intelligence Functions

■ Lesson 6: Year-over-Year and YTD Comparisons

- YOY Sales — Difference and Percentage
- YOY YTD Sales
- Protecting YOY with ISBLANK

■ Lesson 7: Moving Annual Total (MAT)

- DATESINPERIOD — Trailing N Periods
- DATESBETWEEN — Explicit Start and End
- MAT Sales — 12-Month Rolling Sum
- Call Order Risk in Nested Time Intelligence

■ Lesson 8: Semi-Additive Measures

- What Is a Semi-Additive Measure?
- LASTDATE — Last Day of Period
- LASTNONBLANK — Last Day with Data
- Two Grand Total Strategies for LASTNONBLANK
- Opening and Closing Balance Functions

■ Demonstration 1: Building the Date Table and YTD/PY Measures

■ Demonstration 2: Semi-Additive Measures — LASTDATE vs LASTNONBLANK**■ Lab 8: Time Intelligence Calculations**

- Exercise 1: Date Table and YTD Measures
- Exercise 2: Period Comparisons and YOY
- Exercise 3: Moving Annual Total
- Exercise 4: Semi-Additive — Last Balance

■ Module Review and Takeaways

Module Overview — Time Intelligence and the Date Table

Time intelligence is the set of calculations that compare data over time: year-to-date, same period last year, year-over-year growth, moving annual total. Every model that contains transaction data needs at least some of these calculations. They are among the most commonly tested topics in the PL-300 exam.

All time intelligence functions in DAX require one prerequisite: a **properly configured date table**. Without a valid date table, time intelligence functions either fail silently or produce incorrect results. This module begins with the date table — its requirements, how to build it, and why it must be marked — and then covers the full time intelligence function library.

EXAM

PL-300 heavily tests: DATESYTD/TOTALYTD, SAMEPERIODLASTYEAR, DATEADD, YOY (difference and %), DATESINPERIOD, and the concept of marking the date table. Semi-additive measures (LASTNONBLANK) appear in advanced scenarios. Know the difference between PARALLELPERIOD (full period) and DATEADD (partial period shift).

LESSON 1

Building a Proper Date Table

Lesson Objectives

After completing this lesson, you will be able to:

- List the four requirements for a valid date table.
- Create a date table using CALENDAR or CALENDARAUTO.
- Build a date table with ADDCOLUMNS for reuse across projects.
- Mark a table as a Date table in Power BI.
- Explain why Auto Date/Time should be disabled in production models.

Requirements for a Valid Date Table

DAX time intelligence functions require the date table to meet four requirements:

Requirement	Detail	Why It Matters
Contiguous date range	Contains every single date from the first to the last day of the period, with no gaps.	Time intelligence functions scan the date table. A gap causes incorrect period boundaries.
Unique Date/DateTime column	One column of Date or DateTime type. If DateTime, all rows must have identical time parts (preferably 00:00:00).	The relationship from Sales to the date table must be based on a Date or DateTime column for the Auto-ALL mechanism to work.
Covers full calendar years	The table should begin on January 1 of the first year and end on December 31 of the last year.	YTD and PREVIOUSYEAR require all days of each year to be present, even if there is no transaction data on those days.
Marked as a Date table	Use Table Tools > Mark as Date Table and select the Date column.	Marking enables DAX to automatically add ALL('Date') when a DateTime filter is applied to the date column, making time intelligence work correctly with any relationship key type.

CALENDAR and CALENDARAUTO

```
-- CALENDAR: manually specify start and end dates
Date =
CALENDAR (
    DATE ( YEAR ( MIN ( Sales[Order Date] ) ), 1, 1 ), -- Jan 1 of first year in Sales
    DATE ( YEAR ( MAX ( Sales[Order Date] ) ), 12, 31 ) -- Dec 31 of last year in Sales
)
-- Returns a single-column table named [Date] with one row per calendar day.

-- CALENDARAUTO: scans all date columns in the model automatically
Date = CALENDARAUTO() -- finds min year and max year across all date columns
-- CALENDARAUTO always returns full calendar years (Jan 1 to Dec 31).
-- Advantage: automatically extends when data grows.
-- Disadvantage: if any date column has future dates, the calendar extends unnecessarily.
```

ADDCOLUMNS Date Table Pattern

```
-- Full date table with ADDCOLUMNS (reusable across projects)
Date =
```

```

VAR MinYear = YEAR ( MIN ( Sales[Order Date] ) )
VAR MaxYear = YEAR ( MAX ( Sales[Order Date] ) )
RETURN
ADDCOLUMNS (
    FILTER (
        CALENDARAUTO(),
        YEAR ( [Date] ) >= MinYear && YEAR ( [Date] ) <= MaxYear
    ),
    "Year", YEAR ( [Date] ),
    "Quarter Number", INT ( FORMAT ( [Date], "q" ) ),
    "Quarter", "Q" & INT ( FORMAT ( [Date], "q" ) ),
    "Month Number", MONTH ( [Date] ),
    "Month", FORMAT ( [Date], "mmm" ),
    "Week Day Number", WEEKDAY ( [Date] ),
    "Week Day", FORMAT ( [Date], "ddd" ),
    "Is Working Day", IF ( WEEKDAY([Date],2) <= 5, 1, 0 ),
    "Year Month Number", YEAR([Date]) * 100 + MONTH([Date] )
)
-- Tip: mark the [Date] column as the date key, then create the relationship
-- from Sales[Order Date] to Date[Date].

```

TIP

After creating the date table, go to Table Tools > Mark as Date Table and select the [Date] column as the unique date column. This step is required for DATESYTD and all other time intelligence functions to work correctly when the relationship key is not a DateTime column. It also enables the Q&A natural language feature for time queries.

Auto Date/Time — Why to Disable It

Power BI's Auto Date/Time feature creates a hidden date table for every Date column in the model. While convenient for quick exploration, it has two major drawbacks for production models:

- **One hidden table per date column.** If Sales has OrderDate and DeliveryDate, two separate hidden date tables are created. This bloats the model and creates confusion.
- **Cannot be customised.** The hidden tables contain only Year, Quarter, Month, Day. You cannot add IsWorkingDay, Fiscal Year, or custom calendar columns.

WARNING

Disable Auto Date/Time in production models: File > Options and Settings > Options > Data Load > uncheck 'Auto Date/Time'. Create your own date table using the ADDCOLUMNS pattern above. Connect it with the appropriate relationships. Mark it as a Date table. This is the approach expected in PL-300 exam scenarios.

LESSON 2

Multiple Dates and Role-Playing Dimensions

Lesson Objectives

After completing this lesson, you will be able to:

- Create measures for different date roles using USERELATIONSHIP.
- Explain the trade-offs between multiple relationships and multiple date tables.

Multiple Relationships — Single Date Table

```
-- Model: Sales has Order Date and Delivery Date. Date table connected to both.
-- Active relationship: Sales[Order Date] -> Date[Date]
-- Inactive relationship: Sales[Delivery Date] -> Date[Date]

[Ordered Amount] :=
SUMX ( Sales, Sales[Net Price] * Sales[Quantity] ) -- uses active relationship

[Delivered Amount] :=
CALCULATE (
SUMX ( Sales, Sales[Net Price] * Sales[Quantity] ),
USERELATIONSHIP ( Sales[Delivery Date], 'Date'[Date] )
)
-- USERELATIONSHIP temporarily activates the Delivery Date relationship.
-- [Delivered Amount] responds to Date[Year] by DELIVERY year, not order year.

-- Both measures in the same matrix:
-- Year 2007: Ordered Amount = orders placed in 2007.
-- Delivered Amount = orders delivered in 2007 (may include 2006 orders).
```

NOTE

The single-date-table + multiple-relationship approach is the recommended approach for most models. It requires one measure per metric per date role, but keeps the model compact. Use multiple date tables only when you need to compare two different date dimensions in the same visual (e.g. Order Year vs Delivery Year on the same axis).

LESSON 3

Understanding the Auto-ALL Mechanism

Lesson Objectives

After completing this lesson, you will be able to:

- Explain the Auto-ALL mechanism that makes time intelligence work.
- Identify when the Auto-ALL fires and when it does not.
- Explain why an integer relationship key requires the table to be marked as a Date table.

The Auto-ALL Mechanism Explained

Time intelligence functions like `DATESYTD` work by returning a table of dates that replaces the current filter context on the date table. For this to work, the existing date filter must be removed before the new set is applied. DAX handles this automatically for `DateTime` columns used in relationships:

```
-- Manual YTD (no time intelligence function):
[Sales YTD Manual] :=
VAR LastDate = MAX ( 'Date'[Date] )
VAR Year = YEAR ( LastDate )
VAR YTDDates =
FILTER ( ALL ( 'Date'[Date] ),
'Date'[Date] <= LastDate && YEAR ( 'Date'[Date] ) = Year )
RETURN CALCULATE ( [Sales Amount], YTDDates )

-- When placed on April 2007 cell, the filter context is Year=2007, Month=April.
-- The FILTER uses ALL('Date'[Date]) which REMOVES the existing month filter.
-- CALCULATE then applies YTDDates as the new date filter.
-- The month filter is gone; the YTD range (Jan-Apr) is active.

-- What if we use 'Date'[Date] alone (no ALL)?:
VAR YTDDates = FILTER ( 'Date'[Date],
'Date'[Date] <= LastDate && YEAR('Date'[Date]) = Year )
-- DANGEROUS: FILTER('Date'[Date],...) respects the existing Month=April filter!
-- Only April dates pass through -> YTDDates = only April dates.
-- CALCULATE('Date'[Date]=April) AND YTDDates(April) = April only. NOT YTD!

-- The Auto-ALL: DAX auto-adds ALL('Date') when a DateTime column filter is applied
-- to a column used in a relationship. This is why DATESYTD works:
CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) )
-- DAX detects 'Date'[Date] is the relationship column -> adds ALL('Date') automatically.
-- Result: same as using ALL('Date'[Date]) manually. The month filter is removed.
```

The Integer Key Problem

```
-- Model: Sales[DateKey] (integer, e.g. 20070415) -> Date[DateKey] (integer).
-- The Date table has a Date column but the relationship uses DateKey.

-- Problem: DATESYTD returns a set of Date values.
-- CALCULATE(measure, DATESYTD('Date'[Date])) applies a filter on Date[Date] (DateTime).
-- BUT the relationship is on DateKey (integer) -> auto-ALL does NOT fire.
-- The April month filter is still active.
-- YTD and April filter INTERSECT -> effectively April only -> same as [Sales Amount].
```

```
-- Solution 1: add ALL('Date') manually to every time intelligence calculation.
[Sales YTD Int Key] :=
CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ), ALL ( 'Date' ) )

-- Solution 2 (recommended): Mark the table as a Date table.
-- When the table is marked, DAX treats the Date column as special,
-- and the auto-ALL fires even when the relationship uses an integer key.
-- All time intelligence functions work correctly without manual ALL.
```

CONCEPT

Mark every date table as a Date table. This is a single click in the Power BI UI (Table Tools > Mark as Date Table), but it is the key that makes all time intelligence calculations correct. Without it, integer-keyed relationships silently produce wrong results for YTD, PY, and MAT.

LESSON 4

Year-to-Date, Quarter-to-Date, Month-to-Date

Lesson Objectives

After completing this lesson, you will be able to:

- Write CALCULATE + DATESYTD to compute year-to-date measures.
- Explain the difference between DATESYTD and TOTALYTD.
- Use the fiscal year end parameter in DATESYTD for non-calendar fiscal years.
- Write QTD and MTD using their corresponding functions.

DATESYTD, DATESQTD, DATESMTD

```
-- YTD: returns all dates from Jan 1 of the current year through the last visible date
[Sales YTD] :=
CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) )

-- On the April 2007 cell: DATESYTD returns Jan 1 - Apr 30 2007.
-- [Sales Amount] evaluates with those dates active -> cumulative Jan-Apr total.

-- QTD: returns all dates from the first day of the current quarter
[Sales QTD] :=
CALCULATE ( [Sales Amount], DATESQTD ( 'Date'[Date] ) )

-- MTD: returns all dates from the first day of the current month
[Sales MTD] :=
CALCULATE ( [Sales Amount], DATESMTD ( 'Date'[Date] ) )

-- Fiscal year ending June 30 (FY ends in June, not December):
[Sales FYTD] :=
CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date], "6/30" ) )
-- The optional second parameter is the fiscal year end date (month/day).
-- "6/30" means the fiscal year runs July 1 to June 30.
-- On April 2024 cell: fiscal YTD = Jul 2023 through Apr 2024.
```

TOTALYTD, TOTALQTD, TOTALMTD

```
-- TOTALYTD is syntactic sugar for CALCULATE + DATESYTD.
-- These two are equivalent:
[Sales YTD v1] := CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) )
[Sales YTD v2] := TOTALYTD ( [Sales Amount], 'Date'[Date] )

-- TOTALYTD hides the CALCULATE. Since CALCULATE should always be visible,
-- prefer the CALCULATE + DATESYTD form for clarity.
-- Exception: quick one-off measures where brevity is prioritised.

-- Custom fiscal year end with TOTALYTD:
[Sales FYTD v2] := TOTALYTD ( [Sales Amount], 'Date'[Date], "6/30" )
```


TIP

Prefer the `CALCULATE ( [measure], DATESYTD ( ... ) )` form over `TOTALYTD`. It makes the `CALCULATE` explicit — important because `CALCULATE` performs context transition and the reader should always see it. The `TOTAL` functions are legacy shortcuts. For the PL-300 exam, know both forms but understand that they are equivalent.

LESSON 5

Period-over-Period Comparisons

Lesson Objectives

After completing this lesson, you will be able to:

- Use SAMEPERIODLASTYEAR to shift the current period one year back.
- Use DATEADD to shift any number of years, quarters, months, or days.
- Explain the difference between PARALLELPERIOD (full period) and DATEADD (partial shift).
- Use PREVIOUSYEAR, PREVIOUSQUARTER, and PREVIOUSMONTH.
- Combine DATESYTD and SAMEPERIODLASTYEAR for prior-year YTD.

SAMEPERIODLASTYEAR and DATEADD

```
-- Prior year sales: shift current period back by 1 year
[PY Sales] := CALCULATE ( [Sales Amount], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
-- On April 2008: SAMEPERIODLASTYEAR returns April 2007.
-- On Q1 2008: returns Q1 2007.
-- On 2008: returns all of 2007.

-- Equivalent using DATEADD:
[PY Sales DATEADD] := CALCULATE ( [Sales Amount], DATEADD ( 'Date'[Date], -1, YEAR ) )
-- DATEADD(column, number, interval): shifts the current dates.
-- Intervals: YEAR, QUARTER, MONTH, DAY.
-- Negative numbers = back in time. Positive = forward.

-- Shift 2 quarters back:
[Sales 2Q Ago] := CALCULATE ( [Sales Amount], DATEADD ( 'Date'[Date], -2, QUARTER ) )

-- Shift 30 days back:
[Sales 30D Ago] := CALCULATE ( [Sales Amount], DATEADD ( 'Date'[Date], -30, DAY ) )
```

PARALLELPERIOD vs DATEADD — Full Period vs Partial Shift

This is a key distinction tested in PL-300. DATEADD shifts the current dates by the specified amount. If the current filter is a single month (e.g. April), DATEADD returns that same month one year ago (April last year). PARALLELPERIOD returns the *entire* period of the specified unit, shifted by the specified amount:

```
-- Scenario: report is showing Q1 2008 (Jan+Feb+Mar 2008).

-- DATEADD -1 YEAR: shifts Q1 2008 by 1 year -> Q1 2007 (Jan+Feb+Mar 2007)
-- Same number of dates, same relative position in the year. Partial period.
[PY Sales DATEADD] := CALCULATE ( [Sales Amount], DATEADD ( 'Date'[Date], -1, YEAR ) )
-- Result for Q1 2008: sales of Jan+Feb+Mar 2007 (partial Q1).

-- PARALLELPERIOD -1 YEAR: returns the full year 2007 (entire previous year)
[PY Full Year Sales] := CALCULATE ( [Sales Amount], PARALLELPERIOD ( 'Date'[Date], -1, YEAR ) )
-- Result for Q1 2008 cell: ALL of 2007 (full year). Not just Q1!

-- PARALLELPERIOD -1 QUARTER: returns the full previous quarter
-- When Q2 2008 is selected (Apr+May+Jun), returns all of Q1 2008 (full quarter).
[PQ Sales] := CALCULATE ( [Sales Amount], PARALLELPERIOD ( 'Date'[Date], -1, QUARTER ) )

-- PREVIOUSMONTH: returns the single previous month as a full month
```

```
-- When April 2008 is selected, returns March 2008 (full month).
[PM Sales] := CALCULATE ( [Sales Amount], PREVIOUSMONTH ( 'Date'[Date] ) )

-- PREVIOUSYEAR: returns all of the previous calendar year.
[PY Full Year] := CALCULATE ( [Sales Amount], PREVIOUSYEAR ( 'Date'[Date] ) )
```

Combining Time Intelligence Functions — Prior Year YTD

```
-- Prior year YTD: combine DATESYTD and SAMEPERIODLASTYEAR
[PY YTD Sales] :=
CALCULATE (
[Sales Amount],
DATESYTD ( SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
)

-- Time intelligence functions accept a table as their first argument.
-- SAMEPERIODLASTYEAR returns a table (the shifted dates).
-- DATESYTD then takes that shifted table and returns the YTD subset of it.
-- On April 2008: SAMEPERIODLASTYEAR -> April 2007.
-- DATESYTD -> Jan-Apr 2007. Result = YTD sales of prior year.
```

LESSON 6

Year-over-Year and YTD Comparisons

Lesson Objectives

After completing this lesson, you will be able to:

- Build YOY Sales (absolute difference) protected with ISBLANK.
- Build YOY Sales % (percentage change) using DIVIDE.
- Build YOY YTD Sales using [YTD Sales] - [PY YTD Sales].

YOY Sales — Difference and Percentage

```
-- Building blocks (define these first):
[Sales Amount] := SUMX ( Sales, Sales[Quantity] * Sales[Net Price] )
[PY Sales] := CALCULATE ( [Sales Amount], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

-- YOY difference (absolute): only show value when BOTH CY and PY exist
[YOY Sales] :=
VAR CySales = [Sales Amount]
VAR PySales = [PY Sales]
RETURN
IF (
NOT ISBLANK ( CySales ) && NOT ISBLANK ( PySales ),
CySales - PySales
)
-- Returns BLANK when either year has no data.
-- Example: first year in dataset has no prior year -> blank, not -PySales.

-- YOY percentage change:
[YOY Sales %] :=
VAR CySales = [Sales Amount]
VAR PySales = [PY Sales]
RETURN
IF (
NOT ISBLANK ( CySales ) && NOT ISBLANK ( PySales ),
DIVIDE ( CySales - PySales, PySales )
)
-- Format as Percentage. Positive = growth, negative = decline.

-- YOY YTD Sales:
[YTD Sales] := CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) )
[PY YTD Sales] :=
CALCULATE ( [Sales Amount], DATESYTD ( SAMEPERIODLASTYEAR ( 'Date'[Date] ) ) )

[YOY YTD Sales] :=
VAR CyYtd = [YTD Sales]
VAR PyYtd = [PY YTD Sales]
RETURN IF ( NOT ISBLANK(CyYtd) && NOT ISBLANK(PyYtd), CyYtd - PyYtd )
```

CONCEPT

Always use variables for YOY measures. Each of [Sales Amount] and [PY Sales] is already defined as a measure. Store them in variables to avoid re-evaluation, and use the ISBLANK check to prevent misleading negative values when one period has data and the other doesn't (the first year of data has no prior year — without the check, the difference = 0 - PY = negative).

LESSON 7

Moving Annual Total (MAT)

Lesson Objectives

After completing this lesson, you will be able to:

- Use DATESINPERIOD to define a trailing 12-month window.
- Use DATESBETWEEN with LASTDATE, NEXTDAY, and SAMEPERIODLASTYEAR for MAT.
- Explain the call-order risk in nested time intelligence and how to avoid it.

DATESINPERIOD — Trailing N Periods

```
-- DATESINPERIOD: returns dates within a window ending at a reference date
-- Syntax: DATESINPERIOD ( column, start_date, number_of_intervals, interval )

-- Moving Annual Total (MAT): sum of the last 12 months
[MAT Sales] :=
CALCULATE (
[Sales Amount],
DATESINPERIOD (
'Date'[Date],
LASTDATE ( 'Date'[Date] ), -- reference: last date in current filter context
-12, -- 12 intervals BACK from the reference date
MONTH -- interval type: month
)
)

-- On March 2008: LASTDATE = Mar 31 2008.
-- DATESINPERIOD returns Apr 1 2007 through Mar 31 2008 (12 months).
-- [Sales Amount] evaluates over that range.

-- Other useful windows:
[Sales Last 3 Months] :=
CALCULATE ( [Sales Amount], DATESINPERIOD ( 'Date'[Date], LASTDATE('Date'[Date]), -3, MONTH ) )

[Sales Last 30 Days] :=
CALCULATE ( [Sales Amount], DATESINPERIOD ( 'Date'[Date], LASTDATE('Date'[Date]), -30, DAY ) )
```

DATESBETWEEN — Explicit Start and End Dates

```
-- DATESBETWEEN: explicit start and end dates for the period
-- Syntax: DATESBETWEEN ( column, start_date, end_date )

-- MAT Sales using DATESBETWEEN (equivalent to DATESINPERIOD version):
[MAT Sales V2] :=
VAR LastDay = LASTDATE ( 'Date'[Date] )
VAR FirstDay = NEXTDAY ( SAMEPERIODLASTYEAR ( LASTDATE ( 'Date'[Date] ) ) )
VAR Result =
CALCULATE (
[Sales Amount],
DATESBETWEEN ( 'Date'[Date], FirstDay, LastDay )
)
RETURN Result

-- On March 2008:
```

```
-- LastDay = Mar 31 2008
-- LASTDATE('Date'[Date]) = Mar 31 2008
-- SAMEPERIODLASTYEAR(Mar 31 2008) = Mar 31 2007
-- NEXTDAY(Mar 31 2007) = Apr 1 2007
-- DATESBETWEEN returns Apr 1 2007 through Mar 31 2008. Same as DATESINPERIOD.
```

Call Order Risk in Nested Time Intelligence

WARNING

When composing time intelligence functions, the evaluation order matters. The correct order for the first day of MAT is: `NEXTDAY(SAMEPERIODLASTYEAR(LASTDATE('Date'[Date])))` — shift to last year first, then add one day. The wrong order — `SAMEPERIODLASTYEAR(NEXTDAY(...))` — calls `NEXTDAY` before `SAMEPERIODLASTYEAR`. If the `LASTDATE` is December 31 (the last day of the date table), `NEXTDAY` returns an empty table (January 1 of next year is not in the table). `SAMEPERIODLASTYEAR` receives an empty table and returns empty. `DATESBETWEEN` receives a blank start date (interpreted as Dec 30, 1899), and returns the entire date table — producing wildly inflated MAT values on Dec 31.

LESSON 8

Semi-Additive Measures

Lesson Objectives

After completing this lesson, you will be able to:

- Explain what makes a measure semi-additive.
- Use LASTDATE to show the end-of-period balance.
- Use LASTNONBLANK when the last day of the period may have no data.
- Explain the two grand total strategies for LASTNONBLANK and implement both.
- Explain the limitations of CLOSINGBALANCEMONTH/QUARTER/YEAR.

What Is a Semi-Additive Measure?

An additive measure (like Sales Amount) can be summed across any dimension: total sales = sum of daily sales = sum of product sales = sum of customer sales. A **semi-additive** measure is additive across some dimensions (e.g. customers) but not others (e.g. time). Bank balances are the canonical example: the balance at end of Q1 is NOT the sum of Jan+Feb+Mar balances. It is the balance on the last day of March:

```
-- Sample data: Balances table (CustomerName, Date, Balance)
-- Katie Jordan: Jan 31 = 1,687 Feb 28 = 2,812 Mar 15 = 2,531
-- Luis Bonifaz: Jan 31 = 3,100 Feb 28 = 2,800
-- Maurizio: Jan 31 = 1,200 Mar 18 = 1,950

-- WRONG: SUM of all balance rows
[Total Balance WRONG] := SUM ( Balances[Balance] )
-- Q1 total would show: 1687+2812+2531+3100+2800+1200+1950 = 16,080. Meaningless.

-- CORRECT intent: for each time period, show the LAST known balance.
-- For each customer, the last known balance in the period.
```

LASTDATE — Last Day of Period

```
-- LASTDATE returns the last date in the current filter context.
-- Use as a CALCULATE filter to show the balance on the last day of the period.
[Last Balance] :=
CALCULATE (
SUM ( Balances[Balance] ),
LASTDATE ( 'Date'[Date] )
)
-- For April: LASTDATE = Apr 30. CALCULATE filters Date to Apr 30.
-- SUM(Balances[Balance]) = total of all customer balances on Apr 30.

-- Limitation: if data is not available on the exact last day of the period
-- (weekend, holiday, or irregular update frequency),
-- LASTDATE returns that day but SUM returns BLANK (no data on that day).
-- Result: blanks in the matrix on months/quarters where the last day has no data.
```

LASTNONBLANK — Last Day with Data

```
-- LASTNONBLANK scans a table and returns the last row for which the expression
-- evaluates to non-blank.
-- Syntax: LASTNONBLANK ( column, expression )
```



```
[Last Balance NonBlank] :=
CALCULATE (
SUM ( Balances[Balance] ),
LASTNONBLANK (
'Date'[Date],
COUNTROWS ( RELATEDTABLE ( Balances ) ) -- non-zero if data exists on that date
)
)
-- LASTNONBLANK iterates 'Date'[Date] (row context).
-- For each date, RELATEDTABLE(Balances) returns related balance rows.
-- COUNTROWS = 0 (no data) or > 0 (data exists).
-- LASTNONBLANK returns the last date where COUNTROWS > 0.
-- CALCULATE filters the date table to that single last-known date.
-- SUM returns the balance on that date.

-- This correctly handles weekends and holidays:
-- If Apr 30 is a Sunday with no balance data, LASTNONBLANK returns Apr 28 (Friday).
```

Two Grand Total Strategies for LASTNONBLANK

The grand total of a LASTNONBLANK measure requires a business decision: should the total show the balance on the single last date across ALL customers, or the sum of each customer's individual last balance?

```
-- Strategy 1: Grand total = balance on the SINGLE last date with ANY data
-- (last date where at least one customer has a balance)
[Last Balance All Customers] :=
VAR LastDateAllCustomers =
CALCULATETABLE (
LASTNONBLANK ( 'Date'[Date], COUNTROWS ( RELATEDTABLE ( Balances ) ) ),
ALL ( Balances[Name] ) -- remove customer filter: find global last date
)
RETURN CALCULATE ( SUM ( Balances[Balance] ), LastDateAllCustomers )
-- Grand total: sums ALL customers' balances on the global last date (e.g. Jul 18).

-- Strategy 2: Grand total = SUM of each customer's INDIVIDUAL last balance
-- (each customer may have a different last date)
[Last Balance Individual] :=
SUMX (
VALUES ( Balances[Name] ), -- iterate each customer
CALCULATE ( -- CT fires per customer
SUM ( Balances[Balance] ),
LASTNONBLANK ( 'Date'[Date], COUNTROWS ( RELATEDTABLE ( Balances ) ) )
)
)
-- Grand total: Katie's last balance + Luis's last balance + Maurizio's last balance
-- = 2531 + 2800 + 1950 = 7281. Each customer's last date may differ.

-- Which to use? Depends on business requirement:
-- Strategy 1: 'what was the total portfolio value on the last reporting date?'
-- Strategy 2: 'what is the sum of each customer's most recent known balance?'
```

Opening and Closing Balance Functions

DAX provides convenience functions for opening and closing balances. They have the same limitation as LASTDATE — they return a specific calendar date regardless of whether data exists on that date:

```
-- Closing balance functions (equivalent to using LASTDATE-based patterns):
[Closing Month Balance] := CLOSINGBALANCEMONTH ( SUM(Balances[Balance]), 'Date'[Date] )
[Closing Quarter Balance] := CLOSINGBALANCEQUARTER ( SUM(Balances[Balance]), 'Date'[Date] )
[Closing Year Balance] := CLOSINGBALANCEYEAR ( SUM(Balances[Balance]), 'Date'[Date] )

-- Opening balance functions:
[Opening Month Balance] := OPENINGBALANCEMONTH ( SUM(Balances[Balance]), 'Date'[Date] )
[Opening Quarter Balance] := OPENINGBALANCEQUARTER ( SUM(Balances[Balance]), 'Date'[Date] )
[Opening Year Balance] := OPENINGBALANCEYEAR ( SUM(Balances[Balance]), 'Date'[Date] )

-- Limitation: CLOSINGBALANCEMONTH returns the last day of the month.
-- If that day is a weekend or holiday with no data -> result is BLANK.
-- For irregular data, use LASTNONBLANK instead.

-- Better closing balance (works with irregular data):
[Closing Balance Robust] :=
CALCULATE (
SUM ( Balances[Balance] ),
LASTNONBLANK ( 'Date'[Date], COUNTROWS ( RELATEDTABLE ( Balances ) ) )
)
```

DEMONSTRATION 1

Building the Date Table and YTD/PY Measures

Scenario: The instructor builds the date table from scratch using CALENDARAUTO + ADDCOLUMNS, marks it, creates the relationship, and then builds the full suite of YTD, PY, YOY difference, and YOY% measures.

1	Create a new calculated table 'Date' using CALENDARAUTO(). Show the single [Date] column. Expand using ADDCOLUMNS to add Year, Month Number, Month, Quarter, Is Working Day, Year Month Number. Show the resulting table in Data view.
2	Create the relationship: Sales[Order Date] -> Date[Date]. Set Sales[Order Date] as the lookup. Open Table Tools, click Mark as Date Table, select [Date] column. Confirm the message.
3	Build [Sales YTD] = CALCULATE([Sales Amount], DATESYTD('Date'[Date])). Place on matrix with Year > Month. Show cumulative increase within each year. Verify Jan shows just January's sales; Dec shows the full year total.
4	Build [PY Sales] = CALCULATE([Sales Amount], SAMEPERIODLASTYEAR('Date'[Date])). Add to the matrix. Show that 2007 PY = blank (no 2006 data), 2008 PY = 2007 values.
5	Build [YOY Sales] with ISBLANK protection and [YOY Sales %] with DIVIDE. Show positive/negative growth months highlighted with conditional formatting.
6	Build [PY YTD Sales] = CALCULATE([Sales Amount], DATESYTD(SAMEPERIODLASTYEAR('Date'[Date]))). Show all five measures side by side on the matrix.
RESULT	Full date table built and marked. All five time intelligence measures verified. YOY blank protection confirmed on first-year rows.

DEMONSTRATION 2

Semi-Additive Measures — LASTDATE vs LASTNONBLANK

Scenario: The instructor uses the Balances sample data to demonstrate the failure of SUM, the limitation of LASTDATE with irregular data, and the correct LASTNONBLANK pattern with both grand total strategies.

1	Place SUM(Balances[Balance]) on a matrix with Date[Month] on rows and CustomerName on columns. Show the monthly totals look obviously wrong at the quarter and year level (summing all months).
2	Build [Last Balance] using LASTDATE. Show it works for months where the last day has data. Apply a date filter to a month where the last day is a weekend. Show the blank result.
3	Build [Last Balance NonBlank] using LASTNONBLANK + COUNTROWS(RELATEDTABLE(Balances)). Show it correctly handles weekends by finding the last day with actual data.
4	Show the Grand Total issue: the grand total shows only one customer's balance (the customer with the latest date). Explain why: without a customer filter, LASTNONBLANK finds the globally last date, which may only have one customer's data.
5	Build [Last Balance All Customers] with CALCULATETABLE(LASTNONBLANK(...),ALL(Balances[Name])). Show the grand total now shows the total of all customers on the global last date.
6	Build [Last Balance Individual] with SUMX(VALUES(Balances[Name]),CALCULATE(...)). Show the grand total is the sum of each customer's last-known balance. Discuss: which strategy matches the business requirement?
RESULT	LASTDATE vs LASTNONBLANK limitation demonstrated. Both grand total strategies implemented and their differences explained.

LAB 8

Time Intelligence Calculations

Lab Scenario

The Contoso BI team needs a complete time intelligence dashboard: YTD sales, prior year comparisons, YOY growth, a 12-month MAT trend, and a last-known balance for the inventory valuation model. You will build all measures from scratch on the properly marked date table.

NOTE

Open Contoso_Lab08_Start.pbix. The model includes the Contoso Sales model (Sales, Product, Customer, Date) and a separate Balances table for Exercise 4. The Date table is already created but NOT marked. You will mark it in Exercise 1. Save as Contoso_Lab08_Complete.pbix.

Exercise 1: Date Table and YTD Measures

Estimated time: 25 minutes

Task 1: Mark the Date table and verify YTD

1. In Power BI Desktop, select the Date table in the Fields pane. Click Table Tools > Mark as Date Table. Select 'Date' as the date column. Click OK.
2. Add the YTD measure:


```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Sales YTD] :=  
    CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) )" 'frags': [ParaFrag(__tag__='para', bold=0,  
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[Sales YTD] :=",  
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,  
    fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",  
    textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,  
    fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="CALCULATE ( [Sales Amount],  
    DATESYTD ( 'Date'[Date] ) )", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style':  
    'bulletText': None 'debug': 0 ) #Paragraph
```
3. Place on a matrix: Date[Year] and Date[Month Number] on rows, [Sales Amount] and [Sales YTD] as values. Verify [Sales YTD] increases monotonically within each year and resets at January of each new year.

Task 2: QTD and Fiscal YTD

1. Add [Sales QTD] := CALCULATE([Sales Amount], DATESQTD('Date'[Date])).
2. Add [Sales Fiscal YTD] := CALCULATE([Sales Amount], DATESYTD('Date'[Date], "6/30")). (Assume Contoso's fiscal year ends June 30.)
3. Verify [Sales Fiscal YTD] resets at July of each year, not January.

Results: Date table marked. Sales YTD resets correctly at January. Sales Fiscal YTD resets at July. QTD resets at each quarter start.

Exercise 2: Period Comparisons and YOY

Estimated time: 30 minutes

Task 1: Prior year measures

1. Add the following measures:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[PY Sales] := CALCULATE ( [Sales Amount],
SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
[PY YTD Sales] := CALCULATE ( [Sales Amount],
DATESYTD ( SAMEPERIODLASTYEAR ( 'Date'[Date] ) ) )" 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[PY Sales] := CALCULATE (
[Sales Amount], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[PY YTD Sales] := CALCULATE
( [Sales Amount],", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="DATESYTD (
SAMEPERIODLASTYEAR ( 'Date'[Date] ) ) )", textColor=Color(.101961,.227451,.360784,1), us_lines=[])]
'style': 'bulletText': None 'debug': 0 ) #Paragraph
```

2. Add to the matrix. Verify [PY Sales] for 2008 matches [Sales Amount] for 2007 on the same months.

Task 2: YOY difference and percentage

1. Add:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[YOY Sales] :=
VAR Cy = [Sales Amount]
VAR Py = [PY Sales]
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), Cy - Py )

[YOY Sales %] :=
VAR Cy = [Sales Amount]
VAR Py = [PY Sales]
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), DIVIDE(Cy - Py, Py) )" 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="[YOY Sales] :=", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR Cy = [Sales Amount]",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="VAR Py = [PY Sales]",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="RETURN IF ( NOT
ISBLANK(Cy) ", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="&",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="&",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="NOT ISBLANK(Py), Cy - Py )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
```

```

textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[YOY Sales %] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Cy = [Sales Amount]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='VAR Py = [PY Sales]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='RETURN IF ( NOT
ISBLANK(Cy) ', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' NOT ISBLANK(Py), DIVIDE(Cy
- Py, Py) )', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0
) #Paragraph

```

2. Verify [YOY Sales] is blank for the first year. Apply conditional formatting (data bars or color scale) to [YOY Sales %] to highlight growth vs decline months.

Task 3: PARALLELPERIOD comparison

1. Add [PY Full Year Sales] := CALCULATE([Sales Amount], PARALLELPERIOD('Date'[Date],-1,YEAR)).
2. Place on matrix with Year > Quarter on rows. Observe that [PY Full Year Sales] shows the entire prior year's total on every quarter row — not just Q1-of-prior-year.

Results: PY Sales, PY YTD, YOY, YOY%, and PARALLELPERIOD measures all verified. First-year blanks confirmed. PARALLELPERIOD behaviour vs DATEADD observed.

Exercise 3: Moving Annual Total

Estimated time: 20 minutes

Task 1: MAT Sales using DATESINPERIOD

1. Add:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[MAT Sales] :=
CALCULATE (
[Sales Amount],
DATESINPERIOD ( 'Date'[Date], LASTDATE ( 'Date'[Date] ), -12, MONTH )
)" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
rise=0, text='[MAT Sales] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="
", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CALCULATE (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="
",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' [Sales Amount]',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,

```

```

fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" DATESINPERIOD (
'Date'[Date], LASTDATE ( 'Date'[Date] ), -12, MONTH )", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text=')', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

- Place on a line chart with Date[Date] (month level) on X-axis, [Sales Amount] and [MAT Sales] as series. Observe [MAT Sales] is smoother — seasonal spikes are absorbed by the 12-month window.

Task 2: MAT Sales using DATESBETWEEN

- Add the alternative formulation:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[MAT Sales V2] :=
VAR LastDay = LASTDATE ( 'Date'[Date] )
VAR FirstDay = NEXTDAY ( SAMEPERIODLASTYEAR ( LastDay ) )
RETURN CALCULATE ( [Sales Amount], DATESBETWEEN ( 'Date'[Date], FirstDay, LastDay ) )" 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='[MAT Sales V2] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="VAR LastDay = LASTDATE ( 'Date'[Date] )", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='VAR FirstDay = NEXTDAY ( SAMEPERIODLASTYEAR ( LastDay ) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="RETURN CALCULATE ( [Sales
Amount], DATESBETWEEN ( 'Date'[Date], FirstDay, LastDay ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

- Verify [MAT Sales] and [MAT Sales V2] produce identical values on all dates except potentially December 31 of the last year in the data (call-order issue).

Results: MAT Sales smoothing visible on line chart. V1 and V2 produce identical results. The NEXTDAY/SAMEPERIODLASTYEAR order confirmed as correct.

Exercise 4: Semi-Additive — Last Balance

Estimated time: 20 minutes

Task 1: LASTDATE version and confirm the limitation

- Build [Last Balance] = CALCULATE(SUM(Balances[Balance]), LASTDATE('Date'[Date])).
- Place on matrix with Date[Month] on rows, Balances[Name] on columns. Identify any months where the last calendar day has no data -> confirm the blank result.

Task 2: LASTNONBLANK version

- Build:


```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Last Balance NB] :=
  CALCULATE (
    SUM ( Balances[Balance] ),
    LASTNONBLANK ( 'Date'[Date], COUNTROWS ( RELATEDTABLE ( Balances ) ) )
)" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
rise=0, text='[Last Balance NB] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
  ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
  ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CALCULATE (', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' SUM ( Balances[Balance] ),',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" LASTNONBLANK (
'Date'[Date], COUNTROWS ( RELATEDTABLE ( Balances ) ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=')',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Compare [Last Balance] and [Last Balance NB] side by side. Verify NB version fills in where LASTDATE showed blank.

Task 3: Grand total strategies

1. Build the two grand total variants from Lesson 8 (All Customers and Individual). Place both on the matrix. Confirm the totals differ.
2. Save as **Contoso_Lab08_Complete.pbix**.

Results: LASTDATE blank limitation confirmed. LASTNONBLANK fills the gaps. Two grand total strategies demonstrate different business interpretations.

Module Review and Takeaways

Review Questions

What are the four requirements for a valid date table?

Answer: 1. Contiguous dates: no gaps from first to last day. 2. A unique Date or DateTime column (all time parts identical if DateTime). 3. Covers full calendar years (Jan 1 to Dec 31). 4. Marked as a Date table (Table Tools > Mark as Date Table). Without meeting all four requirements, time intelligence functions may silently produce incorrect results.

Explain the Auto-ALL mechanism and why it is necessary.

Answer: When a DateTime column that defines a one-to-many relationship with another table is filtered inside CALCULATE, DAX automatically adds ALL('Date') as an additional filter argument. This removes any existing filter on the date table before the time intelligence filter is applied. Without this, the time intelligence filter would intersect with the existing month/quarter filter instead of replacing it, producing a result that is identical to the base measure. When the relationship uses an integer key (not DateTime), the Auto-ALL does not fire unless the table is marked as a Date table.

What is the difference between DATEADD and PARALLELPERIOD?

Answer: DATEADD shifts the current set of dates by a specified number of intervals. The result has the same number of dates as the input, just shifted in time. Example: on April 2008, DATEADD(-1, YEAR) returns April 2007 (one month, shifted). PARALLELPERIOD returns the entire specified period shifted by the specified number. Example: on April 2008, PARALLELPERIOD(-1, YEAR) returns all of 2007 (a full year, not just April 2007). Use DATEADD for same-length period comparisons. Use PARALLELPERIOD when you want the complete prior period regardless of current selection length.

Why must YOY Sales protect against blank values with ISBLANK?

Answer: The first year in the dataset has no prior year data. Without the ISBLANK check, [PY Sales] returns BLANK for the first year. [Sales Amount] - BLANK = [Sales Amount] (DAX treats blank as zero in arithmetic). The YOY measure would show a misleading large positive value in the first year, as if comparing against zero. The ISBLANK check returns BLANK when either year has no data, preventing the misleading result.

What is the risk in using SAMEPERIODLASTYEAR(NEXTDAY(...)) instead of NEXTDAY(SAMEPERIODLASTYEAR(...))?

Answer: Calling NEXTDAY first on the last date in the table (e.g. Dec 31) produces an empty table because Jan 1 of the next year does not exist in the date table. SAMEPERIODLASTYEAR then receives an empty table and returns an empty table. DATESBETWEEN receives a blank start date, which DAX interprets as Dec 30, 1899, and returns the entire date table. The MAT measure shows the total of all time rather than a 12-month window on Dec 31. The correct order: NEXTDAY(SAMEPERIODLASTYEAR(LASTDATE(...))) shifts to last year first, then adds one day — ensuring the date exists in the table.

Describe the two LASTNONBLANK grand total strategies and when to use each.

Answer: Strategy 1 (All Customers): use CALCULATETABLE(LASTNONBLANK(...), ALL(Balances[Name])) to find the single global last date with any data across all customers. The grand total shows the sum of all customers' balances on that one date. Use this when asking: 'What was the total portfolio value on the last reporting date?' Strategy 2 (Individual): use SUMX(VALUES(CustomerName), CALCULATE(SUM, LASTNONBLANK)) to iterate each customer and find their individual last known balance. Sum those individual balances for the grand total. Use this when asking: 'What is the sum of each customer's most recently known

balance?' These two strategies produce different grand totals when customers have different last-known dates.

Time Intelligence Quick Reference

Function	What It Returns	Typical Usage
DATESYTD(col [, "m/d"])	All dates from Jan 1 (or fiscal start) through last visible date in current year.	YTD measures. CALCULATE([Sales Amount], DATESYTD('Date'[Date]))
DATESQTD(col)	All dates from the start of the current quarter.	QTD measures.
DATESMTD(col)	All dates from the start of the current month.	MTD measures.
TOTALYTD(expr, col [, "m/d"])	Syntactic sugar for CALCULATE + DATESYTD.	Quick YTD — prefer CALCULATE + DATESYTD for clarity.
SAMEPERIODLASTYEAR(col)	Same period shifted 1 year back.	PY Sales, PY YTD. Equivalent to DATEADD(col,-1,YEAR).
DATEADD(col, n, interval)	Current dates shifted n intervals. Same number of dates.	Flexible period shift. Intervals: YEAR, QUARTER, MONTH, DAY.
PARALLELPERIOD(col, n, interval)	Full shifted period (entire year/quarter/month), not just the current partial period.	PY Full Year, PQ Full Quarter.
PREVIOUSYEAR/MONTH/QUARTER(col)	Full previous calendar year/month/quarter.	Single-period prior comparisons.
DATESINPERIOD(col, ref, n, interval)	Dates within n intervals before/after ref.	MAT: -12 months. Trailing windows.
DATESBETWEEN(col, start, end)	All dates between start and end (inclusive).	Custom date windows. MAT V2 pattern.
LASTDATE(col)	Last date in current filter context as a table.	CALCULATE filter for end-of-period value. Limited: may miss weekends.
LASTNONBLANK(col, expr)	Last row where expr is non-blank.	Semi-additive: finds last date with actual data.
FIRSTDATE / FIRSTNONBLANK	First date / first date with data.	Opening balances.
CLOSINGBALANCE...	Closing value at end of Month/Quarter/Year.	Convenience functions. Same limitation as LASTDATE.

Module Summary

- All time intelligence functions require a properly built date table: contiguous dates, unique DateTime column, full calendar years, and marked as a Date table. Disable Auto Date/Time in production models. Use CALENDARAUTO + ADDCOLUMNS for a reusable template.
- The Auto-ALL mechanism: when a DateTime column used in a relationship is filtered by a time intelligence function, DAX automatically adds ALL('Date') before applying the new filter. This removes the existing period filter so the TI function's dates can replace it. The table must be marked or the relationship column must be DateTime for this to work.
- DATESYTD returns all dates from year start to last visible date. Use CALCULATE([measure], DATESYTD('Date'[Date])). Use the optional second parameter for fiscal year end. TOTALYTD is syntactic

sugar — prefer the explicit CALCULATE form.

■ SAMEPERIODLASTYEAR shifts the current period one year back. DATEADD shifts by any number of any interval. PARALLELPERIOD returns the entire period (not just the partial current selection) shifted by one period. PREVIOUSYEAR/QUARTER/MONTH return single full periods.

■ YOY measures should always use variables and ISBLANK protection: IF(NOT ISBLANK(CY) && NOT ISBLANK(PY), CY - PY). Without it, the first year shows misleading values.

■ MAT Sales: use DATESINPERIOD(col, LASTDATE(col), -12, MONTH). Or DATESBETWEEN with NEXTDAY(SAMEPERIODLASTYEAR(LASTDATE(col))) as the start. Call order in nested TI functions matters: NEXTDAY must come AFTER SAMEPERIODLASTYEAR.

■ Semi-additive measures: use LASTNONBLANK (not LASTDATE) when the last day of a period may have no data. LASTNONBLANK is an iterator that finds the last date with actual rows. Two grand total strategies: ALL-customers last date (sum on one date) vs per-customer last date (SUMX over customers with CALCULATE + LASTNONBLANK).

NOTE

Next — Module 9: Calculation Groups. With time intelligence fully mastered, Module 9 covers calculation groups: defining a single set of time intelligence calculations that apply to any measure in the model, eliminating the need to create separate YTD, PY, and YOY versions of every measure.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

9

Calculation Groups

SELECTEDMEASURE · Precedence · Format Strings · Sideways Recursion

The most powerful productivity feature in modern DAX.

Learn how calculation groups eliminate measure explosion by applying one set of calculations (YTD, PY, YOY...) to every measure at once.

Master SELECTEDMEASURE, format string overrides, precedence rules when combining multiple groups, measure-reference semantics, and the two non-negotiable best practices.

Prerequisite: Modules 1–8.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 9: Calculation Groups

Module Contents

■ Module Overview — The Measure Explosion Problem

■ Lesson 1: What Are Calculation Groups?

- The Problem: N Measures × M Variations
- The Solution: Calculation Items and SELECTEDMEASURE
- How Calculation Groups Appear in Power BI Reports

■ Lesson 2: Creating Calculation Groups in Tabular Editor

- Tabular Editor — The Required Tool
- Creating a Calculation Group Table
- Adding Calculation Items with SELECTEDMEASURE
- Naming and Ordinal (Sort Order)

■ Lesson 3: SELECTEDMEASURE and Related Functions

- SELECTEDMEASURE — The Placeholder Function
- SELECTEDMEASURENAME — String Identification
- ISSELECTEDMEASURE — Boolean Gate for Specific Measures
- SELECTEDMEASUREFORMATSTRING — Dynamic Format Strings

■ Lesson 4: Format String Overrides

- Why Format Strings Matter in Calculation Groups
- Fixed Format String Override
- Dynamic Format String with SELECTEDMEASUREFORMATSTRING

■ Lesson 5: How Calculation Items Are Applied — The Substitution Model

- A Calculation Item Replaces Measure References
- What Happens When CALCULATE Filters a Calculation Item
- The Single-Measure-Reference Rule
- What Breaks When the Rule Is Violated

■ Lesson 6: Multiple Calculation Groups and Precedence

- The Precedence Property
- How Precedence Determines Application Order
- The Daily YTD Average — A Precedence Case Study
- ISSELECTEDMEASURE as a Guard Against Unwanted Applications

■ Lesson 7: Sideways Recursion

- What Is Sideways Recursion?
- Using One Calculation Item Inside Another
- Recursion Risks and the Infinite Loop Detection
- When to Use and Avoid Sideways Recursion

■ Lesson 8: Best Practices and Professional Patterns

- Best Practice 1: Only Apply to Single Measure References
- Best Practice 2: Define Precedence Explicitly
- The Complete Time Intelligence Calculation Group
- The Metric Selector Calculation Group

■ Demonstration 1: Building the Time Intelligence Calculation Group

■ Demonstration 2: Combining Two Calculation Groups with Precedence

■ Lab 9: Calculation Groups

- Exercise 1: Time Intelligence Calculation Group
- Exercise 2: Format String Overrides for YOY%
- Exercise 3: Metric Selector Group and Precedence

■ Module Review and Takeaways

Module Overview — The Measure Explosion Problem

Suppose your Contoso model has four base measures: [Sales Amount], [Total Cost], [Gross Margin], and [Order Count]. Your business users need each of these as: Current Year (CY), Year-to-Date (YTD), Quarter-to-Date (QTD), Prior Year (PY), Prior Year YTD (PY YTD), Year-over-Year difference (YOY), and Year-over-Year % (YOY%). That is 7 variations $\times$ 4 measures = **28 measures**. Add a fifth base measure and you need 35. Add a sixth time variation and you need 42. The model bloats. Maintenance is a nightmare. Every time a base measure changes, seven derived measures must change with it.

Calculation groups solve this entirely. Define the seven time variations once, as calculation items in a single group. They apply automatically to every measure in the model. 28 measures collapse to $4 + 7 = 11$ objects. When you add a fifth base measure, the seven variations are free.

EXAM

PL-300 tests calculation groups at the conceptual and applied level: what SELECTEDMEASURE does, why calculation items only apply to measure references, what precedence controls, and how to set up a Time Intelligence calculation group. Tabular Editor is required to create calculation groups — Power BI Desktop alone cannot.

LESSON 1

What Are Calculation Groups?

Lesson Objectives

- Describe the measure explosion problem and how calculation groups solve it.
- Define the terms calculation group, calculation item, and SELECTEDMEASURE.
- Explain how a calculation group appears to users in a Power BI report.

The Structure of a Calculation Group

A **calculation group** is a special table in the data model with a single text column. Each row of that column is a **calculation item** — a named DAX expression. When a user selects a calculation item in a slicer, the engine applies that item's expression to every measure in a visual. The key function inside a calculation item is `SELECTEDMEASURE()`, which acts as a placeholder for whichever measure is being calculated in the current cell:

```
-- Pseudo-code showing the structure of a Time Intelligence calculation group:
CALCULATION GROUP 'Time Intelligence'[Time calc]

-- Calculation Item: CY (Current Year – no modification)
SELECTEDMEASURE()

-- Calculation Item: YTD
CALCULATE ( SELECTEDMEASURE(), DATESYTD ( 'Date'[Date] ) )

-- Calculation Item: QTD
CALCULATE ( SELECTEDMEASURE(), DATESQTD ( 'Date'[Date] ) )

-- Calculation Item: PY
CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

-- Calculation Item: YOY
VAR Cy = SELECTEDMEASURE()
VAR Py = CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), Cy - Py )

-- Calculation Item: YOY%
VAR Cy = SELECTEDMEASURE()
VAR Py = CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), DIVIDE ( Cy - Py, Py ) )

-- SELECTEDMEASURE() is replaced by whatever measure is in the visual.
-- When [Sales Amount] is in the visual and user selects YTD:
-- the YTD item's expression becomes:
-- CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) )
-- When [Gross Margin] is in the visual and user selects YTD:
-- CALCULATE ( [Gross Margin], DATESYTD ( 'Date'[Date] ) )
```

How Calculation Groups Appear in Power BI Reports

The calculation group table appears in the Fields pane like a regular table. The user drags its single column to a slicer or to a row/column in a matrix. When a single calculation item is selected, all measures in the visual are modified by that item. When no item is selected, all measures behave normally (CY / no modification).

NOTE

Calculation groups require **Tabular Editor** (free community version available at tabulareditor.com). Power BI Desktop does not provide a native UI to create calculation groups. In Analysis Services, calculation groups can also be defined via XMLA or TMSL scripts. Once created in Tabular Editor, the calculation group is saved with the .pbix file.

LESSON 2

Creating Calculation Groups in Tabular Editor

Lesson Objectives

- Open a .pbix file in Tabular Editor.
- Create a calculation group table and rename its column.
- Add calculation items and enter their DAX expressions.
- Set the Ordinal property to control item sort order.

Step-by-Step Creation in Tabular Editor

Step	Action	Detail
1	Connect to model	From Power BI Desktop: External Tools > Tabular Editor. Or File > Open > Power BI Desktop file.
2	Create group	Right-click Tables or use Model menu > New Calculation Group. A new table appears with a special icon.
3	Rename group	Click the table name and rename to e.g. 'Time Intelligence'.
4	Rename column	Expand the group. Click the column (named 'Name' by default). Rename to e.g. 'Time calc'. This is what appears in slicers.
5	Add items	Right-click the column > New Calculation Item. Name the item (e.g. 'YTD'). Enter the DAX expression in the expression editor.
6	Set Ordinal	Each item has an Ordinal property (integer). Lower ordinal = appears first in slicers. Set: CY=0, YTD=1, QTD=2, PY=3, YOY=4, YOY%=5.
7	Set Precedence	Click the calculation group table. Set Precedence (integer). Higher value = applied first. Default: 0.
8	Save	Ctrl+S in Tabular Editor saves back to the .pbix model. Return to Power BI Desktop — the new table appears in Fields.

TIP

After saving in Tabular Editor, switch back to Power BI Desktop. The calculation group table now appears in the Fields pane. Drag its column to a slicer. Drag any measure to a matrix. Select a calculation item in the slicer to see the measure change behavior. If the column shows values but the matrix does not change, check that Tabular Editor saved successfully.

LESSON 3

SELECTEDMEASURE and Related Functions**Lesson Objectives**

- Use SELECTEDMEASURE() as the placeholder for the current measure.
- Use SELECTEDMEASURENAME() to branch logic based on the measure's name.
- Use ISSELECTEDMEASURE() as a type-safe boolean guard.
- Use SELECTEDMEASUREFORMATSTRING() to preserve the base measure's format in dynamic strings.

SELECTEDMEASURE() — The Core Placeholder

```
-- SELECTEDMEASURE() stands for 'whichever measure is being evaluated right now'.
-- Inside a calculation item, it is replaced at evaluation time with that measure's expression.

-- YTD calculation item:
CALCULATE ( SELECTEDMEASURE(), DATESYTD ( 'Date'[Date] ) )

-- When [Sales Amount] calls this item:
-- CALCULATE ( [Sales Amount], DATESYTD ( 'Date'[Date] ) ) -- Sales Amount YTD

-- When [Gross Margin] calls this item:
-- CALCULATE ( [Gross Margin], DATESYTD ( 'Date'[Date] ) ) -- Gross Margin YTD

-- SELECTEDMEASURE() can appear multiple times in one calculation item:
-- YOY calculation item:
VAR Cy = SELECTEDMEASURE() -- current year value
VAR Py = CALCULATE ( SELECTEDMEASURE(), -- prior year value
    SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), Cy - Py )
-- Both occurrences are replaced by the same base measure.
-- Cy = [Sales Amount] in current context
-- Py = CALCULATE([Sales Amount], SAMEPERIODLASTYEAR(...))
```

ISSELECTEDMEASURE() and SELECTEDMEASURENAME()

```
-- ISSELECTEDMEASURE: type-safe boolean check for specific measures.
-- Returns TRUE if the current SELECTEDMEASURE is one of the listed measures.

-- Daily Average item – only apply to numeric quantity measures, not percentages:
IF (
    ISSELECTEDMEASURE ( [Sales Amount], [Total Cost], [Gross Margin], [Order Count] ),
    DIVIDE ( SELECTEDMEASURE(), COUNTROWS ( 'Date' ) )
)
-- Returns BLANK for any measure NOT in the list (e.g. [Gross Margin %]).

-- Alternatively, EXCLUDE specific measures:
IF (
    NOT ISSELECTEDMEASURE ( [Gross Margin %] ),
    DIVIDE ( SELECTEDMEASURE(), COUNTROWS ( 'Date' ) )
)

-- SELECTEDMEASURENAME: returns the name of the current measure as a string.
-- Use when the measure name controls logic (less safe than ISSELECTEDMEASURE):
IF (
```

```
SELECTEDMEASURENAME() <> "Gross Margin %",  
DIVIDE ( SELECTEDMEASURE(), COUNTROWS ( 'Date' ) )  
)  
-- WARNING: if the measure is renamed, SELECTEDMEASURENAME silently stops matching.  
-- ISSELECTEDMEASURE raises a parse error on rename, making bugs visible immediately.  
-- PREFER ISSELECTEDMEASURE over SELECTEDMEASURENAME.
```

LESSON 4

Format String Overrides

Lesson Objectives

- Explain why YOY% displays as a decimal without a format string override.
- Set a fixed format string on a calculation item.
- Use SELECTEDMEASUREFORMATSTRING() for dynamic format strings.

Why Format Strings Matter

By default, a calculation item inherits the format string of its base measure. This is correct for CY, YTD, PY, and YOY (all have the same unit as the base — currency). It is **wrong** for YOY%: if [Sales Amount] is formatted as currency (e.g. \$ #,0.00), then the YOY% item produces a result like -0.12 formatted as '-\$0.12' instead of '-12%'.

```
-- In Tabular Editor, each calculation item has a Format String property.
-- Leave blank to inherit the base measure's format string.
-- Set a fixed string to override for ALL measures when this item is active.

-- Fixed format string for YOY% item:
-- Format String property: "0.00%"
-- This makes -0.12 display as -12.00% regardless of the base measure's format.

-- DAX expression for YOY% item:
VAR Cy = SELECTEDMEASURE()
VAR Py = CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), DIVIDE ( Cy - Py, Py ) )
-- Format String: "0.00%"

-- Dynamic format string using SELECTEDMEASUREFORMATSTRING:
-- Useful when you want to PRESERVE the base measure's format but append something.
-- Example: append ' YTD' to whatever the base measure's format is:
-- Format String Expression (DAX):
SELECTEDMEASUREFORMATSTRING() & " YTD"
-- If [Sales Amount] has format '$#,0.00', YTD item shows '$1,234,567.00 YTD'.
-- If [Order Count] has format '#,0', YTD item shows '1,234 YTD'.

-- Best approach for most Time Intelligence groups:
-- CY, YTD, QTD, PY: no override (inherit base format)
-- YOY: no override (same unit as base – difference in same currency)
-- YOY%: fixed "0.00%" override
```



In Tabular Editor, the Format String of a calculation item can be either a literal string (e.g. `<font face="Courier" color="#1A3A5C">"0.00%"</font>`) or a DAX expression that returns a string (toggle between modes in the UI). Use a DAX expression when the format must adapt to which base measure is active.

LESSON 5

How Calculation Items Are Applied — The Substitution Model

Lesson Objectives

- Explain that calculation items replace measure references, not whole expressions.
- Trace the actual code generated when a calculation item is applied to a measure.
- State the single-measure-reference rule and explain what breaks without it.
- Identify the safe and unsafe patterns for CALCULATE with a calculation item filter.

The Substitution Model — Measure References Only

This is the most important concept in this module. A calculation item is applied by **substituting SELECTEDMEASURE() for every measure reference** in the expression being evaluated. This substitution happens at the measure-reference level — not at the expression level. Understanding this is essential for predicting what code actually runs:

```
-- Calculation Item YTD:
-- CALCULATE ( SELECTEDMEASURE(), DATESYTD ( 'Date'[Date] ) )

-- SAFE: single measure reference in CALCULATE
[Sales YTD via CI] :=
CALCULATE ( [Sales Amount], 'Time Intelligence'[Time calc] = "YTD" )
-- Expansion: CALCULATE takes [Sales Amount] and applies the YTD item.
-- Result:
-- CALCULATE (
-- CALCULATE ( [Sales Amount], DATESYTD('Date'[Date]) )
-- )
-- Correct: [Sales Amount] evaluated in YTD filter context. Clean.

-- DANGEROUS: complex expression with multiple measure references
[Cost Ratio YTD] :=
CALCULATE (
DIVIDE ( [Total Cost], [Sales Amount] ), -- TWO measure references
'Time Intelligence'[Time calc] = "YTD"
)
-- The YTD item is applied to BOTH measure references:
-- DIVIDE(
-- CALCULATE ( [Total Cost], DATESYTD('Date'[Date]) ),
-- CALCULATE ( [Sales Amount], DATESYTD('Date'[Date]) )
-- )
-- This happens to produce the correct result in this case –
-- but only by luck. In complex expressions it produces incorrect results.

-- CORRECT approach: encapsulate in a single measure, THEN apply the CI
[Cost Ratio] := DIVIDE ( [Total Cost], [Sales Amount] ) -- base measure

[Cost Ratio YTD Fixed] :=
CALCULATE ( [Cost Ratio], 'Time Intelligence'[Time calc] = "YTD" )
-- Now there is ONE measure reference: [Cost Ratio].
-- The YTD item replaces it:
-- CALCULATE ( CALCULATE ( [Cost Ratio], DATESYTD('Date'[Date]) ) )
-- CORRECT: DATESYTD wraps the entire [Cost Ratio] formula.
```

The Working Days Example — Why the Rule Is Non-Negotiable

```
-- Complex measure [Sales per Working Day]:
[Sales WD] :=
DIVIDE (
[Sales Amount],
SUMX ( VALUES('Date'[Year Month Number]),
IF ( [Sales Amount] > 0, [NumOfWorkingDays] ) )
)
-- Contains multiple measure references: [Sales Amount] (x3), [NumOfWorkingDays]

-- WRONG: applying YTD CI directly to the complex expression
[Sales WD YTD Wrong] :=
CALCULATE ( [Sales WD], 'Time Intelligence'[Time calc] = "YTD" )
-- The CI replaces EVERY measure reference with its CALCULATE+DATESYTD wrapper:
-- [Sales Amount] -> CALCULATE([Sales Amount], DATESYTD(...))
-- [NumOfWorkingDays] -> CALCULATE([NumOfWorkingDays], DATESYTD(...))
-- The working days in the SUMX now accumulate as YTD working days per month,
-- not the actual working days in each individual month.
-- Produces absurdly wrong values at quarter and year level.

-- CORRECT: [Sales WD] is a single measure reference in CALCULATE
-- (which it already is above). This IS the correct pattern.
-- The fix was to ensure [Sales WD] is a proper named measure
-- and to apply the CI to [Sales WD] (one measure ref), not to an inline expression.
```

CONCEPT

The Single-Measure-Reference Rule: When using CALCULATE to apply a calculation item filter, the expression inside CALCULATE must be a single measure reference — nothing else. Never write `CALCULATE(DIVIDE([A],[B]), 'CG'[CI]="X")`. Always write `[Ratio] := DIVIDE([A],[B])` first, then `CALCULATE([Ratio], 'CG'[CI]="X")`. This is the most important best practice in calculation groups. Violating it leads to silently incorrect results that are extremely hard to debug.

LESSON 6

Multiple Calculation Groups and Precedence

Lesson Objectives

- Explain what the Precedence property controls.
- Determine the correct precedence order for Time Intelligence + Averages groups.
- Trace the two-step substitution when both groups are active simultaneously.
- Use ISSELECTEDMEASURE to guard items against unwanted measure types.

Precedence — The Order of Application

When two calculation groups are both active in a report (the user has selected items from both slicers simultaneously), the engine must decide which group's calculation item is applied first. This is controlled by the **Precedence** property of each calculation group — an integer where **higher value = applied first**:

```
-- Two calculation groups:
-- Time Intelligence (Precedence = 20): items YTD, PY, YOY...
-- Averages (Precedence = 10): items Daily AVG, Monthly AVG...

-- User selects YTD from Time Intelligence AND Daily AVG from Averages.
-- Goal: (Sales Amount YTD) / (days in YTD period)

-- Step 1: Higher precedence (Time Intelligence, P=20) applied first.
-- [Sales Amount] -> CALCULATE([Sales Amount], DATESYTD('Date'[Date]))
-- Filter context now contains the YTD date range.

-- Step 2: Lower precedence (Averages, P=10) applied second.
-- The Daily AVG item: DIVIDE(SELECTEDMEASURE(), COUNTROWS('Date'))
-- SELECTEDMEASURE() is now the already-modified [Sales Amount YTD].
-- Final code: DIVIDE(CALCULATE([Sales Amount],DATESYTD(...)), COUNTROWS('Date'))
-- COUNTROWS('Date') also sees the YTD filter -> counts days in YTD period.
-- Result: Sales Amount YTD / number of days in YTD. CORRECT.

-- WRONG precedence (Averages=20, Time Intelligence=10):
-- Step 1: Daily AVG applied first.
-- DIVIDE([Sales Amount], COUNTROWS('Date'))
-- Step 2: YTD applied to the one measure reference in that expression.
-- DIVIDE(CALCULATE([Sales Amount],DATESYTD(...)), COUNTROWS('Date'))
-- COUNTROWS('Date') does NOT get the YTD filter!
-- It counts ALL days in the current year, not just YTD days.
-- In December: denominator = 365 even though YTD = 12 months. Correct.
-- In January: denominator = 365, not 31. WRONG.

-- RULE: Time-shifting groups should have HIGHER precedence than aggregation groups.
-- Change the date filter first, then aggregate within the new date context.
```

CONCEPT

Precedence rule of thumb: Groups that modify the filter context (e.g. Time Intelligence — adds date filter) should have **higher precedence** than groups that aggregate within the context (e.g. Averages — divides by COUNTROWS). This ensures COUNTROWS sees the already-modified date filter, not the original filter context.

LESSON 7

Sideways Recursion

Lesson Objectives

- Define sideways recursion in the context of calculation groups.
- Write a PYTD calculation item that reuses the YTD item.
- Explain why true recursion ($A \rightarrow B \rightarrow A$) causes an infinite loop and how DAX handles it.
- State when sideways recursion is safe and when to avoid it.

Sideways Recursion — Reusing Existing Calculation Items

A calculation item can invoke another calculation item from the same group by filtering the calculation group column inside its expression. This is called **sideways recursion**: item A calls item B, but item B does not call back to item A (no circular dependency):

```
-- Time Intelligence group with: YTD, PY, and now PYTD.

-- Option 1: Direct definition (recommended for readability)
-- Calculation Item: PYTD
CALCULATE ( SELECTEDMEASURE(),
  DATESYTD ( SAMEPERIODLASTYEAR ( 'Date'[Date] ) ) )
-- Straightforward. No sideways recursion needed here.

-- Option 2: Sideways recursion (reusing existing items)
-- Calculation Item: PYTD (alternative)
CALCULATE (
  CALCULATE ( SELECTEDMEASURE(), 'Time Intelligence'[Time calc] = "YTD" ),
  SAMEPERIODLASTYEAR ( 'Date'[Date] )
)
-- First applies SAMEPERIODLASTYEAR to shift to prior year,
-- then calls the YTD item within that prior-year context.
-- More complex. Use Option 1 in practice.

-- DANGEROUS: mutual recursion (A -> B -> A) causes infinite loop
-- Item A calls Item B. Item B calls Item A.
-- DAX detects this at evaluation time and raises a circular reference error.
-- DAX does NOT allow full recursion by design.

-- HIDDEN recursion through measures:
-- Measure [MB] := CALCULATE([Sales Amount], 'CG'[Item]='B')
-- Item A: CALCULATE([MB]) -- MB calls Item B
-- Item B: CALCULATE([MA]) -- MA calls Item A -> infinite loop
-- DAX detects this too and raises an error.
```

WARNING

Limit sideways recursion to hidden calculation groups that are controlled entirely by code, never by user slicer selection. When users can freely combine calculation items through slicers, sideways recursion creates unpredictable interactions. For most models: define each calculation item independently. Only use sideways recursion when you have confirmed there is no risk of circular chains.

LESSON 8

Best Practices and Professional Patterns

Lesson Objectives

- Apply the two non-negotiable best practices for calculation groups.
- Build a complete Time Intelligence calculation group with YTD, QTD, PY, YOY, YOY%.
- Build a Metric Selector calculation group for user-driven measure switching.
- Combine both groups with correct precedence.

The Two Non-Negotiable Best Practices

■ **BP1: Only apply calculation items to a single measure reference.** When using CALCULATE to filter a calculation item, the first argument must be exactly one measure reference. Never pass a DIVIDE, an IF, or any expression with multiple measure references.

■ **BP2: Define the Precedence of every calculation group explicitly.** Never leave Precedence at 0 when multiple groups exist. Document your precedence choices in comments.

The Complete Time Intelligence Calculation Group

```
-- Calculation Group: Time Intelligence
-- Column name: Time calc
-- Precedence: 20

-- CY (Ordinal 0) – no modification, shows base measure as-is
SELECTEDMEASURE()

-- YTD (Ordinal 1)
CALCULATE ( SELECTEDMEASURE(), DATESYTD ( 'Date'[Date] ) )

-- QTD (Ordinal 2)
CALCULATE ( SELECTEDMEASURE(), DATESQTD ( 'Date'[Date] ) )

-- PY (Ordinal 3)
CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

-- PY YTD (Ordinal 4)
CALCULATE ( SELECTEDMEASURE(), DATESYTD ( SAMEPERIODLASTYEAR ( 'Date'[Date] ) ) )

-- YOY (Ordinal 5) | Format String: inherited (same unit as base)
VAR Cy = SELECTEDMEASURE()
VAR Py = CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), Cy - Py )

-- YOY% (Ordinal 6) | Format String: "0.00%" <-- OVERRIDE REQUIRED
VAR Cy = SELECTEDMEASURE()
VAR Py = CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
RETURN IF ( NOT ISBLANK(Cy) && NOT ISBLANK(Py), DIVIDE ( Cy - Py, Py ) )
```

The Metric Selector Calculation Group

```
-- Calculation Group: Metric
-- Column name: Metric
-- Precedence: 10 (lower than Time Intelligence – applied AFTER TI items)
```

```
-- Sales Amount (Ordinal 0)
[Sales Amount]
-- Note: no SELECTEDMEASURE() here. This item REPLACES the base measure.
-- Whatever base measure is in the visual, it is replaced by [Sales Amount].

-- Total Cost (Ordinal 1)
[Total Cost]

-- Gross Margin (Ordinal 2)
[Gross Margin]

-- Order Count (Ordinal 3)
[Order Count]

-- Usage: place a placeholder measure in the visual (e.g. [Sales Amount]).
-- Add both slicers: Time calc and Metric.
-- The Metric slicer selects which measure to show.
-- The Time calc slicer selects which variation.
-- The visual effectively shows any measure × any time variation.
```

DEMONSTRATION 1

Building the Time Intelligence Calculation Group

Scenario: The instructor builds the complete Time Intelligence calculation group from scratch in Tabular Editor, returning to Power BI to verify each item, and adds the YOY% format string override.

1	Open Contoso_Demo09_Start.pbix. Launch Tabular Editor from External Tools. Right-click Tables > New Calculation Group. Rename to 'Time Intelligence'. Rename the column to 'Time calc'. Set Precedence = 20.
2	Add six calculation items: CY, YTD, QTD, PY, YOY, YOY%. Enter DAX for each. Set Ordinals 0-5. For YOY% set Format String = "0.00%".
3	Ctrl+S in Tabular Editor. Switch to Power BI Desktop. Show the new 'Time Intelligence' table in Fields. Add a slicer with 'Time calc' column. Add [Sales Amount] to a matrix.
4	Select YTD in slicer. Verify matrix shows cumulative values. Select PY. Verify prior year values. Select YOY%. Verify percentages with % format.
5	Add [Gross Margin] to the same matrix. Show YTD, PY, YOY% all apply to BOTH measures simultaneously — this is the power of calculation groups.
6	Demonstrate the single-measure rule violation: build a <code>CALCULATE(DIVIDE([A],[B]),CI="YTD")</code> measure. Show it produces a value. Build the correct version with a named [Cost Ratio] measure first. Confirm both produce the same result in simple cases but explain when they diverge.
RESULT	Complete Time Intelligence group operational. Format string override validated. The single-measure rule demonstrated with a concrete example.

DEMONSTRATION 2

Combining Two Calculation Groups with Precedence

Scenario: The instructor adds a Daily Average calculation group and demonstrates the precedence failure (wrong order) and fix (correct order), then adds the Metric Selector for a fully dynamic report.

1	Add a second calculation group 'Averages' with one item: Daily AVG = DIVIDE(SELECTEDMEASURE(), COUNTROWS('Date')). Set Precedence = 10 (lower than Time Intelligence). Save and return to Power BI.
2	Add a 'Averages' slicer. Select Daily AVG + YTD simultaneously. Show result in January vs December. Verify December's denominator = 31 (days in YTD period = days in December for the last month). Confirm January = correct daily average for January YTD.
3	Swap precedences: set Averages = 20, Time Intelligence = 10. Return to Power BI. Repeat the test. Show January value is now wrong (denominator = 365 instead of 31). Explain: Daily AVG runs first before the date filter is set.
4	Fix precedence back: Time Intelligence = 20, Averages = 10. Confirm correct results.
5	Build the Metric Selector group. Add Sales Amount, Total Cost, Gross Margin, Order Count items. Set Precedence = 10. Add a third slicer 'Metric'. Show a single-measure matrix that now lets users choose both metric and time variation.
RESULT	Precedence failure demonstrated visually. Correct precedence order confirmed. Dynamic metric+time selector report built with only two slicers and one matrix measure.

LAB 9

Calculation Groups

Lab Scenario

Contoso's report team needs to eliminate measure explosion: instead of maintaining 28+ time-variant measures, they want a single set of 4 base measures driven by a Time Intelligence slicer, with a correct YOY% format and a Metric slicer. You will build and validate everything from scratch.

NOTE

Open Contoso_Lab09_Start.pbix. The model has Sales Amount, Total Cost, Gross Margin, Order Count as measures. Tabular Editor must be installed (available free at tabulareditor.com). Save as Contoso_Lab09_Complete.pbix.

Exercise 1: Time Intelligence Calculation Group

Estimated time: 35 minutes

Task 1: Create the calculation group in Tabular Editor

1. Launch Tabular Editor from External Tools. Create a new Calculation Group. Name it 'Time Intelligence'. Rename the column to 'Time calc'. Set Precedence = 20.
2. Add the following calculation items with the given expressions and ordinals:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "CY (Ordinal 0): SELECTEDMEASURE()
YTD (Ordinal 1): CALCULATE ( SELECTEDMEASURE(), DATESYTD ( 'Date'[Date] ) )
QTD (Ordinal 2): CALCULATE ( SELECTEDMEASURE(), DATESQTD ( 'Date'[Date] ) )
PY (Ordinal 3): CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )
PY YTD (Ordinal 4): CALCULATE ( SELECTEDMEASURE(),
DATESYTD(SAMEPERIODLASTYEAR('Date'[Date])) )
YOY (Ordinal 5): VAR Cy=SELECTEDMEASURE() VAR
Py=CALCULATE(SELECTEDMEASURE(),SAMEPERIODLASTYEAR('Date'[Date])) RETURN IF(NOT
ISBLANK(Cy)&&NOT; ISBLANK(Py),Cy-Py)
YOY% (Ordinal 6): VAR Cy=SELECTEDMEASURE() VAR
Py=CALCULATE(SELECTEDMEASURE(),SAMEPERIODLASTYEAR('Date'[Date])) RETURN IF(NOT
ISBLANK(Cy)&&NOT; ISBLANK(Py),DIVIDE(Cy-Py,Py))" 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='CY (Ordinal 0):
SELECTEDMEASURE()', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="YTD (Ordinal 1): CALCULATE ( SELECTEDMEASURE(), DATESYTD ( 'Date'[Date] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="QTD (Ordinal 2): CALCULATE (
SELECTEDMEASURE(), DATESQTD ( 'Date'[Date] ) )", textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="PY (Ordinal 3): CALCULATE ( SELECTEDMEASURE(), SAMEPERIODLASTYEAR ( 'Date'[Date] ) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
```

```

fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="PY YTD (Ordinal 4):
CALCULATE ( SELECTEDMEASURE(), DATESYTD(SAMEPERIODLASTYEAR('Date'[Date])) )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="YOY (Ordinal 5): VAR
Cy=SELECTEDMEASURE() VAR
Py=CALCULATE(SELECTEDMEASURE(),SAMEPERIODLASTYEAR('Date'[Date])) RETURN IF(NOT
ISBLANK(Cy)", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&NOT;',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ISBLANK(Py),Cy-Py)',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="YOY% (Ordinal 6): VAR
Cy=SELECTEDMEASURE() VAR
Py=CALCULATE(SELECTEDMEASURE(),SAMEPERIODLASTYEAR('Date'[Date])) RETURN IF(NOT
ISBLANK(Cy)", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='&NOT;',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='
ISBLANK(Py),DIVIDE(Cy-Py,Py))', textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style':
'bulletText': None 'debug': 0 ) #Paragraph

```

3. Set the Format String of the YOY% item to "0.00%".
4. Save in Tabular Editor (Ctrl+S). Return to Power BI Desktop.

Task 2: Verify in Power BI

1. Add a Slicer visual with 'Time Intelligence'[Time calc] as the field. Add a Matrix visual: Date[Year] on rows, Date[Month] on columns, [Sales Amount] as value.
2. Select YTD in the slicer. Confirm values are cumulative within each year.
3. Select PY. Confirm 2008 values equal the 2007 [Sales Amount] values in the same months.
4. Select YOY%. Confirm values display as percentages (e.g. -12.34%). Confirm first-year rows show blank (ISBLANK protection).
5. Add [Gross Margin] alongside [Sales Amount]. Confirm both measures respond to the slicer simultaneously.

Results: All 7 calculation items operational. YOY% shows percentages. Both measures change together with the slicer. First-year blanks confirmed.

Exercise 2: Format String Overrides for YOY%

Estimated time: 15 minutes

Task 1: Verify format string is applied correctly

1. In the matrix from Exercise 1, select YOY% for [Sales Amount]. Confirm values show as percentages. Confirm values do NOT show as currency.

2. Select YOY (not YOY%). Confirm values show as currency (same format as [Sales Amount]).
3. Switch the base measure to [Order Count] (integer). Select YOY%. Confirm percentages still display correctly — the format string override applies to all measures.

Task 2: Dynamic format string (optional advanced)

1. In Tabular Editor, change the YTD item's Format String to a DAX expression:
`SELECTEDMEASUREFORMATSTRING( ) & " YTD"`. Save and return to Power BI.
2. Select YTD. Observe the tooltip or formatted value includes ' YTD' suffix.

Results: Fixed format string override for YOY% confirmed. Dynamic format string demonstrated on YTD.

Exercise 3: Metric Selector Group and Precedence

Estimated time: 25 minutes

Task 1: Create the Metric Selector group

1. In Tabular Editor, create a second Calculation Group 'Metric'. Rename column to 'Metric'. Set Precedence = 10.
2. Add four items (no SELECTEDMEASURE — full replacement):
 Paragraph('caseSensitive': 1 'encoding': 'utf8' 'text': 'Sales Amount (Ordinal 0): [Sales Amount]
 Total Cost (Ordinal 1): [Total Cost]
 Gross Margin (Ordinal 2): [Gross Margin]
 Order Count (Ordinal 3): [Order Count]' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier',
 fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Sales Amount (Ordinal 0): [Sales Amount]',
 textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
 fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
 textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
 fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Total Cost (Ordinal 1): [Total
 Cost]', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
 fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
 textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
 fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Gross Margin (Ordinal 2): [Gross
 Margin]', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
 fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
 textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
 fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='Order Count (Ordinal 3): [Order
 Count]', textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0)
 #Paragraph
3. Save and return to Power BI Desktop. Add a second slicer for 'Metric'[Metric]. Place any placeholder measure (e.g. [Sales Amount]) in a matrix.
4. Select 'Gross Margin' in the Metric slicer AND 'YTD' in the Time calc slicer. Confirm the matrix shows Gross Margin YTD values.

Task 2: Verify precedence

1. With both slicers active (Metric='Sales Amount', Time='YOY%'), confirm YOY% values display as percentages (format string from Time Intel group applies correctly).
2. In Tabular Editor, swap Precedence: Metric=20, Time Intelligence=10. Save. Re-select the same items. Observe that YOY% format string no longer applies correctly (the Metric item runs first, overrides the measure, then YOY% format string is applied to a full-replacement item). Restore correct precedence: Time Intelligence=20, Metric=10. Save.
3. Save the file as **Contoso_Lab09_Complete.pbix**.

Results: Metric Selector group operational. Any measure x any time variation available through two slicers. Precedence swap demonstrated — incorrect results confirmed. Correct precedence restored.

Module Review and Takeaways

Review Questions

What is the measure explosion problem, and how do calculation groups solve it?

Answer: With N base measures and M time variations (YTD, PY, YOY...), maintaining $N \times M$ separate measures becomes unmanageable — every change to a base measure requires updating M derived measures. Calculation groups solve this by defining the M variations once as calculation items. Each item uses `SELECTEDMEASURE()` as a placeholder that is replaced by whatever measure is being evaluated. The result: N base measures + M items = N+M objects, not N×M.

What does `SELECTEDMEASURE()` return, and in what context can it be used?

Answer: `SELECTEDMEASURE()` returns the value of whichever measure is being evaluated at the moment the calculation item is applied. It is a placeholder that is substituted at evaluation time with the base measure's full expression. It can only be used inside calculation item expressions — it is not valid in regular DAX measures.

Why must a `CALCULATE` that filters a calculation item contain only a single measure reference?

Answer: Calculation items are applied by substituting `SELECTEDMEASURE()` for every measure reference in the expression. If the expression contains multiple measure references (e.g. `DIVIDE([A],[B])`), the substitution applies the calculation item to EACH reference independently. This can produce correct results by coincidence in simple cases but generates incorrect results when the item modifies the filter context (e.g. `DATESYTD` wraps each measure reference separately rather than wrapping the whole expression). A single measure reference ensures the calculation item wraps the entire business logic at once.

How does Precedence control the application order of two calculation groups?

Answer: The Precedence property is an integer assigned to each calculation group. Higher value = applied first. When a user selects items from two groups simultaneously, the group with the higher precedence has its item applied to the measure first. The resulting expression then becomes the 'measure' that the lower-precedence group's item is applied to. Time-shifting groups (modifying filter context) should have higher precedence than aggregation groups (dividing by `COUNTROWS`), so that `COUNTROWS` sees the already-filtered date range.

What is sideways recursion, and when is it safe to use?

Answer: Sideways recursion is when a calculation item invokes another calculation item from the same calculation group (not a circular reference — item A calls item B, but B does not call back to A). It allows reusing complex item logic without repeating code. It is safe only in hidden calculation groups controlled entirely by code, not exposed to user slicer selection. When users can combine items through slicers, sideways recursion can create unintended interaction chains. DAX prevents true circular recursion ($A \rightarrow B \rightarrow A$) at evaluation time.

Quick Reference — Calculation Group Functions

Function / Property	Purpose	Usage
<code>SELECTEDMEASURE()</code>	Placeholder: replaced by the current measure being evaluated at runtime.	Inside calculation item expressions only.

<code>SELECTEDMEASURENAME()</code>	Returns the name of the current measure as a string.	Branching logic by measure name. Less safe than <code>ISSELECTEDMEASURE</code> .
<code>ISSELECTEDMEASURE([A], [B], ...)</code>	Returns TRUE if the current measure is one of the listed measures.	Gating — apply item only to specific measures. Raises error on misspelled measure names.
<code>SELECTEDMEASUREFORMATSTRING()</code>	Returns the format string of the current base measure.	Dynamic format strings. Append currency symbol, unit suffix, etc.
<code>Precedence</code> (property)	Integer. Higher value = applied first when multiple groups are active.	Set higher on time-shifting groups, lower on aggregation groups.
<code>Ordinal</code> (property)	Integer. Controls the sort order of calculation items in slicers.	Set CY=0, YTD=1, QTD=2, PY=3, YOY=4, YOY%=5 for logical user experience.
<code>Format String</code> (property)	Override the base measure's format string for a specific item.	Required for YOY% to display as percentage instead of decimal.
<code>Single-measure rule</code>	Best Practice: <code>CALCULATE</code> (CI filter) must have exactly one measure reference as its first argument.	Prevents incorrect multi-substitution bugs.
<code>Sideways recursion</code>	A calculation item referencing another item in the same group.	Use only in hidden, code-controlled groups.

Module Summary

- Calculation groups solve the measure explosion problem. Define M time variations once; they apply to all N base measures automatically. The model shrinks from N×M measures to N+M objects.
- `SELECTEDMEASURE()` is the placeholder for the current base measure inside a calculation item. It is substituted at evaluation time by the measure's expression. `ISSELECTEDMEASURE` provides type-safe branching by measure identity.
- Calculation items are applied by substituting `SELECTEDMEASURE()` for every measure reference. The single-measure-reference rule: always apply calculation items to a single named measure reference — never to an inline expression with multiple measure references.
- The `Precedence` property (integer, higher = applied first) controls application order when multiple groups are active. Time-shifting groups must have higher precedence than aggregation groups to ensure `COUNTROWS` and other context-sensitive functions see the already-modified filter context.
- Format string overrides are essential for YOY%: set `Format String` = "0.00%" so the item displays as a percentage regardless of the base measure's format. Dynamic format strings using `SELECTEDMEASUREFORMATSTRING()` preserve the base format while adding information.
- `Sideways recursion` allows one calculation item to invoke another item in the same group. Use only in hidden, code-controlled groups. Avoid in user-facing groups with slicer selection. DAX prevents true circular recursion at evaluation time.

NOTE

Next — Module 10: Working with the Filter Context. The final module covers the advanced filter-context inspection functions: `HASONEVALUE`, `SELECTEDVALUE`, `ISFILTERED`, `ISCROSSFILTERED`, `VALUES` vs `FILTERS`, the critical `ALLEXCEPT` vs `ALL+VALUES` distinction, `ISEMPTY`, data lineage, `TREATAS`, and arbitrarily shaped filters — the toolkit for writing measures that work correctly in every report, regardless of how the filter context is constructed.

The Definitive Guide to DAX

Business Intelligence with Microsoft Power BI, Analysis Services, and Excel

MODULE

10

Working with the Filter Context

HASONEVALUE · ISFILTERED · VALUES vs FILTERS · ALLEXCEPT Trap · TREATAS

The advanced filter-context toolkit for professional-grade DAX.

Master HASONEVALUE, SELECTEDVALUE, ISFILTERED, ISCROSSFILTERED, the critical ALLEXCEPT-vs-ALL+VALUES distinction, ISEMPY, data lineage, TREATAS, arbitrarily shaped filters, and KEEPFILTERS.

Write measures that produce correct results in every report, regardless of how the user constructs their filter context.

Prerequisite: Modules 1–9. Final module of the series.

Based on: The Definitive Guide to DAX, 2nd Ed. — Russo & Ferrari | Adapted for Educational Use

Module 10: Working with the Filter Context

Module Contents

■ Module Overview — Why Filter Context Awareness Matters

■ Lesson 1: HASONEVALUE and SELECTEDVALUE

- HASONEVALUE — Protecting Calculations for Specific Granularities
- SELECTEDVALUE — The Compact Extraction Pattern
- What-If Analysis with Parameter Tables

■ Lesson 2: ISFILTERED and ISCROSSFILTERED

- ISFILTERED — Is a Column Directly Filtered?
- ISCROSSFILTERED — Is a Table or Column Indirectly Filtered?
- The Limitation: Filtered Does Not Mean Values Are Reduced
- HASONEFILTER vs HASONEVALUE

■ Lesson 3: VALUES vs FILTERS

- VALUES — Visible Values Under the Current Filter Context
- FILTERS — All Actively Filtered Values
- When They Differ — The Category + Color Slicer Scenario
- Using FILTERS for Accurate Selected-Item Labels

■ Lesson 4: The ALLEXCEPT Trap — ALL+VALUES vs ALLEXCEPT

- How ALLEXCEPT Works as a CALCULATE Modifier
- The Hidden Trap — When the Except Column Is Not in the Report
- The Correct Pattern: ALL + VALUES
- How ALL* Modifiers Interact with Context Transition

■ Lesson 5: ISEMPY

- ISEMPY — Testing Whether a Table Has Rows
- Performance Advantage Over COUNTROWS
- The NonBuyingCustomers Pattern

■ Lesson 6: Data Lineage and TREATAS

- What Is Data Lineage?
- Anonymous Values vs Column-Tagged Values
- TREATAS — Reassigning Data Lineage
- TREATAS for Multi-Column Filters
- Data Lineage Rules — Column Reference vs Expression

■ Lesson 7: Arbitrarily Shaped Filters

- Simple Filters vs Arbitrarily Shaped Filters
- How CALCULATE Destroys the Filter Shape
- Fix 1 — Iterate a Unique-Per-Period Column
- Fix 2 — KEEPFILTERS in the Iterator
- Best Practices for Arbitrarily Shaped Filter Safety

■ Lesson 8: Professional Patterns and Series Synthesis

- ISINSCOPE — Hierarchy-Level Detection
- The Ratio-to-Parent Pattern
- The What-If Inflation Pattern
- The NonBuying Customer Segmentation Pattern

- The TREATAS Semi-Additive Optimisation Pattern

■ **Demonstration 1: Filter Inspection Functions in Action**

■ **Demonstration 2: The ALLEXCEPT Trap and TREATAS**

■ **Lab 10: Working with the Filter Context**

- Exercise 1: HASONEVALUE, SELECTEDVALUE, and Parameter Tables
- Exercise 2: ISFILTERED, ISCROSSFILTERED, and VALUES vs FILTERS
- Exercise 3: The ALLEXCEPT Trap and Correct Pattern
- Exercise 4: Data Lineage, TREATAS, and Arbitrarily Shaped Filters

■ **Module Review, Series Recap, and Exam Readiness**

Module Overview — Why Filter Context Awareness Matters

Every DAX measure operates inside a filter context. In most cases that context is well-behaved: a year slicer, a category on the row axis, a single visual applying a clean column filter. But as reports grow in complexity — multiple slicers, hierarchies, cross-filtering, drill-through, hierarchical Excel selections — the filter context becomes more unpredictable. A measure that works correctly on a clean report can silently produce wrong results when placed in a different context.

This module provides the complete toolkit for **reading, testing, and safely responding** to the filter context at runtime: inspection functions (HASONEVALUE, ISFILTERED, ISCROSSFILTERED), value extraction (SELECTEDVALUE, VALUES, FILTERS), the critical ALLEXCEPT vs ALL+VALUES distinction, empty-table testing (ISEMPTY), data lineage and TREATAS, and the defence against arbitrarily shaped filters (KEEPFILTERS).

EXAM

PL-300 tests: HASONEVALUE and SELECTEDVALUE for parameter tables; ISFILTERED to conditionally show/hide calculations; the difference between VALUES (visible) and FILTERS (actively filtered); the ALLEXCEPT pitfall; and the concept of data lineage. These are also the most common sources of hard-to-debug production bugs — mastering this module is essential for professional-grade Power BI development.

LESSON 1

HASONEVALUE and SELECTEDVALUE**Lesson Objectives**

- Use HASONEVALUE to protect a calculation from running at the wrong granularity.
- Use SELECTEDVALUE to extract a single user-selected value with a safe default.
- Build a what-if parameter table and use SELECTEDVALUE to read the parameter.

HASONEVALUE — Protecting Calculations

HASONEVALUE(column) returns TRUE when exactly one value of the column is visible in the current filter context. It is the standard guard for calculations that only make sense at a specific granularity:

```
-- Problem: QTD at year level shows Q4 value, confusing users.
[QTD Sales Raw] := CALCULATE ( [Sales Amount], DATESQTD ( 'Date'[Date] ) )
-- At year level: filter context has all 12 months -> last quarter's total shown.
-- Misleading. Should be blank.

-- Fix: only show QTD when exactly one month is visible
[QTD Sales] :=
IF (
HASONEVALUE ( 'Date'[Month Number] ),
CALCULATE ( [Sales Amount], DATESQTD ( 'Date'[Date] ) )
)
-- At month level: one Month Number visible -> TRUE -> QTD shown.
-- At quarter or year level: multiple months visible -> FALSE -> BLANK.

-- Other common HASONEVALUE guards:
[RANKX Protected] :=
IF ( HASONEVALUE('Product'[Brand]),
RANKX(ALL('Product'[Brand]),[Sales Amount]), BLANK() )

-- Guard for any calculation meaningful only for a single product:
[Margin %] :=
IF (
HASONEVALUE ( 'Product'[Product Name] ),
DIVIDE ( [Gross Margin], [Sales Amount] )
)
```

SELECTEDVALUE — Compact Single-Value Extraction

```
-- Long form: manually check HASONEVALUE, then extract with VALUES
[User Selected Rate] :=
IF (
HASONEVALUE ( 'Discount Rate'[Rate] ),
VALUES ( 'Discount Rate'[Rate] ),
0.05 -- default: 5%
)

-- Compact form: SELECTEDVALUE does the same in one call
[User Selected Rate] := SELECTEDVALUE ( 'Discount Rate'[Rate], 0.05 )
-- Syntax: SELECTEDVALUE ( column, [default_value] )
-- Returns the single visible value if exactly one is visible.
-- Returns the default if zero or more than one value is visible.
```

```
-- Default is optional; omitting it returns BLANK on multi-selection.

-- What-If Inflation Analysis pattern:
-- Step 1: Inflation Rate parameter table (Year=2005..2025, Inflation=0..20%)
-- Step 2: Read the selected rate
[Inflation Rate Selected] := SELECTEDVALUE ( 'Inflation'[Rate], 0 )

-- Step 3: Reporting year (latest order year in the data)
[Reporting Year] := YEAR ( MAX ( Sales[Order Date] ) )

-- Step 4: Multiplier – compound inflation from each year to reporting year
[Inflation Multiplier] :=
VAR ReportYear = [Reporting Year]
VAR SelectedYear = SELECTEDVALUE ( 'Date'[Calendar Year Number], ReportYear )
VAR Rate = [Inflation Rate Selected]
VAR YearsAgo = ReportYear - SelectedYear
RETURN ( 1 + Rate ) ^ YearsAgo

-- Step 5: Inflation-adjusted sales (iterate by year, apply multiplier per year)
[Inflation Adjusted Sales] :=
SUMX (
VALUES ( 'Date'[Calendar Year Number] ),
[Sales Amount] * [Inflation Multiplier]
)
```

TIP

Power BI's What-If Parameter feature (Modeling > New Parameter) auto-generates the parameter table and SELECTEDVALUE measure automatically. Understanding what it generates helps you build custom variations. SELECTEDVALUE is the correct way to read a What-If parameter in any DAX measure.

LESSON 2

ISFILTERED and ISCROSSFILTERED

Lesson Objectives

- Distinguish a direct filter (ISFILTERED) from an indirect filter (ISCROSSFILTERED).
- Explain why ISFILTERED=TRUE does not guarantee that values are reduced.
- Know when to use ISCROSSFILTERED with a table argument vs a column argument.
- Distinguish HASONEFILTER from HASONEVALUE.

ISFILTERED vs ISCROSSFILTERED

ISFILTERED(column) returns TRUE when a direct filter exists on that column — meaning CALCULATE or a slicer/visual has placed an explicit filter on that column. ISCROSSFILTERED(table_or_column) returns TRUE when the column or table is filtered directly OR indirectly (because a related table or column is filtered):

```
-- Simple measures to inspect filter state:
[Is Gender Filtered] := ISFILTERED ( Customer[Gender] )
[Is Customer CrossF] := ISCROSSFILTERED ( Customer )
[Is Sales CrossF] := ISCROSSFILTERED ( Sales )

-- Scenario: Matrix with Customer[Continent] on rows, Customer[Gender] on columns.
-- Cell: Europe / Male
-- ISFILTERED(Customer[Gender]) = TRUE (Gender slicer is active)
-- ISFILTERED(Customer[Continent]) = TRUE (row filter on Continent)
-- ISCROSSFILTERED(Customer) = TRUE (any column of Customer is filtered)
-- ISCROSSFILTERED(Sales) = TRUE (Sales many-side follows Customer filter)

-- Subtotal row: Europe (no column filter)
-- ISFILTERED(Customer[Gender]) = FALSE (no direct filter on Gender)
-- ISFILTERED(Customer[Continent]) = TRUE (Continent still filtered)
-- ISCROSSFILTERED(Customer) = TRUE (Continent is a Customer column)

-- Practical use: show a calculation only when a specific column is filtered
[Conditional Metric] :=
IF (
    ISFILTERED ( 'Product'[Category] ),
    [Sales Amount], -- show when category is actively filtered
    BLANK() -- hide at grand total or non-category context
)
```

The Limitation — Filtered Does Not Mean Values Are Reduced

WARNING

ISFILTERED = TRUE does not mean all values of the column are NOT visible. A filter can be active while still permitting all values to pass through. Example: `CALCULATE([Sales Amount], Customer[City] <> "DAX")` — there is no city named DAX in the table. The filter is active (ISFILTERED = TRUE), but every single city value is still visible. To reliably check whether values are truly reduced, compare `COUNTROWS(Customer)` against `COUNTROWS(ALL(Customer))`. If they are equal, the filter is active but not effective.

HASONEFILTER vs HASONEVALUE

```
-- HASONEFILTER: TRUE when exactly ONE filter value is active on the column.
-- HASONEVALUE: TRUE when exactly ONE value is VISIBLE in the filter context.

-- Difference scenario:
-- Filter: Customer[Gender] = 'Male'
-- But in the current context (Customer Type = Company), NO customers are Male.
-- VALUES(Customer[Gender]) = {} (empty – no Male companies)

-- HASONEFILTER(Customer[Gender]) = TRUE (one filter value: 'Male')
-- HASONEVALUE(Customer[Gender]) = FALSE (zero visible values after cross-filtering)

-- Rule: Use HASONEVALUE when you need to extract the visible value.
-- Use HASONEFILTER when you need to know the user's explicit selection,
-- regardless of whether that value currently exists in the context.
```

LESSON 3

VALUES vs FILTERS

Lesson Objectives

- Explain the difference between VALUES (visible values) and FILTERS (filtered values).
- Identify the scenario where VALUES and FILTERS return different results.
- Choose the correct function for a 'selected items' label measure.

VALUES vs FILTERS — The Precise Difference

VALUES(column) returns the distinct values of a column that are visible in the current filter context — after all cross-filtering and relationship propagation. FILTERS(column) returns the values that an explicit filter has been applied to, regardless of whether those values actually exist in the current context:

```
-- Scenario:
-- Two slicers: Category = 'TV and Video', Color = {Red, Blue, Azure, Silver}
-- No TV and Video products exist in Azure or Silver.

-- VALUES returns only the colors that EXIST for TV and Video products:
[Selected Colors VALUES] :=
CONCATENATEX ( VALUES('Product'[Color]), 'Product'[Color], ", ", 'Product'[Color], ASC )
-- Result: 'Blue, Red' (only 2 of the 4 selected colors exist in this category)

-- FILTERS returns all colors that the slicer filter is set to:
[Selected Colors FILTERS] :=
CONCATENATEX ( FILTERS('Product'[Color]), 'Product'[Color], ", ", 'Product'[Color], ASC )
-- Result: 'Azure, Blue, Red, Silver' (all 4 slicer selections, regardless of category)

-- Which to use?
-- VALUES: 'show what actually exists in this context' -> use for calculations.
-- FILTERS: 'show what the user explicitly selected' -> use for informational labels.

-- Complete Selected Colors label with FILTERS:
[Color Filter Label] :=
VAR FilteredColors = FILTERS ( 'Product'[Color] )
VAR AllColors = COUNTROWS ( ALL ( 'Product'[Color] ) )
VAR SelectedCount = COUNTROWS ( FilteredColors )
VAR ColorList = CONCATENATEX ( FilteredColors, 'Product'[Color], ", ",
'Product'[Color], ASC )
RETURN SWITCH ( TRUE(),
SelectedCount = AllColors, "All " & AllColors & " colors selected.",
SelectedCount > 5, SelectedCount & " colors selected.",
"Showing: " & ColorList
)
```

CONCEPT

The VALUES vs FILTERS distinction only matters when there is a cross-filtering relationship between two tables and the user has selected values that do not all exist in both tables simultaneously. For simple slicers on a single table, VALUES and FILTERS return the same result. Use FILTERS when building labels that must reflect the user's slicer selection truthfully, not just the values that happen to exist after cross-filtering.

LESSON 4

The ALLEXCEPT Trap — ALL+VALUES vs ALLEXCEPT

Lesson Objectives

- Explain how ALLEXCEPT behaves as a CALCULATE modifier (removes filters, never adds).
- Demonstrate the hidden trap when the excepted column is not in the report.
- Write the correct pattern using ALL + VALUES.
- Explain how ALL* modifiers suppress context transition in CALCULATE.

How ALLEXCEPT Works — and Its Hidden Trap

The ALL* functions, when used as filter arguments in CALCULATE, are **CALCULATE modifiers** — they remove filters from the context. They never add filters. ALLEXCEPT(Table, Col1, Col2...) removes all filters from the table **except** direct filters on the listed columns. This sounds correct — but hides a critical trap when those columns are not active in the report:

```
-- Goal: count customers of the same Customer Type as the current row.
-- Report columns: Customer[Continent], Customer[Gender]

-- Attempt with ALLEXCEPT:
[AllExcept Type] :=
CALCULATE (
COUNTROWS ( Customer ),
ALLEXCEPT ( Customer, Customer[Customer Type] )
)

-- When report has Continent on rows and Gender on columns:
-- ALLEXCEPT removes the Continent and Gender filters.
-- Only Customer Type filter remains (if Customer Type is in the report).
-- WORKS correctly when Customer Type is on the report.

-- THE TRAP: remove Customer Type from the report.
-- Now the report only has Gender on the rows.
-- ALLEXCEPT says: 'remove all filters except Customer Type'.
-- But Customer Type has no filter active (it's not on the report).
-- Result: ALL filters are removed -> count = ALL customers = grand total.
-- EVERY row shows the same value: total number of customers. WRONG.

-- Root cause: ALLEXCEPT removes direct filters on Continent and Gender,
-- but does NOT add a filter on Customer Type.
-- The 'except' means 'preserve IF a filter exists' – not 'add a filter'.
-- ALL* functions can ONLY remove filters. They never add them.
```

The Correct Pattern — ALL + VALUES

```
-- CORRECT: ALL removes all filters, then VALUES re-adds only the visible Customer Types
[Correct By Type] :=
CALCULATE (
COUNTROWS ( Customer ),
ALL ( Customer ), -- CALCULATE modifier: remove all filters
VALUES ( Customer[Customer Type] ) -- filter argument: add back visible types
)

-- ALL(Customer) removes all direct filters from Customer.
-- VALUES(Customer[Customer Type]) evaluates AFTER ALL, in the now-unfiltered context.
```

```
-- But Customer[Customer Type] is CROSS-FILTERED by Gender (through the relationship).
-- So VALUES returns only the Customer Types visible given the current Gender filter.
-- The result is: count of all customers of the same type(s) visible in this context.
-- Works correctly whether Customer Type is on the report or not.

-- Key insight: the difference between ALLEXCEPT and ALL+VALUES:
-- ALLEXCEPT: 'remove all DIRECT filters except those on the listed columns'
-- -> If the excepted column has no direct filter, nothing is preserved.
-- ALL + VALUES: 'remove all direct filters, then apply VALUES as a new filter'
-- -> VALUES picks up cross-filters, making the measure context-aware.
```

ALL* Modifiers and Context Transition Suppression

```
-- ALL* functions as CALCULATE modifiers suppress context transition.
-- Reason: CT applies a filter, then the ALL modifier removes it. The optimizer
-- recognizes this as a no-op and skips the CT entirely.

-- Calculated column: % of global sales (no CT effect intended)
'Product'[Global Pct] =
VAR ProductSales = [Sales Amount] -- CT fires: current product's sales
VAR TotalSales =
CALCULATE (
[Sales Amount],
ALL ( 'Product' ) -- removes all Product filters (incl CT)
)
RETURN DIVIDE ( ProductSales, TotalSales )

-- ALL('Product') is a CALCULATE modifier applied AFTER context transition.
-- The optimizer knows CT and ALL cancel out -> skips CT entirely.
-- TotalSales = grand total sales. Correct.

-- % of Category (keep Category filter, remove Product Name filter)
'Product'[Category Pct] =
VAR ProductSales = [Sales Amount]
VAR CategorySales =
CALCULATE (
[Sales Amount],
ALLEXCEPT ( 'Product', 'Product'[Category] )
)
RETURN DIVIDE ( ProductSales, CategorySales )

-- ALLEXCEPT removes all filters except Category.
-- Category was set by CT from the calculated column row context.
-- Result: sales of the current product / total sales of its category.
```

CONCEPT

The ALL* Modifier Rule: ALL, ALLEXCEPT, ALLNOBLANKROW, ALLSELECTED — when used as filter arguments in CALCULATE — are CALCULATE modifiers. They remove filters. They never add them. They are applied AFTER context transition, so they can suppress the effect of CT. VALUES, FILTER, and column predicates are filter arguments — they add filters to the context. Only ALL+VALUES together gives you: remove everything, then re-add based on current cross-filtering state.

LESSON 5

ISEMPTY

Lesson Objectives

- Use ISEMPTY to test whether a table expression has any rows.
- Explain the performance advantage of ISEMPTY over COUNTROWS.
- Build the NonBuyingCustomers measure using ISEMPTY and RELATEDTABLE.

ISEMPTY — Efficient Empty-Table Testing

```
-- Without ISEMPTY: requires a full scan to count
COUNTROWS ( VALUES ( 'Product'[Color] ) ) = 0

-- With ISEMPTY: engine stops at the first row found (no full scan needed)
ISEMPTY ( VALUES ( 'Product'[Color] ) )

-- ISEMPTY is always preferred for performance when you only need TRUE/FALSE.

-- NonBuyingCustomers: customers who made NO purchases in the current filter context
[NonBuying Customers] :=
VAR AllSelectedCustomers =
CALCULATETABLE (
DISTINCT ( Customer[CustomerKey] ),
ALLSELECTED () -- respects slicer selections
)
VAR NoSalesCustomers =
FILTER (
AllSelectedCustomers,
ISEMPTY ( RELATEDTABLE ( Sales ) ) -- row context: current CustomerKey
)
RETURN COUNTROWS ( NoSalesCustomers )

-- For each CustomerKey in AllSelectedCustomers:
-- RELATEDTABLE(Sales) returns the Sales rows related to this customer.
-- ISEMPTY = TRUE if that customer has zero sales rows in this context.
-- Result: count of customers with no sales in the current filter context.

-- Practical use: place alongside [Unique Customers] to show buying vs non-buying split.
[Buying Customers] := [Unique Customers]
[NonBuying Customers] := [NonBuying Customers] -- measure above
[Buying Rate %] := DIVIDE ( [Unique Customers], [Unique Customers] + [NonBuying Customers] )
```

LESSON 6

Data Lineage and TREATAS

Lesson Objectives

- Define data lineage and explain why it is needed for CALCULATE filters.
- Distinguish a column-tagged value from an anonymous value.
- Use TREATAS to assign data lineage to a computed table.
- Apply TREATAS for multi-column filters and semi-additive optimisation.

Data Lineage — The Tag That Makes Filters Work

Every column in a table carries a hidden **data lineage tag** that identifies which column in the data model it came from. When a table is used as a CALCULATE filter argument, DAX uses the lineage tags to know which columns in the model to filter. Values without lineage tags are **anonymous** — they produce no filter effect:

```
-- Anonymous table: no lineage -> no filter effect
[Test Wrong] :=
CALCULATE (
[Sales Amount],
{ "Red", "Blue" } -- anonymous table (no column tag)
)
-- Result: same as [Sales Amount] with no filter. The anonymous values do nothing.

-- Correct: preserve lineage by filtering a column that has it
[Test Correct] :=
CALCULATE (
[Sales Amount],
FILTER (
ALL ( 'Product'[Color] ), -- ALL returns tagged values
'Product'[Color] IN { "Red", "Blue" }
)
)
-- ALL('Product'[Color]) returns all color values WITH the lineage of Product[Color].
-- FILTER preserves the lineage. CALCULATE knows to filter Product[Color]. Correct.

-- Lineage rules:
-- Column reference -> lineage preserved (VALUES, ALL, DISTINCT, FILTER preserve it)
-- Expression result -> anonymous ('Product'[Color] & "" loses lineage)
-- Constant table -> anonymous ({ "Red", "Blue" } has no lineage)

-- ADDCOLUMNS preserves lineage of existing columns but new columns are anonymous:
ADDCOLUMNS (
VALUES ( 'Product'[Color] ), -- [Color] retains lineage
"AnonColor", 'Product'[Color] & "" -- anonymous: expression result
)
```

TREATAS — Assigning Data Lineage

```
-- TREATAS assigns the lineage of specified model columns to an anonymous table.
-- Syntax: TREATAS ( table_expression, column1 [, column2 ...] )

-- Simple TREATAS: make an anonymous constant table filter a model column
[Red and Blue Sales] :=
CALCULATE (
```

```
[Sales Amount],
TREATAS ( { "Red", "Blue" }, 'Product'[Color] )
)
-- TREATAS tags { "Red", "Blue" } with the lineage of Product[Color].
-- CALCULATE now knows to filter Product[Color]. Result = Red + Blue sales.

-- Multi-column TREATAS: filter multiple columns with a single table expression
-- Example: sales for (Red in 2007) and (Blue in 2008) only (NOT red in 2008)
CALCULATE (
[Sales Amount],
TREATAS (
{ ( "Red", "CY 2007" ), ( "Blue", "CY 2008" ) },
'Product'[Color],
'Date'[Calendar Year]
)
)
-- This is an arbitrarily shaped filter – the combination Red+2008 is excluded.
-- Cannot be expressed as two independent column filters.

-- TREATAS for semi-additive last balance per customer (optimised pattern):
[Last Balance Per Customer TREATAS] :=
VAR LastCustomerDate =
ADDCOLUMNS (
VALUES ( Balances[Name] ),
"LastDate",
CALCULATE ( MAX ( 'Date'[Date] ),
LASTNONBLANK ( 'Date'[Date], COUNTROWS(RELATEDTABLE(Balances)) ) )
)
VAR FilterPairs =
TREATAS ( LastCustomerDate, Balances[Name], 'Date'[Date] )
VAR Result =
CALCULATE ( SUM ( Balances[Balance] ), FilterPairs )
RETURN Result
-- FilterPairs is a table of (CustomerName, LastDate) pairs tagged with proper lineage.
-- CALCULATE filters: for each customer, exactly their last known date.
-- One CALCULATE call instead of iterating every customer. More efficient.
```

LESSON 7

Arbitrarily Shaped Filters

Lesson Objectives

- Distinguish a simple column filter from an arbitrarily shaped filter.
- Explain why CALCULATE destroys the shape of an arbitrarily shaped filter.
- Fix the broken Monthly Average using a unique-per-period column.
- Fix it alternatively using KEEPFILTERS.
- State the two best practices for writing measures safe against shaped filters.

Simple Filters vs Arbitrarily Shaped Filters

A **simple filter** is the Cartesian product of individual column filters. If you filter Year IN {2007, 2008} and Month = 'December', every combination (2007+Dec, 2008+Dec) is included — this is a simple filter. An **arbitrarily shaped filter** is a set of row combinations that *cannot* be expressed as a product of independent column filters. For example: {(2007, December), (2008, January)} — December only for 2007, January only for 2008. This shape appears when a user selects items from a hierarchy slicer in Power BI:

```
-- Arbitrarily shaped filter produced by a hierarchy slicer:
-- User selects: 2007 Q4 and 2008 Q1
-- Internal filter context (TREATAS representation):
TREATAS (
{
( "CY 2007", "September" ), ( "CY 2007", "October" ),
( "CY 2007", "November" ), ( "CY 2007", "December" ),
( "CY 2008", "January" ), ( "CY 2008", "February" ),
( "CY 2008", "March" )
},
'Date'[Calendar Year], 'Date'[Month]
)
-- This is NOT the same as (Year IN {2007,2008}) AND (Month IN {Sep..Mar})
-- which would include Sep-2008, Oct-2008, etc. — NOT what the user selected.
```

How CALCULATE Destroys the Filter Shape

```
-- Monthly Average measure:
[Monthly Avg WRONG] :=
AVERAGEX (
VALUES ( 'Date'[Month] ), -- iterates: Sep, Oct, Nov, Dec, Jan, Feb, Mar
[Sales Amount] -- CT fires per month name
)

-- Problem at Total level with the arbitrarily shaped filter:
-- VALUES('Date'[Month]) returns 7 distinct month names.
-- AVERAGEX iterates each month name.
-- CT on 'January' -> CALCULATE adds filter: 'Date'[Month] = 'January'
-- CALCULATE REPLACES the existing filter on [Month] (removes old, adds new).
-- The arbitrarily shaped filter said: January only in 2008.
-- After CALCULATE: [Month]='January' with no Year restriction -> Jan 2007 AND 2008!
-- Sales Amount for January = Jan 2007 + Jan 2008. Way higher than one month.
-- Grand Total AVERAGEX is averaging inflated values -> completely wrong.

-- Fix 1: Iterate a column that is UNIQUE per year+month combination
```

```
[Monthly Avg FIX1] :=
AVERAGEX (
VALUES ( 'Date'[Year Month Number] ), -- unique per year+month (e.g. 200701)
[Sales Amount] -- CT on unique col -> no year bleed
)
-- Year Month Number uniquely identifies each year+month pair.
-- CT on 200801 -> only January 2008 filtered. Correct.

-- Fix 2: KEEPFILTERS prevents CALCULATE from replacing the existing filter
[Monthly Avg FIX2] :=
AVERAGEX (
KEEPFILTERS ( VALUES ( 'Date'[Month] ) ),
[Sales Amount]
)
-- KEEPFILTERS wraps the VALUES table.
-- When AVERAGEX iterates 'January', instead of replacing the Month filter,
-- it INTERSECTS: existing filter AND 'January'.
-- In the shaped filter, January appears only with 2008. Intersection: Jan 2008 only.
-- The shaped filter shape is preserved. Correct.
```

CONCEPT

KEEPFILTERS semantics: Normally, when CALCULATE applies a filter to a column, it removes the previous filter on that column and replaces it with the new one. KEEPFILTERS changes this to: add the new filter ON TOP of the existing filter (intersection, not replacement). This preserves the arbitrarily shaped filter's constraints. Use KEEPFILTERS when an iterator might break hierarchically structured filter contexts.

Best Practices for Shaped Filter Safety

- **BP1 — Unique columns:** When iterating for monthly, quarterly, or weekly aggregations, always use a column that is unique at that granularity (Year Month Number, Year Quarter Number, ISO Week Sequential). Never iterate Month Name or Quarter Name — they repeat across years.
- **BP2 — KEEPFILTERS defence:** If BP1 cannot be applied (e.g. iterating a pre-existing column you cannot modify), wrap the VALUES call in KEEPFILTERS. Understand that KEEPFILTERS changes the semantics: values that are in VALUES but not in the current shaped filter will be excluded.

LESSON 8

Professional Patterns and Series Synthesis

Lesson Objectives

- Use ISINSCOPE to detect the hierarchy level in a matrix.
- Build a single Ratio-to-Parent measure that works at all hierarchy levels.
- Recall and connect the key patterns from all 10 modules.

ISINSCOPE — Hierarchy-Level Detection

ISINSCOPE(column) returns TRUE when the column is part of the current grouping in a matrix or visual. It is used to detect which level of a hierarchy is currently being browsed, enabling a single measure to produce different values at different levels:

```
-- Hierarchy: Product[Category] > Product[Subcategory] > Product[Product Name]
-- Goal: show ratio-to-parent at each level in a single measure.

[Ratio to Parent] :=
VAR CurrentSales = [Sales Amount]
VAR SubcategorySales =
CALCULATE ( [Sales Amount], ALLSELECTED ( 'Product'[Product Name] ) )
VAR CategorySales =
CALCULATE ( [Sales Amount], ALLSELECTED ( 'Product'[Subcategory] ) )
VAR TotalSales =
CALCULATE ( [Sales Amount], ALLSELECTED ( 'Product'[Category] ) )
VAR Result =
SWITCH (
TRUE(),
-- Check from INNERMOST to OUTERMOST level
ISINSCOPE ( 'Product'[Product Name] ),
DIVIDE ( CurrentSales, SubcategorySales ),
ISINSCOPE ( 'Product'[Subcategory] ),
DIVIDE ( CurrentSales, CategorySales ),
ISINSCOPE ( 'Product'[Category] ),
DIVIDE ( CurrentSales, TotalSales ),
BLANK() -- Total row: no scope
)
RETURN Result
-- At product level: product / its subcategory total
-- At subcategory level: subcategory / its category total
-- At category level: category / grand total
-- At total row: blank

-- IMPORTANT: evaluate conditions from innermost to outermost.
-- When Product Name is in scope, Subcategory and Category are ALSO in scope
-- (because the hierarchy always includes all parent levels).
-- Checking innermost first catches the most specific level correctly.
```

Module Series Synthesis — The Complete DAX Mental Model

This module completes a 10-module journey through DAX. The following table maps every major concept to its module, connecting the full knowledge graph:

Module	Core Concept	Key Functions / Patterns
1–2	What DAX is; row vs filter context fundamentals.	SUMX, CALCULATE, SUM, RELATED, measures vs columns
3	Table functions and set operations.	FILTER, ALL, VALUES, DISTINCT, ALLEXCEPT, ALLSELECTED, CALCULATETABLE
4	Evaluation contexts — row context, filter context, context transition.	CT via CALCULATE, measure reference in iterator, EARLIER
5	CALCULATE mechanics — filter arguments, modifiers, KEEPFILTERS.	CALCULATE, CALCULATETABLE, ALL*, KEEPFILTERS, USERRELATIONSHIP
6	Variables — freeze values, debug, performance.	VAR/RETURN, lazy evaluation, ADDCOLUMNS pre-computation
7	Iterators with context transition, granularity, RANKX.	SUMX, AVERAGEX, MAXX, RANKX, CONCATENATEX, granularity rule
8	Time intelligence — date table, YTD, PY, YOY, MAT, semi-additive.	DATESYTD, SAMEPERIODLASTYEAR, DATEADD, LASTNONBLANK, TREATAS
9	Calculation groups — eliminate measure explosion.	SELECTEDMEASURE, precedence, format strings, single-measure rule
10	Filter context inspection — read, test, and safely respond to context.	HASONEVALUE, SELECTEDVALUE, ISFILTERED, VALUES vs FILTERS, ALLEXCEPT trap, TREATAS, KEEPFILTERS

DEMONSTRATION 1

Filter Inspection Functions in Action

Scenario: The instructor builds a live filter-inspection dashboard showing ISFILTERED, ISCROSSFILTERED, HASONEVALUE, HASENEFILTER, VALUES, and FILTERS side by side, demonstrating exactly when each function returns TRUE or FALSE as different slicers and matrix levels are activated.

1	Create a matrix with Customer[Continent] on rows and Customer[Gender] on columns. Add five measures: ISFILTERED(Customer[Gender]), ISFILTERED(Customer[Continent]), ISCROSSFILTERED(Customer), ISCROSSFILTERED(Sales), COUNTROWS(Customer). Show the full matrix. Trace which cells show TRUE vs FALSE for each function.
2	Add a Product Category slicer. Select one category. Show that ISCROSSFILTERED(Sales)=TRUE propagates from Product through Sales to Customer (if relationships are bidirectional). Explain the relationship direction dependency.
3	Demonstrate the ISFILTERED limitation: add a measure CALCULATE(ISFILTERED(Customer[City]),Customer[City]<>'DAX'). Show it returns TRUE even though no city is filtered out. Then add COUNTROWS(Customer) vs COUNTROWS(ALL(Customer)) comparison to show 'truly filtered'.
4	Build the COLOR FILTER LABEL measure with VALUES vs FILTERS. Add a Category slicer and a Color slicer. Select a category that limits colors. Show that VALUES returns fewer colors than FILTERS for the same selection. Toggle between the two in the visual to make the difference vivid.
5	Build HASONEVALUE vs HASENEFILTER comparison. Apply a Gender filter for Male. Switch to a Customer Type = Company context. Show HASENEFILTER(Gender) = TRUE but HASONEVALUE(Gender) = FALSE (no Male companies).
RESULT	Full filter inspection dashboard built. VALUES vs FILTERS difference visually confirmed. HASENEFILTER/HASONEVALUE distinction demonstrated.

DEMONSTRATION 2

The ALLEXCEPT Trap and TREATAS

Scenario: The instructor demonstrates the ALLEXCEPT hidden trap, builds the correct ALL+VALUES pattern, and shows TREATAS for both a simple color filter and the semi-additive last-balance optimisation.

1	Build [AllExcept Type] = CALCULATE(COUNTROWS(Customer), ALLEXCEPT(Customer, Customer[CustomerType])). Place on matrix with Continent rows and Gender columns — results look correct. Then REMOVE Customer Type from the report. Show every row now displays the same grand total. Explain: ALLEXCEPT removed Gender filter but Customer Type was never in the report, so nothing remains to constrain the count.
2	Build [Correct By Type] = CALCULATE(COUNTROWS(Customer), ALL(Customer), VALUES(Customer[CustomerType])). Show the matrix without Customer Type. Confirm each cell shows the count for that type. Explain: ALL removes everything, VALUES re-adds only what is cross-filtered.
3	Demonstrate TREATAS: build [Red Blue Sales] = CALCULATE([Sales Amount], TREATAS({'Red','Blue'},Product[Color])). Show it correctly filters — contrast with the anonymous {"Red","Blue"} version that does nothing.
4	Demonstrate the arbitrarily shaped filter problem: use a hierarchy slicer to select Q4 2007 and Q1 2008. Place [Monthly Avg WRONG] and [Monthly Avg FIX1] on a card. Show they differ at Total. Explain why [Monthly Avg WRONG] is inflated (month name iterates across both years). Show [Monthly Avg FIX1] using Year Month Number is correct.
5	Add [Monthly Avg FIX2] with KEEPFILTERS. Confirm same correct result as FIX1. Explain the semantic difference: FIX1 avoids the problem through unique granularity; FIX2 solves it through intersection semantics.
RESULT	ALLEXCEPT trap demonstrated and fixed. TREATAS filter assignment confirmed. Arbitrarily shaped filter broken/fixed visually confirmed.

LAB 10

Working with the Filter Context

Lab Scenario

The Contoso BI team needs a set of context-aware, production-grade measures: protected QTD, a customer discount what-if slicer, a selected-colors label, a NonBuying customers metric, an ALLEXCEPT-corrected denominator, and a TREATAS-optimised semi-additive last balance. You will build, verify, and deliberately break-then-fix each measure.

NOTE

Open Contoso_Lab10_Start.pbix (extends Lab 9). The model includes Sales, Product, Customer, Date, and Balances tables. Save completed work as Contoso_Lab10_Complete.pbix.

Exercise 1: HASONEVALUE, SELECTEDVALUE, and Parameter Tables

Estimated time: 25 minutes

Task 1: Protect QTD with HASONEVALUE

1. Add [QTD Sales Raw] := CALCULATE([Sales Amount], DATESQTD('Date'[Date])).
2. Place on a matrix with Year > Month hierarchy. Observe the year level shows Q4 value (misleading). Note the value.
3. Add the protected version:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[QTD Sales] :=
  IF (
    HASONEVALUE ( 'Date'[Month Number] ),
    CALCULATE ( [Sales Amount], DATESQTD ( 'Date'[Date] ) )
  )" 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
  rise=0, text="[QTD Sales] :=", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
  ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
  link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
  ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
  text="IF (", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
  fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
  textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
  fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" HASONEVALUE ( 'Date'[Month
  Number] ),", textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
  fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
  textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
  fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=" CALCULATE ( [Sales Amount],
  DATESQTD ( 'Date'[Date] ) )", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
  ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
  link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
  ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
  text=")", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

4. Verify the year-level row is now blank. Month-level values unchanged.

Task 2: Discount What-If with SELECTEDVALUE

1. Create a new table: Discount = {0, 0.02, 0.05, 0.10, 0.15, 0.20} (values 0–20%).

2. Add a slicer on Discount[Discount Rate]. Add the following measures:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Selected Discount] := SELECTEDVALUE (
Discount[Discount Rate], 0 )
[Discounted Sales] := [Sales Amount] * ( 1 - [Selected Discount] )
[Discount Impact] := [Sales Amount] - [Discounted Sales]' 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Selected Discount] :=
SELECTEDVALUE ( Discount[Discount Rate], 0 )', textColor=Color(.101961,.227451,.360784,1),
us_lines=[]), ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0,
lineBreak=True, link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='[Discounted Sales] := [Sales Amount] * ( 1 - [Selected Discount] )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Discount Impact] := [Sales
Amount] - [Discounted Sales]', textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style':
'bulletText': None 'debug': 0 ) #Paragraph
```

3. Place on a card. Change the slicer to 10%. Confirm Discounted Sales = Sales Amount × 0.90.

Results: QTD blank at year level. SELECTEDVALUE reads discount rate correctly. Discounted Sales responds to slicer.

Exercise 2: ISFILTERED, ISCROSSFILTERED, and VALUES vs FILTERS

Estimated time: 20 minutes

Task 1: Filter inspection measures

1. Add these measures:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Brand Filtered?] := ISFILTERED ( 'Product'[Brand] )
[Product CrossF?] := ISCROSSFILTERED ( 'Product' )
[Sales CrossF?] := ISCROSSFILTERED ( Sales )" 'frags': [ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[Brand Filtered?] :=
ISFILTERED ( 'Product'[Brand] )", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="[Product CrossF?] := ISCROSSFILTERED ( 'Product' )",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[Sales CrossF?] :=
ISCROSSFILTERED ( Sales )", textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style':
'bulletText': None 'debug': 0 ) #Paragraph
```

2. Place on a matrix with Product[Category] on rows. Add a Brand slicer. Select one brand. Verify [Brand Filtered?] = TRUE only when Brand is directly filtered. Verify [Product CrossF?] = TRUE whenever any Product column is filtered.

Task 2: VALUES vs FILTERS label

1. Add:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Colors by VALUES] :=
CONCATENATEX(VALUES('Product'[Color]),'Product'[Color],",", 'Product'[Color],ASC)
```

```
[Colors by FILTERS] :=
CONCATENATEX(FILTERS('Product'[Color]),'Product'[Color]," ",'\Product'[Color],ASC)' 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='[Colors by VALUES] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CONCATENATEX(VALUES('Product'[Color]),'Product'[Color]," ",'\Product'[Color],ASC)',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text='[Colors by FILTERS] :=',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CONCATENATEX(FILTERS('Product'[Color]),'Product'[Color]," ",'\Product'[Color],ASC)',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Add a Category slicer (select 'TV and Video') and a Color slicer (select Azure, Blue, Red, Silver). Place both measures on cards. Confirm VALUES shows only 2 colors; FILTERS shows all 4.

Results: ISFILTERED responds correctly to direct vs indirect filters. VALUES vs FILTERS difference visible with Category+Color cross-filtering.

Exercise 3: The ALLEXCEPT Trap and Correct ALL+VALUES Pattern

Estimated time: 20 minutes

Task 1: Demonstrate the ALLEXCEPT trap

1. Add:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Count by Type ALLEXCEPT] :=
CALCULATE ( COUNTROWS(Customer), ALLEXCEPT(Customer, Customer[Customer Type]) )' 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='[Count by Type ALLEXCEPT] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CALCULATE ( COUNTROWS(Customer), ALLEXCEPT(Customer, Customer[Customer Type]) )',
textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

2. Place on a matrix with Customer[Continent] on rows. Add Customer[Customer Type] to the matrix. Values look correct. Then REMOVE Customer Type from the matrix. Observe every row now shows the same grand total. Confirm the ALLEXCEPT trap.

Task 2: Fix with ALL + VALUES

1. Add:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': '[Count by Type CORRECT] :=
CALCULATE (
```

```

COUNTROWS ( Customer ),
ALL ( Customer ),
VALUES ( Customer[Customer Type] )
)' 'frags': [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[],
rise=0, text='[Count by Type CORRECT] :=', textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='br', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True,
link=[], rise=0, text="", textColor=Color(.101961,.227451,.360784,1), us_lines=[]),
ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text='CALCULATE (' , textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br',
bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' COUNTROWS ( Customer ),',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' ALL ( Customer ),',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=' VALUES ( Customer[Customer
Type] )', textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text=')',
textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

- Without Customer Type on the matrix, verify [Count by Type CORRECT] shows correct segment counts for each continent row. Confirm it is stable whether Customer Type is on the report or not.

Results: ALLEXCEPT trap confirmed visually. ALL+VALUES fix validated independently of report columns.

Exercise 4: Data Lineage, TREATAS, and Shaped Filters

Estimated time: 20 minutes

Task 1: TREATAS for a simple filter

- Add:

```

Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Red Blue Sales] := CALCULATE([Sales Amount],
TREATAS({'Red','Blue'},Product[Color]))
[Anonymous Table Sales] := CALCULATE([Sales Amount], {'Red','Blue'}) -- wrong: no lineage" 'frags':
[ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
text="[Red Blue Sales] := CALCULATE([Sales Amount], TREATAS({'Red','Blue'},Product[Color]))",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[Anonymous Table Sales] :=
CALCULATE([Sales Amount], {'Red','Blue'}) -- wrong: no lineage",
textColor=Color(.101961,.227451,.360784,1), us_lines=[]) 'style': 'bulletText': None 'debug': 0 )
#Paragraph

```

2. Place both on cards. Verify [Red Blue Sales] shows correct filtered sales. Verify [Anonymous Table Sales] shows the same as [Sales Amount] (no filter applied).

Task 2: Arbitrarily shaped filter and KEEPFILTERS

1. If a Hierarchy Slicer is available, select Q4 2007 and Q1 2008 simultaneously. Otherwise use a Date table filter for Sep-Dec 2007 and Jan-Mar 2008 combined.

2. Add:

```
Paragraph( 'caseSensitive': 1 'encoding': 'utf8' 'text': "[Monthly Avg WRONG] :=
  AVERAGEX(VALUES('Date'[Month]), [Sales Amount])
  [Monthly Avg FIXED] := AVERAGEX(VALUES('Date'[Year Month Number]), [Sales Amount])" 'frags':
  [ParaFrag(__tag__='para', bold=0, fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0,
  text="[Monthly Avg WRONG] := AVERAGEX(VALUES('Date'[Month]), [Sales Amount])",
  textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='br', bold=0,
  fontName='Courier', fontSize=7.9, greek=0, italic=0, lineBreak=True, link=[], rise=0, text="",
  textColor=Color(.101961,.227451,.360784,1), us_lines=[]), ParaFrag(__tag__='para', bold=0,
  fontName='Courier', fontSize=7.9, greek=0, italic=0, link=[], rise=0, text="[Monthly Avg FIXED] :=
  AVERAGEX(VALUES('Date'[Year Month Number]), [Sales Amount])",
  textColor=Color(.101961,.227451,.360784,1), us_lines=[])] 'style': 'bulletText': None 'debug': 0 )
#Paragraph
```

3. Place both on cards at the Total level of the slicer selection. Confirm [Monthly Avg WRONG] is inflated (months appear in two years). Confirm [Monthly Avg FIXED] is correct.
4. Save as **Contoso_Lab10_Complete.pbix**.

Results: TREATAS lineage confirmed. Anonymous table filter confirmed to be ineffective. Arbitrarily shaped filter bug demonstrated and fixed.

Module Review, Series Recap, and Exam Readiness

Review Questions

What is the difference between HASONEVALUE and HASONEFILTER?

Answer: HASONEVALUE returns TRUE when exactly one value is VISIBLE in the current filter context — after all cross-filtering and relationship propagation. HASONEFILTER returns TRUE when exactly one filter value is ACTIVE on the column — regardless of whether that value currently produces any visible rows. Example: filter Gender = Male, but in the current context only Companies are visible (companies have no gender). HASONEFILTER(Gender) = TRUE (one filter: Male). HASONEVALUE(Gender) = FALSE (zero visible values). Use HASONEVALUE when you need to extract a value; HASONEFILTER when you need to know the user's selection.

What is the difference between VALUES and FILTERS?

Answer: VALUES returns the distinct values of a column that are VISIBLE in the current filter context — after all cross-filtering and relationship propagation. If no TV+Video products exist in Azure, VALUES('Product'[Color]) does not return Azure, even if Azure is selected in a color slicer. FILTERS returns all values that are ACTIVELY FILTERED by a slicer or visual, regardless of whether they currently have corresponding rows in the data. FILTERS('Product'[Color]) returns Azure even if no TV+Video products are Azure-coloured. Use VALUES for calculations; use FILTERS for user-facing labels that must reflect the slicer.

Describe the ALLEXCEPT hidden trap and the correct fix.

Answer: ALLEXCEPT(Table, Col) is a CALCULATE modifier that removes all direct filters from Table except those on Col. The trap: if Col is not in the current report, no filter on Col is active, so ALLEXCEPT removes ALL filters — leaving the measure to return the grand total for every row. The fix is ALL(Table) + VALUES(Col): ALL removes all direct filters, and VALUES re-adds the values that are visible in the cross-filtered context. VALUES picks up indirect filters (cross-filtering), so the measure remains correct regardless of whether Col appears in the report.

What is data lineage, and why does an anonymous table fail as a CALCULATE filter?

Answer: Data lineage is a hidden tag on each column value that identifies which column in the data model it comes from. When CALCULATE uses a table as a filter argument, it uses lineage tags to know which model columns to filter. An anonymous table (e.g. a constant table {"Red","Blue"}) has no lineage tags — DAX does not know which column these values belong to, so no filter is applied. TREATAS assigns lineage to an anonymous table, enabling CALCULATE to use it as a proper filter.

How does CALCULATE destroy an arbitrarily shaped filter, and what are the two fixes?

Answer: An arbitrarily shaped filter is a multi-column filter where not all combinations of values are included (e.g. {(2007, December), (2008, January)}). When CALCULATE iterates months inside AVERAGEX and applies a Month filter, it REPLACES the existing Month portion of the shaped filter — breaking the year+month coupling and allowing the same month from multiple years to be included. Fix 1: iterate a column that is unique per year+month (Year Month Number), so CALCULATE's replacement only affects one specific period. Fix 2: use KEEPFILTERS on VALUES — instead of replacing, KEEPFILTERS intersects the new filter with the existing shaped filter, preserving its constraints.

What is ISINSCOPE and how is it used in a Ratio-to-Parent measure?

Answer: ISINSCOPE(column) returns TRUE when the column is currently being used for grouping in a matrix or visual — meaning the user is browsing the hierarchy at that level. In a Ratio-to-Parent measure,

SWITCH(TRUE(), ISINSCOPE(Product Name), ..., ISINSCOPE(Subcategory), ..., ISINSCOPE(Category), ...) evaluates conditions from innermost to outermost hierarchy level. At product level: product / subcategory total. At subcategory: subcategory / category total. At category: category / grand total. At grand total: BLANK. The innermost check comes first because browsing a product also puts subcategory and category in scope.

Module 10 Quick Reference

Function	Returns / Does	Key Distinction
HASONEVALUE(<i>col</i>)	TRUE if exactly 1 value is visible in the current filter context.	Accounts for cross-filtering. Use to protect calculations from wrong granularities.
SELECTEDVALUE(<i>col</i> , <i>default</i>)	The single visible value, or default if 0 or >1 values visible.	Compact HASONEVALUE+VALUES pattern. Use for parameter/what-if tables.
ISFILTERED(<i>col</i>)	TRUE if a direct filter is active on the column.	Does NOT mean values are reduced. Only means a filter was applied.
ISCROSSFILTERED(<i>table</i>)	TRUE if the table/column is filtered directly or indirectly.	Use with a table argument (not column) for reliable cross-filter detection.
HASONEFILTER(<i>col</i>)	TRUE if exactly 1 filter value is active on the column.	May differ from HASONEVALUE when filter value has no matching rows.
VALUES(<i>col</i>)	Visible distinct values under current filter context.	Respects cross-filtering. May show fewer values than FILTERS if cross-filtering removes rows.
FILTERS(<i>col</i>)	All actively filtered values, regardless of visible rows.	Does NOT respect cross-filtering. Use for accurate slicer-selection labels.
ALLEXCEPT(<i>T</i> , <i>col</i>)	CALCULATE modifier: removes all direct filters on <i>T</i> except on <i>col</i> .	TRAP: if <i>col</i> has no active filter, ALL filters are removed -> grand total result.
ALL(<i>T</i>)+VALUES(<i>col</i>)	Removes all filters, then re-adds what is visible via cross-filtering.	Correct replacement for ALLEXCEPT. VALUES picks up indirect filters.
ISEMPTY(<i>table</i>)	TRUE if the table has zero rows in the current filter context.	More efficient than COUNTROWS=0. Stops scanning at first row found.
TREATAS(<i>table</i> , <i>col1</i> , ...)	Assigns data lineage to table columns.	Required when using computed/anonymous tables as CALCULATE filters.
KEEPFILTERS(<i>table</i>)	Wraps a table so CALCULATE intersects instead of replaces its filter.	Preserves arbitrarily shaped filters. Use in iterators over non-unique columns.
ISINSCOPE(<i>col</i>)	TRUE when <i>col</i> is part of current visual grouping (hierarchy drill-down).	Use to detect hierarchy level. Evaluate from innermost to outermost in SWITCH.

Series Complete — PL-300 Exam Readiness Summary

Congratulations — you have completed all 10 DAX modules. The following summary maps the series to the PL-300 exam domains.

PL-300 Domain	Coverage in This Series	Exam Weight
Prepare and transform data	Power Query M (earlier modules). Date table creation (Module 8). Data type handling (Modules 1–2).	15–20%
Model data	Star schema, relationships, CALCULATE, evaluation contexts (Modules 1–5). Time Intelligence (Module 8). Calculation Groups (Module 9).	30–35%
Visualize and analyze data	Filter context for visuals (Modules 4–5, 10). Hierarchy measures with ISINSCOPE (Module 10). Conditional measures with HASONEVALUE (Module 10).	25–30%

Deploy and maintain assets	Row-level security, RLS DAX filters (Module 5 CALCULATE). Performance and DirectQuery considerations (Modules 6–7).	20–25%
----------------------------	--	--------

Final exam preparation checklist:

- Know all evaluation context rules cold: row context, filter context, context transition, and how CALCULATE modifiers differ from filter arguments.
- Be able to write YTD, PY, YOY, MAT, and LASTNONBLANK from memory. Know the date table requirements and why the table must be marked.
- Know the calculation group single-measure rule and precedence.
- Know the ALLEXCEPT trap, the ALL+VALUES fix, and when to use KEEPFILTERS.
- Understand SELECTEDVALUE and HASONEVALUE for parameter/what-if tables.
- Know TREATAS: what data lineage is, why anonymous tables don't filter, how TREATAS assigns lineage.
- Know the two fixes for arbitrarily shaped filter safety (unique column and KEEPFILTERS).